COMPLETE REFERENCE · 8 TOOLS · ONE TOOLCHAIN

The MultiCloud DevOps Handbook

A single, consolidated reference covering the full delivery pipeline — from source control to infrastructure to a running cluster — assembled from the complete study notes on Git, Terraform, Maven, SonarQube, CI/CD, Docker, Kubernetes, and Ansible. Every theory note, diagram description, command table, and scenario-based interview Q&A from the source material is preserved in full.

Git & GitHub · Terraform · Maven · SonarQube · CI/CD · Docker · Kubernetes · Ansible

INDEX · JUMP TO A SECTION

GIT

Git & GitHub

Distributed version control, branching, and collaboration workflows.

16 sections

TF

Terraform

Infrastructure as Code — provisioning and managing cloud resources declaratively.

18 sections

MVN

Maven

Java project build automation, dependency management, and lifecycle.

07 sections

SQ

SonarQube

Static code analysis, quality gates, and security scanning.

07 sections

CI/CD

CI/CD

Jenkins, GitLab CI, and GitHub Actions — automated build, test & release pipelines.

12 sections

DKR

Docker

Containerization — images, containers, networking, volumes, and Compose.

11 sections

K8S

Kubernetes

Container orchestration at scale — workloads, networking, scaling, and GitOps.

21 sections

ANS

Ansible

Agentless configuration management, provisioning, and automation.

22 sections

GIT

PART 01 OF 08

Git & GitHub

Distributed version control, branching, and collaboration workflows.

Document Legend: 💡 Blue = Layman Explanation | 📝 Green = Theory & Key Points | 🎯 Yellow = Scenario Interview Q&A | ⚠️ Orange = Warning/Caution | Dark = Commands/Code

1. What is Git?

Git is a free, open-source **Distributed Version Control System (DVCS)** created by **Linus Torvalds in 2005** originally for managing the Linux kernel source code. It is now the most widely used version control system in the world, used by millions of developers and companies including Google, Facebook, Microsoft, Netflix, and Amazon.

What does Version Control mean? Version control means tracking every change made to files over time — recording who made the change, when it was made, and why (via commit messages). This allows any version of the project to be retrieved at any point in the future.

Why Distributed? In a distributed system, every developer's machine holds a complete, independent copy of the entire repository — including its full commit history, all branches, and all versions. You can commit, branch, merge, and view logs completely offline. There is no single point of failure.

How Git stores data: Git does NOT store files as incremental differences (deltas). Instead, Git stores a snapshot of the entire project every time you commit. If a file hasn't changed, Git stores a reference

(link) to the previous identical snapshot, not a duplicate copy. This makes Git operations extremely fast.

> 💡 **Layman Explanation:**
>
> Think of Git as a 'time machine' for your code. Every commit = a full photograph of your project at that exact moment. You can travel back to ANY past commit and see exactly what the code looked like. Multiple developers each have a complete copy — no single point of failure. If GitHub goes down, every developer's local machine still has the full history.

> 📝 **Theory & Key Points:**
>
> - Git was created in 2005 by Linus Torvalds after BitKeeper (the previous tool) revoked its free license.
> - Git was designed for speed, data integrity, and support for distributed, non-linear workflows.
> - The `.git` folder contains the entire database — objects, refs, config, HEAD pointer, and the index (staging area).
> - Git uses SHA-1 hashing to generate a 40-character unique ID for every commit, tree, and blob.
> - SHA-1 ensures data integrity — if any file is corrupted, the hash won't match and Git will detect it.
> - The three core Git objects: **Blob** (file content), **Tree** (directory), **Commit** (snapshot + metadata).

Git (Distributed) vs SVN (Centralized):

ASPECT	GIT (DISTRIBUTED)	SVN (CENTRALIZED)
Repository Location	Full copy on every developer's machine	Single central server
Offline Work	Fully supported — commit, branch, log, diff	Requires network for most operations
Speed	Most operations are local (milliseconds)	Network round-trips (seconds)
Failure Risk	Low — any clone is a complete backup	High — server failure stops all work
Branching	Instant and nearly free	Heavyweight and slow
Merge Strategy	Advanced 3-way merge algorithm	Basic merge, conflict-prone
Storage Model	Snapshots of entire project	Delta (differences) between versions

🎯 **Scenario-Based Interview Questions**

Q1: Your GitHub is down. Can your team still work? How? Yes — Git is distributed. Every developer's local machine has the complete repository history. Developers can: commit new changes locally, create/switch branches, view full commit history, merge branches, and resolve conflicts — all without any network. When GitHub comes back, they push their local commits. No work is lost. This is the core distributed advantage.

Q2: A teammate asks 'Why use Git when we can just share a folder on Google Drive?' How do you explain it? A shared folder has no: tracking of who changed what, ability to merge concurrent changes, rollback to a previous version, or conflict detection. With Git: every change is recorded with author + timestamp + message. Two people can edit different files simultaneously and Git merges them. If someone breaks the build, you revert to any past snapshot in seconds. You also get branching — isolated experiments that don't affect others.

---

2. Installation & First-Time Setup

After installing Git, you must configure your identity. This is not for authentication — it is your **author signature** embedded inside every commit you create. Teams use this to track who made which change.

Fig: GIT (Version Control Tool) vs GitHub (Cloud Based Remote Repository) — the workflow shows Working Directory → Staging → Local Repo → git push/pull → GitHub → Local Repo (second

developer) → Staging → Working Directory, using `git status`, `git add`, `git commit`, `git push`, `git pull`.

> 📝 **Theory & Key Points:**
>
> - Git configuration has 3 levels: `--system` (all users on OS), `--global` (current user, all repos), `--local` (current repo only).
> - Local config overrides global, which overrides system — most specific wins.
> - `git config --global` is the most commonly used — sets name/email for all repos on your machine.
> - Config is stored in: `~/.gitconfig` (global) or `.git/config` (local per repo).
> - Without `user.name` and `user.email` configured, Git will block commits or use OS defaults.

Installation:

PLATFORM	COMMAND / URL
Windows / Mac / Linux	https://git-scm.com/downloads
Linux (RHEL/CentOS/Amazon Linux)	`sudo yum install git -y`
Ubuntu/Debian	`sudo apt-get install git -y`
Verify installation	`git --version`

```bash
# Set your name — shown on every commit you create
git config --global user.name "Your Name"

# Set your email — must match your GitHub account email for contribution graphs
git config --global user.email "you@example.com"

# Verify your configuration
git config --global user.name
git config --global user.email
git config --list          # show all config values

# Initialize a brand-new Git repository in current folder
git init
# This creates a hidden .git folder — Git now tracks this directory
```

What is inside the `.git` folder?

FILE / FOLDER	WHAT IT CONTAINS
HEAD	Pointer to the currently active branch (e.g., `ref: refs/heads/main`)
objects/	Database of all commits, files (blobs), and directory structures (trees)
refs/heads/	One file per branch — each file contains the latest commit hash of that branch
refs/tags/	One file per tag — permanent release markers
config	Repository-level configuration (remote URL, branch tracking, etc.)
index	The Staging Area — a binary file tracking what will go in the next commit
COMMIT_EDITMSG	The message from the most recent commit

🎯 **Scenario-Based Interview Questions**

Q1: A new developer joins. They set up Git and make commits, but their name doesn't appear in GitHub's contribution graph. What's wrong? The email in their git config does not match the email registered on their GitHub account. Fix: `git config --global user.email 'dev@company.com'` (must be the same email as GitHub account settings). GitHub uses the commit's author email to map contributions to user profiles.

Q2: What is 'git init' and what exactly does it create? `git init` initialises a new Git repository by creating a `.git` folder in the current directory. Inside `.git`: objects/ (stores all commit, file, tree objects), refs/heads/ (branch pointers), HEAD (points to current branch), index (staging area), config (local repository settings). Deleting the `.git` folder completely removes Git tracking from the project (files remain untouched).

---

3. The Core Git Workflow

Every file change in Git moves through a precise lifecycle. Mastering the four zones — Working Directory, Staging Area, Local Repository, and Remote Repository — is the foundation of understanding all Git commands.

Fig: Complete Git Workflow diagram — Workspace → (git add) → Staging Area (Index) → (git commit -m) → Local Repository → (git push/git fetch/git pull) → Remote Repository (GitHub/GitLab). Also shows git diff, git diff --staged, git diff HEAD, git reset --soft/mixed/hard HEAD~1, git restore, git stash/stash apply/stash pop, git branch commands, and git pull fast-forward/no-rebase/rebase, git revert, git log, git status, .gitignore.

3.1 The Four Zones Explained

- **Zone 1 — Working Directory:** Your local filesystem — the actual files on your computer. When you create a new file or edit an existing one, changes happen here first. Git can **see** these changes but does not track them yet. Files can be 'untracked' (brand new, never seen by Git) or 'modified' (previously committed, now changed).
- **Zone 2 — Staging Area (Index):** A preparation zone between your edits and your commit. You explicitly choose which changes to include in the next commit using `git add`. This lets you commit only a subset of your changes. The staging area is stored as the `.git/index` binary file.
- **Zone 3 — Local Repository:** The permanent commit database stored in the `.git/objects` folder on your machine. When you run `git commit`, Git takes a snapshot of everything in the staging

area and stores it here as a new commit object with a unique SHA-1 hash. Fully offline.

- **Zone 4 — Remote Repository:** A server-hosted copy of your repository (GitHub, GitLab, Bitbucket). `git push` uploads your local commits to the remote. `git pull` downloads and merges remote commits into your local branch.

> 💡 **Layman Explanation:**
>
> - Working Directory = your desk (you write and edit here)
> - Staging Area = the outbox tray (you select what to send — before committing)
> - Local Repository = your personal filing cabinet (committed, permanent on your machine)
> - Remote Repository = the team's shared cloud office (GitHub — everyone syncs here)
> - `git add` moves changes: Working Dir → Staging Area
> - `git commit` moves changes: Staging Area → Local Repository
> - `git push` moves changes: Local Repository → Remote Repository

COMMAND	WHAT IT DOES
`git status`	Show file status — untracked, modified, staged
`git add <file>`	Stage a specific file
`git add .`	Stage ALL modified and new files
`git add -p`	Interactively stage specific hunks (parts) of a file
`git commit -m 'msg'`	Commit staged changes with a message
`git commit -am 'msg'`	Stage all tracked modified files AND commit in one step
`git push`	Upload local commits to remote
`git pull`	Download remote changes and merge into current branch
`git fetch`	Download remote changes WITHOUT merging
`git clone <url>`	Copy a full remote repository locally (first time)
`git log --oneline`	Compact one-line commit history
`git log --oneline --graph`	Visual ASCII branch/merge graph
`git show <hash>`	Show full details of a specific commit

📝 **Theory & Key Points:**

- `git status` is the single most important command — always run it before staging or committing.
- Write meaningful commit messages: 'Fix null pointer exception in user auth service' — not 'fix' or 'update'.
- Commit often in small, logical units. Each commit should represent one complete thought or change.
- `git add -p` (patch mode) lets you stage specific lines within a file — powerful for clean commits.
- `git commit -am` skips `git add` but only works for files already tracked (not brand-new files).
- `git log --oneline --graph --all` gives a complete visual picture of all branches and merges.

🎯 **Scenario-Based Interview Questions**

Q1: You changed 5 files. Only 3 are related to the bug fix you're committing. How do you commit only those 3?

```text
git add file1.py file2.py file3.py   # stage only the 3 relevant files
git status                           # verify only those 3 are staged
git commit -m 'Fix authentication bug'
```

The other 2 files stay in your Working Directory, unstaged. This keeps commit history clean and logical. For finer control: `git add -p` allows staging specific lines within a file.

Q2: You accidentally staged a file with 'git add .' that you didn't mean to include. How do you fix it?

```text
git restore --staged <filename>   # un-stages the file, moves it back to Working
Directory
git status                        # verify it's no longer staged
git add file1.py file2.py
git commit -m 'Correct commit'
```

The file's changes are preserved — only the staging is undone.

---

4. GitHub & Remote Repositories

GitHub is a cloud-based platform (owned by Microsoft) that hosts Git repositories and adds powerful collaboration features on top of plain Git: web interface, team management, pull request workflows, code review, CI/CD automation (GitHub Actions), issue tracking, wikis.

Other platforms: **GitLab** (self-hosted or cloud, strong CI/CD), **Bitbucket** (Atlassian ecosystem, Jira integration), **Azure DevOps Repos** (Microsoft enterprise), **AWS CodeCommit** (AWS-native).

📝 **Theory & Key Points:**

- Git ≠ GitHub. Git is the version control engine (local). GitHub is a hosting platform (remote).
- A Remote is any Git repository hosted on a server — not necessarily GitHub.
- 'origin' is the default name (alias) for the remote you cloned from — just a shortcut for the URL.
- You can have multiple remotes: origin (your fork), upstream (original repo), production (deploy server).
- `git remote -v` shows all configured remote names and their URLs.
- GitHub Actions can automatically trigger builds, tests, and deployments when you push or create a tag.

4.1 SSH Authentication — How It Works

SSH (Secure Shell) uses a key pair: a private key (stays secret on your machine) and a public key (given to GitHub). When you push, your machine proves it holds the matching private key without ever sending it.

BASH

```
# Step 1: Generate key pair
ssh-keygen -t ed25519 -C 'your@email.com'
# Saves to: ~/.ssh/id_ed25519 (private) and ~/.ssh/id_ed25519.pub (public)

# Step 2: Display and copy the public key
cat ~/.ssh/id_ed25519.pub

# Step 3: Add to GitHub
# GitHub → Settings → SSH and GPG Keys → New SSH Key → Paste

# Step 4: Test the connection
ssh -T git@github.com
# Expected: Hi username! You've successfully authenticated.

# Step 5: Set remote to SSH format
git remote add origin git@github.com:username/repo.git
```

4.2 Personal Access Token (HTTPS)

GitHub deprecated password authentication in August 2021. Personal Access Tokens (PATs) replace passwords. A PAT (`ghp_...`) can have specific permissions (scopes), expiry dates, and can be revoked individually.

```bash
# Generate: GitHub → Settings → Developer Settings → Personal Access Tokens
# Select scopes: repo (full), workflow (for Actions), etc.

# Use inline with push:
git push https://<TOKEN>@github.com/username/repo.git

# Set as permanent remote (token embedded in URL):
git remote set-url origin https://<TOKEN>@github.com/username/repo.git
```

4.3 Remote Management Commands

COMMAND	PURPOSE
`git remote -v`	List all remotes with their URLs
`git remote add origin <url>`	Add new remote named 'origin'
`git remote remove origin`	Remove the 'origin' remote
`git remote set-url origin <url>`	Change the URL of 'origin'
`git remote rename origin upstream`	Rename remote from 'origin' to 'upstream'
`git remote add upstream <url>`	Add a second remote (common in fork workflows)

🎯 **Scenario-Based Interview Questions**

Q1: You cloned a repo using HTTPS. Now you want to switch to SSH to avoid entering tokens repeatedly. Walk through the steps.

```text
git remote -v                                          # check current URL (shows
https://...)
git remote set-url origin git@github.com:username/repo.git
git remote -v                                          # verify SSH URL
git push                                               # now uses SSH — no
password/token prompt
```

Also verify SSH is set up: `ssh -T git@github.com`

Q2: Your Personal Access Token expired and all git push commands fail. What do you do?

1. GitHub → Settings → Developer Settings → Personal Access Tokens → Generate new token
2. Select required scopes (repo, workflow) and set an appropriate expiry
3. `git remote set-url origin https://<NEW-TOKEN>@github.com/username/repo.git`
4. `git push` — should work now Best practice: store tokens in a credential manager or use SSH to avoid this recurring issue.

---

5. Git Branches

A branch in Git is an extremely lightweight **pointer (reference)** to a specific commit. Creating a branch does not copy any files — Git simply writes a tiny file in `.git/refs/heads/` containing the commit hash. This makes branch creation practically instant and free, regardless of project size.

Why branches matter: Without branches, all developers would commit to a single line of history. Branches give each developer (or feature) an isolated workspace that is completely independent of the main codebase until intentionally merged.

HEAD: HEAD is a special pointer stored in `.git/HEAD`. It always points to your currently active branch. When you make a new commit, your current branch pointer moves forward automatically, and HEAD moves with it.

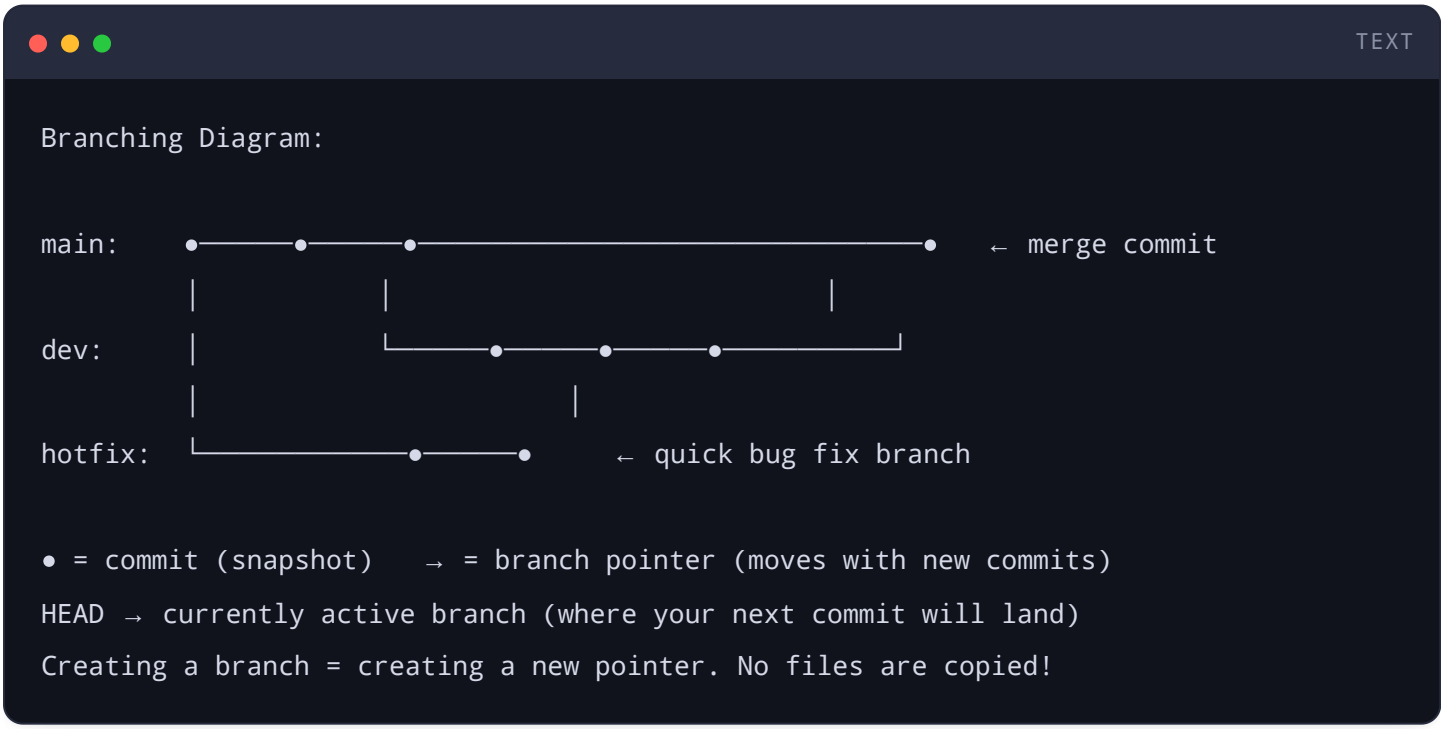

Fig: Real-World Git Branch Strategy — Dev / Test / Prod Environments with PR Workflow. Shows three environments (Prod, Testing, Development) each fed by feature branches. Real-world flow noted in the diagram: change instance type t2.medium → t2.large; git pull from main branch; create individual branch; instance type modification; git add; git commit; git push to individual branch; raise the pull request to your senior to merge into main.

5.1 Branch Commands

COMMAND	WHAT IT DOES
`git branch`	List all local branches (* = current)
`git branch -a`	List local AND remote tracking branches
`git branch -r`	List only remote branches
`git branch dev`	Create a new branch called 'dev'
`git checkout dev`	Switch to existing 'dev' branch
`git checkout -b dev`	Create AND switch to 'dev' in one command
`git switch dev`	Modern way to switch branches (Git 2.23+)
`git switch -c dev`	Modern way to create and switch
`git branch -m old new`	Rename a branch
`git branch -d dev`	Safe-delete (only if fully merged)
`git branch -D dev`	Force-delete (even if unmerged)
`git merge dev`	Merge 'dev' into current branch
`git push -u origin dev`	Push branch to remote + set upstream tracking
`git push origin --delete dev`	Delete a remote branch

📝 **Theory & Key Points:**

- Branch naming conventions: `feature/login-page`, `bugfix/null-pointer`, `hotfix/payment-crash`, `release/v2.0`
- HEAD → main means you are on main branch. Detached HEAD means HEAD points to a commit (not a branch).
- Fast-forward merge: when main has no new commits since branching — Git moves the pointer forward. No merge commit.
- 3-way merge: when both branches diverged — Git finds the common ancestor and creates a merge commit.
- `git push -u origin dev` sets the upstream tracking — future `git push` / `git pull` work without arguments.
- Real-world: never commit directly to main. Always use feature branches + Pull Requests.
- In GitFlow: main = production, develop = integration, feature/* = individual work, release/* = QA, hotfix/* = urgent fixes.

5.2 Real-World Branch Workflow

BASH

```
# Standard real-world feature development flow:
git checkout main
git pull                              # always start from latest main
git checkout -b feature/login         # create your feature branch

# ... write code ...
git add .
git commit -m 'Add login form with validation'

# ... more commits ...
git push -u origin feature/login      # push to remote

# On GitHub: open Pull Request from feature/login → main
# Teammates review, approve, CI/CD passes → Merge!

# After merge, clean up:
git checkout main
git pull
git branch -d feature/login               # delete local branch
git push origin --delete feature/login    # delete remote branch
```

🎯 **Scenario-Based Interview Questions**

Q1: Your team has three environments — Development, Testing, and Production. How do Git branches map to this real-world setup?

- `main` branch → Production (only stable, tested, reviewed code)
- `staging/test` branch → Testing/QA environment
- `develop/dev` branch → Development environment (integrated features)
- `feature/*` branches → Individual developer workspaces

Flow: Developer creates feature/xyz branch → commits → pushes → raises PR → Tech Lead reviews → merges to dev → QA tests on staging → Approved → merged to main → deployed to production. Branch protection rules prevent direct pushes to main and staging.

Q2: A junior dev committed directly to main by accident. You discover this after they pushed. What do you do and how do you prevent it in future? Immediate fix:

```text
git log --oneline        # find the bad commit hash (e.g., abc1234)
git revert abc1234       # create a new commit that undoes it (safe — history preserved)
git push
```

Prevent in future: Enable Branch Protection on main (require PR before merging, require 1+ approvals, require CI status checks). Now nobody — not even admins — can push directly to main.

---

6. Push Issues & Pull Strategies

When two or more developers work on the same branch, their local commit histories can diverge. Git prevents a push if the remote has commits your local branch doesn't have — otherwise you would silently overwrite your teammate's work.

Why does Git reject the push? Git uses a fast-forward push by default. A fast-forward means your branch is simply ahead of the remote. If the remote has its own new commits (diverged), a fast-forward is impossible. Git rejects the push to protect the remote branch's history.

Fig: Git Pull Methods — Fast-Forward, Merge & Rebase Compared with Real Scenario. Shows two developers (HYD, BLR) working via a shared repo "ProjectK", illustrating git clone, ssh-keygen, git pull = fetch + merge, and the difference between merge (creates a new extra commit on top of the recent commit, keeps history) and rebase (linear operation, no extra history).

📝 **Theory & Key Points:**

- `git pull` = `git fetch` + `git merge` (by default). It downloads AND immediately merges.
- `git fetch` = download only. Remote-tracking branches (origin/main) are updated but YOUR branch is unchanged.
- Fast-forward pull: only works when you have no new local commits.
- `git pull --no-rebase`: creates a merge commit. Full history preserved. Safe for teams. Messy graph.
- `git pull --rebase`: replays your commits on top. Linear, clean history. Rewrites your local commit hashes.
- RULE: Never rebase commits that have already been pushed to a shared/public branch — causes history divergence.
- `git push --force-with-lease`: safer than `--force`. Fails if the remote changed since your last fetch.

STRATEGY	RESULT	WHEN TO USE
`git pull` (fast-forward)	Pointer moves forward, no merge commit	You have no local commits — remote has new ones only
`git pull --no-rebase`	Merge commit created — joins both histories	Team collaboration, want full history preserved
`git pull --rebase`	Your commits replayed on top — linear history	Solo branches, before raising a PR for clean history

🎯 **Scenario-Based Interview Questions**

Q1: Dev A and Dev B both pull main at 9 AM. Dev A pushes at 10 AM. Dev B pushes at 11 AM and gets a non-fast-forward rejection. Walk through exactly what Dev B should do.

```text
git pull --no-rebase   # downloads Dev A's commit and merges with Dev B's local commit
# If no conflict: auto-merge with a merge commit created
# If conflict: resolve manually (open file, fix, git add, git commit)
git push               # now succeeds
```

Root cause: Both started from the same base. Dev A pushed first. Dev B should pull more frequently to stay in sync.

Q2: What is the difference between 'git fetch' and 'git pull'? When would you use fetch? `git fetch`: Downloads all remote changes into remote-tracking branches but does NOT touch your working branch. Completely safe. `git pull`: fetch + merge (or rebase); immediately applies remote changes. Use fetch to preview remote changes before merging: `git fetch && git diff main origin/main`, or to check out a remote branch without merging: `git fetch origin && git checkout -b dev origin/dev`.

---

7. Merge Conflicts

A merge conflict occurs when Git tries to automatically combine two branches but finds **contradictory changes in the same lines** of the same file. Git's automatic merge algorithm cannot decide which version is correct, so it pauses the merge, marks the conflicting sections, and requires a human to resolve them.

When do conflicts happen?

- Same branch conflict: Dev A and Dev B both edit line 42 of app.py on the same branch, Dev A pushes first
- Branch merge conflict: merging feature/login into main, both branches modified the same lines
- Rebase conflict: your commit modifies a line also changed in the commits you are rebasing onto

- Cherry-pick conflict: the commit you're cherry-picking touches lines already modified in the target branch

How Git marks conflicts:

```text
CONFLICT MARKERS explained:

<<<<<<< HEAD
console.log('Version from YOUR current branch');
=======
console.log('Version from the branch being merged in');
>>>>>>> feature/login

<<<<<<< HEAD   = start of YOUR version (current branch — where HEAD points)
=======        = divider separating the two versions
>>>>>>> branch = end of INCOMING version (the branch being merged)

You must: choose one version, combine them, or write something entirely new
Then DELETE all three marker lines (<<<<, ====, >>>>) before staging
```

📝 **Theory & Key Points:**

- `git status` shows conflicted files as 'both modified' — these are the files you need to fix.
- Never commit with conflict markers still in the file — it will break your application.
- VS Code shows both versions side-by-side with 'Accept Current', 'Accept Incoming', 'Accept Both' buttons.
- `git mergetool` opens a visual 3-way diff tool (vimdiff, kdiff3, etc.) — configured via `git config merge.tool`.
- After resolving ALL conflicts in ALL files, run `git add <file>` for each, then `git commit`.
- Prevention: Pull frequently, use short-lived branches, coordinate who owns which files.

7.1 Step-by-Step Conflict Resolution

STEP	ACTION
Step 1 — Find conflicts	`git status` → files listed under 'both modified'
Step 2 — Open each file	Use VS Code / IntelliJ — locate the `<<<<<<<` markers
Step 3 — Resolve	Edit: keep right code, combine, or rewrite. Remove ALL 3 marker lines
Step 4 — Stage the fix	`git add <filename>` — mark this file as conflict-resolved
Step 5 — Commit	`git commit -m 'Resolve merge conflict in auth.py'`
Step 6 — Push	`git push`
Step 7 — Test	Run the application and tests — merged logic may have new bugs

⚠️ **Important Warning:**

Never leave conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`) in committed code — it will crash your app. After resolving, ALWAYS test the application. In VS Code: open the conflicted file → click 'Resolve in Merge Editor' for a visual side-by-side resolution.

🎯 **Scenario-Based Interview Questions**

Q1: You're merging feature/payment into main and get a conflict in payment.py. Walk through exactly what you do, including commands.

```text
git merge feature/payment    # conflict triggers, merge pauses
git status                   # shows: both modified: payment.py
# Open payment.py, find markers, decide final code, delete marker lines
git add payment.py
git commit -m 'Resolve conflict: merge payment module'
git push
# Then: run full test suite to verify merged code works correctly
```

Q2: You and your teammate both updated the same config file this week and now conflicts appear every merge. How do you prevent this long-term?

1. Split config into module-specific files — each team owns their section
2. Use feature flags or environment variables instead of shared config changes
3. Communicate changes via PR description before merging
4. Use CODEOWNERS file in GitHub — assign config file ownership to one person
5. Merge more frequently — smaller PRs reduce conflict surface area
6. Consider a config management tool (Ansible, Terraform) for infrastructure configs

---

8. Git Diff, Reset & Revert

These three commands deal with inspecting and undoing changes. Understanding the **exact zone** where your change currently lives determines which command to use. Using the wrong command — especially `git reset --hard` on shared branches — can cause serious data loss and team disruption.

Fig: Git Diff, Revert & Reset — Complete Visual Reference of All Undo Commands. Shows Working Directory ↔ Staging Area (Index) ↔ Local Repo ↔ Remote Repo, with git diff / git diff --staged / git diff HEAD / git diff main origin/main comparing each zone; git restore / git restore --staged discarding

or unstaging; git reset --soft/--mixed/--hard HEAD~1 acting on Local Repo; git revert creating a new commit that cancels an older commit's changes; plus git add, git commit, git push, git merge, git pull, git fetch, git checkout, git clone.

8.1 Git Diff — Inspect Changes Before Acting

`git diff` is a diagnostic tool. Lines prefixed with `+` are additions; lines with `-` are removals. The output is called a 'patch' or 'diff'.

COMMAND	WHAT IT COMPARES
`git diff`	Working Dir ↔ Staging Area — what you changed but haven't staged
`git diff --staged`	Staging Area ↔ Last Commit — exactly what WILL be in next commit
`git diff HEAD`	Working Dir ↔ Last Commit — total pending changes
`git diff main origin/main`	Local main ↔ Remote main — what a pull would bring
`git diff branch1 branch2`	Compare any two branches
`git diff <hash1> <hash2>`	Compare two specific commits
`git diff --name-only`	Show only file names that changed
`git diff --stat`	Show file names + lines added/removed summary

8.2 Restore — Undo Before Committing

COMMAND	EFFECT
`git restore <file>`	Discard Working Dir changes — revert file to last committed version
`git checkout -- <file>`	Older equivalent of git restore (still works)
`git restore --staged <file>`	Unstage a file — move it back from Staging Area to Working Dir
`git restore --staged .`	Unstage ALL staged files at once
`git clean -fd`	Delete all untracked files and directories (no undo!) ⚠️

8.3 Git Reset — Undo Commits (Local Only)

`git reset` moves the current branch pointer backward to a previous commit, effectively removing newer commits from the branch's history.

```text
BEFORE RESET:
main: A ── B ── C ── D   (HEAD at D)

git reset --soft HEAD~2:
main: A ── B   (C and D changes are IN STAGING AREA — ready to re-commit)

git reset --mixed HEAD~2 (DEFAULT):
main: A ── B   (C and D changes are in WORKING DIR — unstaged)

git reset --hard HEAD~2:
main: A ── B   (C and D changes are PERMANENTLY DELETED from all areas ⚠️)
```

COMMAND	EFFECT
`git reset --soft HEAD~1`	Undo last commit. Changes land in Staging Area.
`git reset --mixed HEAD~1`	Undo last commit. Changes land in Working Dir. (DEFAULT)
`git reset --hard HEAD~1`	Undo last commit AND delete all changes permanently
`git reset <commitID>`	Reset HEAD to any specific commit
`git reset HEAD~3`	Go back 3 commits (mixed mode by default)

⚠️ **Important Warning:**

`git reset --hard` permanently deletes your changes from ALL zones. No undo. Only on local unpushed commits. NEVER use `git reset` on commits already pushed to a shared branch — it rewrites history and causes divergence for teammates. Recovery: `git reflog` shows all HEAD movements; recover 'lost' commits within 90 days using `git reset --hard <old-hash>`.

8.4 Git Revert — Safe Undo on Shared Branches

`git revert` creates a brand new commit that introduces the exact opposite changes of a specified commit. The original commit still exists in history. This is the safe way to undo changes already pushed to shared branches — it **never rewrites history**.

```text
BEFORE: A ── B ── C ── D   (D has a bug)
git revert D (creates D' which reverses D's changes):
AFTER:  A ── B ── C ── D ── D'
D still exists in history. D' cancels its effect. Everyone can safely git pull.
```

COMMAND	PURPOSE
`git revert <hash>`	Create new 'undo' commit reversing the specified commit
`git revert --no-edit <hash>`	Revert without opening editor for commit message
`git revert HEAD`	Revert the most recent commit
`git revert abc^..def`	Revert a range of commits (each gets a revert commit)

📝 **Reset vs Revert decision tree:**

Is the commit pushed to a shared branch? → YES: use `git revert` (safe). → NO: use `git reset` (OK because nobody else has it yet). `git reset --soft` is useful for 'amending' multiple commits into one before pushing. `git revert` is the standard way to 'undo' a deployment.

🎯 **Scenario-Based Interview Questions**

Q1: A bad feature was pushed to main last Friday and is now in production breaking users. Other developers have already pulled it. How do you fix it? Use `git revert` — NOT `git reset`:

```text
git log --oneline       # find the bad commit hash (e.g., bad1234)
git revert bad1234      # creates new commit 'Revert: bad feature'
git push                # push the revert commit to main
```

All teammates simply run `git pull` and the bug is gone. NEVER use `git reset` here — it would rewrite history, forcing every teammate to re-clone. After the revert: fix the bug properly on a feature branch, test, merge via PR.

Q2: You accidentally ran 'git reset --hard HEAD~3' locally and lost 3 commits. You haven't pushed. Can you recover them? YES — use `git reflog`:

```text
git reflog
# abc1234 HEAD@{0}: reset: moving to HEAD~3   ← current (after reset)
# def5678 HEAD@{1}: commit: Add user dashboard ← where you were before
git reset --hard def5678            # restore to your 3 commits
# OR: git checkout -b recovery-branch def5678
```

Reflog tracks all HEAD movements locally for 90 days. `git reset --hard` only moves the branch pointer — the actual commit objects remain until garbage collection.

---

9. Git Stash

`git stash` temporarily saves your uncommitted work (both staged and unstaged changes) to a special internal stack, and reverts your working directory to a clean state (matching the last commit). Your changes are not lost.

Why is it needed? Git blocks branch switching if you have uncommitted changes that conflict with the target branch. Stash allows you to save your WIP, switch to another branch, do urgent work, then restore your WIP exactly as it was.

💡 **Layman Explanation:**

Stash = a temporary locker for half-finished code.

```text
git stash              → locks your WIP away, working dir is now clean
git checkout main      → switch to main safely
... fix bug, commit, push ...
git checkout feature/my-branch
git stash pop          → unlocks your WIP — exactly where you left off
```

📝 **Theory & Key Points:**

- The stash is stored as a stack (LIFO). `git stash pop` restores the most recent.
- `git stash` saves: tracked modified files + staged changes. Does NOT save untracked files by default.
- `git stash -u` (`--include-untracked`): also stashes untracked (new) files.
- `git stash -a` (`--all`): stashes everything including files in `.gitignore`.
- You can have multiple stashes simultaneously. `git stash list` shows all of them.
- Stashes are not pushed to remote — local only. Auto-named: 'WIP on branchname: commitHash commit message'.

COMMAND	PURPOSE
`git stash`	Save all uncommitted changes (tracked files)
`git stash -u`	Save including untracked (new) files
`git stash push -m 'msg'`	Stash with a custom descriptive label
`git stash list`	Show all stash entries with index
`git stash show stash@{0}`	Summary of what's in stash@{0}
`git stash show -p stash@{0}`	Full diff of stash@{0}
`git stash pop`	Restore latest stash AND remove from list
`git stash apply stash@{1}`	Restore specific stash but KEEP it in list
`git stash drop stash@{2}`	Delete a specific stash without restoring
`git stash clear`	Delete ALL stashes permanently
`git stash branch newbranch`	Create new branch from stash + apply it

🎯 **Scenario-Based Interview Questions**

Q1: You are midway through implementing a feature on 'feature/payments'. Suddenly you need to fix a critical bug on 'main'. Your code is not ready to commit. Walk through the complete stash workflow.

```text
git stash push -m 'WIP: payment gateway integration'
git stash list                          # verify: stash@{0}: WIP: payment...
git checkout main
git pull
git checkout -b hotfix/null-pointer
# ... fix the bug ...
git add . && git commit -m 'Fix null pointer in order service'
git push  # raise PR, get approved, merge
git checkout feature/payments
git stash pop                           # restore your WIP
```

---

10. Git Cherry-Pick

`git cherry-pick` applies the changes introduced by one or more specific commits from any branch onto your current branch. Unlike merge (which brings in ALL commits from a branch), cherry-pick is **surgical**. Each cherry-picked commit gets a new commit hash on the target branch but contains identical code changes.

How it works internally: Git takes the diff introduced by the selected commit, and re-applies that diff on top of your current HEAD. Conflicts are resolved the same way as a merge conflict.

💡 **Layman Explanation:**

Cherry-pick = pick ONE specific commit from any branch and apply it to your branch. The whole branch is NOT merged. Example: Dev branch has 20 commits, only commit #7 is a critical bug fix needed in main — cherry-pick just #7.

```text
dev branch:   A ── B ── C ── D ── E   (C = critical bug fix)
                        |
main branch:  X ── Y ──── C'          ← C' is cherry-picked C (new hash, same
changes)

Steps:
git log dev --oneline          # find hash of commit C (e.g., abc1234)
git checkout main              # switch to target branch
git cherry-pick abc1234        # apply commit C's changes to main
git push                       # push cherry-picked commit to remote

Multiple commits:
git cherry-pick abc123 def456      # cherry-pick two specific commits
git cherry-pick abc123^..def456    # cherry-pick a range of commits
```

FEATURE	GIT CHERRY-PICK	GIT MERGE
Scope	One or more SPECIFIC commits	ALL commits from a branch
History	New hash assigned on target branch	Original hashes preserved + merge commit
Use case	Hotfix port to production, backport	Integrate complete feature to main
Selectivity	High — you choose exactly what to apply	Low — all or nothing
Conflict risk	Lower (smaller change set)	Higher (full branch delta)

📝 **Theory & Key Points:**

- Cherry-pick creates a DUPLICATE commit — same code but different hash. If the source branch is later merged, Git sees both commits and may create confusion.
- Always note cherry-picks in commit messages: 'Fix null pointer (cherry-picked from dev abc1234)'.
- Common uses: hotfixes (dev → main), backporting fixes to old release branches, recovering commits from deleted branches.
- `git cherry-pick --no-commit`: applies changes to Working Dir + Staging without creating a commit.
- If cherry-pick conflicts: resolve like merge conflict → `git add` → `git cherry-pick --continue`

🎯 **Scenario-Based Interview Questions**

Q1: A critical security fix was committed to the 'dev' branch 3 days ago (hash: sec4567). Production ('main') needs this fix immediately, but dev has 15 other unfinished commits that cannot go to production yet. What do you do?

TEXT

```
git checkout main
git pull
git cherry-pick sec4567          # apply only the security fix
# If conflict: resolve it, git add, git cherry-pick --continue
git push origin main             # deploy the fix to production
```

Document it in the commit message: 'Security: fix SQL injection (cherry-picked from dev sec4567)'. When dev is fully ready and merged, the cherry-pick may create a minor duplicate — acceptable. Consider a hotfix/* branch instead of committing directly to main.

11. .gitignore — Exclude Files from Git

`.gitignore` is a plain text file placed at the root of your repository that tells Git which files and directories to completely ignore. Ignored files never appear in `git status`, are never staged by `git add .`, and are never committed.

Why it matters: Many files generated by your tools, OS, or runtime should never enter version control: compiled binaries, dependency folders (node_modules, .venv), editor settings (.idea, .vscode), environment secrets (.env), and log files. Committing these creates bloated repos, security breaches, OS-specific conflicts, and meaningless diffs.

📝 **Theory & Key Points:**

- `.gitignore` patterns use glob syntax: `*` (wildcard), `**` (any directory depth), `?` (single char), `!` (negate).
- `*.log` — ignore all .log files anywhere in the repo
- `temp/` — ignore the entire temp directory (trailing `/` means directory)
- `!important.log` — un-ignore a specific file even if `*.log` is listed
- `/config.txt` — ignore only config.txt at the root level (not in subdirectories)
- The `.gitignore` file itself IS tracked by Git — it is committed and shared with the team.
- Global gitignore: `git config --global core.excludesfile ~/.gitignore_global`
- Template `.gitignore` files for all languages/frameworks at: https://gitignore.io

BASH

```
# Create the .gitignore file
touch .gitignore

# --- Common .gitignore content ---
*.log               # ignore all log files
*.tmp               # ignore temporary files
temp/               # ignore entire temp/ directory
.env                # ignore environment variables / secrets file
.env.*              # ignore all .env variants (.env.local, .env.prod)
node_modules/       # ignore Node.js dependencies
dist/               # ignore build output
__pycache__/        # ignore Python bytecode cache
.venv/              # ignore Python virtual environment
.DS_Store           # ignore macOS metadata files
*.class             # ignore Java compiled class files
.idea/              # ignore IntelliJ IDE settings
.vscode/            # ignore VS Code settings (or commit it for team consistency)

# Un-ignore a specific file inside an ignored folder:
!logs/critical.log

# Stop tracking a file that was already committed:
git rm --cached .env
git commit -m 'Stop tracking .env — added to .gitignore'
```

🎯 **Scenario-Based Interview Questions**

Q1: A developer pushed their AWS credentials (.env file with AWS_SECRET_KEY) to a public GitHub repo. It's already been there for 2 days. What do you do? CRITICAL — treat as a security incident immediately:

1. FIRST: Revoke/rotate the AWS credentials in the AWS IAM console NOW.
2. Remove from the repo: `git rm --cached .env`, `echo '.env' >> .gitignore`, commit, push
3. Remove from Git history entirely: `git filter-repo --invert-paths --path .env`, force push to all branches
4. Contact GitHub: their secret scanning may have already flagged it
5. Audit AWS CloudTrail logs for any unauthorized API calls in the past 2 days Lesson: ALWAYS add `.env` to `.gitignore` BEFORE the first commit. Use `.env.example` for documenting required variables.

---

12. Git Fork

Forking creates a **server-side copy** of someone else's repository under your own GitHub account. The fork is completely independent. Forking is the backbone of open-source contribution: you fork, contribute changes, and propose them back via a Pull Request.

Fork vs Clone vs Branch:

- **Fork:** Creates a remote copy on your GitHub account — a completely separate repo.
- **Clone:** Creates a local copy on your machine.
- **Branch:** Creates a parallel line of development WITHIN the same repository.

```text
FORK WORKFLOW — Contributing to open-source:

Original Repo (you don't own)          Your Forked Repo (your GitHub)
github.com/original/project   →FORK→    github.com/you/project
                                                  |
                                              git clone
                                                  |
                                            Local Machine
                                        (create feature branch)
                                        (make changes, commit)
                                                  |
                                              git push
                                                  |
                                          Your Fork on GitHub
                                                  |
                                    Pull Request → Original Repo
                                    (owner reviews + merges)
```

📝 **Theory & Key Points:**

- 'upstream' = the original repository you forked from. 'origin' = your fork on GitHub.
- Add upstream remote: `git remote add upstream <original-url>`
- Sync your fork regularly: `git fetch upstream && git merge upstream/main && git push origin main`
- Pull Requests (PRs) are a GitHub concept (not a Git command) — they request a repo owner to 'pull' your changes.
- A PR shows: all commits, all file diffs, reviewer comments, CI/CD status, approval status.
- Fork-based workflows protect the original repo — contributors cannot accidentally break main.

BASH

```bash
# After forking on GitHub:
git clone https://github.com/YOUR-USERNAME/repo.git
cd repo

# Add the ORIGINAL as 'upstream':
git remote add upstream https://github.com/ORIGINAL-OWNER/repo.git
git remote -v   # shows both origin (your fork) and upstream (original)

# Create a feature branch, make changes:
git checkout -b feature/add-dark-mode
git add . && git commit -m 'Add dark mode toggle'
git push origin feature/add-dark-mode

# On GitHub: Open Pull Request from your fork's branch to original/main

# Keep your fork in sync with upstream:
git fetch upstream
git checkout main
git merge upstream/main
git push origin main
```

🎯 **Scenario-Based Interview Questions**

Q1: You want to contribute a feature to a popular open-source project on GitHub that you have no write access to. Walk through the complete process.

1. Fork on GitHub → creates github.com/you/project
2. Clone: `git clone https://github.com/you/project.git`
3. Add upstream: `git remote add upstream https://github.com/original/project.git`
4. Create branch: `git checkout -b feature/dark-mode`
5. Code, commit: `git add . && git commit -m 'Add dark mode toggle'`
6. Push to YOUR fork: `git push origin feature/dark-mode`
7. Open Pull Request: your fork → original/main
8. Address review comments: push more commits to the same branch — PR updates automatically
9. Owner approves and merges
10. Sync your fork: `git fetch upstream && git merge upstream/main && git push origin main`

---

13. Git Tags — Marking Releases

A Git tag is a **permanent, immovable reference** to a specific commit. While a branch pointer moves forward with every new commit, a tag always points to the same commit forever. Tags mark significant points in history — typically official software releases (v1.0.0, v2.3.1). In CI/CD pipelines, pushing a tag often triggers an automated release workflow.

Semantic Versioning (SemVer): vMAJOR.MINOR.PATCH

- **MAJOR:** Breaking changes (v1.x.x → v2.0.0)
- **MINOR:** New features, backward-compatible (v1.0.x → v1.1.0)
- **PATCH:** Bug fixes only, backward-compatible (v1.0.0 → v1.0.1)

```text
Commit history with tags:

[c3d4e5f] Initial project setup    ←── v1.0.0 (permanent label)
    |
[b2c3d4e] Add login feature        ←── v1.1.0
    |
[a1b2c3d] Fix session timeout bug  ←── v1.1.1
    |
[9f8e7d6] Add payment module       ←── v2.0.0 (HEAD)

Branch 'main' → moves to 9f8e7d6 with each new commit
Tag 'v1.0.0'  → always points to c3d4e5f — NEVER MOVES
```

📝 **Theory & Key Points:**

- Two types of tags: **Lightweight** (just a pointer, no metadata: `git tag v1.0`) and **Annotated** (full Git object with tagger name, email, date, message, optional GPG signature: `git tag -a v1.0 -m 'msg'`).
- Always use ANNOTATED tags for official releases.
- Tags are NOT pushed automatically with `git push` — must be pushed explicitly.
- `git push origin --tags` pushes ALL local tags. Use `git push origin v1.0` to push just one.
- GitHub automatically creates a Release page and downloadable zip/tar for each pushed tag.
- In GitHub Actions: `on: push: tags: [v*.*.*]` triggers a release workflow on any version tag push.
- `git checkout v1.0` lets you inspect code at a specific release (puts you in detached HEAD).

COMMAND	PURPOSE
`git tag`	List all tags
`git tag v1.0`	Create lightweight tag on current commit
`git tag -a v1.0 -m 'message'`	Create annotated tag with metadata
`git tag -a v1.1 <hash> -m 'msg'`	Tag a specific older commit
`git push origin v1.0`	Push one specific tag to remote
`git push origin --tags`	Push ALL tags to remote
`git tag -d v1.0`	Delete tag locally
`git push origin --delete v1.0`	Delete tag from remote
`git show v1.0`	Show tag details and the tagged commit
`git describe --tags`	Show nearest tag + commits since (e.g., v1.0-3-gabc123)

🎯 Scenario-Based Interview Questions

Q1: Your team tagged and released v2.0.0. Three hours later, a critical crash is reported in production. How do you handle the release versioning?

1. Identify the bug — find the commit causing the crash
2. Create a hotfix branch from the v2.0.0 tag: `git checkout -b hotfix/v2.0.1 v2.0.0`
3. Fix the bug, commit: `git commit -m 'Fix crash: null check in payment processor'`
4. Tag the fix: `git tag -a v2.0.1 -m 'Hotfix: payment crash (critical)'`
5. Push branch and tag: `git push origin hotfix/v2.0.1 && git push origin v2.0.1`
6. Deploy v2.0.1 to production
7. Merge hotfix back to main AND dev

14. Branch Protection Rules

Branch protection rules are **policy settings configured in GitHub** that restrict actions on specific branches. They prevent accidental or unauthorized changes to production-critical branches, enforce code review processes, and guarantee automated quality checks pass before any code is merged.

Why they are essential: Without branch protection, any developer can push directly to main at any time — bypassing code review, breaking tests, and potentially deploying broken code to production.

> 💡 **Layman Explanation:**
>
> Branch protection = automatically enforced rules on your most important branches. No one — not even repo owners — can bypass them (when 'Include administrators' is enabled). Every merge must pass through: code review + automated tests + up-to-date check.

RULE	WHAT IT ENFORCES
Require PR before merging	Blocks all direct pushes. All changes must come through a Pull Request.
Require N approvals	PR needs minimum N reviewer approvals before merging.
Dismiss stale reviews	If new commits are pushed to the PR, previous approvals are invalidated.
Require status checks to pass	CI/CD pipelines (tests, linting, security scan) must all be green.
Require branches to be up-to-date	Branch must include all latest base branch commits before merging.
Restrict who can push/merge	Only specific people or teams can merge (e.g., senior engineers only).
Require signed commits	All commits must be GPG-verified to prevent impersonation.
Require linear history	No merge commits allowed — enforces squash or rebase merge strategy.
Include administrators	Rules apply even to repository owners and admins — no exceptions.
Allow force pushes (disabled)	Prevents `git push --force` which would overwrite shared history.

📝 **Theory & Key Points:**

- CODEOWNERS file (`.github/CODEOWNERS`): automatically assigns reviewers based on which files changed. Example: `/backend/ @backend-team`.
- With branch protection: only CODEOWNERS can approve changes to their designated paths.
- Status checks come from: GitHub Actions, Jenkins, CircleCI, SonarQube, or any CI tool via the GitHub API.
- Required status checks ensure: unit tests pass, code coverage is maintained, security scans are clean.
- Branch protection is the enforcement of your PR/review workflow — define the workflow first, then protect.

🎯 **Scenario-Based Interview Questions**

Q1: Your team has been having incidents where developers push broken code directly to main. What GitHub settings do you configure to enforce a proper workflow? In GitHub → Repository Settings → Branches → Add rule for 'main':

- ☑ Require a pull request before merging → Minimum 2 approving reviews → Dismiss stale approvals when new commits pushed
- ☑ Require status checks to pass before merging → 'unit-tests', 'lint', 'security-scan' → Require branches up to date
- ☑ Restrict who can push to matching branches → Only DevOps/SRE team can merge
- ☑ Include administrators (no exceptions) Result: No code reaches main without 2 code reviews + all tests passing + up-to-date with main. Direct pushes are completely blocked.

---

15. Git Reflog — The Safety Net

`git reflog` (reference log) is a local journal of every HEAD movement in your repository. Every time HEAD moves — due to a commit, checkout, reset, merge, rebase, or cherry-pick — a reflog entry is recorded. It is your ultimate recovery tool for accidental data loss. The reflog shows history that `git log` cannot — including commits that were removed by `git reset`.

Why git log is not enough: `git log` shows only commits reachable from the current HEAD forward. After a `git reset --hard`, those commits are no longer reachable via `git log`. But they still physically exist in `.git/objects` — and `git reflog` can find them.

> 💡 **Layman Explanation:**
>
> Reflog = Git's 'recently visited history' — like a browser's back button for your repo. Even after `git reset --hard`, your commits are NOT deleted immediately. They remain in `.git/objects` for 90 days (default garbage collection time). Reflog is LOCAL only — never pushed to remote.

BASH

```
# View the full reflog (all HEAD movements):
git reflog

# Example output:
a1b2c3d (HEAD -> main) HEAD@{0}: commit: Add payment module
f4e5d6c                HEAD@{1}: reset: moving to HEAD~1
9g8h7i6                HEAD@{2}: commit: Add login feature
c5d4e3f                HEAD@{3}: checkout: moving from dev to main
8f7e6d5                HEAD@{4}: merge feature/ui: Fast-forward

# Recover a specific lost commit:
git reset --hard 9g8h7i6         # restore to a specific past HEAD position

# Or create a branch at the lost state:
git checkout -b recovered 9g8h7i6

# View reflog for a specific branch:
git reflog show dev

# Reflog with timestamps:
git reflog --date=iso
```

📝 **Theory & Key Points:**

- `git reflog` is available even after: `git reset --hard`, deleted branches, bad rebases.
- Garbage collection removes unreferenced objects after 90 days (`git gc --prune=now` removes them immediately).
- Reflog is stored in `.git/logs/HEAD` (and per-branch in `.git/logs/refs/heads/`).
- `git reflog expire --expire=now` clears all reflog entries (use with extreme caution).
- This is why `git reset --hard` is recoverable short-term but should still be used with caution.

🎯 **Scenario-Based Interview Questions**

Q1: A developer ran 'git reset --hard HEAD~5' to remove 5 commits, then realized those commits had important work. They haven't pushed. Can the commits be recovered? YES — using git reflog:

```text
git reflog
# abc1234 HEAD@{1}: commit: Add API integration ← this was the 5th commit (most recent)
git reset --hard abc1234   # restores to exactly that state — all 5 commits are back
# OR: git checkout -b recovery-branch abc1234
```

Why this works: `git reset --hard` only moves the branch pointer — the actual commit objects in `.git/objects/` are NOT deleted yet. Reflog is your map to find them.

---

16. Quick Command Reference Card

Setup & Init

COMMAND	DESCRIPTION
`git init`	Initialize new Git repository
`git clone <url>`	Clone remote repository locally
`git config --global user.name`	Set global username
`git config --global user.email`	Set global email
`git config --list`	View all configuration

Daily Workflow

COMMAND	DESCRIPTION
`git status`	Check file status
`git add <file>`	Stage specific file
`git add .`	Stage all changes
`git commit -m 'msg'`	Commit staged changes
`git push`	Push to remote
`git pull`	Pull from remote
`git fetch`	Download without merging
`git log --oneline`	Compact history
`git log --oneline --graph --all`	Visual branch history

Branching & Merging

COMMAND	DESCRIPTION
`git branch`	List local branches
`git checkout -b <name>`	Create + switch to branch
`git merge <branch>`	Merge branch into current
`git cherry-pick <hash>`	Apply specific commit
`git branch -D <name>`	Force delete branch
`git push -u origin <name>`	Push branch + set tracking

Undoing Changes

COMMAND	DESCRIPTION
`git restore <file>`	Discard working dir changes
`git restore --staged <f>`	Unstage a file
`git reset --soft HEAD~1`	Undo commit — keep staged
`git reset --mixed HEAD~1`	Undo commit — keep in working dir
`git reset --hard HEAD~1`	Undo commit + delete ⚠️
`git revert <hash>`	Safe undo — new commit
`git reflog`	Find lost commits
`git stash` / `git stash pop`	Save and restore WIP

Inspect & Compare

COMMAND	DESCRIPTION
`git diff`	Working dir vs staging
`git diff --staged`	Staging vs last commit
`git diff main origin/main`	Local vs remote
`git show <hash>`	Full commit details
`git blame <file>`	Who changed which line
`git log --author='name'`	Commits by a specific author

Source document: "MultiCloud DevOps — Git & GitHub Complete Notes — by Veera Sir" (Theory + Diagrams + Commands + Scenario-Based Interview Q&A — Version 4.0)

TF

PART 02 OF 08

Terraform

Infrastructure as Code — provisioning and managing cloud resources declaratively.

1. What is Terraform?

Fig: "IaC - VPC Creation Using Terraform" — a workflow diagram showing a laptop with Code/VPC/IaC pushed to GitHub ("Create a new repo terraform-practice" → "Clone the repo into the local"), opened in VS Code as Terraform Configuration Files (provider.tf, main.tf). The Terraform Workflow box shows terraform init → terraform plan → terraform apply feeding into the AWS Cloud (using CLI Keys for account authentication / AWS Credentials — access key & secret access key — via AWS CLI) which results in a VPC Created in the AWS Account console. A separate panel shows the Terraform Registry with Providers for AWS, Azure, and GCP, reached via terraform init.

Terraform is an open-source **Infrastructure as Code (IaC)** tool created by **HashiCorp** in 2014. It allows you to define, provision, and manage cloud infrastructure using a high-level declarative configuration language called **HCL (HashiCorp Configuration Language)**. Terraform is cloud-agnostic — the same workflow manages resources on AWS, Azure, Google Cloud, on-premises, or any combination.

What does "Infrastructure as Code" mean? Instead of manually clicking through web consoles to create servers, databases, and networks, you write code that describes your desired infrastructure. Terraform reads this code and makes the real-world infrastructure match your description automatically. This code can be versioned in Git, reviewed, tested, and shared, just like application code.

Written in Go, runs everywhere: Terraform is compiled into a single binary. It communicates with cloud provider APIs using **Providers** — plugins that understand how to talk to AWS, Azure, GCP, Kubernetes, GitHub, Datadog, and 3,000+ other services. When you run `terraform init`, Terraform downloads the required provider plugins automatically.

💡 Layman Explanation:

- Think of Terraform as a 'universal remote control' for cloud infrastructure.
- Instead of clicking buttons in 5 different cloud consoles, you write one config file.
- Terraform figures out what needs to be created, changed, or deleted — and does it in order.
- If something goes wrong, the state file tracks exactly what was created so cleanup is clean.
- Team members can share the same config in GitHub — everyone provisions identical environments.

📝 Theory & Key Points:

- Terraform uses a DECLARATIVE approach — you describe WHAT you want, not HOW to do it.
- Imperative (Ansible/scripts): 'Step 1: create VPC. Step 2: create subnet.' — you specify steps.
- Declarative (Terraform): 'I want a VPC, subnet, and EC2 instance.' — Terraform figures out steps.
- Terraform tracks infrastructure state in `terraform.tfstate` — its source of truth.
- Terraform computes a DIFF between desired state (.tf files) and actual state (state file) on every plan.
- Terraform supports 3,000+ providers: AWS, Azure, GCP, Kubernetes, GitHub, Cloudflare, Datadog, etc.
- HCL (HashiCorp Configuration Language) is human-readable and designed for infrastructure descriptions.
- Terraform is idempotent — running it multiple times produces the same result with no duplicates.
- The Terraform Registry (registry.terraform.io) hosts providers and reusable modules.

ASPECT	TERRAFORM (IAC) VS MANUAL / CLICKOPS
Speed	Entire infrastructure in minutes with one command vs hours of manual clicking
Repeatability	Identical environments every time vs human error causes configuration drift
Version Control	Full Git history of every infra change vs no history — impossible to audit
Disaster Recovery	Re-run plan+apply to recreate everything vs manual rebuild takes days
Multi-Cloud	One tool, one workflow for all providers vs different console/SDK per provider
Cost Control	`terraform destroy` cleans up everything cleanly vs forgotten resources keep billing
Team Collaboration	Code reviews, PRs, shared Git repo vs no standard process, tribal knowledge

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your team manages 20 AWS accounts manually through the console. A new region needs the same setup. How does Terraform help?

- Write the infrastructure config once in Terraform HCL files.
- Use workspaces or variable files to parameterize region, account, and environment names.
- Run `terraform apply` for the new region — identical infrastructure is provisioned in minutes.
- Store the code in GitHub — every future environment reuses the same templates.
- This eliminates: manual errors, hours of clicking, inconsistent environments, and undocumented changes.

Q2: Scenario: 'Why use Terraform instead of AWS CloudFormation?'

- CloudFormation: AWS-only, JSON/YAML syntax, limited multi-provider support.
- Terraform: Cloud-agnostic (AWS + Azure + GCP + 3000+ providers in ONE config), cleaner HCL syntax, better state management.
- Terraform has a larger community, public module registry, and workspace environment management.
- If you ONLY use AWS and never will: CloudFormation is fine.
- If you use multiple clouds or services: Terraform is the industry standard.

Q3: Scenario: A developer asks "What happens when Terraform runs terraform plan — is anything changed in AWS?"

- NO — `terraform plan` is a completely read-only, dry-run operation.
- Terraform reads the current state file and queries the cloud API (no writes).
- It computes and displays: + (resources to create), ~ (resources to update), - (resources to destroy).
- Nothing in AWS is changed until `terraform apply` is executed.
- Best practice: always run plan and review output carefully before apply.

Q4: Scenario: Your organization is evaluating Terraform vs Pulumi. What are the key differences?

- Terraform: uses HCL (domain-specific language), declarative, huge community and provider ecosystem.
- Pulumi: uses general-purpose languages (Python, TypeScript, Go), code-first IaC approach.

- Terraform: better for infrastructure-focused teams, simpler syntax, more mature for multi-cloud.
- Pulumi: better for developer teams already proficient in programming languages.
- For most DevOps teams: Terraform is the industry standard with the largest adoption.

---

2. Benefits of Infrastructure as Code (IaC)

IaC transforms infrastructure provisioning from a manual, error-prone process into a software engineering discipline. Instead of writing runbooks and SOPs that humans follow, you write code that machines execute — consistently, reliably, and at any scale.

BENEFIT	EXPLANATION
Speed & Simplicity	Entire infrastructure created by running a single command. Environments that took days now take minutes.
Team Collaboration	Infrastructure configs are stored in Git. Teams collaborate with PRs, code reviews, and change history — same as application code.
Error Reduction	Removes human error from configuration. Same code = same infrastructure every time. No more 'it works on staging but not production'.
Disaster Recovery	When a region fails, re-run Terraform in another region. Infrastructure is rebuilt exactly. No manual reconstruction from memory.
Enhanced Security	Security configurations are codified and reviewable. No undocumented port openings or manual IAM changes that get forgotten.
Cost Visibility	`terraform plan` shows exactly what will be created. `terraform destroy` cleans up everything with no forgotten resources.
Idempotency	Running Terraform multiple times with the same config produces the same result. No duplicates, no side effects.
Documentation as Code	The .tf files ARE the documentation. They show exactly what infrastructure exists and why.
Auditability	Git history provides a full audit trail: who changed what, when, and why — with PR reviews and approvals.
Environment Parity	Dev, staging, and production environments are provisioned from identical code — eliminating environment-specific bugs.

🔶 **Terraform Concept — Idempotency:**

- Running the same Terraform code 10 times produces the exact same infrastructure as running it once.
- If the resource already exists and matches config → Terraform does nothing.
- If it doesn't exist → Terraform creates it.
- If it exists but differs → Terraform updates it.
- This makes Terraform safe to run repeatedly — no surprises, no duplicates.

🎯 Scenario-Based Interview Questions

Q1: Scenario: A junior engineer asks 'What is the difference between IaC and configuration management tools like Ansible?'

- Terraform (IaC / Provisioning): Creates and manages INFRASTRUCTURE — VMs, networks, databases, cloud services.
- Ansible (Configuration Management): Configures SOFTWARE on existing infrastructure — installs packages, deploys apps.
- They are complementary: Use Terraform to provision the EC2 instance, then Ansible to configure the app on it.
- Other IaC tools: AWS CloudFormation (AWS-only), Pulumi (code-first IaC), Azure ARM/Bicep (Azure-specific).

Q2: Scenario: Management asks "How does IaC help our compliance and audit team?"

- Every infrastructure change is a Git commit with author, timestamp, and PR review history.
- Security policies (encryption, IAM, security groups) are codified — cannot be bypassed without a code change.
- `terraform plan` output shows exactly what will change — reviewable before execution.
- Automated policy enforcement with tools like Sentinel or OPA integrates with Terraform pipelines.
- Full reproducibility: any historical version of infrastructure can be rebuilt from Git history.

Q3: Scenario: A developer says "Why not just use Bash scripts to automate AWS CLI commands instead of Terraform?"

- Bash scripts are imperative — they break if partial failures occur halfway through execution.
- Scripts have no concept of state — they cannot detect what already exists vs what needs creating.
- Bash scripts are NOT idempotent — running twice often creates duplicates or errors.
- Terraform tracks state, handles dependencies automatically, and shows a preview before changes.
- Terraform handles rollback by restoring desired state — Bash scripts have no such mechanism.

3. Terraform Core Workflow

The Terraform workflow follows four main commands: **init → plan → apply → destroy**. This sequence ensures infrastructure changes are always previewed before being applied — like a dry run before the real thing. Understanding this workflow is the foundation of working with Terraform.

> 💡 **Layman Explanation:**
>
> - Terraform workflow = Write → Init → Plan → Apply (→ Destroy when done)
> - Init = set up the project (download plugins) — do this once per project setup
> - Plan = 'what WILL happen?' — always run before apply, never skip
> - Apply = 'do it now' — actually creates/changes/destroys cloud resources
> - Destroy = tear down everything — clean up when done, stops cloud billing

TEXT

```
Terraform Core Workflow — init > plan > apply > destroy

[Write .tf Files]  --terraform init-->  [terraform init]  --terraform plan
                                                                -out=plan.tfplan-->
  # Write HCL                                Downloads providers      [terraform plan]
  provider.tf                                Sets up backend           Reads state file
  variables.tf                               Creates .terraform/       Queries cloud API
  main.tf                                                              Shows +/-/~ diff

  [terraform plan] --terraform apply plan.tfplan--> [terraform apply] --terraform
destroy
                                                                                --auto-
approve--> [terraform destroy]
                                             Executes the plan
Removes all resources
                                             Updates state file
Stops cloud billing
                                             Approve: yes/no
Cleans state file

  (modify .tf -> re-run workflow: terraform destroy loops back to Write .tf Files)
```

Fig 3: Terraform Core Workflow — Write, Init, Plan, Apply, Destroy

3.1 Core Commands

COMMAND	WHAT IT DOES
`terraform version`	Show installed Terraform version
`terraform init`	Initialize working directory — downloads provider plugins, sets up backend
`terraform init -upgrade`	Re-download latest provider versions
`terraform fmt`	Auto-format .tf files to canonical style — always run before committing
`terraform validate`	Check syntax and logical errors in configuration files (no cloud credentials needed)
`terraform plan`	Preview changes — shows +create, ~update, -destroy (dry run, nothing changes)
`terraform plan -out=plan.tfplan`	Save plan to file — use this file in apply for exact execution
`terraform apply`	Execute the plan — prompts for 'yes' confirmation before making changes
`terraform apply --auto-approve`	Apply without confirmation prompt (use in CI/CD pipelines)
`terraform apply -target=aws_instance.web`	Apply changes to ONE specific resource only
`terraform destroy`	Destroy ALL resources managed by this configuration
`terraform destroy --auto-approve`	Destroy without confirmation prompt
`terraform show`	Show current state in human-readable format
`terraform output`	Display all output values from state file
`terraform refresh`	Sync state file with actual real-world infrastructure
`terraform graph`	Generate DOT format dependency graph of all resources
`terraform taint <resource>`	Mark resource for forced replacement on next apply (deprecated in v0.15+)

COMMAND	WHAT IT DOES
`terraform state list`	List all resources tracked in the state file

📝 **Theory & Key Points:**

- ALWAYS run `terraform plan` before `terraform apply` — review every + (create), ~ (change), - (destroy).
- `terraform plan -out=myplan.tfplan` saves the plan. `terraform apply myplan.tfplan` executes EXACTLY that plan.
- This two-step approach is essential in CI/CD: Plan in PR pipeline, Apply after PR approval.
- `terraform fmt`: run this before committing — keeps all team members' code consistently formatted.
- `terraform validate`: catches syntax errors before plan. Fast and does not need cloud credentials.
- `terraform apply -target`: powerful but use with caution — can create state inconsistencies if overused.
- `terraform graph | dot -Tsvg > graph.svg` generates a visual dependency diagram of all resources.
- The `terraform refresh` command is implicitly run during plan — it syncs actual cloud state into the state file.

🎯 Scenario-Based Interview Questions

Q1: Scenario: A new engineer asks 'What happens if I run terraform apply without running terraform plan first?'

- Terraform automatically runs an embedded plan before apply — it will show changes and ask for "yes" confirmation.
- However, using `terraform plan -out=plan.tfplan` + `terraform apply plan.tfplan` is safer in team environments.
- Reason: The plan is reviewed and saved; apply executes EXACTLY the reviewed plan — no risk of changes in between.
- In CI/CD: plan runs in the PR pipeline (visible to reviewers), apply runs after PR approval.
- RISK: Skipping plan review — you might miss a -destroy that would delete a production database!

Q2: Scenario: Your CI/CD pipeline runs terraform apply in production. How do you ensure it's safe?

- Step 1: `terraform fmt` and `terraform validate` in CI — fail fast on syntax errors.
- Step 2: `terraform plan -out=plan.tfplan` — save the plan as a pipeline artifact.
- Step 3: Manual approval gate — team reviews the plan output in the PR/pipeline before proceeding.
- Step 4: `terraform apply plan.tfplan` — apply EXACTLY the reviewed plan, no surprises.
- Step 5: Post-apply tests (Terratest or Inspec) to validate infrastructure is correct.

Q3: Scenario: terraform plan shows a resource will be destroyed. You were not expecting this. How do you investigate?

- Read the plan output carefully — look for the reason (configuration change, resource drift, etc.).
- Run `terraform state show <resource>` to see the current state attributes.
- Compare against your .tf file — find what changed that triggered the replacement.
- Check if someone manually modified the resource in the console (use `terraform refresh` first).
- If destructive change is unintended — update your .tf code to match desired state before applying.

Q4: Scenario: What is the difference between terraform validate and terraform plan?

- `terraform validate`: checks HCL syntax and logical errors ONLY — does NOT contact the cloud API.

- `terraform plan`: performs a real diff — contacts cloud API to compare desired vs actual state.
- validate: fast, no credentials needed, catches typos and missing required variables.
- plan: slow, requires cloud credentials, catches real configuration problems (wrong AMI IDs, missing permissions).
- Best practice: run validate first in CI (fast feedback), then plan (full check).

---

4. Terraform Project Structure

Terraform does not enforce a specific file structure — it reads **all .tf files** in a directory. However, a consistent, well-organized structure is essential for maintainability, especially in team environments. The file names do not matter to Terraform — but they matter enormously for humans reading and maintaining the code.

💡 **Layman Explanation:**

- Terraform reads ALL .tf files in a folder simultaneously — order does not matter.
- Split your config into logical files so teammates can quickly find what they need.
- Standard structure is like organizing a kitchen: everything in its logical place.
- provider.tf = brand and model of the kitchen (which cloud and version)
- variables.tf = the settings/ingredients you can customize
- main.tf = the actual cooking instructions (resources)
- outputs.tf = what comes out of the oven (useful values after provisioning)

TEXT

```
Standard Terraform Project Structure:

project-name/
 ├── provider.tf         # Terraform block + provider configuration (AWS/Azure/GCP)
 ├── backend.tf          # Remote state configuration (S3 + DynamoDB)
 ├── variables.tf        # Input variable declarations (type, description, default)
 ├── terraform.tfvars    # Actual values for variables (NOT committed to Git if
sensitive)
 ├── main.tf             # Core resources being created (EC2, VPC, RDS, etc.)
 ├── data.tf             # Data sources (fetching existing resources)
 ├── locals.tf           # Local computed values (avoid repeating expressions)
 ├── outputs.tf          # Values to display after apply (IPs, ARNs, DNS names)
 └── modules/            # Reusable child modules
     ├── vpc/
     │   ├── main.tf
     │   ├── variables.tf
     │   └── outputs.tf
     └── ec2/
         ├── main.tf
         ├── variables.tf
         └── outputs.tf
```

FILE	PURPOSE
provider.tf	Terraform version constraints + provider config (region, credentials source)
backend.tf	Remote state backend config (S3 bucket, DynamoDB lock table)
variables.tf	Declare all input variables with type, description, default, validation
terraform.tfvars	Assign values to variables — environment-specific, kept out of Git if secret
main.tf	Primary resource definitions — EC2, VPC, S3, RDS, IAM, etc.
data.tf	Data source blocks — fetch existing resources (AMI IDs, VPC info, etc.)
locals.tf	Local value calculations — computed expressions used multiple times
outputs.tf	Output values displayed after apply — IPs, ARNs, endpoints
.terraform.lock.hcl	Provider version lock file — ALWAYS commit this to Git
.gitignore	Exclude: .terraform/ folder, *.tfstate, *.tfstate.backup, terraform.tfvars

⚠️ **Important Warning:**

- NEVER commit terraform.tfstate to Git — it may contain secrets and causes team conflicts.
- NEVER commit terraform.tfvars if it contains passwords, API keys, or sensitive values.
- ALWAYS commit .terraform.lock.hcl — it ensures consistent provider versions across the team.
- DO commit: *.tf files, .terraform.lock.hcl, example.tfvars (with placeholder values).
- Use .gitignore template: .terraform/, *.tfstate, *.tfstate.backup, *.tfvars (if sensitive).

🎯 Scenario-Based Interview Questions

Q1: Scenario: A new team member asks 'Why do we have both variables.tf and terraform.tfvars files?'

- variables.tf DECLARES variables — defines the variable name, type, description, and optional default.
- terraform.tfvars ASSIGNS values — provides the actual values for the declared variables.
- Separation of concerns: declarations go in variables.tf (committed to Git), sensitive values go in terraform.tfvars (git-ignored).
- Different environments: create dev.tfvars, staging.tfvars, prod.tfvars with different values.
- Use: `terraform apply -var-file=prod.tfvars` to apply with production values.

Q2: Scenario: What files should you NEVER commit to Git in a Terraform project?

- terraform.tfstate and terraform.tfstate.backup — contain real resource IDs, IPs, and possibly secrets.
- terraform.tfvars — if it contains API keys, passwords, or sensitive configuration.
- .terraform/ directory — contains downloaded provider binaries (large, reproducible via init).
- Always commit: *.tf files, .terraform.lock.hcl, example.tfvars (with placeholder/dummy values).
- Use a .gitignore file with: .terraform/, *.tfstate*, *.tfvars (evaluate per project).

Q3: Scenario: Your Terraform project has grown to 50 resources in one main.tf file. How do you organize it better?

- Split by concern: networking.tf (VPC, subnets), compute.tf (EC2), database.tf (RDS), security.tf (IAM, SGs).
- Extract reusable patterns into modules: modules/vpc/, modules/ec2/, modules/rds/.
- Create separate module calls for each logical application tier.
- Consider separating into multiple Terraform root configurations if the project is very large.
- Rule of thumb: if a file is over 150-200 lines, it is time to split it.

5. Terraform State File

The Terraform state file (**terraform.tfstate**) is the most critical file in any Terraform project. It is Terraform's source of truth about what infrastructure currently exists. Every `terraform plan` and `apply` reads this file to compute the difference between desired state (.tf files) and actual state (what is in the cloud).

What the state file contains: For every resource Terraform manages, the state file records: the resource type, its unique ID, all attribute values (IP addresses, ARNs, names), dependencies between resources, and metadata. Without the state file, Terraform does not know which real-world resources it created — it would try to create duplicates.

🔶 **Terraform Concept — State File:**

- State file is Terraform's 'memory' of what it has created.
- Desired state = your .tf files (what you WANT)
- Actual state = terraform.tfstate (what Terraform THINKS exists)
- Real state = actual infrastructure in the cloud
- `terraform plan` = compares desired state vs actual state → shows the diff
- `terraform refresh` = syncs actual cloud state → into the state file

5.1 Remote State (Essential for Teams)

By default, Terraform stores state locally in terraform.tfstate. This **breaks completely in team environments**: two developers running `terraform apply` simultaneously will corrupt the state. The solution is **remote state** — storing the state file in a shared, locked location accessible to all team members.

Fig 1: "Terraform Team Collaboration — GitHub, S3 State, and Multi-Developer Workflow" — a diagram showing Dev A, Dev B, and Dev C each with their own .tf files pushing via git push/PR review into a shared GitHub Repo (Shared .tf files), which triggers the CI/CD Pipeline (fmt → validate → plan → apply). The pipeline runs `terraform apply` against an S3 Bucket (holding terraform.tfstate, versioned + AES-256 encrypted) which coordinates with a DynamoDB table (state lock [LockID], prevents concurrent apply) before writing into the AWS Cloud, which contains VPC, EC2, RDS, S3, IAM, and ALB resources. A legend maps colors to Developer, Git, CI/CD, Lock, and State.

Fig 2: "Terraform Remote State — AWS S3 + DynamoDB Backend Architecture" — a diagram showing the Terraform CLI (running terraform init / plan / apply) reading/writing state to/from an AWS S3 Bucket (Remote State) that contains: the terraform.tfstate object (resource_type, id, attributes, IPs,

ARNs, dependencies, metadata, outputs — with a "release lock after apply" note), S3 Versioning ENABLED (full history of every state change, restored on corruption / accidental delete), and AES-256 Encryption (server-side encryption at rest, enforce SSL = true in transit). The Terraform CLI is also connected to a backend.tf Config block (bucket = "tf-state", key = "prod/tf.tfstate", dynamodb_table = "lock"), and to an AWS DynamoDB (State Lock) box containing a LockID Item (created on plan/apply start, deleted on completion/failure) using PAY_PER_REQUEST mode (hash_key = LockID (String), Attribute type = S, no capacity planning needed). Two contrasting panels at the bottom compare "Without Locking (DANGER)" — two 'terraform apply' at the same time → state corruption → duplicate/conflicting resources — versus "With DynamoDB Lock (SAFE)" — Dev A acquires lock first, Dev B sees "Error acquiring state lock", Dev B must wait, state stays consistent.

💡 **Layman Explanation:**

- Local state = disaster in a team. Two people run "apply" simultaneously → state corruption.
- Remote state = state file stored in S3 (or Azure Blob, GCS, Terraform Cloud).
- DynamoDB table provides STATE LOCKING — only one person can apply at a time.
- Everyone on the team reads the same state — no "works on my machine" problem.
- Remote state enables state sharing between Terraform projects (terraform_remote_state data source).

5.2 Setting Up S3 Remote Backend

Step 1: Create the S3 bucket and DynamoDB table first (chicken-and-egg: you can use Terraform itself or the AWS console for the initial bootstrap).

HCL

```hcl
# Step 1: Create S3 bucket for state storage
resource "aws_s3_bucket" "terraform_state" {
  bucket = "mycompany-terraform-state-prod"
  # Enable versioning — keeps history of state changes, allows rollback
  versioning { enabled = true }
  # Enable server-side encryption
  server_side_encryption_configuration {
    rule {
      apply_server_side_encryption_by_default {
        sse_algorithm = "AES256"
      }
    }
  }
}

# Step 2: Create DynamoDB table for state locking
resource "aws_dynamodb_table" "terraform_state_lock" {
  name         = "terraform-state-lock"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"
  attribute { name = "LockID"; type = "S" }
}

# backend.tf — configure remote state
terraform {
  backend "s3" {
    bucket         = "mycompany-terraform-state-prod"
    key            = "prod/terraform.tfstate"   # path inside the bucket
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "terraform-state-lock"      # enables state locking
  }
}
```

5.3 State Locking

When `terraform plan` or `apply` runs, Terraform **acquires a lock** on the state file. Any other Terraform operation trying to run simultaneously will see the lock and wait (or fail). This prevents state corruption from concurrent applies. DynamoDB provides the locking mechanism for S3 backends.

COMMAND	PURPOSE
`terraform state list`	List all resources tracked in the state file
`terraform state show <resource>`	Show detailed attributes of one resource in state
`terraform state mv`	Rename/move a resource in state (without recreating it)
`terraform state rm <resource>`	Remove resource from state (stops managing it, does NOT delete the real resource)
`terraform state pull`	Download and display remote state locally
`terraform state push`	Manually update remote state (use with extreme caution)
`terraform force-unlock <lockID>`	Release a stuck state lock (only if apply crashed mid-run)

⚠️ **Important Warning:**

- NEVER manually edit terraform.tfstate — it is JSON but direct edits will corrupt it.
- If state becomes corrupted: restore from the S3 versioning backup immediately.
- `terraform state rm` removes a resource from tracking — the REAL resource still exists.
- Use `terraform force-unlock` ONLY if a terraform apply crashed and left a stale lock.
- DO NOT use force-unlock if another apply is still running — that will corrupt state.

🎯 Scenario-Based Interview Questions

Q1: Scenario: Two engineers on your team run 'terraform apply' at the same time. What happens and how do you prevent it?

- Without remote state + locking: Both apply operations read the same local state file simultaneously.
- Both think they need to create the same resources — result: duplicate resources and corrupted state file.
- With S3 + DynamoDB backend: Engineer A acquires the DynamoDB lock first.
- Engineer B's apply sees the lock and fails with: 'Error acquiring the state lock'.
- DynamoDB item with LockID is created during A's apply and deleted when A finishes.
- This is why remote state + DynamoDB locking is MANDATORY for any team using Terraform.

Q2: Scenario: Someone accidentally deleted a resource directly in the AWS console that Terraform manages. What does terraform plan show?

- Terraform compares its state file (which shows the resource exists) against reality (it is gone).
- `terraform plan` will show: + aws_instance.web (to add) — it plans to CREATE it again.
- Running `terraform apply` will recreate the deleted resource exactly as defined in .tf files.
- This is Terraform "reconciling" desired state with actual state.
- To prevent accidental deletion: use `lifecycle { prevent_destroy = true }` for critical resources.

Q3: Scenario: You need to move a Terraform-managed resource to a different Terraform project without recreating it. How?

- Step 1: Run `terraform state mv <source_resource> <dest_resource>` to rename in source state.
- OR: `terraform state pull > state.json` — edit the JSON to move the resource.
- Step 2: In the destination project, write the resource block matching the existing resource.
- Step 3: `terraform import <resource_type>.<name> <real_resource_id>` in destination project.
- Step 4: Verify `terraform plan` in destination shows "No changes" — perfect alignment.
- Step 5: `terraform state rm` in source project to stop managing it there.

Q4: Scenario: Your S3 remote state file is corrupted. What is your recovery procedure?

- Step 1: Do NOT run any terraform commands — stop all operations immediately.

- Step 2: Go to S3 console → your state bucket → enable/check versioning.
- Step 3: Restore the previous version of terraform.tfstate from S3 version history.
- Step 4: Run `terraform plan` to verify the restored state matches your actual infrastructure.
- Step 5: If no backup exists — use `terraform import` to re-import critical resources.
- Prevention: Always enable S3 versioning on your state bucket — it is your safety net.

---

6. Providers

A **Provider** is a plugin that allows Terraform to interact with a specific API or cloud platform. Providers are responsible for understanding how to create, read, update, and delete resources for their respective platform. Terraform downloads providers automatically during **terraform init**.

The **Terraform Registry** (registry.terraform.io) hosts thousands of providers — AWS, Azure, Google Cloud, Kubernetes, GitHub, Datadog, Cloudflare, PagerDuty, and more. Each provider is versioned and maintained either by HashiCorp, the cloud provider, or the community.

```hcl
# provider.tf — standard setup
terraform {
  required_version = ">= 1.5.0"    # minimum Terraform version
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"           # any 5.x version (not 6.x)
    }
    azurerm = {
      source  = "hashicorp/azurerm"
      version = ">= 3.0, < 4.0"
    }
  }
}

# AWS provider — region from variable, credentials from environment
provider "aws" {
  region = var.aws_region
  # Credentials: set env vars AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY
  # OR use AWS CLI configured profile
  # OR use IAM role attached to EC2/Lambda (recommended for CI/CD)
}
```

6.1 Multi-Region / Multi-Provider (Provider Aliases)

When you need to create resources in **multiple regions or multiple accounts** in the same Terraform config, you use **provider aliases**. Each provider block gets a unique alias, and resources reference the alias to specify which provider instance to use.

```hcl
# Default provider — ap-south-1 (India)
provider "aws" {
  region = "ap-south-1"
}

# Aliased provider — us-east-1 (USA)
provider "aws" {
  region = "us-east-1"
  alias  = "us_east"
}

# Resource uses DEFAULT provider (ap-south-1)
resource "aws_s3_bucket" "india_bucket" {
  bucket = "company-data-india"
}

# Resource uses ALIASED provider (us-east-1)
resource "aws_s3_bucket" "usa_bucket" {
  bucket   = "company-data-usa"
  provider = aws.us_east    # references the alias
}
```

6.2 Provider Version Constraints

CONSTRAINT SYNTAX	MEANING
`version = "5.0.0"`	Exact version only — very restrictive, rarely recommended
`version = ">= 5.0"`	Version 5.0.0 or higher — allows any future major version
`version = "~> 5.0"`	Any 5.x version but not 6.x — patch + minor updates allowed (recommended)
`version = ">= 4.0, < 6.0"`	Between 4.0 and 6.0 exclusive — range constraint
`version = "!= 5.1.0"`	Any version except 5.1.0 — useful to avoid a known broken release

🎯 Scenario-Based Interview Questions

Q1: Scenario: You need to deploy resources in both us-east-1 and eu-west-1 in the same Terraform run. How do you configure this?

- Define two AWS provider blocks — one default and one with an alias: `provider "aws" { region = "us-east-1" }` `provider "aws" { region = "eu-west-1"; alias = "europe" }`
- Then reference `aws.europe` in resources that should go to EU: `resource "aws_vpc" "eu_vpc" { cidr_block = "10.1.0.0/16"; provider = aws.europe }`
- Both are created in a single `terraform apply` — resources are created in their respective regions.

Q2: Scenario: Your provider plugin suddenly breaks after a team member ran terraform init -upgrade. How do you fix it?

- The .terraform.lock.hcl file records the exact provider version that was previously working.
- If it was committed to Git: `git checkout .terraform.lock.hcl` to restore the known-good version.
- Then run `terraform init` to download the pinned version from the lock file.
- To prevent: pin provider versions with `~>` in required_providers (e.g., "~> 5.0" not ">= 5.0").
- Always commit .terraform.lock.hcl — it is the provider equivalent of package-lock.json.

Q3: Scenario: How do you authenticate Terraform to AWS securely in a CI/CD pipeline?

- Option 1 (Best): Use OIDC/AssumeRole — GitHub Actions or Jenkins assumes an IAM role via OIDC. No static credentials stored.
- Option 2: Store AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY as pipeline secrets (environment variables).
- Option 3: For EC2-hosted runners — attach an IAM Instance Profile. Terraform auto-discovers it.
- NEVER hardcode credentials in .tf files or commit them to Git.
- Use short-lived credentials via STS AssumeRole for production pipelines — rotate regularly.

7. Data Sources

A **data source** allows Terraform to **fetch information about existing infrastructure** — resources that were NOT created by Terraform, or resources created in a different Terraform workspace/project. Data sources are **read-only**: they never create, modify, or destroy anything. They query the provider API and return information for use in your configuration.

💡 **Layman Explanation:**

- Data source = a READ operation against the cloud API.
- Resource block = CREATES something new.
- Data block = READS an existing thing (does not touch it).
- Example: 'resource aws_vpc' creates a new VPC.
- Example: 'data aws_vpc' looks up an existing VPC by its ID.
- Data sources are refreshed on every terraform plan/apply.
- Use data sources to bridge manually-created resources with Terraform-managed ones.

HCL

```hcl
# data.tf — Example 1: Use existing VPC and Subnet (created manually)
data "aws_vpc" "existing_vpc" {
  id = "vpc-0abc12345"    # VPC ID from AWS console
}

data "aws_subnet" "existing_subnet" {
  id = "subnet-0xyz78901"
}

# Example 2: Dynamic AMI lookup — always get the latest Amazon Linux 2 AMI
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter { name = "name"; values = ["amzn2-ami-hvm-*-x86_64-gp2"] }
  filter { name = "virtualization-type"; values = ["hvm"] }
}

# Reference: data.aws_ami.amazon_linux.id
# Returns the current latest AMI ID automatically on every apply
```

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your team created a VPC manually 2 years ago (before Terraform was adopted). Now you want to create new subnets inside that VPC using Terraform. How?

- Use a data source to reference the existing VPC without importing it: `data "aws_vpc" "legacy_vpc" { tags = { Name = "production-vpc" } }` `resource "aws_subnet" "new_subnet" { vpc_id = data.aws_vpc.legacy_vpc.id; cidr_block = "10.0.10.0/24" }`
- Terraform manages the new subnet, but does NOT touch the original VPC (no risk of deleting it).
- This is the safest way to work with pre-existing infrastructure.

Q2: Scenario: You are hardcoding AMI IDs like ami-0440d3b780d96b29d in your Terraform code. Why is this a problem?

- AMI IDs are region-specific — the same AMI ID does not work in different regions.
- AMI IDs become outdated — old AMIs may have unpatched security vulnerabilities.
- Fix: Use a data source for dynamic AMI lookup: `data "aws_ami" "latest" { most_recent = true; owners = ["amazon"] }` `ami = data.aws_ami.latest.id` — always resolves to the current latest AMI in any region.

Q3: Scenario: How do you share outputs between two separate Terraform projects (e.g., network project and app project)?

- Network project stores its state in S3. In outputs.tf: `output "vpc_id" { value = aws_vpc.main.id }`
- App project uses terraform_remote_state data source:

```text
data "terraform_remote_state" "network" {
  backend = "s3"
  config  = { bucket = "tf-state"; key = "network/terraform.tfstate"; region = "us-east-1" }
}
vpc_id = data.terraform_remote_state.network.outputs.vpc_id
```

- This enables decoupled projects to share outputs without combining into one monolithic config.

8. Variables & Outputs

Variables make Terraform configurations **reusable and environment-agnostic**. Instead of hardcoding values like instance types, region names, or CIDR blocks directly in your resources, you declare variables and pass in different values for different environments (dev/staging/prod). This is the single most important practice for writing maintainable Terraform code.

8.1 Input Variables (variables.tf)

HCL

```hcl
# variables.tf — declare variables with type, description, validation
variable "aws_region" {
  description = "AWS region to deploy resources"
  type        = string
  default     = "us-east-1"
  validation {
    condition     = contains(["us-east-1", "us-west-2", "eu-west-1"], var.aws_region)
    error_message = "Region must be us-east-1, us-west-2, or eu-west-1."
  }
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}

variable "environment" {
  description = "Deployment environment (dev/staging/prod)"
  type        = string
}

variable "allowed_cidrs" {
  type    = list(string)
  default = ["10.0.0.0/8"]
}

variable "tags" {
  type    = map(string)
  default = { Project = "MyApp", Owner = "DevOps" }
}
```

8.2 Assigning Values (terraform.tfvars)

HCL

```hcl
# terraform.tfvars — values for the current environment
aws_region    = "us-east-1"
instance_type = "t3.small"
environment   = "production"
allowed_cidrs = ["10.0.0.0/8", "172.16.0.0/12"]
tags = {
  Project    = "EcommerceApp"
  Owner      = "platform-team"
  Environment = "production"
  CostCenter = "engineering"
}
# Apply with specific file: terraform apply -var-file=prod.tfvars
```

8.3 Output Values (outputs.tf)

HCL

```hcl
# outputs.tf — expose useful values after apply
output "instance_public_ip" {
  description = "Public IP address of the web server"
  value       = aws_instance.web.public_ip
}

output "rds_endpoint" {
  description = "RDS database connection endpoint"
  value       = aws_db_instance.main.endpoint
  sensitive   = true   # hides value in logs and plan output
}
```

📝 **Theory & Key Points:**

- Variable types: string, number, bool, list(string), map(string), set(string), object({}), any
- Variable precedence (highest wins): -var flag > *.auto.tfvars > terraform.tfvars > env vars > defaults
- Environment variable syntax: `export TF_VAR_aws_region=us-east-1`
- `sensitive = true`: hides output values in plan/apply output. Still stored in state file — encrypt state!
- Always add descriptions to variables and outputs — they appear in terraform-docs generation.
- validation blocks catch errors BEFORE plan runs — fail fast with clear error messages.
- Use object() type for complex structured variables (e.g., database configuration objects).

🎯 Scenario-Based Interview Questions

Q1: Scenario: You need different EC2 instance sizes for dev, staging, and prod. How do you manage this with variables?

- Declare the variable: `variable "instance_type" { type = string; default = "t3.micro" }`
- Create environment-specific tfvars files: dev.tfvars: `instance_type = "t3.micro"` staging.tfvars: `instance_type = "t3.small"` prod.tfvars: `instance_type = "t3.large"`
- Apply with: `terraform apply -var-file=prod.tfvars`
- Alternative: use locals with conditional: `instance_type = local.is_prod ? "t3.large" : "t3.micro"`

Q2: Scenario: You have a sensitive database password in your Terraform variables. How do you handle it securely?

- Declare with sensitive = true: `variable "db_password" { type = string; sensitive = true }`
- Never put the real value in terraform.tfvars committed to Git.
- Pass at runtime: `terraform apply -var="db_password=$DB_PASS"` from a secrets manager.
- Use AWS Secrets Manager or SSM Parameter Store and fetch with a data source.
- In CI/CD: inject as environment variable TF_VAR_db_password from vault/secrets store.

Q3: Scenario: Another Terraform project needs the VPC ID created by your project. How do you expose and consume it?

- In your project outputs.tf: `output "vpc_id" { value = aws_vpc.main.id }`
- In the consuming project — use remote state data source: `data "terraform_remote_state" "network" { backend = "s3"; config = { bucket = "tf-state"; key = "vpc/terraform.tfstate" } }` `vpc_id = data.terraform_remote_state.network.outputs.vpc_id`
- This creates a clean dependency between projects without hardcoding IDs.

9. Meta-Arguments

Meta-arguments are **special arguments** that can be used in ANY resource block — they control the behavior of how resources are created, updated, and destroyed, rather than configuring the resource itself. They are built into Terraform's core and work across all providers.

META-ARGUMENT	PURPOSE
`depends_on`	Explicitly declare resource dependencies that Terraform cannot auto-detect
`count`	Create N identical copies of a resource based on a number
`for_each`	Create one resource per item in a map or set — each with its own config
`lifecycle`	Control create/destroy behavior: prevent_destroy, create_before_destroy, ignore_changes
`provider`	Specify which provider alias to use for this resource (for multi-region/account)
`provisioner`	Run scripts on a resource after creation (file, local-exec, remote-exec)

9.1 depends_on — Explicit Dependencies

HCL

```hcl
# EC2 needs S3 bucket to exist BEFORE starting (startup script downloads from S3)
# Without depends_on, Terraform might try to create them simultaneously
resource "aws_s3_bucket" "config_bucket" {
  bucket = "myapp-config-bucket"
}

resource "aws_instance" "app_server" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t3.micro"
  user_data     = file("startup.sh")    # script downloads from S3 on boot
  depends_on    = [aws_s3_bucket.config_bucket]
}
```

9.2 count — Create Multiple Identical Resources

HCL

```
# Create 3 identical EC2 instances, each with a unique name
resource "aws_instance" "web_servers" {
  count         = 3
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t3.micro"
  tags = {
    Name = "web-server-${count.index + 1}"   # web-server-1, web-server-2, web-server-
3
  }
}
# Reference specific: aws_instance.web_servers[0].public_ip
# Reference all:      aws_instance.web_servers[*].public_ip
```

⚠️ **Important Warning — count LIMITATION:**

- If you remove an item from the middle, ALL subsequent resources are recreated.
- count=3 creates web-server-0, web-server-1, web-server-2.
- If you remove server-1 (reorder), server-2 becomes server-1 — RECREATED!
- Use for_each instead of count when resources need stable identities.

9.3 for_each — Create Resources from a Map or Set

HCL

```
# Create S3 buckets for multiple environments using for_each + map
resource "aws_s3_bucket" "env_buckets" {
  for_each = {
    dev     = "myapp-dev-bucket-2024"
    staging = "myapp-staging-bucket-2024"
    prod    = "myapp-prod-bucket-2024"
  }
  bucket = each.value    # the map value (bucket name)
  tags = {
    Name        = each.value
    Environment = each.key    # "dev", "staging", or "prod"
  }
}
# Reference: aws_s3_bucket.env_buckets["prod"].arn
```

9.4 lifecycle — Control Resource Behavior

HCL

```
resource "aws_instance" "production_db" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t3.xlarge"
  lifecycle {
    create_before_destroy = true  # Create new BEFORE destroying old (reduces
downtime)
    prevent_destroy       = true  # BLOCK terraform destroy for this critical
resource
    ignore_changes = [
      tags["LastModified"],       # Ignore external tag changes
      user_data,                  # Ignore user_data changes after first apply
    ]
  }
}
```

LIFECYCLE RULE	EFFECT
`create_before_destroy = true`	Create replacement FIRST, then destroy old. Reduces downtime during forced replacements.
`prevent_destroy = true`	Block ALL destroy operations on this resource. Terraform errors if destroy is attempted.
`ignore_changes = [attr1]`	Ignore specified attribute changes — Terraform will not try to reconcile them.
`ignore_changes = all`	Never update this resource after initial creation. Terraform can only create/destroy it.

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your EC2 instance has an Auto Scaling group that modifies the 'desired_count' tag automatically. Terraform keeps reverting it. How do you fix this?

- Add ignore_changes to the lifecycle block for those specific tags: `lifecycle { ignore_changes = [ tags["desired_count"], tags["last_scaled_at"] ] }`
- Now `terraform apply` will NOT revert the Auto Scaling tags.
- Terraform still manages all other attributes — only the specified tag keys are ignored.
- This is the correct approach for any attributes managed by external systems.

Q2: Scenario: What is the difference between count and for_each? When would you use each?

- count: Use when you need N identical resources with sequential numbering.
- for_each: Use when each resource has a unique identity (name, region, environment).
- Removing one for_each item destroys ONLY that resource — count destroys all after that index.
- Rule: 3 identical web servers → count = 3
- Rule: S3 bucket per environment (dev/staging/prod) → for_each = { dev=..., staging=..., prod=... }
- Always prefer for_each for production resources with stable identities.

Q3: Scenario: You want to ensure a production RDS database is never accidentally deleted. How do you configure this?

- Add `lifecycle { prevent_destroy = true }` to the aws_db_instance resource.
- Terraform will throw an error if anyone tries to run terraform destroy on it.
- Error: "Instance cannot be destroyed".
- To actually delete it (intentionally): remove the prevent_destroy block first, then run destroy.
- Also useful for: S3 buckets with critical data, DynamoDB tables, Route53 hosted zones.

Q4: Scenario: You are replacing an EC2 instance with a new AMI version. How do you minimize downtime?

- By default Terraform destroys the old instance first, then creates new — causes downtime.
- Add `lifecycle { create_before_destroy = true }` to the EC2 resource.
- Terraform creates the replacement instance first, then terminates the old one.
- Combine with an ALB/Load Balancer to route traffic to new instance before old is removed.
- For zero-downtime: use blue/green deployment with Auto Scaling Groups instead.

10. Provisioners

Provisioners are a **last resort** mechanism in Terraform for executing actions on local or remote machines after a resource is created. HashiCorp recommends avoiding provisioners when possible — use them only when there is no native Terraform resource or provider for what you need.

Why avoid provisioners? Provisioners break Terraform's **idempotency** guarantee. If a provisioner script fails halfway, Terraform marks the resource as "tainted" and tries to destroy + recreate it on the next apply. Better alternatives: AWS user_data, cloud-init, Packer AMIs, or Ansible post-provisioning.

🔶 **Terraform Concept — 3 Types of Provisioners:**

- **file** = copy files/directories from your machine TO the remote resource
- **local-exec** = run a command on YOUR LOCAL machine after resource creation
- **remote-exec** = run commands ON THE REMOTE RESOURCE (requires SSH/WinRM connection)

HCL

```hcl
# remote-exec Provisioner – runs on EC2 via SSH
resource "aws_instance" "web" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t3.micro"
  key_name      = aws_key_pair.deployer.key_name

  connection {
    type        = "ssh"
    user        = "ec2-user"
    private_key = file("~/.ssh/id_rsa")
    host        = self.public_ip
  }

  provisioner "remote-exec" {
    inline = [
      "sudo yum update -y",
      "sudo yum install -y nginx",
      "sudo systemctl start nginx",
    ]
  }

  # local-exec – runs on YOUR machine after EC2 is created
  provisioner "local-exec" {
    command = "echo ${self.public_ip} >> inventory.txt"
  }
}
```

⚠️ **Important Warning:**

- Avoid provisioners when possible — use AWS user_data (cloud-init) for instance bootstrapping.
- If a provisioner fails, the resource is marked "tainted" — next apply destroys and recreates it.
- remote-exec requires SSH access — make sure security group allows inbound port 22.
- local-exec is NOT idempotent — it runs every time the resource is created.
- Better alternatives: AWS Launch Templates with user_data, Packer for pre-baked AMIs.

🎯 Scenario-Based Interview Questions

Q1: Scenario: When should you use a provisioner vs user_data for bootstrapping an EC2 instance?

- user_data (preferred): Cloud-init script runs at first boot, idempotent, no SSH needed.
- user_data is baked into the instance launch — Terraform has no SSH dependency.
- Provisioner: Use ONLY when user_data is not possible (e.g., running Ansible post-provisioning).
- Provisioner risk: if SSH fails (security group, key mismatch), Terraform marks resource as tainted.
- Best practice: Build immutable AMIs with Packer (pre-configured) and use them directly.

Q2: Scenario: A provisioner in your Terraform config fails midway. What is the resource state and how do you recover?

- The resource is marked as "tainted" in the state file — Terraform considers it incomplete.
- On next `terraform apply`, Terraform will destroy the tainted resource and create a new one.
- To manually untaint: `terraform untaint <resource_address>` (if the resource is actually fine).
- Check the provisioner script for idempotency issues — ensure it can run safely multiple times.
- Prevention: Use null_resource with triggers instead of directly attaching provisioners to instances.

Q3: Scenario: You need to run an Ansible playbook after Terraform creates an EC2 instance. How do you integrate them?

- Option 1 (local-exec): Use provisioner "local-exec" to trigger Ansible after instance creation. `command = "ansible-playbook -i ${self.public_ip}, playbook.yml"`
- Option 2 (CI/CD pipeline): Terraform apply creates EC2, then pipeline step runs Ansible separately.
- Option 2 is recommended — keeps concerns separated and both tools in their natural roles.
- Add a null_resource with depends_on the EC2 instance to trigger the Ansible run at the right time.

11. Terraform Modules

A Terraform module is a **reusable package of Terraform resources** grouped together. Every Terraform configuration is technically a module — the "root module". Modules allow you to encapsulate a pattern (like "an EC2 instance with security group") into a reusable component that can be used multiple times with different parameters.

Fig 4: "Terraform Module Architecture — Root + Child Modules" — a diagram showing a Root Module box (provider.tf, main.tf, variables.tf, outputs.tf, backend.tf) with arrows fanning out to four child modules: module/vpc, module/rds, module/ec2, and module/iam (each containing main.tf, variables.tf, outputs.tf). Below, all modules connect down into "AWS Cloud Resources (Provisioned)": VPC, Subnets, EC2, ALB, RDS, IAM Roles, and S3.

Why modules are essential: Without modules, every team creating an EC2 instance writes the same resource blocks from scratch — with slightly different configurations, missing best practices, and no consistency. With modules, the platform team defines a **vetted, compliant EC2 module**, and other teams call it with just a few lines of code.

> 💡 **Layman Explanation:**
>
> - Module = a reusable Terraform component, like a function in programming.
> - Instead of 100 lines of EC2 configuration repeated everywhere...
> - ...a module reduces it to 10 lines: `module "my_ec2" { source="./modules/ec2"; ... }`
> - Platform team maintains the module. App teams just call it with their parameters.
> - Modules can be local (./modules/vpc) or remote (GitHub, Terraform Registry).
> - Terraform Registry has thousands of community modules: registry.terraform.io/modules

HCL

```hcl
# modules/ec2/variables.tf
variable "ami_id"         { type = string }
variable "instance_type"  { type = string; default = "t3.micro" }
variable "subnet_id"      { type = string }
variable "name"           { type = string }
variable "environment"    { type = string }

# modules/ec2/main.tf
resource "aws_instance" "this" {
  ami           = var.ami_id
  instance_type = var.instance_type
  subnet_id     = var.subnet_id
  tags = { Name = var.name; Environment = var.environment }
}

# modules/ec2/outputs.tf
output "instance_id" { value = aws_instance.this.id }
output "private_ip"  { value = aws_instance.this.private_ip }

# root/main.tf — calling the module
module "web_server" {
  source        = "./modules/ec2"
  ami_id        = data.aws_ami.amazon_linux.id
  instance_type = "t3.small"
  subnet_id     = aws_subnet.public.id
  name          = "web-server-prod"
  environment   = "production"
}

# From Terraform Registry (official AWS VPC module):
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "5.1.0"
  name    = "production-vpc"
  cidr    = "10.0.0.0/16"
}
```

🎯 Scenario-Based Interview Questions

Q1: Scenario: 5 different teams each write their own EC2 Terraform code with inconsistent configurations. How do you standardize?

- Create a company-wide internal Terraform module in a shared Git repo.
- Platform team builds modules/ec2/ with all required standards: encryption, mandatory tags, SSM agent.
- Publish to private Git repo or Terraform Cloud private registry.
- All teams call: `module "app" { source = "git::https://company-git/tf-modules//ec2" }`
- Teams provide only: name, instance_type, subnet_id.
- All compliance defaults are enforced by the module — teams cannot accidentally skip them.

Q2: Scenario: When should you use a local module vs a public Terraform Registry module?

- Public Registry (terraform-aws-modules): battle-tested, community-maintained, great for common patterns (VPC, EKS, RDS).
- Local modules: for company-specific patterns, proprietary standards, or highly customized configurations.
- Rule: Start with public Registry modules for generic infrastructure. Build local modules for custom business logic.
- Pin version for Registry modules: `version = "~> 5.1.0"` — prevents surprise breaking changes.
- For sensitive environments: copy and vendor Registry modules into your own repo for full control.

Q3: Scenario: You updated a shared Terraform module that is used by 10 different teams. How do you manage the rollout safely?

- Use semantic versioning for your module (v1.0.0, v1.1.0, v2.0.0).
- Teams pin the module version: `source = "git::https://.../vpc?ref=v1.0.0"`
- Release v1.1.0 with the new feature — backward compatible. Teams upgrade at their own pace.
- For breaking changes: create v2.0.0 with a migration guide. Never silently modify an existing version.
- Test module changes with terraform plan in a sandbox environment before releasing the new version.

Q4: Scenario: What is the difference between calling a module and calling a resource directly?

- Resource: directly creates one AWS resource. module: calls a reusable package that may create multiple resources.
- Module provides abstraction — consumers pass high-level inputs, module handles all the detail.
- Module outputs are referenced as: `module.<name>.<output>` (e.g., module.vpc.vpc_id).
- After adding or changing a module source, always run `terraform init` to download it.
- Modules enable DRY (Do Not Repeat Yourself) principles in infrastructure code.

---

12. Terraform Import

`terraform import` brings **existing cloud resources** (created manually or by another tool) under Terraform management. It reads the real resource from the cloud API and writes its configuration into the Terraform state file. After importing, Terraform can plan and apply changes to that resource just like any resource it originally created.

💡 **Layman Explanation:**

- terraform import = 'Hey Terraform, this resource already exists — start managing it.'
- Import does NOT generate .tf configuration files — you must write them manually first.
- Import only writes to the state file — the .tf code must match, or plan will show changes.
- Goal after import: 'terraform plan' shows 'No changes.' — perfect alignment.

HCL

```hcl
# IMPORT STEPS:

# Step 1: Write skeleton resource block in main.tf
resource "aws_instance" "legacy_server" {
  ami           = "unknown"    # placeholder — update after import
  instance_type = "unknown"
}

# Step 2: Run terraform init
# Step 3: Import the resource
terraform import aws_instance.legacy_server i-0b9be609418aa0609
# Format: terraform import <resource_type>.<name> <real_resource_id>

# Step 4: terraform show — read state to get all attribute values
# Step 5: Update main.tf to match state file values
resource "aws_instance" "legacy_server" {
  ami           = "ami-00f22f6155d6d92c5"  # from state
  instance_type = "t2.micro"
  tags = { Name = "LegacyServer" }
}

# Step 6: terraform plan → should show 'No changes'
```

RESOURCE TYPE	IMPORT COMMAND
aws_instance	`terraform import aws_instance.web i-0abc12345`
aws_s3_bucket	`terraform import aws_s3_bucket.data my-bucket-name`
aws_security_group	`terraform import aws_security_group.sg sg-0abc12345`
aws_vpc	`terraform import aws_vpc.main vpc-0abc12345`
aws_iam_role	`terraform import aws_iam_role.deployer role-name`
aws_route53_record	`terraform import aws_route53_record.www ZONEID_www.example.com_A`

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your company has 50 EC2 instances created manually over 3 years. Management wants everything managed by Terraform. What is your approach?

- Phased migration: Do not try to import all 50 at once.
- Use "terraformer" tool — auto-generates both import commands AND .tf code from existing AWS resources.
- For each resource: write skeleton .tf block → terraform import → terraform show → update .tf → verify plan shows no changes.
- Group similar resources into modules (all web servers, all DB servers, etc.) as you import.
- Enable policy: all future changes must go through Terraform — block console modifications.

Q2: Scenario: After running terraform import, terraform plan shows changes to the resource. What does that mean and how do you fix it?

- It means your .tf file does not perfectly match the real resource attributes.
- Run `terraform state show <resource>` to see all actual attribute values.
- Update your .tf resource block to exactly match every attribute in the state output.
- Some attributes (like tags with auto-generated values) may need lifecycle { ignore_changes } blocks.
- Keep iterating: import → show → update .tf → plan → repeat until "No changes".

Q3: Scenario: Terraform 1.5+ introduced "import blocks" — how is that different from terraform import command?

- Old way: `terraform import` command is imperative — run once, not repeatable, not in version control.
- New way (v1.5+): `import {}` block in .tf file — declarative, version-controlled, repeatable. `import { to = aws_instance.web; id = "i-0abc12345" }`
- With v1.6+ `terraform generate config`: Terraform can auto-generate the .tf resource block too.
- Import blocks are the modern standard — more team-friendly and auditable than CLI commands.

13. Terraform Workspaces

Terraform workspaces allow you to maintain **multiple, independent state files** within a single Terraform configuration directory. Each workspace has its own state file, so the same code can provision separate infrastructure for different environments (dev, staging, prod) without any code changes.

HCL

```hcl
# Workspace commands:
terraform workspace list             # list all workspaces (* = current)
terraform workspace show             # show current workspace name
terraform workspace new dev          # create and switch to "dev" workspace
terraform workspace select prod      # switch to existing "prod" workspace
terraform workspace delete dev       # delete a workspace (must be empty)

# Use workspace name in resources:
resource "aws_instance" "web" {
  instance_type = terraform.workspace == "prod" ? "t3.large" : "t3.micro"
  tags = {
    Name        = "web-server-${terraform.workspace}"
    Environment = terraform.workspace
  }
}
```

📝 **Theory & Key Points:**

- Default workspace is called 'default' — it always exists and cannot be deleted.
- `terraform.workspace` variable contains current workspace name (e.g., "dev", "prod").
- Workspaces are simpler than maintaining separate directories per environment.
- Limitation: Workspaces still use the SAME .tf code — if dev and prod need very different configs, use separate directories.
- In remote S3 backend, workspace state paths: `s3://bucket/env:/workspace-name/terraform.tfstate`
- Terraform Cloud offers more powerful workspaces with team-level access controls and run history.

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your team uses the same Terraform code for dev, staging, and production. How do you manage separate state files?

- Approach 1 — Workspaces: `terraform workspace new dev && terraform apply`
- Use `terraform.workspace` in resource names and conditional instance sizes.
- Pro: single code directory. Con: all envs share same .tf code (hard for major differences).
- Approach 2 — Separate directories: environments/dev/ and environments/prod/ — both call shared modules.
- Best practice: Use workspaces for similar environments; separate directories for environments with major structural differences.

Q2: Scenario: What happens to the state file when you switch workspaces in Terraform?

- Terraform uses a completely separate state file for each workspace.
- Local backend: state stored in `terraform.tfstate.d/<workspace>/terraform.tfstate`
- S3 backend: state stored at `env:/<workspace-name>/<key>` in the bucket.
- Switching workspaces does NOT affect the other workspace's state or infrastructure.
- Always verify your current workspace before running apply: `terraform workspace show`

Q3: Scenario: When would you choose Terraform workspaces over separate directories for environment management?

- Use workspaces: environments are nearly identical (same resources, same structure, different sizes).
- Use separate directories: environments have fundamentally different architectures (e.g., dev has no DR, prod does).
- Workspaces simplify: single terraform apply with workspace selection manages all environments.
- Separate directories: more explicit, independent lifecycle per environment, clearer blast radius.
- Large teams often prefer separate directories — clearer ownership and harder to accidentally deploy to wrong env.

14. Locals

Local values (**locals**) allow you to assign a name to an expression used multiple times within a module. They are computed internally within Terraform — unlike variables (which are set by the user), locals are set **by the configuration itself**. Locals reduce repetition, improve readability, and allow complex expression results to be reused.

HCL

```hcl
# locals.tf
locals {
  name_prefix  = "${var.project}-${var.environment}-${var.region}"
  common_tags = {
    Project     = var.project
    Environment = var.environment
    ManagedBy   = "Terraform"
    LastUpdated = formatdate("YYYY-MM-DD", timestamp())
  }
  is_prod       = var.environment == "prod"
  instance_type = local.is_prod ? "t3.large" : "t3.micro"
}

# Use locals in resources:
resource "aws_s3_bucket" "app_data" {
  bucket = "${local.name_prefix}-data"   # myapp-prod-us-east-1-data
  tags   = local.common_tags
}

resource "aws_instance" "web" {
  instance_type = local.instance_type    # t3.large in prod, t3.micro in dev
  tags          = merge(local.common_tags, { Role = "WebServer" })
}
```

🎯 **Scenario-Based Interview Questions**

Q1: Scenario: You repeat the same tag block in 30 different resources. How do locals help?

- Define common_tags in locals.tf once: `locals { common_tags = { Project = var.project, ManagedBy = "Terraform" } }`
- Apply to every resource: `tags = local.common_tags`
- To add resource-specific tags: `tags = merge(local.common_tags, { Role = "WebServer" })`
- If tag requirements change, update in ONE place — all 30 resources get the change automatically.
- Locals also ensure consistency — no typos in repeated tag keys across resources.

Q2: Scenario: What is the difference between locals and variables in Terraform?

- Variables: set externally by the user (CLI, tfvars, environment variables). User controls the value.
- Locals: computed internally by the configuration. The config controls the value. Cannot be overridden from outside.
- Use variables for: things users need to customize (instance_type, region, environment name).
- Use locals for: derived values, complex expressions, values computed from other variables.
- Example: `variable "environment"` (set by user) → `local "is_prod" = var.environment == "prod"` (computed from it).

---

15. Terraform Functions

Terraform includes a rich library of **built-in functions** that can be used in expressions throughout your configuration. These functions enable string manipulation, collection operations, math, type conversions, and file/encoding operations. Functions are called with the syntax: `function_name(argument1, argument2)`.

FUNCTION CATEGORY	COMMON EXAMPLES
String	format(), upper(), lower(), replace(), trimspace(), join(), split(), startswith(), endswith()
Numeric	abs(), ceil(), floor(), min(), max(), pow(), log()
Collection	length(), toset(), tolist(), tomap(), concat(), flatten(), merge(), keys(), values(), zipmap()
Encoding	base64encode(), base64decode(), jsonencode(), jsondecode(), yamlencode(), yamldecode()
Filesystem	file(), filebase64(), templatefile(), fileset(), pathexpand()
Date/Time	timestamp(), formatdate(), timeadd()
IP Network	cidrsubnet(), cidrhost(), cidrnetmask(), cidrrange()
Type Conversion	tostring(), tonumber(), tobool(), can(), try()

HCL

```hcl
# Common function examples:

# String functions
locals {
  upper_env   = upper(var.environment)                      # "PROD"
  bucket_name = lower("${var.project}-${var.env}")           # "myapp-prod"
  joined      = join(", ", ["us-east-1", "eu-west-1"])       # "us-east-1, eu-west-1"
}

# cidrsubnet — calculate subnet CIDRs automatically
resource "aws_subnet" "public" {
  cidr_block = cidrsubnet("10.0.0.0/16", 8, 1)   # "10.0.1.0/24"
}

# templatefile — render a template with variables
user_data = templatefile("${path.module}/userdata.tpl", {
  app_name = var.app_name
  db_host  = aws_db_instance.main.endpoint
})

# jsonencode — encode complex objects as JSON string
policy = jsonencode({
  Version   = "2012-10-17"
  Statement = [{ Effect = "Allow"; Action = "s3:*"; Resource = "*" }]
})
```

🎯 Scenario-Based Interview Questions

Q1: Scenario: You need to create subnets across 3 AZs with sequential CIDR blocks. How do you use cidrsubnet?

- `cidrsubnet(iprange, newbits, netnum)` calculates a subnet CIDR from a parent range.
- Example: `cidrsubnet("10.0.0.0/16", 8, 1) = "10.0.1.0/24"`, `cidrsubnet("10.0.0.0/16", 8, 2) = "10.0.2.0/24"`
- With for_each and index:

```TEXT
resource "aws_subnet" "subnets" {
  for_each   = toset(["us-east-1a", "us-east-1b", "us-east-1c"])
  cidr_block = cidrsubnet("10.0.0.0/16", 8, index(tolist(toset(...)),
each.key))
}
```

- This eliminates hardcoded CIDR blocks and makes the config dynamic and scalable.

Q2: Scenario: How do you use templatefile() to generate dynamic user_data scripts for EC2?

- Create a template file: userdata.tpl with `${variable}` placeholders:

```TEXT
#!/bin/bash
echo "APP_NAME=${app_name}" >> /etc/environment
echo "DB_HOST=${db_host}" >> /etc/environment
```

- In Terraform resource:

```TEXT
user_data = templatefile("${path.module}/userdata.tpl", {
  app_name = var.app_name
  db_host  = aws_db_instance.main.endpoint
})
```

- `templatefile()` is preferred over the deprecated template_file data source.

16. Terraform in CI/CD Pipelines

Integrating Terraform into CI/CD pipelines is a **critical DevOps practice** that enforces code review for infrastructure changes, prevents manual console modifications, and provides a full audit trail of every infrastructure change. The standard approach separates the Plan stage (reviewable) from the Apply stage (controlled).

16.1 Standard CI/CD Pipeline Stages

PIPELINE STAGE	TERRAFORM COMMAND & PURPOSE
1. Code Lint & Format	`terraform fmt -check` — fail if code is not properly formatted
2. Validate	`terraform validate` — check syntax without cloud credentials
3. Security Scan	tfsec / checkov — scan for security misconfigurations in .tf files
4. Plan	`terraform plan -out=plan.tfplan` — generate and save the execution plan
5. Plan Review	Human/team reviews the plan output in the PR/pipeline UI
6. Approval Gate	Manual approval required before apply (protected branch or pipeline gate)
7. Apply	`terraform apply plan.tfplan` — execute EXACTLY the reviewed plan
8. Post-Apply Tests	Terratest / Inspec — verify infrastructure is correctly provisioned

YAML

```yaml
# GitHub Actions — Terraform CI/CD Pipeline (simplified)
name: Terraform
on: [pull_request, push]

jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: "1.7.0"

      - name: Terraform Init
        run: terraform init

      - name: Terraform Format Check
        run: terraform fmt -check

      - name: Terraform Validate
        run: terraform validate

      - name: Terraform Plan
        run: terraform plan -out=tfplan

      # Apply only on merge to main branch
      - name: Terraform Apply
        if: github.ref == 'refs/heads/main'
        run: terraform apply -auto-approve tfplan
```

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your team wants to ensure all Terraform changes go through code review before being applied to production. How do you implement this?

- Set up branch protection on "main" — direct pushes blocked, PRs required.
- CI pipeline runs `terraform plan` on every PR — plan output visible to reviewers.
- Manual approval gate in the pipeline before terraform apply is triggered.
- Apply only triggers on merge to main — not on PRs.
- Use Atlantis or Terraform Cloud for PR-based plan/apply workflow with Slack notifications.

Q2: Scenario: How do you prevent sensitive secrets from appearing in terraform plan output in CI/CD logs?

- Mark sensitive variables with `sensitive = true` — Terraform hides their values in plan output.
- Inject secrets as environment variables (TF_VAR_db_password) from vault/secrets manager.
- Use AWS Secrets Manager or SSM Parameter Store with data sources — never pass raw secrets.
- Mask secrets in CI/CD: GitHub Actions and GitLab CI auto-mask registered secret values in logs.
- Review terraform plan output before committing pipelines to ensure no accidental secret exposure.

Q3: Scenario: Your terraform apply in CI/CD fails midway. Infrastructure is partially created. How do you handle this?

- Do NOT immediately re-run apply without investigating.
- Run `terraform state list` to see which resources were successfully created.
- Run `terraform plan` to see what Terraform thinks needs to happen next.
- In most cases: re-running `terraform apply` will continue from where it left off (Terraform is idempotent).
- If state is corrupted: restore from S3 version history of the state file.
- If resources are in bad state: manually fix in AWS console and run `terraform import` or `terraform state rm`.

17. Terraform Security Best Practices

Security in Terraform covers three main areas: **credentials management**, **state file security**, and **infrastructure security scanning**. Ignoring any of these areas can lead to credential exposure, state file tampering, or misconfigured cloud resources.

17.1 Credentials Management

CREDENTIALS METHOD	SECURITY LEVEL & NOTES
IAM Role (EC2/Lambda)	⭐⭐⭐ Best — no credentials stored, auto-rotated by AWS. Use for CI/CD runners on AWS.
OIDC / AssumeRole (GitHub Actions)	⭐⭐⭐ Best — short-lived tokens, no static keys stored in pipeline secrets.
AWS CLI Profile (local dev)	⭐⭐ Good — credentials in ~/.aws/credentials, not in code. Use for local development only.
Environment Variables	⭐ Acceptable — AWS_ACCESS_KEY_ID + AWS_SECRET_ACCESS_KEY as CI secrets. Rotate regularly.
Hardcoded in .tf files	❌ NEVER — credentials committed to Git are permanently compromised.

17.2 State File Security

⚠️ **State File Security Critical Points:**

- The state file stores real resource IDs, IP addresses, and potentially PASSWORDS in plaintext JSON.
- Always enable encryption on your S3 state bucket (AES-256 or KMS).
- Enable S3 bucket versioning — enables recovery from corruption or accidental deletion.
- Restrict S3 bucket access to only Terraform execution roles — block public access.
- Never store state in Git — even accidentally committed state files can expose secrets.
- Use DynamoDB state locking to prevent concurrent modifications.

17.3 Security Scanning Tools

TOOL	PURPOSE
tfsec	Static analysis of Terraform code — finds security misconfigurations before apply
checkov	Multi-framework IaC scanner — supports Terraform, CloudFormation, Kubernetes
terrascan	Scans for compliance violations against CIS, GDPR, HIPAA policies
Infracost	Cost estimation for Terraform plans — shows cost impact of changes in CI/CD
Sentinel (HashiCorp)	Policy-as-Code framework for Terraform Cloud/Enterprise — enforces rules before apply
OPA / Conftest	Open Policy Agent — custom policy enforcement for Terraform plan JSON output

🎯 Scenario-Based Interview Questions

Q1: Scenario: A developer accidentally committed AWS credentials in a terraform.tfvars file to a public GitHub repo. What do you do immediately?

- Step 1: Rotate the compromised credentials IMMEDIATELY in AWS IAM — do not wait.
- Step 2: Revoke the exposed key and create a new one.
- Step 3: Check CloudTrail for unauthorized API calls using the compromised credentials.
- Step 4: Remove the credentials from Git history using git filter-branch or BFG Repo Cleaner.
- Step 5: Make the GitHub repository private or delete and recreate it if public.
- Prevention: Add .gitignore for *.tfvars files. Use pre-commit hooks with git-secrets to scan before commit.

Q2: Scenario: How do you enforce that all Terraform-managed S3 buckets have encryption enabled?

- Option 1: tfsec scan in CI pipeline — tfsec will flag unencrypted S3 buckets before apply.
- Option 2: Sentinel policy (Terraform Cloud): enforce "all aws_s3_buckets must have encryption block".
- Option 3: Terraform module — company-wide S3 module that always includes encryption by default.
- Option 4: AWS SCP (Service Control Policy) — deny s3:CreateBucket without encryption at the AWS org level.
- Best practice: Combine all four — defense in depth for compliance.

Q3: Scenario: Explain why Terraform state files should never be committed to Git.

- State files contain real resource IDs, IP addresses, and cloud credentials in PLAINTEXT JSON.
- Database connection strings, passwords marked as sensitive may still appear in state.
- Once committed to Git, the secrets are in history permanently — even if later deleted from the file.
- Multiple developers with local state files causes corruption when they simultaneously apply.
- Solution: Use S3 remote backend with encryption enabled + DynamoDB locking.

18. Quick Command Reference

Core Workflow

COMMAND	DESCRIPTION
`terraform init`	Initialize project — download providers + modules
`terraform fmt`	Auto-format all .tf files
`terraform validate`	Check syntax and configuration errors
`terraform plan`	Preview changes (dry run)
`terraform apply`	Apply changes (with confirmation)
`terraform apply -auto-approve`	Apply without confirmation (CI/CD)
`terraform destroy`	Destroy all managed resources
`terraform output`	Show all output values
`terraform show`	Show current state in readable format
`terraform refresh`	Sync state with real infrastructure

State Management

COMMAND	DESCRIPTION
`terraform state list`	List all resources in state
`terraform state show <res>`	Show attributes of one resource
`terraform state mv`	Rename resource in state
`terraform state rm <res>`	Remove resource from state tracking
`terraform force-unlock <id>`	Release stuck state lock

Import & Workspaces

COMMAND	DESCRIPTION
`terraform import <type>.<name> <id>`	Import existing resource into state
`terraform workspace list`	List all workspaces
`terraform workspace new dev`	Create + switch to "dev" workspace
`terraform workspace select prod`	Switch to "prod" workspace
`terraform workspace show`	Show current workspace

CI/CD & Security

COMMAND / TOOL	DESCRIPTION
`terraform fmt -check`	Check formatting (returns non-zero if unformatted — use in CI)
`terraform plan -out=plan.tfplan`	Save plan artifact for later apply
`terraform apply plan.tfplan`	Apply EXACTLY the saved plan (use after review)
`tfsec .`	Scan Terraform code for security issues
`checkov -d .`	Multi-framework compliance scanning
`infracost breakdown --path .`	Estimate monthly cost of Terraform configuration

---

by Veera Sir — MultiCloud DevOps — Terraform Study Notes

MVN

PART 03 OF 08

Maven

Java project build automation, dependency management, and lifecycle.

Source: *Maven_SonarQube_Final_Notes.pdf* (MultiCloud DevOps by Veera Sir — "Maven & SonarQube: Build Automation & Code Quality — Complete Study Notes"), Part A, pages 2–19.

1. What is Maven?

Maven is a **build automation and project management tool**, used primarily for Java projects, although it can also build C#, Ruby, Scala, and other language projects. Maven is hosted and maintained by **The Apache Software Foundation** and is itself written in Java. The name 'Maven' is a Yiddish word meaning 'accumulator of knowledge' — fitting, as the tool centralizes everything needed to build a project.

The core problem Maven solves: Before build tools like Maven existed, developers manually downloaded JAR files for every library their project needed, manually set classpaths, and wrote custom scripts to compile, test, and package code. This was error-prone and impossible to standardize across teams. Maven introduced **Convention over Configuration** — a standard project structure and build process that works the same way for every Maven project, anywhere.

> 💡 **Layman Explanation:**
>
> Maven is like a restaurant kitchen with a strict recipe book (pom.xml). You don't manually go buy ingredients (libraries) — you list them in the recipe, and Maven fetches them automatically. Every Maven 'kitchen' (project) follows the same layout: ingredients here, recipe steps there, finished dish goes there. Any chef (developer) who knows Maven can walk into ANY Maven project and immediately know where everything is.

1.1 Why Use Maven? — Key Advantages

ADVANTAGE	EXPLANATION
Dependency Management	Automatically downloads required libraries (JARs) from remote repositories. No manual JAR downloading or classpath management.
Standardized Build Process	Same commands (`mvn compile`, `mvn test`, `mvn package`) work identically across every Maven project, regardless of size or team.
Project Management	Centralizes project metadata: version, name, dependencies, plugins — all in one pom.xml file.
Build Automation	Single command runs the ENTIRE build process: compile → test → package → install, automatically in correct order.
Report Generation	Generates project reports: test coverage, code quality metrics, dependency reports, javadocs.
Plugin Ecosystem	Thousands of plugins extend Maven: SonarQube scanner, Docker builds, deployment plugins, code coverage tools.
Multi-Module Support	Large projects can be split into multiple modules, all managed and built together via a parent POM.
Written in Java	Cross-platform — runs anywhere Java runs (Windows, Linux, Mac).

🎯 **Scenario-Based Interview Questions**

Q1: Scenario: A new developer joins your team and says 'I don't understand why we need Maven — can't we just manually compile Java files with javac?' How do you explain the need for Maven? `javac` works for a single file, but real projects have problems Maven solves:

1. **DEPENDENCIES:** A typical enterprise app uses 20-50 third-party libraries (Spring, Hibernate, logging, JSON parsers). Manually downloading and managing JAR versions for all of them — and their own dependencies (transitive dependencies) — is unmanageable. Maven downloads everything automatically based on pom.xml.
2. **STANDARDIZATION:** With javac, every developer might organize folders differently. Maven enforces ONE standard structure (`src/main/java` , `src/test/java`) so any Maven project is immediately navigable by any developer.
3. **BUILD STAGES:** Compiling is just one step. You also need: run tests, package into JAR/WAR, copy to a repository. Maven automates this entire pipeline with one command: `mvn install` .
4. **CI/CD INTEGRATION:** Jenkins, GitHub Actions, GitLab CI all have native Maven support — making automated builds trivial.

Q2: Scenario: What is the difference between Maven and Gradle? When would you choose one over the other? Maven: → XML-based configuration (pom.xml) — verbose but very explicit and structured → Convention-driven — strict standard lifecycle (clean, compile, test, package...) → Mature, huge community, extremely stable → Slower builds (no build caching by default in older versions)

Gradle: → Groovy/Kotlin DSL — more concise, programmable → Flexible — you can customize the build logic extensively → Faster builds — incremental compilation, build cache, parallel execution → Used by Android development (official build tool)

Choose Maven when: Enterprise Java projects, team prefers explicit XML config, need maximum stability/maturity, working with legacy codebases. Choose Gradle when: Android projects (mandatory), need faster build times, want flexible custom build logic, modern microservices with complex build requirements.

---

2. Maven Project Structure & Archetypes

Maven enforces a **Standard Directory Layout** — every Maven project follows the exact same folder structure. This means tools, IDEs, and other developers can predict where to find source code, tests, and resources without any project-specific documentation.

2.1 Generating a Project with Archetypes

A **Maven Archetype** is a project template. Running the `archetype:generate` goal scaffolds a brand-new project with the correct folder structure and a starter pom.xml — eliminating manual setup.

BASH

```
# Interactive mode — choose archetype from a list
mvn archetype:generate

# Non-interactive — specify everything via flags (used in scripts/CI)
mvn archetype:generate \
  -DgroupId=com.apple \
  -DartifactId=java_project \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DinteractiveMode=false
```

2.2 Standard Folder Structure

```
my-app/
├── pom.xml                          ← Project Object Model (build config)
└── src/
    ├── main/
    │   ├── java/                    ← APPLICATION source code
    │   │   └── com/mycompany/app/
    │   │       └── App.java
    │   └── resources/               ← config files, 3rd-party files for app
    │       └── application.properties
    └── test/
        ├── java/                    ← UNIT TEST source code
        │   └── com/mycompany/app/
        │       └── AppTest.java
        └── resources/                ← 3rd-party files needed FOR TESTING

target/                             ← BUILD OUTPUT (auto-generated, never committed)
├── classes/                        ← compiled .class files
├── test-classes/                   ← compiled test .class files
├── my-app-1.0.jar                  ← final packaged artifact
└── surefire-reports/               ← test execution reports
```

FOLDER/FILE	PURPOSE
src/main/java	Application source code — your actual program logic
src/main/resources	Non-code files needed by the app: config, properties, XML, images
src/test/java	Unit test source code (JUnit/TestNG test classes)
src/test/resources	Files needed ONLY during testing (test data, mock configs)
target/	Auto-generated by Maven during build — NEVER manually edited, NEVER committed to Git
pom.xml	The single configuration file controlling the entire build

⚠️ **Important Warning:**

The `target/` folder is regenerated on every `mvn clean` — never store anything important there manually. Always add `target/` to `.gitignore` — it should never be committed to version control. Each Maven project requires EXACTLY ONE pom.xml — you cannot share a pom.xml between multiple unrelated projects.

🎯 Scenario-Based Interview Questions

Q1: Scenario: A teammate manually created folders for a new project but used 'source' instead of 'src/main/java'. Maven commands fail with 'no sources to compile'. Why, and how do you fix it? Maven follows STRICT convention over configuration — it expects source code EXACTLY at `src/main/java/` (by default). If folders don't match this convention, Maven cannot find the source code.

Fix Option 1 (recommended — follow convention):

```bash
mv source/* src/main/java/
rmdir source
mvn compile   # now works
```

Fix Option 2 (override convention in pom.xml — NOT recommended unless necessary):

```xml
<build>
  <sourceDirectory>source</sourceDirectory>
</build>
```

Best practice: ALWAYS use Maven's standard structure. Overriding it makes the project confusing for any new developer or IDE that expects standard Maven conventions.

Q2: Scenario: How do you generate a brand-new Maven project from the command line for a CI/CD pipeline (non-interactive, scriptable)? Use `archetype:generate` with `-DinteractiveMode=false` to avoid any prompts (critical for automation):

```bash
mvn archetype:generate \
  -DgroupId=com.company.myapp \
  -DartifactId=payment-service \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DarchetypeVersion=1.4 \
  -DinteractiveMode=false
```

This creates:

```text
payment-service/
├── pom.xml (with groupId=com.company.myapp, artifactId=payment-service)
└── src/main/java/com/company/myapp/App.java
```

Why `-DinteractiveMode=false` matters: Without it, Maven prompts for confirmation — which HANGS any CI/CD pipeline waiting for input that never comes. Always use this flag in scripts.

---

3. POM.xml — Project Object Model

POM stands for **Project Object Model**. The pom.xml file is the single most important file in any Maven project — it contains **complete metadata** about the project: its name, version, dependencies (libraries it needs), plugins (build behavior), and build configuration. Without a valid pom.xml in the project root, Maven refuses to execute ANY goal.

📝 **Theory & Key Points:**

- POM.xml is the 'recipe card' for your entire project.
- It answers: What is this project? What version? What libraries does it need? How should it be built?
- Maven reads pom.xml FIRST before doing anything else — without it, `mvn compile` fails immediately.
- Each project needs exactly ONE pom.xml. Multiple unrelated projects cannot share the same pom.xml.
- Extension is always .xml (Extensible Markup Language) — human-readable structured format.

3.1 Complete pom.xml Example

XML

```xml
<?xml version='1.0' encoding='UTF-8'?>
<project xmlns='http://maven.apache.org/POM/4.0.0'>
  <modelVersion>4.0.0</modelVersion>

  <!-- Project Coordinates — uniquely identifies this artifact -->
  <groupId>com.mycompany.app</groupId>       <!-- organization/company -->
  <artifactId>payment-service</artifactId>   <!-- project name -->
  <version>1.0.0</version>                   <!-- release version -->
  <packaging>jar</packaging>                 <!-- jar / war / ear / pom -->

  <!-- Project Info -->
  <name>Payment Service</name>
  <description>Handles payment processing for the e-commerce app</description>

  <!-- Properties — reusable values -->
  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <!-- DEPENDENCIES — third-party libraries this project needs -->
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
      <version>3.1.0</version>
    </dependency>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>          <!-- only needed during testing -->
    </dependency>
  </dependencies>
```

```
  <!-- PLUGINS — configure HOW the project is built -->
  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.11.0</version>
        <configuration>
          <source>17</source>
          <target>17</target>
        </configuration>
      </plugin>
      <plugin>
        <groupId>org.sonarsource.scanner.maven</groupId>
        <artifactId>sonar-maven-plugin</artifactId>
        <version>3.10.0.2594</version>
      </plugin>
    </plugins>
  </build>
</project>
```

3.2 The Two Key Sections — Dependencies vs Plugins

ASPECT	DEPENDENCIES	PLUGINS
Purpose	Libraries your CODE needs to compile/run	Tools that control the BUILD PROCESS
Example	Spring, JUnit, Log4j, Jackson	compiler plugin, surefire (test), sonar-scanner
Effect	Added to classpath — code can 'import' them	Executes actions during build phases
Source	Downloaded from Maven repositories	Also downloaded from repositories, but RUN as tools
Analogy	Ingredients you cook WITH	Kitchen appliances that DO the cooking

3.3 Maven Coordinates — groupId, artifactId, version (GAV)

Every artifact in a Maven repository is uniquely identified by three coordinates, commonly called **GAV**:

COORDINATE	MEANING
groupId	Identifies the organization/company. Convention: reverse domain name. `com.mycompany.app`
artifactId	The name of THIS specific project/module. `payment-service`, `user-api`
version	The release version. `1.0.0`, `2.3.1-SNAPSHOT` (SNAPSHOT = in-development, unstable)

📝 **Theory & Key Points:**

- GAV coordinates uniquely identify any artifact in any Maven repository — like a postal address.
- SNAPSHOT versions (1.0-SNAPSHOT) indicate active development — these get overwritten/updated frequently.
- Release versions (1.0.0) are immutable once published — never change after release.
- `<scope>test</scope>` means the dependency is ONLY available during testing, not packaged into the final JAR.
- `<scope>provided</scope>` means the dependency is available at compile time but provided by the runtime (e.g., servlet-api in a web server).
- Transitive dependencies: if A depends on B, and B depends on C, your project automatically gets C too.

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your build fails with 'package org.springframework does not exist' even though you wrote the import correctly. What's the likely cause? This means the Spring dependency is MISSING from pom.xml, or has the wrong scope.

Diagnosis:

1. Check pom.xml: is spring-boot-starter present under `<dependencies>`?
2. Check scope: if scope is 'test' but you're using it in main code, it won't be available
3. Check Maven offline mode: `mvn compile -o` (offline) fails if dependency isn't already cached locally

Fix:

```xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <version>3.1.0</version>
  <!-- NO scope = compile scope (default) = available everywhere -->
</dependency>
```

Then force re-download:

```bash
mvn clean install -U   # (-U forces update of dependencies, bypassing local cache)
```

Verify the dependency downloaded: `ls ~/.m2/repository/org/springframework/boot/`

Q2: Scenario: Your team has 5 microservices, each with nearly identical dependency versions defined in their own pom.xml. A security vulnerability is found in one shared library. Updating 5 files individually is risky. How do you centralize this? Use a PARENT POM with `dependencyManagement`:

```xml
<!-- parent/pom.xml -->
<packaging>pom</packaging>
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.15.2</version>   <!-- centralized version -->
    </dependency>
  </dependencies>
</dependencyManagement>
```

```xml
<!-- each microservice's pom.xml -->
<parent>
  <groupId>com.company</groupId>
  <artifactId>parent</artifactId>
  <version>1.0.0</version>
</parent>
<dependencies>
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <!-- NO version needed — inherited from parent! -->
  </dependency>
</dependencies>
```

Now: update version ONCE in parent pom.xml → all 5 services inherit the fix after rebuild.

---

4. Maven Build Lifecycle

Maven's build process is organized into **Lifecycles**, each composed of sequential **Phases**. The **default lifecycle** is the one used for building and deploying a project. Each phase MUST complete

successfully before the next one runs. Running a later phase automatically runs all earlier phases first.

Fig: "Maven Build Lifecycle" diagram — a vertical flow of connected boxes reading top to bottom: clean ("remove target/ folder") → validate ("check project is correct") → compile (".java to .class in target/") → test ("runs unit tests (JUnit)") → package ("builds JAR / WAR / EAR") → verify ("checks integration tests pass") → install ("copies to local repo (.m2)"), with install also branching right via an arrow to deploy ("push to remote repo"). Caption: "Fig 1: Maven Build Lifecycle — Sequential Phases from clean to deploy".

4.1 The Default Lifecycle Phases — Explained in Depth

PHASE	WHAT HAPPENS
clean	Deletes the `target/` directory — removes all previously compiled classes and packaged artifacts. Fresh start.
validate	Verifies the project is correct and all necessary information (pom.xml structure) is available and valid.
compile	Compiles the application source code. Converts .java files in `src/main/java` into .class files in `target/classes`.
test	Runs unit tests using a testing framework (JUnit/TestNG). Tests run against COMPILED code — no packaging needed yet.
package	Takes compiled code and packages it into a distributable format: JAR (library/backend), WAR (web app), or EAR.
verify	Runs any checks on the results of integration tests to ensure quality criteria are met before installing.
install	Installs the package into the LOCAL repository (`~/.m2`). Makes it available for OTHER local projects to use as a dependency.
deploy	Copies the final package to a REMOTE repository (Nexus, Artifactory, or Maven Central) — shares with other developers/teams.

4.2 Lifecycle Execution Order — Critical Rule

📝 **Theory & Key Points:**

- Running ANY phase automatically executes ALL preceding phases first.
- `mvn package` → automatically runs: validate → compile → test → package
- `mvn install` → automatically runs: validate → compile → test → package → verify → install
- `mvn test` → automatically runs: validate → compile → test
- You CANNOT skip earlier phases — Maven enforces this sequential dependency.
- If 'test' fails, the build STOPS — 'package' will NOT run (unless `-DskipTests` is used).

BASH

```
# Common Maven commands
mvn clean                                  # delete target/ folder
mvn compile                                # compile source code only
mvn test                                   # compile + run unit tests
mvn package                                # compile + test + create JAR/WAR
mvn install                                # full build + install to local .m2 repo
mvn deploy                                 # full build + push to remote repository

mvn clean install                          # most common — fresh full build +
install
mvn clean package -DskipTests              # build without running tests (use
cautiously)
mvn clean install -X                       # debug mode — verbose output
mvn clean install -o                       # offline mode — use only cached
dependencies
mvn dependency:tree                        # show full dependency tree (including
transitive)
mvn versions:display-dependency-updates    # check for newer dependency versions
```

4.3 Other Maven Lifecycles

LIFECYCLE	PURPOSE
default	The main build lifecycle: validate → compile → test → package → verify → install → deploy
clean	Cleans up build artifacts: pre-clean → clean → post-clean
site	Generates project documentation site: pre-site → site → post-site → site-deploy

🎯 Scenario-Based Interview Questions

Q1: Scenario: A teammate says 'I'll just run mvn package, no need to run mvn test first.' Is this correct? YES — this is correct, but for a subtle reason most developers misunderstand.

Maven lifecycles are SEQUENTIAL — running a LATER phase automatically executes ALL EARLIER phases first. `mvn package` executes in order: validate → compile → test → package

So 'mvn package' ALREADY includes the 'test' phase. You don't need to run 'mvn test' separately — it happens automatically as part of reaching 'package'.

IMPORTANT: If tests FAIL during this automatic test phase, the build STOPS — package will NOT be created. This is why you sometimes see teams use: `mvn package -DskipTests` (skip tests — fast, but risky — only for quick local iteration, NEVER in CI/CD for production)

Q2: Scenario: Your CI/CD pipeline runs 'mvn clean install' but it's installing snapshot artifacts to the LOCAL repo only — other team members can't access them. How do you make builds shareable across the team? `mvn install` only installs to YOUR LOCAL `~/.m2` repository — it is NOT shared with anyone else.

To share artifacts across the team, use `mvn deploy` to a REMOTE repository:

1. Set up a remote repository (Nexus or JFrog Artifactory)
2. Configure pom.xml with distribution management:

```xml
<distributionManagement>
  <repository>
    <id>company-releases</id>
    <url>https://nexus.company.com/repository/releases/</url>
  </repository>
  <snapshotRepository>
    <id>company-snapshots</id>
    <url>https://nexus.company.com/repository/snapshots/</url>
  </snapshotRepository>
</distributionManagement>
```

3. Configure credentials in `~/.m2/settings.xml` (server id, username, password)
4. Run: `mvn clean deploy`

Now ANY team member can declare this artifact as a dependency in their own pom.xml, and Maven fetches it from the shared Nexus repository.

Q3: Scenario: A build that worked yesterday fails today with no code changes, showing 'Could not resolve dependencies'. What happened? Common causes:

1. **SNAPSHOT dependency changed:** If you depend on a -SNAPSHOT version of another internal artifact, and a teammate pushed a breaking change to that SNAPSHOT, your build now fails even though YOUR code didn't change. Fix: Pin to a specific release version instead of SNAPSHOT for stability.
2. **Remote repository down:** Maven Central or your internal Nexus might be temporarily unreachable. Fix: `mvn clean install -o` (offline mode, uses local .m2 cache if dependency was already downloaded before)
3. **Local .m2 cache corrupted:** A partially-downloaded JAR can corrupt the cache. Fix: `rm -rf ~/.m2/repository/com/problematic/package/` then `mvn clean install -U` (force re-download)
4. **Network/proxy/firewall change:** Corporate network changes can block Maven Central access. Fix: configure proxy in `~/.m2/settings.xml`

---

5. Maven Repositories

Maven uses a **repository system** to store and retrieve project dependencies (libraries) and plugins. Understanding the three types of repositories — Local, Central (Remote), and Private/Internal — is essential for managing dependencies efficiently in both individual and enterprise environments.

REPOSITORY TYPE	DESCRIPTION
Local Repository (.m2)	Located at `~/.m2/repository` on YOUR machine. Caches every dependency ever downloaded. Maven checks here FIRST before going to remote.
Central Repository	The default public remote repository: `repo.maven.apache.org` / `mvnrepository.com`. Contains millions of open-source libraries.
Remote/Private Repository	Company-hosted repository (Nexus, JFrog Artifactory) for internal/proprietary artifacts not meant for public access.

🏗️ **Architecture:**

Dependency Resolution Order:

1. Maven checks LOCAL repository (`~/.m2/repository`) first — if found, use it (fast, no network needed)
2. If not found locally, check configured REMOTE repositories (Central, or company Nexus)
3. Download from remote → cache it in LOCAL repository → use it
4. Next build: dependency is now in LOCAL repo — no network call needed

This caching is why the FIRST build after `mvn clean` on a new dependency takes longer, but SUBSEQUENT builds are much faster (everything is cached locally).

BASH

```
# Location of local repository (default)
~/.m2/repository/

# Search remote repository for a library:
# https://mvnrepository.com/artifact/org.springframework/spring-core

# View dependency tree (shows ALL transitive dependencies)
mvn dependency:tree

# Force re-download (bypass local cache):
mvn clean install -U

# Clear local repository cache for a specific artifact:
rm -rf ~/.m2/repository/com/company/myapp
```

🎯 **Scenario-Based Interview Questions**

Q1: Scenario: Your company is concerned about using public Maven Central directly — security and reliability risks. How do you set up a controlled, internal dependency source? Set up Nexus Repository Manager (or JFrog Artifactory) as a PROXY + PRIVATE repository:

1. Install Nexus on an internal server
2. Configure a 'proxy' repository that mirrors Maven Central — all public dependency requests go through Nexus, which caches them
3. Configure a 'hosted' repository for YOUR company's internal/proprietary artifacts
4. Update every developer's `~/.m2/settings.xml` to point to Nexus as the mirror:

```xml
<mirror>
  <id>company-nexus</id>
  <mirrorOf>*</mirrorOf>
  <url>https://nexus.company.com/repository/maven-public/</url>
</mirror>
```

Benefits: → All dependency downloads go through Nexus (auditable, controllable) → Nexus caches everything — faster builds, works even if Maven Central is down → Internal artifacts stay completely private → Can scan for vulnerable dependencies before they're cached

---

6. Maven Artifact Types — JAR, WAR, EAR

When Maven packages your project, it produces an **artifact** — the final, distributable output. The type of artifact depends on the application's purpose, configured via the `<packaging>` tag in pom.xml.

ARTIFACT TYPE	FULL NAME	CONTAINS / USE CASE
JAR	Java ARchive	Backend code only — libraries, standalone applications, microservices
WAR	Web ARchive	Frontend + Backend — full web applications deployed to Tomcat/web servers
EAR	Enterprise ARchive	Multiple JARs + WARs bundled — large enterprise apps deployed to app servers (WebLogic, WebSphere)

📝 **Theory & Key Points:**

- JAR: Used for libraries (other projects depend on them) and standalone apps (`java -jar myapp.jar`).
- WAR: Deployed INTO a servlet container (Tomcat, Jetty) which provides the HTTP server functionality.
- EAR: Used in legacy enterprise Java EE applications — bundles multiple WARs/JARs with shared resources.
- Most modern microservices use JAR with an embedded server (Spring Boot's embedded Tomcat) instead of WAR.

6.1 Build Tools Across Languages

Maven is Java's build tool, but every language ecosystem has its equivalent:

LANGUAGE	BUILD TOOL(S)
Java	Maven, Gradle — dependency management, compile, package, test
Node.js	NPM, Yarn — package.json defines dependencies
Python	pip + setuptools, Poetry — requirements.txt or pyproject.toml
.NET	MSBuild, dotnet CLI — .csproj files
C / C++	Make, CMake — Makefile defines build rules
Ruby	Bundler, Rake — Gemfile defines dependencies
Go	go mod — go.mod defines dependencies

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your team built a simple REST API as a JAR with Spring Boot's embedded server, but a legacy enterprise client insists on deploying it as a WAR into their existing Tomcat cluster. How do you support both? Make the packaging configurable using Maven profiles or change packaging:

```java
// To convert to WAR, extend SpringBootServletInitializer:
@SpringBootApplication
public class MyApp extends SpringBootServletInitializer {
  @Override
  protected SpringApplicationBuilder configure(SpringApplicationBuilder builder)
{
    return builder.sources(MyApp.class);
  }
}
```

```xml
<!-- In pom.xml, change packaging: -->
<packaging>war</packaging>

<!-- Add provided-scope dependency for embedded Tomcat (since external Tomcat
will provide it): -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-tomcat</artifactId>
  <scope>provided</scope>
</dependency>
```

Result: `mvn package` now produces a WAR that can be deployed to the client's existing Tomcat, while you can still run it standalone with embedded Tomcat for local development.

13. Quick Command Reference — Maven Commands

COMMAND	DESCRIPTION
`mvn clean`	Delete target/ folder
`mvn compile`	Compile source code
`mvn test`	Compile + run unit tests
`mvn package`	Compile + test + create JAR/WAR
`mvn install`	Full build + install to local .m2
`mvn deploy`	Full build + push to remote repo
`mvn clean install -DskipTests`	Build without running tests
`mvn dependency:tree`	Show full dependency tree
`mvn archetype:generate`	Generate new project from template
`mvn sonar:sonar`	Run SonarQube code analysis

☁ MultiCloud DevOps — Maven & SonarQube Complete Notes — by Veera Sir

Build Lifecycle + POM + Quality Gates + Jenkins Integration + Scenario-Based Interview Q&A — Version 1.0

SQ

PART 04 OF 08

SonarQube

Static code analysis, quality gates, and security scanning.

Source: *Maven_SonarQube_Final_Notes.pdf* (MultiCloud DevOps by Veera Sir — "Maven & SonarQube: Build Automation & Code Quality — Complete Study Notes"), Part B, pages 20–38.

7. What is SonarQube?

SonarQube is a **web-based, open-source platform for continuous code quality inspection**. It performs automated **static code analysis** — examining source code WITHOUT executing it — to detect bugs, code smells, security vulnerabilities, and code duplication across **25+ programming languages** including Java, JavaScript, C#, Python, Kotlin, and Scala. Other tools in this category include Veracode and Coverity, but SonarQube is the most widely adopted in the open-source/enterprise DevOps ecosystem.

💡 **Layman Explanation:**

SonarQube is like a strict English teacher grading an essay before it's submitted. It doesn't run your program — it READS your code and flags problems: typos (bugs), confusing writing (code smells), and dangerous statements (vulnerabilities). Quality Gates are the 'pass/fail' grade — if the essay doesn't meet the bar, it's rejected before submission (deployment). This catches problems BEFORE code reaches production, not after a customer reports a bug.

7.1 Key Terminology

🔍 **SonarQube Concept:**

Code Smell: A maintainability issue — code that WORKS but is confusing, poorly structured, or hard to maintain. Example: a method with 15 parameters, deeply nested if-statements, duplicate code blocks.

Bug: An actual coding error that WILL cause incorrect behavior at runtime. Example: null pointer dereference, infinite loop, off-by-one array access.

Vulnerability: A security flaw that could be exploited by an attacker. Example: SQL injection risk, hardcoded passwords, weak cryptography, XSS vulnerability.

Code Coverage: Percentage of code executed by automated tests. Low coverage = high risk of undetected bugs.

Technical Debt: The estimated effort needed to fix all code smells — 'cost' of maintaining messy code.

7.2 Benefits of SonarQube

BENEFIT	EXPLANATION
Improve Quality	Catches bugs and code smells before they reach production — shift-left testing philosophy.
Grow Developer Skills	Developers learn best practices by seeing WHY their code is flagged — continuous education.
Continuous Quality Management	Every commit/PR is automatically scanned — quality is monitored constantly, not just at release time.
Reduce Risk (Vulnerabilities)	Identifies security flaws (SQL injection, XSS, hardcoded secrets) before attackers can exploit them.
Scale with Ease	Works across 25+ languages and integrates with any CI/CD tool — one platform for all projects.

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your application passed all unit tests but crashed in production with a NullPointerException that SonarQube had flagged 2 weeks earlier as a 'Bug'. Why wasn't this caught before deployment? Root cause: SonarQube findings were likely IGNORED or there was NO Quality Gate enforcing failure on bugs.

SonarQube identifies issues but does NOT automatically block deployment unless:

1. A Quality Gate is configured with a condition like 'New Bugs > 0 → FAIL'
2. The CI/CD pipeline actually CHECKS the quality gate result (`waitForQualityGate()` in Jenkins) and STOPS the pipeline if it fails

Fix going forward:

1. Configure Quality Gate: Reliability Rating must be A, 0 new bugs allowed
2. In Jenkinsfile, add quality gate enforcement:

```groovy
stage('Quality Gate') {
  steps {
    script {
      def qg = waitForQualityGate()
      if (qg.status != 'OK') {
        error 'Pipeline failed: Quality Gate not passed'
      }
    }
  }
}
```

3. This BLOCKS deployment until the bug is fixed — preventing this exact scenario in future.

Q2: Scenario: SonarQube flags 500 'code smells' on a legacy codebase that has been running fine in production for 5 years. The team is overwhelmed. How do you prioritize? Don't try to fix everything at once. Prioritize using SonarQube's severity classification:

1. **BLOCKER/CRITICAL first:** These represent genuine bugs or security vulnerabilities — fix immediately
2. **Set Quality Gate to apply ONLY to NEW code:** 'New Code' Quality Gates only flag issues in code changed since a baseline date — doesn't punish for legacy debt Administration → Quality Gates → 'Sonar way' (default) already uses this approach

3. **Create a technical debt backlog:** track MAJOR/MINOR smells as backlog items, address gradually during refactoring sprints
4. **Don't block releases on legacy debt:** focus Quality Gate enforcement on NEW commits only, not the entire codebase history

This 'new code' strategy is the industry-standard approach — it stops the bleeding (prevents NEW bad code) without requiring an unrealistic big-bang fix of all historical debt.

---

8. SonarQube Architecture

SonarQube's architecture consists of three core components working together: the **Scanner** (which analyzes code), the **Server** (which applies rules and stores results), and the **Database** (which persists analysis history). Understanding this architecture clarifies how code flows from your repository into actionable quality reports.

Fig: "SonarQube Architecture" diagram — a left-to-right flow. "Source Code (Java, Python, etc.)" box arrows into "Sonar Scanner (scans + sends results)", which arrows into a large "SonarQube Server (port 9000)" container. Inside that container, top to bottom: "Rules Engine (applies quality rules + gates)" arrows down into "Database (H2 (default) or PostgreSQL)" arrows down into "Web Dashboard (Quality Gates)". Below the diagram: "Default login: admin / admin (change after first login)". Caption: "Fig 2: SonarQube Architecture — Scanner → Server → Database → Dashboard".

8.1 The Three Components

COMPONENT	ROLE
SonarQube Server	The central application. Hosts the web dashboard (port 9000), applies quality rules, manages quality gates, stores configuration.
Rules Engine	Applies language-specific static analysis rules to detect bugs, smells, vulnerabilities. 25+ language analyzers built in.
Database	Stores all analysis history, issues, metrics. Default: embedded H2 (testing only). Production: PostgreSQL (recommended).
Scanner	Runs on the developer machine or CI server. Gathers rules from the server, scans source code, sends results back to the server.

⚠️ **Important Warning:**

H2 database (the default) is ONLY suitable for evaluation/testing — NOT for production use. Production SonarQube MUST use PostgreSQL — H2 cannot handle concurrent access or large analysis history reliably. SonarQube requires minimum 4GB RAM and 2 CPUs — undersized servers will crash or hang during analysis.

🎯 **Scenario-Based Interview Questions**

Q1: Scenario: Your SonarQube server has been running for 8 months using the default H2 database. It just crashed and all historical quality data is lost. What went wrong and how do you prevent it? Root cause: H2 is an EMBEDDED, file-based database meant only for EVALUATION purposes. It cannot handle: → Concurrent scans from multiple projects/branches → Large volumes of historical analysis data over time → Production-level reliability (no replication, no backup tooling)

Fix — migrate to PostgreSQL:

1. Install PostgreSQL on a separate server (or RDS if on AWS)
2. Create database: `CREATE DATABASE sonarqube;`
3. Configure sonar.properties:

```
sonar.jdbc.username=sonar
sonar.jdbc.password=<secure-password>
sonar.jdbc.url=jdbc:postgresql://db-host:5432/sonarqube
```

4. Restart SonarQube — it will initialize fresh schema in PostgreSQL
5. NOTE: There is no automated migration FROM H2 — historical data is lost in this transition. This must be planned BEFORE going to production, not after.

Lesson: NEVER run SonarQube in production with the default H2 database.

9. SonarQube Installation

SonarQube requires specific minimum resources: **4GB RAM, 2 CPUs**, and **Java 11 or 17**. On AWS, the recommended instance type is **t2.medium** or larger. SonarQube runs on **port 9000** by default.

9.1 Manual Installation on EC2

BASH

```
# Launch t2.medium EC2 instance, open port 9000 in security group

# Install Java (Amazon Linux 2)
sudo amazon-linux-extras install java-openjdk11 -y

# Install Java (Amazon Linux 2023)
sudo dnf install java-11-amazon-corretto -y

# Switch to /opt directory
cd /opt/

# Download SonarQube
sudo wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-
8.9.6.50800.zip

# Unzip
sudo unzip sonarqube-8.9.6.50800.zip

# SonarQube MUST run as a non-root user — create dedicated 'sonar' user
sudo useradd sonar
sudo chown sonar:sonar sonarqube-8.9.6.50800 -R
sudo chmod 777 sonarqube-8.9.6.50800 -R

# Set password and login as sonar user
passwd sonar
su - sonar

# Start SonarQube (run as sonar user)
sh /opt/sonarqube-8.9.6.50800/bin/linux-x86-64/sonar.sh start

# Check status
sh sonar.sh status

# Access via browser:
# http://<public-ip>:9000
```

```
# Default login: admin / admin
# IMPORTANT: Change the default password immediately after first login!
```

⚠️ **Important Warning:**

Why run as 'sonar' user instead of root? SECURITY: If a vulnerability is exploited, the attacker only gets the limited permissions of the 'sonar' user — NOT full root access to modify the entire system. SEPARATION OF CONCERNS: A dedicated user separates SonarQube's processes and files from other applications — easier to manage, secure, and audit independently. ALWAYS change the default admin/admin password immediately — leaving default credentials is a critical security risk.

9.2 SonarQube via Docker (Faster Setup)

BASH

```
# Single command — runs SonarQube in a container
docker run -d --name sonar -p 9000:9000 sonarqube:lts-community

# Check container status
docker ps
docker logs -f sonar    # watch startup logs

# Access at http://<host-ip>:9000
```

YAML

```yaml
# Production Docker Compose with PostgreSQL:
# docker-compose.yml
version: '3'
services:
  sonarqube:
    image: sonarqube:lts-community
    ports:
      - '9000:9000'
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://db:5432/sonar
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonarpass
    depends_on:
      - db
  db:
    image: postgres:13
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonarpass
      POSTGRES_DB: sonar
```

🎯 Scenario-Based Interview Questions

Q1: Scenario: After installing SonarQube manually with the steps above, 'sonar.sh start' fails with 'max virtual memory areas vm.max_map_count too low'. How do you fix it? This is a well-known SonarQube + Elasticsearch (its search engine) requirement on Linux.

Fix:

```bash
sudo sysctl -w vm.max_map_count=262144
```

Make it permanent (survives reboot):

```bash
echo 'vm.max_map_count=262144' | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

Also check other common startup failures:

1. Insufficient RAM: SonarQube needs minimum 4GB — check with `free -h`
2. Wrong Java version: SonarQube 8.9 needs Java 11; SonarQube 10+ needs Java 17 — `java -version` to verify
3. Port 9000 already in use: `sudo netstat -tlnp | grep 9000`
4. Running as root: SonarQube REFUSES to start as root — must run as 'sonar' user

After fixing, retry: `su - sonar && sh /opt/sonarqube*/bin/linux-x86-64/sonar.sh start`

Q2: Scenario: Compare manual EC2 installation vs Docker installation for SonarQube. When would you choose each in a real project? Manual EC2 Installation: ✓ Full control over the OS, Java version, file system ✓ Better for understanding internals during learning ✓ No Docker dependency on the host ✗ More setup steps, more error-prone (vm.max_map_count, user permissions) ✗ Harder to reproduce identical setups across environments

Docker Installation: ✓ Single command — running in seconds ✓ Consistent, reproducible across any environment ✓ Easy to combine with PostgreSQL via docker-compose ✓ Easy upgrades: `docker pull sonarqube:lts-community && docker-compose up -d` ✗ Requires understanding Docker volumes for data persistence ✗ Slightly more resource overhead from containerization

Recommendation: For production, use Docker + docker-compose with PostgreSQL — easier to maintain, version, and migrate. For learning/understanding internals, manual installation teaches more about how SonarQube actually works.

10. SonarQube Integration with Jenkins

Integrating SonarQube into a Jenkins CI/CD pipeline enables **automated code quality gates** — every code commit is automatically scanned, and the pipeline can be configured to **FAIL the build** if quality standards aren't met. This is the cornerstone of **'Shift-Left' testing** — catching quality and security issues as early as possible in the development cycle.

Fig: "Jenkins + Maven + SonarQube CI Pipeline" diagram — a vertical flow: "Git Checkout (SCM stage)" → "mvn clean" → "mvn sonar:sonar (runs code quality scan)" → "Quality Gate Check (waitForQualityGate())", which then branches two ways: labeled "OK" to "Deploy Stage", and labeled "failed" to "Pipeline Fails". Caption: "Fig 3: Jenkins + Maven + SonarQube CI Pipeline — Quality Gate Pass/Fail Flow".

10.1 Setup Steps

BASH

```
# Step 1: Generate a token in SonarQube
# Login to SonarQube → Administration → Security → Users → Create Token

# Step 2: Install required Jenkins plugins
# Manage Jenkins → Plugins → 'SonarQube Scanner for Jenkins'
# Optional: 'Quality Gates' plugin

# Step 3: Add SonarQube token as Jenkins credential
# Manage Jenkins → Credentials → Add Credential
# Kind: Secret text → paste token → ID: 'sonar-token'

# Step 4: Configure SonarQube server in Jenkins
# Manage Jenkins → System → SonarQube servers
# Name: 'sonar-server' (this exact name is used in withSonarQubeEnv())
# Server URL: http://<sonarqube-ip>:9000
# Server authentication token: select the credential created above
```

10.2 Complete Jenkinsfile with Quality Gate Enforcement

GROOVY

```
pipeline {
    agent any

    environment {
        SONARQUBE = 'sonar-server'  // matches the name configured in Jenkins
    }

    stages {
        stage('SCM') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/nareshdevopscloud/project-1-maven-
jenkins.git'
            }
        }

        stage('Clean') {
            steps {
                sh 'mvn clean'
            }
        }

        stage('Code Quality') {
            steps {
                // withSonarQubeEnv injects SONAR_HOST_URL and SONAR_AUTH_TOKEN
                withSonarQubeEnv(SONARQUBE) {
                    sh 'mvn sonar:sonar'
                }
            }
        }

        stage('Quality Gate') {
            steps {
                script {
                    // Waits for SonarQube server to compute Quality Gate result
                    def qg = waitForQualityGate()
```

```
                    echo "Quality Gate Response: ${qg}"

                    if (qg.status != 'OK') {
                        error '❌ Pipeline failed: Quality Gate requirements not
met.'
                    } else {
                        echo '✅ Quality Gate Passed!'
                    }
                }
            }
        }
    }

    post {
        success {
            echo '🎉 Build + SonarQube analysis completed with passing Quality Gate.'
        }
        failure {
            echo '❌ Build or SonarQube Quality Gate validation failed.'
        }
    }
}
```

📝 **Theory & Key Points:**

- `withSonarQubeEnv('sonar-server')` injects `SONAR_HOST_URL` and `SONAR_AUTH_TOKEN` as environment variables automatically — no hardcoded URLs/tokens in the Jenkinsfile.
- `waitForQualityGate()` PAUSES the pipeline until SonarQube finishes processing and returns a pass/fail status — this requires a webhook configured in SonarQube pointing back to Jenkins.
- Without the webhook, `waitForQualityGate()` will TIMEOUT waiting indefinitely for a response.
- Configure webhook: SonarQube → Administration → Configuration → Webhooks → Add: `http://<jenkins-url>/sonarqube-webhook/`

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your Jenkinsfile calls waitForQualityGate() but the pipeline hangs forever at that stage, never returning a result. What's misconfigured? Root cause: Missing SonarQube → Jenkins WEBHOOK configuration.

How `waitForQualityGate()` actually works:

1. `mvn sonar:sonar` sends analysis results to SonarQube server
2. SonarQube processes results ASYNCHRONOUSLY (takes a few seconds)
3. When done, SonarQube must NOTIFY Jenkins via a WEBHOOK callback
4. `waitForQualityGate()` is literally WAITING for this webhook notification

If the webhook isn't configured, SonarQube never tells Jenkins it's done — Jenkins waits forever (or until pipeline timeout).

Fix:

1. Login to SonarQube → Administration → Configuration → Webhooks
2. Add webhook: Name: Jenkins URL: `http://<jenkins-server>:8080/sonarqube-webhook/`
3. Save
4. Re-run the pipeline — `waitForQualityGate()` will now receive the callback and return promptly

Verify: Check SonarQube webhook delivery logs to confirm Jenkins received the POST request.

Q2: Scenario: Your Jenkinsfile previously had hardcoded SonarQube URL and token directly in 'sh mvn sonar:sonar -Dsonar.host.url=... -Dsonar.login=...'. Security flagged this as a risk. How do you fix it using best practices? Replace hardcoded values with `withSonarQubeEnv()` — the secure, recommended approach:

BAD (hardcoded — security risk, also breaks if URL changes):

```groovy
sh 'mvn sonar:sonar -Dsonar.host.url=http://43.204.112.71:9000 -
Dsonar.login=squ_abc123token'
```

GOOD (using Jenkins-managed credentials):

```groovy
withSonarQubeEnv('sonar-server') {
    sh 'mvn sonar:sonar'
    // SONAR_HOST_URL and SONAR_AUTH_TOKEN are injected automatically
    // No token visible in Jenkinsfile or console logs
}
```

Setup required (one-time):

1. Store token as Jenkins Secret Text credential
2. Configure SonarQube server in Manage Jenkins → System with name 'sonar-server'
3. Reference that name in `withSonarQubeEnv()`

Benefit: Token is encrypted in Jenkins credential store, never appears in pipeline logs, and can be rotated without touching any Jenkinsfile.

11. Quality Gates — Enforcing Standards

A **Quality Gate** is a set of conditions that a project's code must meet before it can be considered 'passing'. If ANY condition fails, the entire Quality Gate fails — and (when integrated with CI/CD) the **pipeline can be configured to stop the deployment**. Quality Gates turn subjective code review opinions into **objective, automated, enforceable standards**.

11.1 Worked Example: Calculating Code Duplication

Consider this sample Java code with intentional duplication:

JAVA

```java
package com.example;

public class App {
    private static final String MESSAGE = "Hello World!";

    public App() {}

    public static void main(String[] args) {
        System.out.println(MESSAGE);
        System.out.println(MESSAGE); // Duplicate line
    }

    public String getMessage() {
        return MESSAGE;
    }

    public String getMessage() {  // Duplicate method (won't even compile,
        return MESSAGE;           // but illustrates the duplication concept)
    }
}
```

Duplication Calculation:

📦 **Maven Concept:**

Total lines of code (LOC): 17 (including comments and blank lines) Duplicated lines: 5 → 2 lines: `System.out.println(MESSAGE);` printed twice → 2 `getMessage()` methods + their bodies counted as duplicate block

Duplication Percentage = (Duplicated Lines / Total Lines) × 100 = (5 / 17) × 100 = **29.41%**

This is FAR above any reasonable threshold (typically 3-5%) — Quality Gate should FAIL.

11.2 Configuring a Quality Gate in SonarQube

BASH

```
# Step-by-step Quality Gate configuration:

# 1. Login to SonarQube console (http://server:9000)

# 2. Navigate to: Quality Gates (top menu)

# 3. Click 'Create' to make a new gate, or edit existing

# 4. Add Condition:
#    Click 'Add Condition'
#    Metric: Duplicated Lines (%)
#    Operator: is greater than
#    Value: 1   (fails if duplication exceeds 1%)

# 5. Save the Quality Gate configuration

# 6. Assign this Quality Gate to your project (or set as default)
```

11.3 Common Quality Gate Conditions (Real-World)

METRIC	CONDITION	EFFECT
Coverage	is less than 80%	→ FAIL — ensures sufficient test coverage on new code
Duplicated Lines (%)	is greater than 3%	→ FAIL — prevents copy-paste code
Maintainability Rating	is worse than A	→ FAIL — limits technical debt accumulation
Reliability Rating	is worse than A	→ FAIL — blocks code with bugs
Security Rating	is worse than A	→ FAIL — blocks code with vulnerabilities
New Bugs	is greater than 0	→ FAIL — zero tolerance for new bugs in changed code
New Vulnerabilities	is greater than 0	→ FAIL — zero tolerance for new security flaws
New Security Hotspots Reviewed	is less than 100%	→ FAIL — every flagged hotspot must be reviewed

BASH

```
# Running analysis with Maven (after Quality Gate is configured):
mvn clean verify sonar:sonar \
  -Dsonar.projectKey=sample-java-project \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.login=<your-sonar-token>

# Check result on dashboard:
# SonarQube → Your Project → Quality Gate status (Passed/Failed badge)
```

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your default Quality Gate ('Sonar way') checks the ENTIRE codebase history, causing every legacy project to fail immediately upon first scan. How do you fix this without ignoring quality standards? SonarQube's built-in "Sonar way" Quality Gate by default focuses on "New Code" conditions — it should NOT fail on entire legacy history unless misconfigured.

Verify/Fix:

1. Check Quality Gate conditions: Administration → Quality Gates → "Sonar way"
2. Ensure conditions use "On New Code" scope, NOT "Overall Code": ✓ New Coverage is less than 80% (only checks code changed since baseline) ✗ Overall Coverage is less than 80% (checks ENTIRE codebase — unfair to legacy)
3. Set the "New Code" baseline period: Administration → New Code → "Previous version" or "Number of days: 30"

Result: Only code CHANGED in the current analysis is held to the standard. Old legacy code that was already there is not punished — but any NEW code added must meet quality standards. This is the standard "stop the bleeding" strategy for legacy codebases.

Q2: Scenario: A developer wants to bypass the Quality Gate just this once because of a 'critical production hotfix that can't wait.' How should this be handled organizationally and technically? This requires a BALANCE between speed and quality control — never simply disable the gate.

Organizational process:

1. Define an 'emergency hotfix' exception process requiring Tech Lead/Manager sign-off
2. Document WHY the bypass was needed and create a follow-up ticket to address the quality issue post-hotfix

Technical implementation options:

Option A — Separate pipeline for hotfixes with relaxed (not removed) gate:

```groovy
pipeline {
  parameters { booleanParam(name: 'SKIP_QUALITY_GATE', defaultValue: false) }
  stages {
    stage('Quality Gate') {
      when { expression { !params.SKIP_QUALITY_GATE } }
      steps { ... waitForQualityGate() ... }
    }
  }
}
```

Option B — Require manual approval to proceed despite gate failure:

```groovy
stage('Quality Gate Override') {
    when { expression { qg.status != 'OK' } }
    steps { input 'Quality Gate failed. Override and deploy anyway?' }
}
```

NEVER permanently disable the gate — use temporary, audited, approval-gated bypasses only.

12. Sonar Scanner — For Non-Maven Projects

While Maven projects use the **sonar-maven-plugin** (`mvn sonar:sonar`), projects in other languages (Python, JavaScript, Go, C#) use the standalone **SonarScanner CLI tool**. This is a command-line utility that can run on any developer's machine or any CI server, independent of any specific build tool.

12.1 Installing SonarScanner (Linux/Mac)

BASH

```
# Download SonarScanner CLI
wget https://binaries.sonarsource.com/Distribution/sonar-scanner-cli/sonar-scanner-cli-4.8.0.2856-linux.zip

# Unzip
unzip sonar-scanner-cli-4.8.0.2856-linux.zip

# Move to standard location
sudo mv sonar-scanner-4.8.0.2856-linux /opt/sonar-scanner

# Add to PATH — edit ~/.bashrc, ~/.zshrc, or ~/.profile
export PATH=$PATH:/opt/sonar-scanner/bin

# Reload shell config
source ~/.bashrc

# Verify installation
sonar-scanner --version
```

12.2 Running Analysis for Non-Java Projects

```bash
# Example: Scanning a Python project
sonar-scanner \
  -Dsonar.projectKey=python1 \
  -Dsonar.sources=. \
  -Dsonar.host.url=http://43.204.112.71:9000 \
  -Dsonar.login=<your-sonar-token>

# Common parameters:
# sonar.projectKey  = unique identifier for this project in SonarQube
# sonar.sources     = path to source code (. means current directory)
# sonar.host.url    = your SonarQube server URL
# sonar.login       = authentication token (NOT username/password)

# Optional: Use a sonar-project.properties file instead of CLI flags
# Create this file in project root:
sonar.projectKey=python1
sonar.sources=.
sonar.host.url=http://43.204.112.71:9000

# Then just run:
sonar-scanner
```

PROJECT TYPE	HOW TO SCAN	COMMAND
Java (Maven)	sonar-maven-plugin	`mvn sonar:sonar`
Java (Gradle)	sonarqube Gradle plugin	`gradle sonarqube`
Python/JS/Go/etc.	SonarScanner CLI	`sonar-scanner -Dsonar.projectKey=...`
.NET/C#	SonarScanner for MSBuild	`dotnet sonarscanner begin/end`

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your company has a polyglot tech stack — Java backend (Maven), Python data pipelines, and a React frontend (JavaScript). How do you set up SonarQube analysis for ALL of them consistently? Use the appropriate scanner method for each project type, but configure them consistently:

1. Java backend (Maven):

```bash
mvn sonar:sonar -Dsonar.projectKey=backend-api -Dsonar.host.url=http://sonar:9000
-Dsonar.login=$TOKEN
```

2. Python data pipelines (standalone SonarScanner):

```bash
sonar-scanner -Dsonar.projectKey=data-pipeline -Dsonar.sources=. -
Dsonar.host.url=http://sonar:9000 -Dsonar.login=$TOKEN
```

3. React frontend (standalone SonarScanner, JS/TS analyzer built-in):

```bash
sonar-scanner -Dsonar.projectKey=frontend-app -Dsonar.sources=src -
Dsonar.exclusions=**/node_modules/** -Dsonar.host.url=http://sonar:9000 -
Dsonar.login=$TOKEN
```

All three appear as SEPARATE projects in the same SonarQube dashboard, each with its own Quality Gate, but managed from a single central server — giving unified visibility across the entire polyglot stack.

13. Quick Command Reference — SonarQube Commands

COMMAND	DESCRIPTION
`sh sonar.sh start`	Start SonarQube server
`sh sonar.sh stop`	Stop SonarQube server
`sh sonar.sh status`	Check SonarQube server status
`sh sonar.sh restart`	Restart SonarQube server
`sonar-scanner --version`	Check SonarScanner CLI version
`sonar-scanner -Dsonar.projectKey=X`	Run analysis for non-Maven project
`docker run -d -p 9000:9000 sonarqube:lts-community`	Quick Docker setup

CI/CD

PART 05 OF 08

CI/CD

Jenkins, GitLab CI, and GitHub Actions — automated build, test & release pipelines.

📘 **Document Legend** (from original PDF): 💡 Blue = Layman · 📝 Green = Theory & Key Points · 🔴 Red = Jenkins Concept · 🚀 Orange = CI/CD Flow · 🎯 Yellow = Scenario Interview Q&A · ⚠️ Warning

1. What is CI/CD?

CI/CD stands for **Continuous Integration / Continuous Delivery (or Deployment)**. It is a modern software engineering practice that automates the process of building, testing, and deploying code changes. Rather than manually compiling, testing, and deploying code every few weeks, CI/CD pipelines do this automatically every time a developer pushes code — enabling teams to release software faster, more frequently, and with higher confidence.

Continuous Integration (CI): The practice of automatically building and testing code every time a developer commits changes to the shared repository. The goal is to detect integration errors (conflicts between developers' code) as early as possible — within minutes of the commit, not days or weeks later.

Continuous Delivery (CD): Extends CI by automatically preparing the tested build for release to any environment (staging, UAT, production). Every build that passes CI is a candidate for release — the deployment itself may require a manual approval step. The key is: the software is ALWAYS in a deployable state.

Continuous Deployment: Takes CD one step further — every change that passes all automated tests is automatically deployed to production with **no human intervention**. Companies like Netflix, Amazon, and Google use this to deploy hundreds of times per day. Requires extremely mature automated testing.

💡 **Layman Explanation:**

- Before CI/CD: Developers code for 2 weeks. Integration day is a nightmare — nothing works together.
- With CI/CD: Every commit is built, tested, and ready to deploy automatically within minutes.
- It's like a factory assembly line: raw materials (code) go in one end, finished product (deployed app) comes out the other.
- Catch bugs in minutes instead of discovering them 3 weeks later during 'integration hell'.

Fig 1: CI/CD Pipeline diagram — Source → Build → Test → Code Quality → Deployment → Production. A second embedded diagram in this figure shows the Jenkins role-based authorization setup process (Security → Authentication and User Management plugin install → Security config with Role-Based Strategy → creating users → managing/assigning roles → global roles matrix → item roles with pattern-based per-job permissions) — this content is covered in full later in Section 8.

🚀 **CI/CD Flow:**

CI Pipeline stages (automated, runs on every commit):

1. **Source** → Developer pushes to GitHub
2. **Build** → Maven/Gradle compiles code + packages (JAR/WAR/Docker image)
3. **Unit Test** → JUnit runs all automated tests
4. **Code Quality** → SonarQube checks code coverage, bugs, security vulnerabilities
5. **Artifact** → Package stored in Nexus/JFrog/S3

CD Pipeline stages (automated deployment): 6. **Deploy to Staging** → Automated deployment to staging environment 7. **Integration Tests** → API tests, smoke tests on staging 8. **Manual Approval** → (Optional) Human review for production 9. **Deploy to Prod** → Blue/green or rolling deployment

CI/CD Tooling Landscape

TOOL CATEGORY	TOOLS	PURPOSE
Build Tools	Maven, Gradle, Ant	Compile source code + package into deployable artifact
Testing	JUnit, TestNG, Selenium	Unit tests, integration tests, UI/browser tests
Code Quality	SonarQube, Checkstyle	Static analysis, coverage, security scanning
Artifact Storage	Nexus, JFrog Artifactory, S3	Store and version build artifacts (JAR, WAR, Docker images)
CI/CD Orchestration	Jenkins, GitHub Actions, GitLab CI, Azure DevOps	Automate and orchestrate the entire pipeline
Containerization	Docker, Podman	Package app + dependencies into portable containers
Deployment	Kubernetes, Ansible, Terraform	Deploy and manage applications in target environments

🎯 **Scenario-Based Interview Questions**

Q1: Scenario: Your company releases software once a month. Every release takes 3 days of manual testing and deployment. Management wants faster releases. How does CI/CD help?

Current state: 1 release/month, 3 days of manual work per release, bugs found late.

With CI/CD:

- Every commit triggers automatic build + test within 10 minutes
- Bugs caught immediately — not 3 weeks later
- Automated deployment to staging: zero manual steps
- Production deployment: push a button (or automatic after approval)
- Frequency increases from monthly to daily or weekly
- Each release is smaller, safer, and easier to roll back

Key metrics improvement:

- Lead time: 3 weeks → hours
- Deployment frequency: monthly → daily
- MTTR (Mean Time To Recovery): days → minutes

Q2: Scenario: What is the difference between Continuous Delivery and Continuous Deployment?

Continuous Delivery: Every successful CI build is READY to deploy to production but requires a MANUAL approval step. A human decides when to push the deploy button. → Use when: Regulated industries (banking, healthcare), legal/business reviews needed

Continuous Deployment: Every successful CI build is automatically deployed to production. No human intervention after code is pushed. → Use when: Netflix, Amazon, web startups wanting multiple deploys per day

Analogy: → Delivery = car is always fueled and ready to drive — you decide when to go → Deployment = car automatically drives to destination when ready

Most enterprises use Continuous DELIVERY with manual production approval gate.

Q3: Scenario: A developer asks 'Why do we need a separate artifact repository like Nexus? Why not just rebuild from source every time we deploy?'

Rebuilding from source for every deployment is DANGEROUS:

1. Build is not reproducible: different dependency versions may be downloaded each time

2. Slow: every environment (dev, staging, prod) waits for a full build
3. Not immutable: if source changes between staging and prod build, you're deploying different code

With Nexus/JFrog:

1. Build ONCE: artifact (myapp-2.1.0.jar) created in CI pipeline
2. Same artifact promoted through: dev → staging → prod
3. Guaranteed: exactly the same binary goes to production as was tested in staging
4. Fast: deployments pull the artifact from Nexus in seconds

This is the 'build once, deploy many' principle — fundamental to proper CI/CD.

Q4: Scenario: How do you handle a failed deployment in production using CI/CD?

With a proper CI/CD pipeline:

1. DETECT: Monitoring alerts (CloudWatch, Prometheus) detect the failure within seconds
2. ROLLBACK: Blue/green deployment → switch traffic back to previous (blue) environment instantly OR: Jenkins rollback pipeline → deploy previous artifact version
3. ROOT CAUSE: Check pipeline logs — which stage failed? Which test caught it?
4. FIX: Developer fixes the code, commits → CI/CD runs again → if all tests pass → redeploy

Rollback strategies:

- Blue/Green: instant, zero downtime rollback (switch DNS/ALB)
- Canary: gradually roll forward or back (10% → 0% traffic to new version)
- Feature flags: disable feature without redeploying
- Git revert + pipeline: revert commit → CI/CD redeploys previous version

---

2. What is Jenkins?

Jenkins is the world's most widely used **open-source CI/CD automation server**. It orchestrates the entire software delivery pipeline — from pulling code from Git, to building, testing, quality checks, and deploying to any environment. Jenkins was originally created by **Kohsuke Kawaguchi** in 2004 as 'Hudson' while working at Sun Microsystems. After Oracle acquired Sun, the community forked it and renamed it **Jenkins** in 2011.

Why Jenkins became the standard: Jenkins has **1,800+ plugins** covering every tool in the DevOps ecosystem. It integrates with Git, Maven, Docker, Kubernetes, AWS, Ansible, Terraform, Slack, Jira, and virtually anything else. Its plugin architecture means you're never blocked by missing features — if the plugin doesn't exist, you write one.

🔴 **Jenkins Concept:**

Key Jenkins Facts:

- Created by: Kohsuke Kawaguchi (originally as Hudson at Sun Microsystems)
- Language: Written in Java — runs on any JVM-capable platform
- License: MIT Open Source — completely free
- Port: Runs on port 8080 by default
- Workspace: `/var/lib/jenkins/workspace/` (where job files are stored)
- Java requirement: Java 11, 17, or 21
- Min resources: 2 GB RAM, 2 CPUs for basic usage
- Plugins: 1,800+ official plugins on plugins.jenkins.io

ADVANTAGE	EXPLANATION
Open Source	Completely free. No licensing fees. Large global community provides support and plugins.
1800+ Plugins	Integrates with every tool in DevOps: Git, Maven, Docker, K8s, AWS, Slack, Jira, and more.
Written in Java	Runs on Windows, Linux, Mac. Cross-platform. Any JVM platform supported.
Automates SDLC	Automates the entire Software Development Life Cycle from code commit to production deployment.
Pipeline as Code	Jenkinsfile stores pipeline definition in Git alongside application code — version controlled.
Distributed Builds	Master-Agent architecture scales horizontally — run 100s of builds in parallel across agents.
Strong Community	Active since 2004. Extensive documentation, tutorials, and community support.
Self-hosted	You control the infrastructure. Data stays in your environment — important for compliance.

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your team is choosing between Jenkins and GitHub Actions for CI/CD. How do you compare them?

Jenkins:

- ✓ Self-hosted — full control, data stays on-premise (compliance/security requirement)
- ✓ 1800+ plugins — integrates with anything
- ✓ Complex pipelines — advanced branching, parallel stages, shared libraries
- ✓ Better for: enterprises, regulated industries, complex multi-team pipelines
- ✗ You maintain the infrastructure, Java, plugins, upgrades

GitHub Actions:

- ✓ Zero infrastructure management — GitHub manages it
- ✓ Native GitHub integration — triggers on push, PR, tag automatically
- ✓ YAML syntax — simpler than Groovy/Jenkinsfile
- ✓ Better for: GitHub-native teams, open source, simpler pipelines
- ✗ Data passes through GitHub's servers — not suitable for strict on-premise requirements

Recommendation: Jenkins for enterprise/on-premise. GitHub Actions for cloud-native/startup teams.

Q2: Scenario: Jenkins is running but a new developer can't find the initial admin password. Where is it stored?

Initial admin password is stored in: `/var/lib/jenkins/secrets/initialAdminPassword`

To retrieve:

```bash
cat /var/lib/jenkins/secrets/initialAdminPassword
```

This is needed only during FIRST setup. After the admin account is configured with a username/password, this file is no longer needed.

Access Jenkins: `http://<public-ip>:8080` Paste the initialAdminPassword → Install suggested plugins → Create admin user → Ready to use

Note: The security group for the EC2 instance must allow inbound TCP port 8080 from your IP.

3. Jenkins Installation

Jenkins runs on Java, so Java must be installed first. Jenkins supports Java 11, 17, and 21. The installation process involves: installing Java, adding the Jenkins package repository, installing Jenkins, and starting the service. Below are instructions for both Amazon Linux versions commonly used in AWS EC2.

3.1 Amazon Linux 2023 (Recommended)

BASH

```
# Step 1: Switch to root
sudo su -

# Step 2: Update system packages
sudo dnf update -y

# Step 3: Install Java 17 (Jenkins requires Java 11, 17, or 21)
sudo dnf install java-17-amazon-corretto -y
java -version   # verify: openjdk version 17.x.x

# Step 4: Add Jenkins repository
sudo wget -O /etc/yum.repos.d/jenkins.repo \
  https://pkg.jenkins.io/redhat-stable/jenkins.repo

# Step 5: Import Jenkins signing key
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key

# Step 6: Install Jenkins
sudo dnf install jenkins -y

# Step 7: Enable Jenkins to start on boot + start service
sudo systemctl enable jenkins
sudo systemctl start jenkins

# Step 8: Verify Jenkins is running
sudo systemctl status jenkins

# Step 9: Get initial admin password
cat /var/lib/jenkins/secrets/initialAdminPassword

# Step 10: Open browser
# http://<EC2-PUBLIC-IP>:8080
```

3.2 Amazon Linux 2 (Legacy)

```bash
sudo su -
sudo yum update -y
sudo amazon-linux-extras install java-openjdk11 -y
sudo wget -O /etc/yum.repos.d/jenkins.repo \
  https://pkg.jenkins.io/redhat-stable/jenkins.repo
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
sudo yum install jenkins -y
sudo systemctl enable jenkins && sudo systemctl start jenkins
sudo systemctl status jenkins
cat /var/lib/jenkins/secrets/initialAdminPassword
```

3.3 Important Paths and Ports

ITEM	DETAILS
Default Port	8080 — change in `/etc/default/jenkins` (JENKINS_PORT=8080)
Home Directory	`/var/lib/jenkins/` — contains jobs, plugins, secrets, workspace
Workspace	`/var/lib/jenkins/workspace/<job-name>/` — job files cloned here
Initial Password	`/var/lib/jenkins/secrets/initialAdminPassword`
Logs	`/var/log/jenkins/jenkins.log`
Config File	`/etc/sysconfig/jenkins` — port, Java, memory settings
Plugins	`/var/lib/jenkins/plugins/`
Java Requirement	Java 11, 17, or 21 — must be installed before Jenkins

3.4 Terraform Installation on Jenkins Server

When Jenkins pipelines need to run Terraform commands, Terraform must be installed on the Jenkins server (or agent):

BASH

```bash
# Install Terraform on Amazon Linux / RHEL
sudo yum install -y yum-utils shadow-utils
sudo yum-config-manager --add-repo \
  https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
sudo yum install terraform -y
terraform version   # verify installation
```

🎯 Scenario-Based Interview Questions

Q1: Scenario: Jenkins starts successfully but you can't access it at http://IP:8080. What do you check?

Checklist:

1. EC2 Security Group: Is inbound TCP 8080 allowed from your IP? (Most common issue) → AWS Console → EC2 → Security Groups → Inbound Rules → Add TCP 8080
2. Jenkins Service Running: `sudo systemctl status jenkins` → should show 'active (running)'
3. Java Version: `java -version` → must be 11, 17, or 21
4. Port Conflict: `sudo netstat -tlnp | grep 8080` → Is another process using 8080?
5. Firewall: `sudo iptables -L` → Check if firewall blocks port 8080
6. Jenkins Log: `sudo tail -100 /var/log/jenkins/jenkins.log` → Check error messages
7. Memory: `free -h` → Jenkins needs at least 2 GB RAM

Fix security group: `aws ec2 authorize-security-group-ingress --group-id sg-xxx --protocol tcp --port 8080 --cidr 0.0.0.0/0`

Q2: Scenario: How do you upgrade Jenkins from version 2.387 to 2.420 without losing existing jobs and configurations?

Jenkins upgrade process (WAR replacement method):

1. BACKUP FIRST: `cp -r /var/lib/jenkins /var/lib/jenkins.backup`
2. Stop Jenkins: `sudo systemctl stop jenkins`
3. Navigate to Jenkins directory: `cd /usr/share/java`
4. Remove old WAR: `rm -rf jenkins.war`
5. Download new version: `wget https://updates.jenkins.io/download/war/2.420/jenkins.war`
6. Move to correct location: `mv jenkins.war /usr/share/java/`
7. Restart Jenkins: `sudo systemctl start jenkins`
8. Verify: `jenkins --version` in terminal + check version in Jenkins UI (Manage Jenkins → About)

Jobs and configurations are stored in `/var/lib/jenkins/` — they are NOT inside the WAR file. Upgrade is safe as long as you don't touch `/var/lib/jenkins/`.

Q3: Scenario: Jenkins service starts but crashes after 2 minutes. What are the likely causes and how do you diagnose?

Diagnosis: `sudo journalctl -u jenkins -n 100 --no-pager` OR: `cat /var/log/jenkins/jenkins.log | tail -200`

Common causes:

1. OutOfMemory Error: Jenkins needs more heap. Fix: edit `/etc/sysconfig/jenkins` `JENKINS_JAVA_OPTIONS='-Djava.awt.headless=true -Xmx2g'` (increase from 256m to 2g)
2. Java version incompatibility: Jenkins 2.4+ requires Java 17+ Fix: `sudo dnf install java-17-amazon-corretto -y`
3. Corrupted plugin: rename plugin file: `mv /var/lib/jenkins/plugins/broken-plugin.jpi broken-plugin.jpi.bak`
4. Disk full: `df -h` → `/var/lib/jenkins` full → clear old builds
5. Port already in use: another process on 8080 Fix: change `JENKINS_PORT` in `/etc/sysconfig/jenkins`

---

4. Build Triggers

A build trigger defines **what event causes a Jenkins job to run automatically**. Instead of manually clicking 'Build Now' every time, triggers enable Jenkins to react automatically to code commits, time schedules, external events, or completion of other jobs. Triggers are the automation heart of CI/CD — they turn Jenkins from a manual build tool into a fully automated pipeline.

Fig 2: Jenkins Build Triggers screenshots showing several concepts in sequence: (1) "Build after other projects are built" trigger configuration — pipeline-2 configured to trigger after pipeline-1 build succeeds, shown with the Jenkins console output of pipeline-1 completing and automatically triggering a new build of pipeline-2; (2) "Build Periodically" trigger with a cron schedule of `* * * * *` (every 1 minute), shown alongside the Jenkins Builds history panel with builds firing every minute automatically; (3) Webhook setup — GitHub "Add webhook" screen with Payload URL `http://3.95.209.244:8080/github-webhook/`, Content type `application/json`, alongside the Jenkins "Poll SCM" trigger configuration, annotated "Jenkins → Poll SCM → detects your commit → triggers build" and noting the build runs on next poll interval even if the change was found earlier; (4) GitHub Integration Plugin note ("Without this plugin, build won't trigger on push using webhook"), the "GitHub hook trigger for GITScm polling" checkbox, and a Webhook Delivery Logs panel showing

successful POST deliveries from GitHub to Jenkins, with a build started by GitHub push shown in the Jenkins console output.

4.1 Trigger Types

TRIGGER TYPE	DESCRIPTION
Manual (Build Now)	Click 'Build Now' in Jenkins UI. Used for ad-hoc or on-demand builds. No automation — developer-initiated.
Build Periodically	Run on a cron schedule regardless of code changes. Example: `0 2 * * *` = every day at 2 AM. Even if no code changed.
Poll SCM	Jenkins polls the Git repository at a defined interval. If changes detected → triggers build. Example: `H/5 * * * *` = check every 5 minutes.
GitHub Webhook	GitHub sends a POST notification to Jenkins when code is pushed. Jenkins builds immediately. Faster than Poll SCM — no delay.
Build After Other Projects	Job B automatically triggered when Job A succeeds. Creates a job chain (pipeline). Job 1 → Job 2 → Job 3.
Remote API Trigger	Trigger via HTTP API call. Used by external systems, scripts, or other CI tools to start Jenkins jobs programmatically.
Parameterized Build	Trigger with custom parameters (e.g., environment=prod, version=2.1). Allows customized builds for different scenarios.

4.2 Cron Syntax

Jenkins uses Unix cron syntax for scheduled triggers: **MINUTE HOUR DAY MONTH DAY_OF_WEEK**:

EXPRESSION	SCHEDULE	EXAMPLE USE
`0 2 * * *`	Every day at 2:00 AM	Nightly build, backup jobs
`H/5 * * * *`	Every 5 minutes	Poll SCM for changes
`0 8 * * 1-5`	8 AM, weekdays only	Business hours build
`* * * * *`	Every 1 minute	Build periodically (frequent test)
`0 0 * * 0`	Every Sunday midnight	Weekly full test suite
`H 8,12,16 * * 1-5`	8AM, 12PM, 4PM on weekdays	3x daily builds
`H/30 * * * *`	Every 30 minutes	Poll SCM — large team

4.3 Webhook Trigger Setup

Webhooks are the **recommended and fastest trigger mechanism** for GitHub-integrated Jenkins. When a developer pushes to GitHub, GitHub immediately sends a POST request to Jenkins — build starts within seconds. Requires: **GitHub Integration plugin** installed in Jenkins.

BASH

```
# Setup steps:
# 1. Install 'GitHub Integration Plugin' in Jenkins
#    Manage Jenkins → Plugins → Available → GitHub Integration

# 2. In GitHub Repository:
#    Settings → Webhooks → Add webhook
#    Payload URL: http://<jenkins-ip>:8080/github-webhook/
#    Content type: application/json
#    Trigger: Just the push event

# 3. In Jenkins Job:
#    Build Triggers → ✓ GitHub hook trigger for GITScm polling

# Result: Developer pushes to GitHub → GitHub notifies Jenkins
#         Jenkins starts build within seconds
```

4.4 Upstream/Downstream Job Chains

Jenkins jobs can be linked in chains where one job's success triggers the next. This creates a simple **deployment pipeline** without using Jenkinsfile:

🔴 **Jenkins Concept:**

Job Chain Example: `Job1 (Build) → Job2 (Test) → Job3 (Deploy to Staging) → Job4 (Deploy to Prod)`

Terminology:

- Upstream job = triggers the next job (Job1 is upstream of Job2)
- Downstream job = gets triggered by previous job (Job2 is downstream of Job1)

If any job FAILS: the chain stops — downstream jobs do NOT execute. Configure: Job2 → Build Triggers → 'Build after other projects are built' → Enter 'Job1'

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your team wants Jenkins to build automatically when anyone pushes to the 'main' branch on GitHub, but NOT on feature branches. How do you configure this?

1. Configure Webhook Trigger: → GitHub repo → Settings → Webhooks → Add webhook → URL: `http://jenkins-ip:8080/github-webhook/` → Content type: application/json → Push events only
2. In Jenkins Pipeline Job: → Build Triggers: ✓ GitHub hook trigger for GITScm polling
3. In Jenkinsfile — filter by branch:

```groovy
pipeline {
    agent any
    stages {
        stage('Build') {
            when { branch 'main' }   // only run when on main branch
            steps { sh 'mvn package' }
        }
    }
}
```

OR use multibranch pipeline — Jenkins auto-discovers branches and applies branch-specific logic.

**Q2: Scenario: Poll SCM is set to 'H/5 * * * '. A developer commits at 10:03 AM. When does Jenkins start the build?*

Poll SCM checks for changes at 5-minute intervals (H means Jenkins picks a hash-distributed minute to spread load).

Scenario: Poll interval fires at 10:05 AM → detects commit made at 10:03 → TRIGGERS BUILD at ~10:05 AM. Delay: Up to 5 minutes from commit to build start.

Problem with Poll SCM: → Always a delay (up to the poll interval) → Jenkins queries GitHub even when nothing changed (wasteful API calls) → Scales poorly with many repos

Webhook is better: → Build starts IMMEDIATELY when developer pushes (seconds, not minutes) → No polling — GitHub pushes notification to Jenkins → Zero wasted API calls → Preferred for production CI/CD pipelines

Q3: Scenario: You have 3 Jenkins jobs: Build → Test → Deploy. You want Deploy to run ONLY if both Build AND Test succeed. How do you configure this?

Option 1 — Job chain (simple): Test job → Build Triggers → 'Build after other projects are built' → 'Build' job Deploy job → Build Triggers → 'Build after other projects are built' → 'Test' job → If Test fails: Deploy does not trigger

Option 2 — Jenkinsfile pipeline (better, recommended):

```groovy
pipeline {
    stages {
        stage('Build') { steps { sh 'mvn package' } }
        stage('Test') { steps { sh 'mvn test' } }
        stage('Deploy') {
            when { expression { currentBuild.result == null ||
currentBuild.result == 'SUCCESS' } }
            steps { sh './deploy.sh' }
        }
    }
}
```

If any stage fails, subsequent stages are automatically skipped.

Q4: Scenario: Jenkins is configured with 'Build Periodically' but you realize it's running builds at 2 AM even when nothing changed. How do you optimize this?

Build Periodically: Builds on schedule REGARDLESS of changes. Wasteful if codebase hasn't changed.

Fix: Switch to Poll SCM: → Jenkins checks repository at the schedule interval → Builds ONLY if there are new commits since last build → Same cron syntax but condition-based

Best optimization: Webhook trigger → No polling at all → GitHub notifies Jenkins ONLY when push happens → No unnecessary builds, no delay

For nightly full test suites (you WANT to run even without changes): Keep Build Periodically at `H 2 * * *` for comprehensive regression tests Combine with webhook for incremental CI builds during working hours.

5. Jenkins Pipelines

A Jenkins Pipeline is a suite of **automated steps defined as code** that models a software delivery process. Rather than configuring separate Jenkins jobs for each step, a pipeline defines the entire CI/CD process in a single **Jenkinsfile** — a text file committed alongside application code in Git. This is known as '**Pipeline as Code**' or '**Infrastructure as Code for CI/CD**'.

Two Syntax Types:

- **Declarative Pipeline:** Structured, opinionated syntax using the `pipeline{}` block. Easier to learn, recommended for new users. Provides better error messages and IDE support.
- **Scripted Pipeline (Groovy):** Uses `node{}` block with full Groovy programming language. More flexible but more complex. Allows any Groovy code for advanced logic.

5.1 Declarative Pipeline

Declarative is the **recommended approach** for most teams. Its structured syntax makes pipelines readable, maintainable, and consistent across the organization.

GROOVY

```groovy
// Basic Declarative Pipeline structure
pipeline {
    agent any    // run on any available agent

    environment {   // global environment variables
        APP_NAME = 'myapp'
        DEPLOY_ENV = 'staging'
    }

    stages {
        stage('Checkout') {      // stage 1: pull code from Git
            steps {
                git branch: 'main',
                    url: 'https://github.com/org/repo.git'
            }
        }

        stage('Build') {         // stage 2: compile and package
            steps {
                sh 'mvn clean package -DskipTests'
            }
        }

        stage('Test') {          // stage 3: run unit tests
            steps {
                sh 'mvn test'
            }
            post {
                always {
                    junit 'target/surefire-reports/*.xml'   // publish test results
                }
            }
        }

        stage('Code Quality') {  // stage 4: SonarQube scan
            steps {
                sh 'mvn sonar:sonar'
```

```
            }
        }

        stage('Approval') {      // stage 5: manual gate before deploy
            steps {
                input 'Deploy to production? Approve to continue.'
            }
        }

        stage('Deploy') {        // stage 6: deploy application
            steps {
                sh './deploy.sh ${DEPLOY_ENV}'
            }
        }
    }

    post {                       // post-build actions
        success { echo 'Pipeline PASSED!' }
        failure { echo 'Pipeline FAILED — check logs'; mail to: 'team@company.com' }
        always { cleanWs() }     // clean workspace after every build
    }
}
```

5.2 Declarative Pipeline with Terraform

Deploying Terraform infrastructure through Jenkins pipelines is a common DevOps pattern. Jenkins provides the automation, Terraform provides the infrastructure provisioning:

GROOVY

```groovy
// Complete Terraform CI/CD Pipeline with manual approval gate
pipeline {
    agent any

    parameters {
        choice(
            name: 'ACTION',
            choices: ['apply', 'destroy'],
            description: 'Select Terraform action'
        )
    }

    stages {
        stage('Git Checkout') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/org/terraform-infra.git'
            }
        }

        stage('Terraform Init') {
            steps {
                dir('environments/prod') {
                    sh 'terraform init -reconfigure'
                }
            }
        }

        stage('Terraform Plan') {
            steps {
                dir('environments/prod') {
                    sh 'terraform plan -out=tfplan'
                }
            }
        }

        stage('Approve Plan') {   // Manual approval — human reviews plan output
```

```
            steps {
                input 'Review the terraform plan above. Approve to ${ACTION}?'
            }
        }

        stage('Terraform Apply/Destroy') {
            steps {
                dir('environments/prod') {
                    sh "terraform ${ACTION} -auto-approve"
                }
            }
        }
    }
}
```

5.3 Scripted Pipeline (Groovy)

Scripted pipelines use the full power of the Groovy programming language. Useful for complex conditional logic that Declarative syntax can't express easily:

```groovy
// Scripted Pipeline — Groovy syntax
node {
    stage('Git Checkout') {
        git branch: 'main',
            url: 'https://github.com/org/repo.git'
    }

    stage('Build') {
        sh 'mvn clean package'
    }

    stage('Test') {
        sh 'mvn test'
    }

    stage('Deploy') {
        if (env.BRANCH_NAME == 'main') {
            sh './deploy-prod.sh'
        } else {
            sh './deploy-staging.sh'
        }
    }
}
```

5.4 Pipeline as Code — Jenkinsfile in GitHub

The most mature approach: store the Jenkinsfile in the same Git repository as your application code. Configure Jenkins job with **'Pipeline script from SCM'** — Jenkins automatically reads the Jenkinsfile from Git on every build:

GROOVY

```groovy
// Jenkinsfile stored in repository root
// Jenkins job: Pipeline → Definition: Pipeline script from SCM
//              SCM: Git, Repository URL: <github-url>
//              Script Path: Jenkinsfile

pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                checkout scm   // checkout the current repo (where Jenkinsfile lives)
            }
        }

        stage('Terraform Init') {
            steps { sh 'terraform init -reconfigure' }
        }

        stage('Terraform Plan') {
            steps { sh 'terraform plan' }
        }

        stage('Terraform Action') {
            steps {
                echo "Running: terraform ${action}"
                sh 'terraform ${action} --auto-approve'
            }
        }
    }
}
```

Declarative vs Scripted — Comparison

FEATURE	DECLARATIVE	SCRIPTED (GROOVY)
Syntax	Structured `pipeline{}` block	Full Groovy: `node{}` block
Learning Curve	Easier — opinionated, clear structure	Harder — need Groovy knowledge
Error Messages	Better, more descriptive	Generic Groovy errors
Flexibility	Good for most cases	Full programming power
IDE Support	Excellent (VS Code, IntelliJ)	Basic
Recommended For	Most teams, standard pipelines	Complex logic, migration from old Jenkins
`post{}` block	Built-in — always/success/failure	Manual try-catch-finally

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your Jenkinsfile worked in staging but fails in production with 'mvn: command not found'. Why and how do you fix it?

Root cause: Maven is installed on the staging agent but NOT on the production agent (or they use different agents).

Fix Options:

1. Install Maven on all agents: `sudo yum install maven -y` (on all Jenkins agent servers)
2. Use Jenkins tool configuration: Manage Jenkins → Tools → Maven installations → Add Maven 3.9.x In Jenkinsfile:

```groovy
pipeline {
    agent any
    tools { maven 'Maven-3.9' }   // Jenkins installs Maven automatically
    stages { stage('Build') { steps { sh 'mvn package' } } }
}
```

3. Use Docker agent (most portable): `agent { docker { image 'maven:3.9-jdk-17' } }` → Runs in a Maven Docker container — no installation needed on agent

Q2: Scenario: A Jenkins pipeline has 5 stages. Stage 3 (Tests) fails. What happens to stages 4 and 5, and how do you configure it to always run stage 5 (cleanup)?

Default behavior: Stage 3 fails → stages 4 and 5 are SKIPPED. Build marked FAILED.

To always run stage 5 (cleanup) even when earlier stages fail:

```groovy
pipeline {
    stages {
        stage('Build') { steps { sh 'mvn package' } }
        stage('Test')  { steps { sh 'mvn test' } }
        stage('Deploy') { steps { sh './deploy.sh' } }
    }
    post {
        always {
            // This ALWAYS runs regardless of success or failure
            cleanWs()          // clean workspace
            sh './cleanup.sh'  // any cleanup commands
        }
        success { echo 'Pipeline passed!' }
        failure { mail to: 'team@company.com', subject: 'Build Failed', body:
'Check Jenkins logs' }
    }
}
```

Q3: Scenario: Your pipeline takes 45 minutes because Build, Test, and Code Quality run sequentially. How do you speed it up?

Use parallel stages — run Test and Code Quality simultaneously:

```groovy
pipeline {
    stages {
        stage('Build') { steps { sh 'mvn package' } }

        stage('Parallel Validation') {
            parallel {
                stage('Unit Tests') {
                    steps { sh 'mvn test' }
                }
                stage('Code Quality') {
                    steps { sh 'mvn sonar:sonar' }
                }
                stage('Security Scan') {
                    steps { sh 'mvn dependency-check:check' }
                }
            }
        }

        stage('Deploy') { steps { sh './deploy.sh' } }
    }
}
```

Result: Test (15min) + SonarQube (15min) + Security (10min) run in parallel → Total wall time: 15 minutes instead of 40 minutes.

---

6. Jenkins Master-Agent (Master-Slave) Architecture

When a single Jenkins server needs to run many builds simultaneously — Docker pipelines, Terraform pipelines, Python tests, Java builds all at once — one server's CPU and memory becomes a bottleneck. The solution is Jenkins' **Master-Agent architecture** (previously called Master-Slave): one central **Jenkins Master** server that manages and orchestrates everything, and multiple **Agent servers** that actually execute the build jobs.

Fig 3: Jenkins Master-Agent Architecture diagram. Top portion titled "Jenkins master and slave" with note "If multiple process wants to automat through Jenkins we can use Jenkins master slave concept". It contrasts a "Not a good approach" — installing Jenkins into each server (shown with icons for Docker, Terraform, Python each getting their own full Jenkins install, crossed out) with the note "This approach is not Recommended installing Jenkins into each every server which result multiple dashboards create and also Jenkins takes more hardware each server". Below, an arrow leads to the recommended approach: one Jenkins master (ip:8080) connects via SSH/controller to multiple target nodes running docker pipeline, Terraform pipeline, and python pipeline, each also connected to GitHub. Annotation: "here Jenkins server connect target node and run required process as mentioned in pipeline". A parallel comparison on the right shows GitHub Actions doing the same thing — a GitHub Actions controller connects to self-hosted runner nodes (Docker, Terraform, Python) each linked to GitHub, with a YAML snippet `jobs: terraform: runs-on: ubuntu-latest` *annotated "Use this YAML in your workflow file for each job" and "runs-on: self-hosted", labeled "Manage by GitHub providers" and "Your task".*

🔴 **Jenkins Concept:**

Jenkins Master: The central control server. → Hosts the Jenkins web UI (port 8080) → Stores all job configurations, build history, credentials → Schedules and dispatches jobs to available agents → Never (ideally) executes build jobs itself — delegates to agents

Jenkins Agent (Slave): Worker servers that execute build jobs. → Receives job instructions from Master via SSH or JNLP → Has the required tools installed (Maven, Terraform, Docker, Python) → Sends results back to Master when done → Multiple agents can run jobs in parallel

6.1 Why Master-Agent?

BENEFIT	EXPLANATION
Parallel Builds	Run 10 builds simultaneously on 10 agents. Single server can only run one at a time (or is severely slowed).
Specialized Agents	Docker agent has Docker installed. Terraform agent has Terraform. Python agent has Python. Each tool isolated.
Scalability	Add more agents when load increases. Remove agents during off-hours. Scale horizontally.
Resource Isolation	A crashed or overloaded agent doesn't affect Master or other agents.
OS Diversity	Linux agent for Java builds, Windows agent for .NET builds, Mac agent for iOS — same Master orchestrates all.
Security	Agents in different network segments (DMZ, private, production). Master never directly touches production.

6.2 Agent Configuration Methods

METHOD	HOW IT WORKS	USE CASE
SSH Agent	Master SSHes into agent server and runs Jenkins agent process	Linux/Mac agents — most common
JNLP/WebSocket	Agent initiates outbound connection to Master (no inbound SSH needed)	Agents behind firewall, Windows agents
Docker Agent	`pipeline { agent { docker { image 'maven:3.9' } } }`	Ephemeral agents — best for CI
Kubernetes Pod	Jenkins creates K8s pod per build, deletes after completion	Cloud-native, auto-scaling agents
Cloud (AWS EC2)	Jenkins provisions EC2 instance on demand, terminates after build	Cost-optimized, auto-scaling

6.3 Setting Up SSH Agent

BASH

```
# On the Agent EC2 server:
# 1. Install Java (same version as Master)
sudo dnf install java-17-amazon-corretto -y

# 2. Create jenkins user
sudo useradd jenkins
sudo mkdir -p /home/jenkins/.ssh

# On the Master server:
# 3. Generate SSH key pair
ssh-keygen -t rsa -b 4096 -C 'jenkins-master-key'

# 4. Copy public key to agent
ssh-copy-id -i ~/.ssh/id_rsa.pub jenkins@<agent-ip>

# In Jenkins UI:
# 5. Manage Jenkins → Nodes → New Node
#    Node name: terraform-agent
#    Type: Permanent Agent
#    Remote root directory: /home/jenkins
#    Launch method: Launch agents via SSH
#    Host: <agent-private-ip>
#    Credentials: Add SSH private key
#    Labels: 'terraform' (use this label in Jenkinsfile)
```

GROOVY

```
// In Jenkinsfile — run job on specific agent:
pipeline {
    agent { label 'terraform' }   // run on agent labeled 'terraform'
    stages { ... }
}
```

6.4 GitHub Actions vs Jenkins Master-Agent

GitHub Actions provides cloud-hosted runners (agents) as a service. Jenkins requires you to manage your own agents. Both support the same master-agent concept:

CONCEPT	JENKINS	GITHUB ACTIONS
Master/Controller	Jenkins Server	GitHub Actions Service
Agents	Jenkins Agents (EC2, pods)	GitHub-hosted runners (ubuntu-latest)
Self-hosted agents	Configure via SSH	`runs-on: self-hosted`
Agent labels	`agent { label 'docker' }`	`runs-on: [self-hosted, docker]`
Job isolation	Each agent has tools installed	Each runner is fresh VM/container

🎯 Scenario-Based Interview Questions

Q1: Scenario: Jenkins is running 10 jobs simultaneously on one server. CPU is at 95% and builds are timing out. What is your solution?

Implement Master-Agent architecture:

1. Keep Jenkins Master for: orchestration only (no build execution) Manage Jenkins → Manage Nodes → Built-In Node → # of executors = 0 (Master runs NO builds — only schedules)
2. Provision Agent EC2 instances (t3.large with 4 CPU, 8 GB RAM)
3. Install Java + required tools on each agent
4. Add agents: Manage Jenkins → Nodes → New Node → SSH
5. Update Jenkinsfiles with agent labels: `agent { label 'java-agent' }` // specific to Java builds `agent { label 'terraform' }` // specific to Terraform
6. Add more agents as load grows

For AWS cost optimization: Use EC2 Fleet plugin → agents are created on demand and terminated after build. Pay only when building.

Q2: Scenario: Your security team says Jenkins builds should not run on the same server as the Jenkins UI because a compromised build could access Jenkins secrets. How do you fix this?

Implement Master-Agent isolation:

1. Jenkins Master: Only UI, job config, credential management. Set executor count to 0 (no builds).
2. Agent Servers: Only execute builds. No access to Jenkins secrets storage.
3. Network segmentation: Master → Private subnet (not internet-accessible) Agents → DMZ subnet (can pull from internet for dependencies)
4. Credential management: Use Jenkins Credentials plugin — agents receive secrets at build time via encrypted channel, not stored on agent
5. Ephemeral agents (most secure): Docker or Kubernetes agents created fresh per build and destroyed immediately after — nothing persists

This is the 'least privilege' principle applied to CI/CD infrastructure.

Q3: Scenario: How do you configure a Jenkins agent using GitHub Actions self-hosted runner instead of a separate Jenkins agent?

GitHub Actions self-hosted runner concept:

```bash
# Install self-hosted runner on your EC2:
# GitHub → Settings → Actions → Runners → New self-hosted runner
# Download and run the runner agent:
./config.sh --url https://github.com/org/repo --token <TOKEN>
./run.sh   # or configure as systemd service
```

```yaml
# In GitHub Actions workflow:
name: Build
on: push:
jobs:
  build:
    runs-on: self-hosted   # uses YOUR server instead of GitHub's
    steps:
      - uses: actions/checkout@v3
      - run: mvn package
```

Equivalent to Jenkins:

```groovy
pipeline {
    agent { label 'my-ec2-agent' }
    stages { ... }
}
```

Both allow running builds on your own infrastructure while the master/controller (GitHub/Jenkins) is cloud-hosted.

7. Jenkins Workspace Management

Every Jenkins job uses a **workspace** — a dedicated directory on the agent (or master) where the job's files are stored during a build. When a pipeline checks out code from Git, the repository is cloned into this workspace directory. Understanding workspace management is important for debugging build failures and preventing disk space issues on Jenkins servers.

🔴 **Jenkins Concept:**

Default workspace location: `/var/lib/jenkins/workspace/<job-name>/` Each job has its own workspace directory. Files from the previous build may still exist in the workspace (not automatically cleaned). Multiple executors on the same node use `workspace@2`, `workspace@3` to avoid conflicts. Large projects accumulate GBs of build artifacts — regular cleanup is essential.

PROPERTY	DETAILS
Default Location	`/var/lib/jenkins/workspace/<job-name>/` on the Jenkins master or agent
Contents	Cloned Git repository, compiled classes, JAR/WAR files, test reports, logs
Persistence	Workspace persists between builds by default — previous build files remain
Cleanup	`cleanWs()` in `post{}` block OR configure 'Workspace Cleanup Plugin' per job
Multiple Executors	`workspace@2`, `workspace@3` are created for parallel builds on same agent
Custom Path	Configure custom workspace path per job in job config → Advanced → Use custom workspace
Disk Management	Monitor with: `df -h /var/lib/jenkins` — set up alerting when >80% full

GROOVY

```groovy
// Workspace management in Jenkinsfile
pipeline {
    agent any

    options {
        // Discard old builds to save disk space
        buildDiscarder(logRotator(numToKeepStr: '10', artifactNumToKeepStr: '5'))

        // Workspace is cleaned at the START of build (fresh checkout)
        skipDefaultCheckout(true)
    }

    stages {
        stage('Clean Checkout') {
            steps {
                cleanWs()      // clean workspace before checkout
                checkout scm
            }
        }

        stage('Build') {
            steps {
                sh 'mvn package'
                // Artifacts stored in workspace/target/
            }
        }
    }

    post {
        always {
            // Archive artifacts before cleaning workspace
            archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
            junit 'target/surefire-reports/*.xml'   // publish test results
            cleanWs()   // clean workspace after build completes
        }
    }
}
```

🎯 **Scenario-Based Interview Questions**

Q1: Scenario: Jenkins build fails with 'No space left on device'. The /var/lib/jenkins disk is 100% full. How do you fix this and prevent recurrence?

Immediate fix:

1. Find largest directories:

```bash
du -sh /var/lib/jenkins/workspace/* | sort -rh | head -20
du -sh /var/lib/jenkins/jobs/*/builds/* | sort -rh | head -20
```

2. Delete old builds:

```bash
find /var/lib/jenkins/jobs -name 'builds' -type d | xargs -I{} find {} -mindepth 1 -maxdepth 1 -type d | sort -r | tail -n +6 | xargs rm -rf
```

3. Clean workspaces: Manage Jenkins → Workspace cleanup
4. Delete Docker images if Docker is installed: `docker system prune -af`

Prevent recurrence:

1. Configure log rotation in ALL jobs: `buildDiscarder(logRotator(numToKeepStr: '10'))`
2. Add `cleanWs()` in `post { always { ... } }` block
3. Increase disk size (EBS volume expansion)
4. Set up CloudWatch alarm on disk usage > 80%
5. Use S3 for artifact storage instead of local disk

---

8. Securing Jenkins — Authentication & Authorization

By default, Jenkins has minimal security configured. In any real-world deployment, **securing Jenkins is critical**: unauthenticated access means anyone can view your code, run builds, access credentials,

and modify configurations. Jenkins security has three layers: Authentication (who are you?), Authorization (what can you do?), and Confidentiality (are secrets protected?).

Fig on p.31: Two CI/CD flow diagrams. The first, titled "CICD - Continues Integration Continues Deployment", shows a developer pushing to GitHub (Source) which flows through Build → Testing → code quality → Deployment, annotated "Create final package by adding required dependencies and libraries", "Test code functionalities", "package= source code+ dependencies + libraries", "executable format", ending at a Tomcat deployment icon. The second, titled "CICD -- Continues Integration and Continues Delivery", is a fuller version showing Jenkins/GitHub Actions/CI-CD tool icons, with the CI half (GitHub → Build with Maven/ant → Testing with Junit → code quality with SonarQube, labeled "pipeline-1") feeding into an artifact tool (S3 bucket, JFrog, Nexus) and then a CD half (pipeline-2) going to CDeployment/Deployment via Azure DevOps-style icons, ending at a Tomcat icon. Below this, a Jenkins summary box lists: "Jenkins — Fully Open source CICD tool", "Jenkins port 8080", "/var/lib/Jenkins --Default work space", "resources: 2gb ram, 2cpu's" alongside a screenshot of the Jenkins Wikipedia infobox showing Original author: Kohsuke Kawaguchi, Initial release: 2 February 2011, Stable release: 2.542, Repository: github.com/jenkinsci/jenkins, Written in: Java, Platform: Java 11, Java 17, Java 21, Type: Continuous delivery.

Fig 4: Jenkins User Authorization — Role-Based Security Setup, a step-by-step annotated screenshot walkthrough: Step-1 shows the "Role-based Authorization Strategy" plugin install screen (Security / Authentication and User Management category) with description "Enables user authorization using a Role-Based strategy. Roles or nodes selected by regular expressions." Step-2 shows Manage Jenkins → Security → Authorization with the "Role-Based Strategy" option selected among choices (Anyone can do anything, Legacy mode, Logged-in users can do anything, Matrix based security, Project based Matrix Authorization Strategy, Role-Based Strategy), annotated "apply+save". Step-3 shows the Users panel with "Create User" form fields (name, pass, username, email) and notes an "Access Denied" error with the comment "user 1 is missing the Overall/Read permission — we need to give permission to add user". Step-4-A shows "Manage Roles" — creating a global role (e.g. "admin", "read") with a permissions matrix (Overall/Create/Read/Administer columns), annotated "give a role name and click add", "select permissions you want to give", "apply+save". Step-4-B shows "Assign Roles" — a Global roles matrix table (User/Group columns for Anonymous, Authenticated Users, admin, user-1) with checkboxes for login/admin roles, annotated "manually add user and select the role", "apply+save", "# now user can see pipelines". Below this, a second annotated panel explains Item Roles in more depth: "global (Authentication + basic access to Jenkins) + item role (Authorization at job/pipeline level)"; the process is: item roles → role to add → name-of-role → pattern → job-name (so the user can access only this job by name) → select permission for the role → apply+save; then assign role → item roles → select role in user section → apply+save; comment: "#now user-1 can access only job 'only-user-1'". This is illustrated with an "Item roles" table showing a "user-1-specific" role with pattern "only-user-1" and Credentials columns (Create/Delete/ManageDomains/Update/View/Build) checked for View and Build only, plus an icon diagram showing user-1 (with only-user-1 pipeline access, green

check) unable to access pipeline-3 (red X), captioned "#Giving specific permission to access specific job".

8.1 Authentication — Who Are You?

Authentication verifies user identity. Jenkins supports multiple authentication methods:

METHOD	DESCRIPTION
Jenkins Own Database	Built-in user management. Users and passwords stored in Jenkins. Simple, good for small teams.
LDAP / Active Directory	Integrate with company LDAP/AD. Users log in with their corporate credentials. Required for enterprises.
GitHub OAuth	Users log in with their GitHub account. Grants access based on GitHub org membership.
Google OAuth	Users log in with Google accounts. Common in Google Workspace organizations.
SAML 2.0	Enterprise SSO integration (Okta, Azure AD). Single sign-on for entire organization.
Matrix Security	Fine-grained control: define permissions per user per action. Without Role-Based plugin.

8.2 Authorization — Role-Based Access Control (RBAC)

The **Role-Based Authorization Strategy plugin** is the standard way to implement authorization in Jenkins. It allows defining roles (Admin, Developer, Read-Only) and assigning them to users or groups — both globally and at the job level.

```bash
# RBAC Setup Process:

# Step 1: Install Plugin
#   Manage Jenkins → Plugins → Available → 'Role-based Authorization Strategy'

# Step 2: Enable RBAC
#   Manage Jenkins → Security → Authorization → Role-Based Strategy → Save

# Step 3: Create Roles
#   Manage Jenkins → Manage and Assign Roles → Manage Roles
#   Global Roles: admin (all permissions), developer (build + read), viewer (read-only)
#   Item Roles: restrict specific users to specific jobs/pipelines
#   Example: user-1-specific role with pattern 'team-A-.*' → matches all jobs starting with 'team-A-'

# Step 4: Create Users
#   Manage Jenkins → Users → Create User
#   Fill: Full name, username, password, email

# Step 5: Assign Roles to Users
#   Manage and Assign Roles → Assign Roles
#   Global roles: Check 'developer' for developer users, 'viewer' for read-only users
#   Item roles: Assign specific job patterns per user
```

ROLE	GLOBAL PERMISSIONS	USE CASE
Admin	All permissions — full control	Jenkins administrators, DevOps team leads
Developer	Build, Read, Workspace, Cancel	Developers — can trigger and view builds
Viewer	Read only — view jobs and builds	Managers, stakeholders — view pipeline status
Item-Specific	View/Build ONLY their team's jobs	Multi-team: Team A sees only Team A jobs

8.3 Confidentiality — Managing Secrets

Jenkins stores credentials (passwords, API keys, SSH keys, tokens) in its **Credentials Store**. Credentials are stored encrypted in Jenkins' home directory. In pipelines, credentials are accessed via environment variables — never hardcoded in Jenkinsfiles.

GROOVY

```groovy
// Using credentials in Jenkinsfile — NEVER hardcode passwords
pipeline {
    agent any

    environment {
        // Bind credential to environment variable (masked in logs)
        AWS_CREDS = credentials('aws-production-credentials')
        GITHUB_TOKEN = credentials('github-token')
        DB_PASSWORD = credentials('db-password-prod')
    }

    stages {
        stage('Deploy') {
            steps {
                // AWS credentials auto-set as AWS_ACCESS_KEY_ID +
AWS_SECRET_ACCESS_KEY
                sh 'aws s3 cp artifact.jar s3://my-bucket/'

                // withCredentials block for more control
                withCredentials([string(credentialsId: 'api-key', variable:
'API_KEY')]) {
                    sh 'curl -H "Authorization: Bearer $API_KEY"
https://api.example.com'
                }
            }
        }
    }
}

// Credential types:
// Secret text: API keys, tokens
// Username with password: Git credentials, Docker registry
// SSH Username with private key: Agent authentication, deployment keys
// Certificate: SSL/TLS certificates
// Secret file: Kubeconfig, certificates files
```

🎯 Scenario-Based Interview Questions

Q1: Scenario: A developer can see ALL Jenkins jobs including production deployment jobs they should not have access to. How do you restrict access so each team sees only their jobs?

Implement Item Roles with RBAC:

1. Install Role-Based Authorization Strategy plugin
2. Manage Roles → Item Roles → Add role: Role name: 'team-payments-role' Pattern: `payments-.*` (matches all jobs starting with 'payments-') Permissions: Build, Read, Cancel, Workspace
3. Assign Roles → Item Roles section: Add team member → select 'team-payments-role'

Result: → Payment team member logs in → sees ONLY jobs matching `payments-.*` → All other jobs are invisible to them → They cannot trigger, view, or know about other teams' pipelines

Naming convention required: all jobs must follow the pattern: `payments-build`, `payments-test`, `payments-deploy` `infra-terraform-prod`, `infra-k8s-staging`

Q2: Scenario: A Jenkins admin left the company. The company's Jenkins has 50 jobs but the admin account is the only one configured. How do you regain access?

Recovery steps (requires server SSH access):

1. SSH into Jenkins server: `ssh ec2-user@jenkins-ip`
2. Stop Jenkins: `sudo systemctl stop jenkins`
3. Edit config.xml to disable security: `sudo nano /var/lib/jenkins/config.xml` Change: `<useSecurity>true</useSecurity>` → `<useSecurity>false</useSecurity>`
4. Start Jenkins: `sudo systemctl start jenkins`
5. Access Jenkins at port 8080 — NO login required now
6. Manage Jenkins → Security → Create new admin user
7. Re-enable security: set authorization back to Role-Based or Matrix
8. Verify new admin login works
9. Lock down the old admin account

Prevention: Always have at least 2 admin accounts. Document all credentials in a password manager (Vault, 1Password).

Q3: Scenario: A Jenkinsfile has the line: `sh 'aws s3 sync . s3://bucket/ --access-key AKIAIOSFODNN7EXAMPLE --secret-key wJalrXUtnFEMI/K7MDENG'`. What are the problems and how do you fix this?

Critical problems:

1. SECURITY: Credentials hardcoded in Jenkinsfile → committed to Git → visible to ANYONE with repo access
2. GIT HISTORY: Even if removed later, credentials are in Git history forever
3. NOT ROTATABLE: Changing the key requires editing every Jenkinsfile that uses it
4. AUDIT: No audit trail of which builds used the credentials

Fix:

1. Revoke the exposed AWS key IMMEDIATELY in IAM
2. Store credentials in Jenkins Credential Store: Manage Jenkins → Credentials → Add Credential: AWS Credentials type
3. Reference in Jenkinsfile:

```groovy
environment {
    AWS_CREDS = credentials('aws-prod-s3-key')
}
// Jenkins auto-sets AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY
steps { sh 'aws s3 sync . s3://bucket/' }
```

4. Best option: Use IAM Role on the EC2 Jenkins agent → no credentials needed at all!

---

9. Jenkins Plugins

Plugins are the **extension mechanism** that makes Jenkins capable of everything. The core Jenkins engine provides scheduling and execution. Every integration with external tools — Git, Maven, Docker, SonarQube, Slack, AWS — is added via plugins. There are **1,800+ official plugins** on plugins.jenkins.io, plus community plugins for virtually every tool in the DevOps ecosystem.

9.1 Essential Plugins

PLUGIN	WHAT IT ENABLES
Git Plugin	Enables SCM integration with Git repositories. Required for 'git' step in pipelines.
GitHub Integration	Enables webhook triggers from GitHub. Required for 'GitHub hook trigger for GITScm polling'.
Maven Integration	Provides Maven build tool integration. Required for maven-based build steps.
Pipeline	Core pipeline plugin — enables Declarative and Scripted pipelines. Usually pre-installed.
Credentials Plugin	Secure storage and management of passwords, API keys, SSH keys.
Role-Based Auth Strategy	Implements RBAC — role-based user access control for jobs and folders.
Blue Ocean	Modern, visual pipeline UI. Shows pipeline stages graphically with detailed logs.
SonarQube Scanner	Integrates SonarQube code quality analysis into pipelines.
Docker Pipeline	Enables `agent { docker {} }` — run pipeline stages inside Docker containers.
Kubernetes	Automatically provision Kubernetes pods as Jenkins agents on demand.
Slack Notification	Send build notifications to Slack channels on success/failure.
Email Extension	Send HTML email notifications with build results and logs.
AWS Steps	Native AWS CLI integration in pipeline steps (S3, EC2, Lambda, etc.)
Workspace Cleanup	Automatically clean workspace before/after builds to save disk space.

9.2 Installing Plugins

BASH

```
# Method 1: Jenkins UI (recommended for most installations)
# Manage Jenkins → Plugins → Available plugins → Search → Install
# Restart Jenkins after installing (or check 'Restart after install')

# Method 2: Jenkins CLI
java -jar jenkins-cli.jar -s http://localhost:8080 install-plugin plugin-name --
restart

# Method 3: Directly download .jpi file
# plugins.jenkins.io → Download .jpi → upload in Manage Plugins → Advanced → Deploy

# View installed plugins version:
# Manage Jenkins → Plugins → Installed plugins

# Update all plugins:
# Manage Jenkins → Plugins → Updates → Select All → Update
# (Test in non-prod first — plugin updates can break pipelines)
```

🎯 **Scenario-Based Interview Questions**

Q1: Scenario: After updating Jenkins plugins, all pipelines fail with 'ClassNotFoundException'. What happened and how do you recover?

Cause: Plugin update introduced a breaking API change. A plugin that other plugins depend on changed its interface, causing ClassNotFoundException at runtime.

Recovery:

1. Identify the problem plugin from the stack trace: `tail -100 /var/log/jenkins/jenkins.log | grep 'ClassNotFound'`
2. Roll back the specific plugin: Stop Jenkins: `sudo systemctl stop jenkins` `cd /var/lib/jenkins/plugins/` Remove new version: `rm plugin-name.jpi` Restore backup: `cp plugin-name.jpi.bak plugin-name.jpi` (if available) OR: Download specific older version from plugins.jenkins.io → /changelog
3. Restart Jenkins and verify

Best practice: → Test plugin updates in a non-prod Jenkins instance first → Enable 'Plugin Manager → Advanced → Check now' for notifications → Keep backups: `cp -r /var/lib/jenkins/plugins /var/lib/jenkins/plugins.backup`

---

10. GitLab CI/CD

GitLab CI/CD is a **built-in CI/CD system** integrated directly into GitLab (both gitlab.com and self-hosted). Unlike Jenkins which requires a separate server, GitLab CI is activated simply by creating a **.gitlab-ci.yml** file in the root of your repository. GitLab then automatically detects this file and runs the pipeline when code is pushed. GitLab CI is particularly popular in organizations already using GitLab for source control.

💡 **Layman Explanation:**

- GitLab CI = like Jenkins but built inside GitLab itself. No separate server to maintain.
- You write a .gitlab-ci.yml file in your repo root. GitLab reads it and runs your pipeline.
- Every push to GitLab → GitLab CI automatically picks up changes → runs pipeline stages.
- GitLab Runners execute the jobs (like Jenkins Agents). They can be hosted by GitLab or self-hosted.

YAML

```yaml
# .gitlab-ci.yml — GitLab CI pipeline definition
# Place this file in the ROOT of your GitLab repository

stages:              # define pipeline stages (run in order)
  - build
  - test
  - quality
  - deploy

variables:            # global variables
  MAVEN_OPTS: '-Dmaven.repo.local=$CI_PROJECT_DIR/.m2'

# Build stage
build-job:
  stage: build
  image: maven:3.9-jdk-17    # run in Docker container
  script:
    - mvn clean package -DskipTests
  artifacts:
    paths:
      - target/*.jar          # save artifact for next stages
    expire_in: 1 hour
  only:
    - main
    - merge_requests

# Test stage
test-job:
  stage: test
  image: maven:3.9-jdk-17
  script:
    - mvn test
  coverage: '/Total.*?([0-9]{1,3})%/'

# Code Quality stage
sonarqube-scan:
  stage: quality
```

```
  image: sonarsource/sonar-scanner-cli
  script:
    - sonar-scanner -Dsonar.projectKey=$CI_PROJECT_NAME
  only:
    - main

# Deploy to staging — runs automatically
deploy-staging:
  stage: deploy
  script:
    - echo 'Deploying to staging...'
    - ./deploy.sh staging
  environment:
    name: staging
  only:
    - main

# Deploy to production — requires MANUAL approval
deploy-production:
  stage: deploy
  script:
    - ./deploy.sh production
  environment:
    name: production
  when: manual        # requires human click to trigger
  only:
    - main
```

GitLab CI vs Jenkins

GITLAB CI	JENKINS	NOTES
`.gitlab-ci.yml`	Jenkinsfile	Both are Pipeline as Code — YAML vs Groovy/Declarative
GitLab Runner	Jenkins Agent	Both execute build jobs on separate machines
GitLab.com hosted	No hosted option	GitLab offers free CI minutes; Jenkins is always self-hosted
`stages: []`	`stages {}`	Similar concept — sequential stage groups
`when: manual`	`input 'Approve?'`	Manual approval gate before deploying to production
`artifacts:`	`archiveArtifacts`	Pass files between stages
`environment:`	No native equivalent	GitLab has built-in environment tracking

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your team is moving from Jenkins to GitLab CI. How do you migrate a Jenkinsfile to .gitlab-ci.yml?

Mapping from Jenkinsfile to .gitlab-ci.yml:

```text
Jenkinsfile                →   .gitlab-ci.yml:
pipeline {                 →   stages: [build, test, deploy]
  agent any                →   runs on GitLab Runner
  stages {                 →   individual job with 'stage:' key
    stage('Build')         →   build-job:
      steps {              →     stage: build
        sh 'mvn...'        →     script: [mvn...]
      }
  }
}
```

Key differences:

1. YAML instead of Groovy — simpler syntax
2. Docker images per job: `image: maven:3.9-jdk-17`
3. Artifacts passed between stages explicitly: `artifacts: { paths: [target/] }`
4. Environment protection: GitLab has built-in environment dashboard
5. Variables stored in: GitLab → Settings → CI/CD → Variables (equivalent to Jenkins Credentials)

Migration usually takes 1-2 days for a standard pipeline.

Q2: Scenario: GitLab CI pipeline runs for every push including feature branches, causing excessive CI costs. How do you limit which branches trigger the pipeline?

Use 'rules' or 'only/except' in .gitlab-ci.yml:

Method 1 — only/except (simple):

```yaml
deploy-prod:
  stage: deploy
  script: ./deploy.sh production
  only:
    - main       # only run on main branch
    - tags       # and on version tags
  except:
    - feature/*  # never on feature branches
```

Method 2 — rules (advanced, recommended):

```yaml
deploy-prod:
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: always
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
      when: manual
    - when: never   # skip for everything else
```

Result: → Feature branches: No deploy job (only build + test) → Merge Requests: Deploy available as manual → Main branch: Deploy runs automatically

---

11. GitHub Actions

GitHub Actions is GitHub's **native CI/CD platform**, launched in 2018. Like GitLab CI, it is built into GitHub itself — no separate CI server needed. Workflows are defined in **YAML files** stored in `.github/workflows/` in your repository. GitHub provides free-tier runners (virtual machines) that execute your workflows, or you can use **self-hosted runners** on your own infrastructure.

💡 **Layman Explanation:**

- GitHub Actions = Jenkins + GitHub built together as one platform.
- No server to install — GitHub manages the infrastructure (runners).
- Push to GitHub → GitHub reads your .github/workflows/*.yml → runs automatically.
- Marketplace has 10,000+ pre-built Actions for every tool — like Jenkins plugins.
- Free tier: 2,000 minutes/month for public repos; 500 minutes for private repos.

YAML

```yaml
# .github/workflows/ci-cd.yml
# GitHub Actions workflow file

name: CI/CD Pipeline

on:                          # TRIGGERS — what events start this workflow
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
  schedule:
    - cron: '0 2 * * *'      # also run nightly at 2 AM

env:                          # global environment variables
  JAVA_VERSION: '17'

jobs:                         # JOBS run in parallel by default

  build:                      # JOB 1: Build and Test
    name: Build and Test
    runs-on: ubuntu-latest    # GitHub-hosted runner (free)

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Java 17
        uses: actions/setup-java@v3
        with:
          java-version: ${{ env.JAVA_VERSION }}
          distribution: corretto

      - name: Build with Maven
        run: mvn clean package

      - name: Run Tests
        run: mvn test
```

```
      - name: Upload artifact
        uses: actions/upload-artifact@v3
        with:
          name: app-jar
          path: target/*.jar

  deploy-staging:              # JOB 2: Deploy to Staging
    name: Deploy to Staging
    needs: build                # waits for 'build' job to succeed
    runs-on: ubuntu-latest
    environment: staging        # requires environment approval if configured

    steps:
      - uses: actions/checkout@v4

      - name: Download artifact
        uses: actions/download-artifact@v3
        with:
          name: app-jar

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v2
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: us-east-1

      - name: Deploy to EC2
        run: |
          aws s3 cp target/app.jar s3://my-deploy-bucket/
          aws ssm send-command --instance-ids ${{ secrets.EC2_ID }} \
            --document-name AWS-RunShellScript \
            --parameters 'commands=["aws s3 cp s3://my-deploy-bucket/app.jar
/app/","sudo systemctl restart myapp"]'

  deploy-production:           # JOB 3: Deploy to Production
    name: Deploy to Production
    needs: deploy-staging
```

```
    runs-on: ubuntu-latest
    environment:                # requires manual approval from reviewer
      name: production
    if: github.ref == 'refs/heads/main'   # only on main branch

    steps:
      - uses: actions/checkout@v4
      - name: Deploy to Production
        run: ./scripts/deploy-prod.sh

  terraform:                    # JOB 4: Terraform Infrastructure
    name: Terraform Plan and Apply
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
      - uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v2
        with:
          terraform_version: '1.6.0'

      - name: Terraform Init
        run: terraform init

      - name: Terraform Plan
        run: terraform plan -out=tfplan

      - name: Terraform Apply
        run: terraform apply tfplan
        if: github.event_name != 'pull_request'   # don't apply on PRs
```

11.1 Secrets in GitHub Actions

```bash
# Store secrets in GitHub: Repo → Settings → Secrets and Variables → Actions
# Reference in workflow:

${{ secrets.AWS_ACCESS_KEY_ID }}
${{ secrets.DB_PASSWORD }}
${{ secrets.DOCKER_TOKEN }}

# Environment secrets (per deployment environment):
# Repo → Settings → Environments → production → Add secret
# These secrets only available when job targets that environment
```

11.2 Self-Hosted Runners

```bash
# Use your own EC2/server as GitHub Actions runner
# Useful for: accessing private resources, specific hardware, no minute limits

# Install runner on EC2:
# GitHub → Repo Settings → Actions → Runners → New self-hosted runner
# Follow the download + configure steps shown on screen

./config.sh --url https://github.com/org/repo --token <TOKEN>
./run.sh   # OR install as service: sudo ./svc.sh install && sudo ./svc.sh start
```

```yaml
# In workflow — use self-hosted runner:
jobs:
  terraform:
    runs-on: self-hosted    # your EC2 server with Terraform installed
    # OR with labels:
    runs-on: [self-hosted, linux, terraform]
```

```groovy
// Equivalent Jenkins pipeline:
// pipeline { agent { label 'terraform-agent' } ... }
```

GitHub Actions vs Jenkins — Concept Mapping

GITHUB ACTIONS CONCEPT	EQUIVALENT JENKINS CONCEPT	DESCRIPTION
`.github/workflows/*.yml`	Jenkinsfile	Pipeline as Code definition file
`on: push/PR`	Build Trigger (webhook)	What event starts the pipeline
`jobs:`	`stages {}`	Groups of work in the pipeline
`steps:`	`steps {}`	Individual commands within a job
`runs-on: ubuntu-latest`	`agent any`	Where to execute the job
`uses: actions/xxx@v3`	Plugin (pre-installed)	Pre-built reusable action
`secrets.MY_SECRET`	`credentials('my-cred')`	Secure credential reference
`environment: production`	`input 'Approve?'`	Manual approval gate
`needs: build`	'Build after Build' trigger	Job dependency

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your GitHub Actions workflow is running on GitHub's free runners but needs to access your company's internal database (not internet-accessible). How do you solve this?

GitHub-hosted runners cannot reach private/internal resources — they run on GitHub's public cloud.

Solution: Self-hosted runner on your internal network:

1. Set up an EC2 instance or on-prem server in the same VPC as your database
2. Install GitHub Actions runner on it: `./config.sh --url https://github.com/org/repo --token <TOKEN>` `sudo ./svc.sh install && sudo ./svc.sh start`
3. Update workflow:

```yaml
jobs:
  db-migration:
    runs-on: self-hosted   # your internal server
    steps:
      - run: ./run-db-migration.sh   # can reach internal DB
```

Security considerations: → Self-hosted runner has access to your internal network — secure it carefully → Ensure runner doesn't have more permissions than needed → Use IAM roles, not hardcoded credentials → Consider Vault Agent for secrets injection

Q2: Scenario: You want to deploy to production using GitHub Actions ONLY when a senior engineer approves. How do you implement this mandatory approval gate?

Use GitHub Environments with Required Reviewers:

1. Create environment: Repo → Settings → Environments → New environment: 'production'
2. Add protection rules: ✓ Required reviewers: add senior engineer's GitHub username ✓ Prevent self-review: requester cannot be their own reviewer
3. In workflow:

```yaml
jobs:
  deploy-prod:
    environment:
      name: production
      url: https://myapp.company.com
    runs-on: ubuntu-latest
    steps:
      - name: Deploy
        run: ./deploy-prod.sh
```

4. Flow: → Developer pushes to main → workflow runs build + test automatically → Reaches deploy-prod job → PAUSES with 'Waiting for approval' → Senior engineer gets email + GitHub notification → Reviews the deployment details → Approves → deploy-prod step executes → If rejected: workflow fails with 'Rejected by reviewer'

Q3: Scenario: Compare Jenkins pipeline with GitHub Actions for deploying Terraform infrastructure. When would you choose each?

Jenkins Terraform Pipeline:

```groovy
pipeline {
    agent { label 'terraform-agent' }   // EC2 with Terraform installed
    stages {
        stage('Plan') { steps { sh 'terraform plan' } }
        stage('Approve') { steps { input 'Apply?' } }
        stage('Apply') { steps { sh 'terraform apply -auto-approve' } }
    }
}
```

→ Choose Jenkins when: On-premise infra, compliance requirement, complex shared library pipelines

GitHub Actions Terraform Workflow:

```yaml
jobs:
  terraform:
    runs-on: self-hosted   # EC2 in your AWS account (OIDC auth — no hardcoded keys!)
    steps:
      - uses: hashicorp/setup-terraform@v2
      - run: terraform init && terraform plan
    environment: production   # manual approval here
```

→ Choose GitHub Actions when: Already using GitHub, cloud-native, simpler setup

Key GitHub Actions advantage for Terraform: OIDC authentication — no AWS access keys needed:

```yaml
uses: aws-actions/configure-aws-credentials@v2
with:
  role-to-assume: arn:aws:iam::123456789012:role/github-actions-role
  aws-region: us-east-1
```

→ IAM Role assumed via OIDC — most secure approach, zero long-lived credentials

Q4: Scenario: GitHub Actions is showing 'Error: Billing: out of free minutes'. Development has stopped. What are your options?

Immediate options:

1. Add payment method: GitHub → Settings → Billing → Add credit card Pay-as-you-go: $0.008/minute (Linux), $0.016/minute (Windows)
2. Self-hosted runners (free alternative): Add your own EC2 instances as runners — no minute limit `jobs: { run: runs-on: self-hosted }`
3. Optimize workflows to use fewer minutes: → Add path filters: only trigger on changes to relevant files `on: push: paths: ['src/**', 'pom.xml']` → Cache dependencies (Maven/npm) — saves minutes on dependency download

```yaml
uses: actions/cache@v3
with: { path: ~/.m2, key: ${{ hashFiles('**/pom.xml') }} }
```

→ Reduce parallel jobs → Use smaller runners (ubuntu-latest uses fewer minutes than macos)

4. GitHub Pro/Team/Enterprise: includes more free minutes (3,000-50,000/month)

---

12. Quick Reference — Commands & Cron Syntax

Jenkins Service Commands

COMMAND	ACTION
`sudo systemctl start jenkins`	Start Jenkins service
`sudo systemctl stop jenkins`	Stop Jenkins service
`sudo systemctl restart jenkins`	Restart Jenkins (after config changes)
`sudo systemctl status jenkins`	Check Jenkins service status
`sudo systemctl enable jenkins`	Auto-start Jenkins on server reboot
`jenkins --version`	Show Jenkins version
`cat /var/lib/jenkins/secrets/initialAdminPassword`	Get initial admin password
`sudo tail -f /var/log/jenkins/jenkins.log`	Watch Jenkins logs in real-time

Cron Expression Reference

EXPRESSION	MEANING
`* * * * *`	Every minute (5 stars = every minute)
`0 * * * *`	Every hour at minute 0
`0 2 * * *`	Every day at 2:00 AM
`0 2 * * 1`	Every Monday at 2:00 AM
`0 8,12,16 * * 1-5`	8AM, 12PM, 4PM on weekdays
`H/15 * * * *`	Every 15 minutes (H = hash for load distribution)
`H H * * *`	Once per day at a random hour
`@hourly`	Shorthand for `0 * * * *`
`@daily`	Shorthand for `0 0 * * *`
`@weekly`	Shorthand for `0 0 * * 0`

GitHub Actions Key Syntax

SYNTAX	PURPOSE
`on: push: branches: [main]`	Trigger on push to main branch
`on: pull_request: branches: [main]`	Trigger on PR targeting main
`on: schedule: - cron: '0 2 * * *'`	Schedule trigger — nightly at 2AM
`runs-on: ubuntu-latest`	Use GitHub-hosted Linux runner
`runs-on: self-hosted`	Use your own EC2/server as runner
`uses: actions/checkout@v4`	Checkout repository code
`${{ secrets.MY_SECRET }}`	Reference a repository secret
`needs: build-job`	This job waits for build-job to complete
`environment: production`	Target environment (enables approval gates)
`when: manual` (GitLab CI)	Requires manual click to run

DKR

PART 06 OF 08

Docker

Containerization — images, containers, networking, volumes, and Compose.

Document Legend: 💡 Blue = Layman | 📝 Green = Theory | 🐳 Teal = Docker Concept | 🏛️ Purple = Architecture | 🎯 Yellow = Interview Q&A | ⚠️ Warning (Dark Background = Commands/Code)

1. Monolithic vs Microservices Architecture

Before understanding Docker, we need to understand why we need it. This comes down to how applications are architected. The two main approaches are Monolithic and Microservices — and the choice between them directly drives the adoption of containers.

1.1 Monolithic Architecture

A Monolithic application is a single, tightly-coupled unit where all components — user interface, business logic, database layer, and integrations — are bundled together and deployed as one large package. All functions run in a single process on a single server.

📝 **Theory & Key Points:**

- All parts of the application are in one codebase: one deployment, one database.
- Examples: Early Facebook, early Amazon, traditional banking applications.
- Works well for small teams and simple applications — fast to start, easy to develop initially.
- Problems emerge at scale: a bug in one module crashes the WHOLE application.
- Scaling is wasteful: even if only the payment module needs more resources, you must scale the entire app.
- Deployment is risky: changing one feature requires redeploying the entire application.
- Technology lock-in: the entire app is written in one language/framework.

1.2 Microservices Architecture

Microservices break an application into small, independent services — each responsible for a specific business function. Each service runs in its own process, can be deployed independently, and communicates with others via APIs or message queues. This is where Docker becomes essential — each microservice runs in its own container.

💡 **Layman Explanation:**

- Monolith = a giant factory where ALL machines are in ONE building. One fire shuts everything down.
- Microservices = many small specialized workshops. Each can work independently. One fire is contained.
- Docker = the shipping container standard. Each microservice is packed into its own container.
- Kubernetes = the container ship captain. Manages hundreds of containers across multiple ships (servers).

ASPECT	MONOLITHIC	MICROSERVICES
Deployment	One large package deployed together	Each service deployed independently
Scaling	Scale everything, even what isn't busy	Scale only the service under load
Technology	Single language/framework for all	Each service can use the best tool
Failure Impact	One bug can crash everything	Failure isolated to one service
Team Size	Works for small teams	Better for large teams (each owns a service)
Development Speed	Fast initially, slows down at scale	Slower to set up, faster at scale
Database	One shared database	Each service has its own database
Complexity	Simple to start	More complex infrastructure (Docker, K8s)
Example	Traditional banking app	Netflix, Amazon, Uber

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your company's e-commerce app is a monolith. Every deployment takes 2 hours and any bug brings down the entire store. Management asks you to propose a solution. What do you recommend? Recommend a phased migration to Microservices with Docker:

Phase 1 — Identify service boundaries:

- Break the monolith by business domain: User Service, Product Service, Order Service, Payment Service
- Each service gets its own codebase, database, and Docker container

Phase 2 — Containerize (Docker):

- Each service wrapped in a Dockerfile → Docker image → Container
- Services communicate via REST APIs or message queues (RabbitMQ/Kafka)

Phase 3 — Orchestrate (Kubernetes):

- K8s manages container deployment, scaling, health checks
- If Payment service gets 10× traffic on sale day: only scale Payment pods
- Other services remain unaffected

Benefit: Payment team deploys their service independently without affecting Product or User teams. Deployment time: 2 hours → 5 minutes per service.

Q2: Scenario: When should you NOT use microservices and stay with a monolith? Stay monolithic when:

1. SMALL TEAM: If you have 2-5 developers, microservices infrastructure (Docker, K8s, API gateway, service mesh) is overkill
2. EARLY STAGE: Startup discovering product-market fit — move fast, the overhead isn't worth it
3. SIMPLE DOMAIN: The application doesn't have clear, separable business boundaries
4. INSUFFICIENT DEVOPS MATURITY: Microservices require CI/CD, containerization, monitoring, distributed tracing — if the team can't manage this, microservices create more problems than they solve
5. LOW TRAFFIC: If you have 100 users, scaling individual services is unnecessary

Rule of thumb: Start monolith, extract services when pain points (scaling, team, deployment frequency) make it necessary. Amazon and Netflix both started as monoliths.

2. What is Docker?

Docker is an open-source **containerization platform** created by Solomon Hykes and launched in 2013. It allows you to **package an application with all its dependencies — code, runtime, libraries, environment variables, and configuration — into a standardized unit called a container**. Containers are portable, lightweight, and consistent across any environment: your laptop, a CI/CD server, or a production cloud instance.

The problem Docker solves: "It works on my machine but not in production." Before Docker, different environments (dev laptop, staging server, production) had different OS versions, different library versions, different configurations. Docker eliminates this by packaging the application WITH its environment. If it works in the container on your laptop, it works identically anywhere.

Fig 1: Docker Containerization — Multiple App Containers on One EC2 with Kubernetes Orchestration. The diagram contrasts traditional Virtualization (a laptop connecting to an AWS Data Centre full of separate EC2 instances, each with its own OS + HW, labeled "container count depends on host (ec2) hardware" and "if I want to deploy 100 applications? minimum 100 if HA requires 200 this count will be increased — 100 LB and 100 ASG required — if app count increase Virtualization concept is not recommended to manage the infra and expensive") against a Docker-based layout: two EC2 stacks each running 4 "APP" containers (APP1–APP4), each with its own thin OS/HW slice (~25mb), sitting on top of a shared docker layer, then EC2/OS, then "AWS Hardware" at the base. Arrows show "Decrease container HW" / "increase host HW" and "To create many containers".

Fig 2: Docker + Kubernetes Architecture — HA Setup with Containers Across Multiple AZs. Two EC2 hosts (us-east-1a and us-east-1b) each running two rows of 4 app containers on top of Docker, connected upward to a central "kubernetes" icon labeled "Orchestration / HA / Scalability" — showing Kubernetes managing containers spread across multiple Availability Zones. Below that, a second small diagram shows a stick-figure "Performance" user pointing at an APP1/OS/HW stack connected to "dockerhub" (labeled "light weight", "No dependencies", "Very less dependencies") which links to a "Docker file" box listing: base image (without dependencies), install python, install dependencies, copy the code, run the code.

🐳 **Docker Concept:**

Docker = shipping container for software.

- A physical shipping container holds ANY goods (electronics, clothes, food) in a standard box.
- A Docker container holds ANY application (Python, Java, Node.js) in a standard package.
- A ship (Kubernetes) can carry hundreds of containers without caring what's inside.
- Ports (Kubernetes nodes/EC2) load and unload containers exactly as the captain (K8s master) commands.

2.1 Docker Architecture — How It Works Internally

🏛️ **Architecture:**

Docker uses a CLIENT-SERVER architecture:

Docker Client (docker CLI) → Docker Daemon (dockerd)

- `docker run nginx` / `docker build -t myapp .` / `docker push myapp:v1` — sent to the daemon
- Docker Daemon receives commands via REST API
- Docker Daemon manages: images, containers, networks, volumes
- Docker Daemon does the actual work

↓

Docker Registry (Docker Hub / ECR / Private Registry)

- Stores Docker images
- Public: hub.docker.com (nginx, ubuntu, mysql)
- Private: AWS ECR, your own registry

🏛️ **Container Runtime: containerd / runc**

- The actual low-level tool that creates and runs containers.
- Docker Daemon talks to containerd, which talks to runc (Linux namespaces + cgroups).

2.2 VM vs Container vs Container Architecture

Fig 3: Custom EC2 vs Container — Traditional VM Requires Full OS per App vs Shared OS in Containers. Top half "CUSTOM EC2": an Application + AMI(-OS) + HW stack transforms into "Application + Java + Java Dependency + Default Dependency" sitting on "AMI (-OS) By Default AWS" on "HOST OS". Bottom half "WITH CONTAINER": the same Application + AMI(-OS) + HW stack transforms into four separate slim containers (Application / AMI(-OS)/Libraries-Deps / OS Files each) all sharing one "OS", sitting on "Docker HUB" then "HW", labeled "CUSTOMIZED WHAT YOU WANT".

ASPECT	VIRTUAL MACHINE (EC2)	DOCKER CONTAINER
Size	GBs — full OS per VM	MBs — shares host OS kernel
Startup Time	Minutes — boots full OS	Seconds — process-level start
OS	Own OS kernel per VM	Shares host OS kernel (Linux)
Isolation	Strong — separate kernel	Process-level — namespaces/cgroups
Portability	Tied to hypervisor (Xen, KVM)	Runs identically anywhere Docker is installed
Resource Usage	High — OS overhead per VM	Low — minimal per-container overhead
Use Case	Long-running stateful services	Stateless microservices, CI/CD jobs
Count per host	~10-50 VMs (AWS EC2)	Hundreds of containers on one EC2
Cost (AWS)	1 EC2 per app = expensive	Many apps on 1 EC2 = cost efficient

📝 **Theory & Key Points:**

- Containers are NOT mini VMs. They share the HOST OS kernel — no separate OS per container.
- Linux namespaces provide isolation: each container has its own PID, network, filesystem, user namespace.
- cgroups (control groups) limit resources: how much CPU, memory, disk I/O each container can use.
- A container is simply a PROCESS running with namespaces and cgroups applied.
- Docker requires Linux kernel. On Mac/Windows, Docker Desktop runs a lightweight Linux VM underneath.
- An EC2 can run hundreds of containers (limited by RAM and CPU, not by OS overhead).
- Containers start in milliseconds — the application process starts directly, no OS boot sequence.

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your team wants to run 100 microservices. Management says "buy 100 EC2 instances". What is the cost-effective Docker-based alternative? With traditional VMs: 100 EC2 t3.micro = ~$750/month. Each needs its own OS, patching, maintenance.

With Docker + fewer EC2s:

- 10 × t3.large EC2 instances (~$600/month)
- Each EC2 runs 10 containers (100 total)
- Docker handles isolation between containers on same host
- Kubernetes manages placement, health, scaling

Additional savings:

- No 100 separate OSes to maintain and patch
- Containers start in seconds (not minutes like VMs)
- When one microservice needs more resources, K8s moves other containers to different nodes
- At 2 AM: scale down to 3 EC2 nodes. At 9 AM: scale up automatically

Savings: 60% cost reduction. Operations: managed by K8s, not manual SSH.

Q2: Scenario: A developer says "Docker containers aren't secure — they share the OS kernel. Let's use VMs." How do you respond? The developer makes a valid point but overestimates the risk in typical scenarios.

Container security mechanisms:

1. Linux Namespaces: process (PID), network, filesystem, user are all isolated
2. cgroups: resource limits — a container can't consume all CPU/memory
3. Seccomp profiles: restrict dangerous system calls
4. AppArmor/SELinux: mandatory access control
5. Read-only containers: rootfs mounted read-only
6. Non-root user: don't run containers as root (USER directive in Dockerfile)

When VMs are better:

- Different OS kernels needed (Windows + Linux on same host)
- Hardcore multi-tenant isolation (shared hosting, public cloud)
- Applications that need direct hardware access (GPU, network cards)

Hybrid (best of both):

- Run containers on separate EC2 instances (container hosts) — physical isolation at host level
- AWS ECS/EKS: each customer's workload on separate EC2 nodes

---

3. Docker Installation

Docker is available on Linux, macOS, and Windows. For production on AWS, the standard is Amazon Linux 2 or Amazon Linux 2023 running on EC2. The installation process installs Docker Engine (daemon) and the Docker CLI.

3.1 Amazon Linux 2023 / Amazon Linux 2

BASH

```
# Switch to root
sudo su -

# Install Docker
sudo yum install docker -y

# Start Docker service
sudo systemctl start docker

# auto-start on reboot
sudo systemctl enable docker

# verify running
sudo systemctl status docker

# Check Docker version
docker --version
docker version   # full details (API version, Go version, Engine)
```

3.2 Allow Non-Root User Access

By default, Docker requires root/sudo for every command. To allow ec2-user (or any user) to run Docker without sudo:

```bash
# Create docker group (usually auto-created during install)
sudo groupadd docker

# Add current user to docker group
sudo usermod -aG docker ec2-user

# Apply group changes (log out and back in, OR run:)
newgrp docker

# Alternative: give docker socket access
sudo chmod 666 /var/run/docker.sock

# Verify -- should run without sudo
docker ps
```

⚠️ **Important Warning:**

`chmod 666 /var/run/docker.sock` is convenient but a security risk in production (any user can run Docker). The recommended approach is: `sudo usermod -aG docker <username>` + logout + login. In production: use IAM roles + Docker socket proxy to restrict access to Docker daemon. The docker group gives root-equivalent power — only add trusted users.

🎯 Scenario-Based Interview Questions

Q1: Scenario: After installing Docker and running "docker ps", you get "permission denied while trying to connect to Docker daemon socket". How do you fix it? Error: `Got permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock`

Fix Option 1 (recommended — persistent):

```bash
sudo usermod -aG docker $USER
newgrp docker   # apply without logout
docker ps       # test
```

Fix Option 2 (quick but less secure):

```bash
sudo chmod 666 /var/run/docker.sock
```

Fix Option 3 (one-time): Use sudo with every docker command: `sudo docker ps`

Why this happens: Docker daemon runs as root and the socket file (`/var/run/docker.sock`) is owned by `root:docker`. Non-root users not in the "docker" group cannot access it.

Verify group membership: `groups $USER` — should show "docker" in the list

Q2: Scenario: You installed Docker but "systemctl start docker" fails. What do you diagnose? Diagnose step by step:

1. Check service status: `sudo systemctl status docker` — look for specific error messages in the output
2. Check logs: `sudo journalctl -u docker -n 50 --no-pager`
3. Common causes:
   - Port conflict: another process using Docker's ports
   - Previous Docker installation conflict: remove old versions first — `sudo yum remove docker docker-client docker-common`
   - Storage driver issue: check `/etc/docker/daemon.json`
   - Low disk space: `df -h` — Docker needs space in `/var/lib/docker`

- SELinux conflict: `sudo setenforce 0` (temporarily)
- Missing dependencies: `sudo yum install -y device-mapper-persistent-data lvm2`

4. Fresh install fix:

```bash
sudo yum remove docker* -y
sudo rm -rf /var/lib/docker
sudo yum install docker -y
sudo systemctl start docker
```

---

4. Docker Images & Containers

Understanding the difference between an Image and a Container is the most fundamental Docker concept. They have a relationship similar to a class and an object in object-oriented programming: an image is the template/blueprint, and a container is the running instance created from that template.

🐳 **Docker Concept:**

DOCKER IMAGE:

- A READ-ONLY template that contains everything needed to run an application.
- Includes: OS files, runtime (Python/Java/Node), application code, dependencies, configuration.
- Built from a Dockerfile. Stored in Docker Hub or a private registry.
- Like a snapshot/blueprint. Like an AWS AMI for an EC2 instance.
- Images are made of LAYERS — each Dockerfile instruction creates a cached layer.

DOCKER CONTAINER:

- A RUNNING INSTANCE of a Docker image — a live process with its own isolated environment.
- Multiple containers can be created from the same image, each independent.
- Has a writable layer on top of the read-only image layers.
- Like a running server — has network, filesystem, processes.
- Containers are ephemeral by default — data is lost when container is removed (use volumes to persist).

4.1 Image Commands

BASH

```
# Pull image from Docker Hub (public registry)
docker pull nginx
docker pull ubuntu:20.04
docker pull mysql:8.0
docker pull python:3.12-slim

# List all local images
docker images
docker image ls

# Inspect image details (layers, env vars, cmd, entrypoint)
docker inspect nginx
docker image inspect nginx

# Image history (see each layer)
docker history nginx

# Search for images on Docker Hub
docker search nginx

# Delete image
docker rmi nginx                # delete if no container uses it
docker rmi nginx -f             # force delete even if container exists
docker image prune              # remove all dangling (untagged) images
docker system prune -a          # remove ALL unused images, containers, networks
```

4.2 Container Commands — The Complete Reference

BASH

```
# ── CREATE & RUN ──
docker run ubuntu                          # create + run (stops immediately if no
foreground process)
docker run -it ubuntu /bin/bash            # interactive terminal -- enter the
container
docker run -dt ubuntu                      # detached + pseudo-TTY (RECOMMENDED --
runs in background)
docker run -dt --name myapp ubuntu         # with custom name
docker run -dt -p 8080:80 nginx            # port mapping: host:container
docker run -dt -e MY_VAR=value ubuntu      # set environment variable
docker run -dt -v myvolume:/data ubuntu    # mount volume
docker run -dt --memory='512m' ubuntu      # limit memory
docker run -dt --cpus='1.5' ubuntu         # limit CPU (1.5 cores)
docker run --rm ubuntu echo hello          # auto-remove container after it exits
docker run -dt --restart=always nginx      # auto-restart if container stops

# ── INSPECTION ──
docker ps                                  # list RUNNING containers
docker ps -a                               # list ALL containers (running + stopped)
docker ps -a | grep Exited                 # show only stopped containers
docker inspect myapp                       # full JSON details of container
docker logs myapp                          # view container logs
docker logs -f myapp                       # follow/stream logs (like tail -f)
docker logs --tail 100 myapp               # last 100 log lines
docker stats                               # real-time CPU/memory/network usage
docker top myapp                           # processes running inside container

# ── ACCESS CONTAINER ──
docker exec -it myapp /bin/bash            # open bash inside RUNNING container
docker exec -it myapp sh                   # if bash not available, use sh
docker exec myapp cat /etc/nginx/nginx.conf  # run single command inside container

# ── LIFECYCLE ──
docker start myapp                         # start a stopped container
docker stop myapp                          # graceful stop (SIGTERM -> SIGKILL after
10s)
```

```
docker restart myapp                    # stop + start
docker kill myapp                       # immediate stop (SIGKILL)
docker pause myapp                      # freeze container (suspend processes)
docker unpause myapp                    # resume frozen container

# ── DELETE ──
docker rm myapp                         # remove STOPPED container
docker rm -f myapp                      # force remove (even if running)
docker container prune                  # remove ALL stopped containers
docker rm -f $(docker ps -aq)           # force remove ALL containers
```

4.3 The Three Container Run Modes Explained

FLAG	BEHAVIOR	WHEN TO USE
(no flag) `docker run ubuntu`	Runs in foreground. You see output. Container exits when process ends.	Not recommended — blocks terminal
`-it` (interactive)	Opens interactive terminal. You CAN type commands inside container. Exits when you type 'exit'.	Debugging, exploration, manual testing
`-dt` (detached, RECOMMENDED)	Runs in background. Terminal is free. Container keeps running. You attach later with exec.	All production and development containers

📝 **Theory & Key Points:**

- Ctrl+P, Ctrl+Q: Detach from a container without stopping it (when connected via exec or run -it).
- 'exit' typed inside container: Exits the shell. If the shell was PID 1, the container STOPS.
- `docker run --rm`: Container is automatically deleted after it exits — great for one-off tasks.
- `docker run --restart=always`: Container restarts automatically if it crashes or EC2 reboots.
- `ps -ef` inside a container shows very few processes — containers are lightweight by design.
- Each container has its own IP address in the Docker bridge network (172.17.0.x by default).

🎯 Scenario-Based Interview Questions

Q1: Scenario: You run "docker run -dt nginx" and it works. But later you run "docker run ubuntu" and the container immediately stops. Why? Root cause: `ubuntu:latest` image has no long-running foreground process. The default CMD is 'bash', which exits immediately when there's no interactive terminal.

Fix:

1. Run with -dt flag: `docker run -dt ubuntu` — the -t flag allocates a pseudo-TTY, keeping the container alive
2. Override the command: `docker run -d ubuntu sleep infinity` — keeps the container alive by running "sleep infinity"
3. Create a Dockerfile with a foreground process: `CMD ["sh", "-c", "while true; do sleep 3600; done"]`

Why nginx stays running: nginx image has `CMD ["nginx", "-g", "daemon off;"]`. The "daemon off" flag runs nginx in foreground — container stays alive as long as nginx runs.

Key rule: A container lives ONLY as long as its main process (PID 1) runs.

Q2: Scenario: How do you run 100 Jenkins containers on one EC2 so each developer has their own isolated Jenkins? Run 100 containers from the jenkins/jenkins image, each on a different port:

```bash
# Jenkins containers for team members on different ports
for port in $(seq 8081 8180); do
  name="jenkins-dev-$port"
  docker run -dt \
    --name $name \
    -p $port:8080 \
    --restart=always \
    -v jenkins-data-$port:/var/jenkins_home \
    jenkins/jenkins:lts
  echo "Jenkins for dev-$port running at http://IP:$port"
done
```

Each developer accesses: http://ec2-ip:8081, http://ec2-ip:8082 etc. Each container has its own volume (persistent data). All 100 containers share the same EC2 hardware.

Compare:

- Traditional: 100 EC2 instances for 100 Jenkins = $800+/month
- Docker: 1 large EC2 (16GB RAM) for 100 Jenkins containers = ~$120/month

Q3: Scenario: "docker stop container1" takes 30 seconds and seems slow. How do you speed it up? `docker stop` sends SIGTERM to the container's main process and waits 10 seconds (default timeout) for graceful shutdown, then sends SIGKILL.

If taking 30 seconds: the app inside is ignoring SIGTERM and Docker is waiting the full timeout before SIGKILL.

Fix Options:

1. Reduce stop timeout: `docker stop --time=5 container1` (wait only 5 seconds)
2. Use `docker kill` (immediate, no waiting): `docker kill container1` (sends SIGKILL directly — no graceful shutdown)
3. Fix the application to handle SIGTERM: application should catch SIGTERM and shut down cleanly. Add to Dockerfile: `STOPSIGNAL SIGTERM` (or SIGINT for some apps)
4. For bulk killing: `docker rm -f $(docker ps -aq)` (force removes all containers immediately)

Q4: Scenario: A container is "healthy" in docker ps but the application inside isn't responding. How do you diagnose? Docker's default "healthy" means the container is RUNNING (PID 1 alive), not that the APP inside is healthy.

Diagnosis steps:

1. Check logs: `docker logs -f myapp | tail -50` — look for errors, exceptions, crash loops
2. Check processes inside: `docker top myapp` — is the application process actually running?
3. Enter the container: `docker exec -it myapp /bin/bash` — try `curl localhost:8080` or `wget -O- localhost:8080`
4. Check resource usage: `docker stats myapp` — is the app consuming 100% CPU? Out of memory?
5. Implement Docker HEALTHCHECK in Dockerfile:

DOCKERFILE

```
HEALTHCHECK --interval=30s --timeout=3s \
  CMD curl -f http://localhost:8080/health || exit 1
```

Now `docker ps` shows "healthy" or "unhealthy" based on actual app response. 6. Check Docker events: `docker events --filter container=myapp`

---

5. Dockerfile — Building Custom Images

A Dockerfile is a plain text file containing a series of instructions that Docker reads top-to-bottom to build a custom Docker image. Each instruction creates a new layer in the image. Layers are cached — if a layer's instruction hasn't changed, Docker reuses the cached layer, making rebuilds fast. Dockerfiles are the "recipe" for creating your application's environment.

5.1 Dockerfile Instructions Reference

INSTRUCTION	WHAT IT DOES
FROM	REQUIRED FIRST: specifies the base image. Every Dockerfile must start with FROM. `FROM scratch` = empty image.
RUN	Executes commands during IMAGE BUILD. Runs in a shell (sh -c). Each RUN creates a new layer. Used for installations.
CMD	Default command when container STARTS. Can be overridden by docker run argument. Only last CMD is used.
ENTRYPOINT	Main executable of the container. CMD arguments are appended to it. Hard to override (needs --entrypoint flag).
WORKDIR	Sets working directory for RUN, CMD, ENTRYPOINT, COPY, ADD. Created if it doesn't exist.
COPY	Copies files/directories from BUILD CONTEXT (local machine) into image. Preferred over ADD for local files.
ADD	Like COPY but also supports: URL downloads and automatic tar extraction. Use COPY unless you need ADD features.
EXPOSE	Documents which port the container listens on. Does NOT actually publish the port. Informational only.
ENV	Sets environment variables available at runtime AND build time. Seen by running containers.
ARG	Build-time variable only. Not available after image is built. Used for build arguments passed via --build-arg.
VOLUME	Creates a mount point. Tells Docker this path should be a volume (persisted outside container).
USER	Sets the user for subsequent RUN, CMD, ENTRYPOINT commands. Use to run as non-root for security.
HEALTHCHECK	Defines how to test if the container is healthy. 'docker ps' shows healthy/unhealthy.
LABEL	Adds metadata key-value pairs to the image (maintainer, version, description).
ONBUILD	Instruction added to image that runs when image is used as a base for another build.

5.2 Complete Dockerfile Examples

Python Flask Application

DOCKERFILE

```
# Best practice: multi-stage for smaller final image
# Build stage: install dependencies
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy only requirements first (leverage layer caching)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Final stage: just the app + installed dependencies
FROM python:3.9-slim

WORKDIR /app

# Copy installed packages from builder
COPY --from=builder /usr/local/lib/python3.9 /usr/local/lib/python3.9

# Copy application code
COPY . .

# Add metadata
LABEL maintainer='veera@company.com' version='1.0'

# Run as non-root user (security best practice)
RUN useradd -m appuser
USER appuser

EXPOSE 5000

# Health check
HEALTHCHECK --interval=30s --timeout=5s \
  CMD curl -f http://localhost:5000/health || exit 1

ENTRYPOINT ['python']
CMD ['app.py']
```

Node.js Application

DOCKERFILE

```
# alpine = minimal size (~5MB vs ~900MB for full node image)
FROM node:18-alpine

WORKDIR /app

# Copy package files first (caching optimization)
COPY package*.json ./

# Install production dependencies only
RUN npm ci --only=production

# Copy application code
COPY . .

# Non-root user for security
USER node

EXPOSE 3000

CMD ['npm', 'start']
```

Apache Web Server (Two Approaches)

DOCKERFILE

```
# Approach 1: Use official httpd image (simpler)
FROM httpd:2.4
COPY ./public-html/ /usr/local/apache2/htdocs/
EXPOSE 80

# Approach 2: Build from Ubuntu scratch (more control)
FROM ubuntu:22.04

RUN apt-get update && \
    apt-get install -y apache2 && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

COPY index.html /var/www/html/

EXPOSE 80

CMD ['/usr/sbin/apache2ctl', '-D', 'FOREGROUND']
```

MySQL Database

DOCKERFILE

```
FROM mysql:8.0

# Environment variables for MySQL initialization
ENV MYSQL_ROOT_PASSWORD=SecureRoot123
ENV MYSQL_DATABASE=myapp
ENV MYSQL_USER=appuser
ENV MYSQL_PASSWORD=AppPass456

EXPOSE 3306

# SQL file copied here is AUTO-EXECUTED on first container start
COPY init.sql /docker-entrypoint-initdb.d/

# mysqld is the default CMD in official MySQL image
# No need to specify CMD here
```

Multi-Stage Build: Maven + Tomcat (Production Pattern)

DOCKERFILE

```
# ═══ STAGE 1: BUILD ═══
# Use Maven image to compile Java application
FROM maven:3.9-eclipse-temurin-17 AS build

WORKDIR /app

# Copy pom.xml first to cache dependency download
COPY pom.xml .
RUN mvn dependency:go-offline -B

# Copy source code and build
COPY src ./src
RUN mvn package -DskipTests

# ═══ STAGE 2: RUNTIME ═══
# Only copy the built WAR file -- no Maven, no source code
FROM tomcat:10.1-jdk17

# Remove default webapps
RUN rm -rf /usr/local/tomcat/webapps/*

# Copy WAR from build stage
COPY --from=build /app/target/webapp.war /usr/local/tomcat/webapps/ROOT.war

EXPOSE 8080

# tomcat image has default CMD -- starts Tomcat automatically
```

5.3 CMD vs ENTRYPOINT — The Critical Difference

FEATURE	CMD	ENTRYPOINT
Purpose	Default command or default arguments	The main executable — the process the container IS
Override	Fully replaced by: `docker run image <command>`	Only arguments appended; use `--entrypoint` to override executable
Combination	When used together: CMD provides default ARGS to ENTRYPOINT	ENTRYPOINT runs, CMD provides its default arguments
Best Use	Optional default commands, flexible behavior	Mandatory fixed executable (run this tool always)
Example	`CMD ['python', 'app.py']`	`ENTRYPOINT ['python']; CMD ['app.py']`

DOCKERFILE

```
# CMD -- fully overridable
FROM ubuntu:latest
CMD ['echo', 'Hello World']

docker run myimage                  # -> Hello World
docker run myimage echo 'Override'  # -> Override (CMD replaced)

# ENTRYPOINT -- executable is fixed, args can be added
FROM ubuntu:latest
ENTRYPOINT ['echo', 'Hello']

docker run myimage                  # -> Hello
docker run myimage World            # -> Hello World (World APPENDED)

# CMD + ENTRYPOINT combination (most flexible)
FROM ubuntu:latest
ENTRYPOINT ['echo']
CMD ['Default Message']

docker run myimage                  # -> Default Message (CMD used as args to
ENTRYPOINT)
docker run myimage 'Custom'         # -> Custom (CMD overridden, ENTRYPOINT unchanged)
```

5.4 Dockerfile Best Practices

📝 **Theory & Key Points:**

1. Use official base images — smaller, tested, security-patched (python:3.12-slim, node:18-alpine).
2. Use specific version tags, not 'latest' — ensures reproducible builds (`FROM node:18.16.0-alpine3.17`).
3. Combine RUN commands with `&&` to reduce layers: `RUN apt update && apt install -y nginx && rm -rf /var/lib/apt/lists/*`
4. Copy requirements/package.json BEFORE source code — leverages layer caching for faster rebuilds.
5. Use .dockerignore to exclude: `.git` , `node_modules` , `__pycache__` , `*.log` (reduces build context size).
6. Use multi-stage builds — keep final image small by discarding build tools.
7. Run as non-root user — USER directive for security.
8. Set HEALTHCHECK — enables container health monitoring in Docker/K8s.
9. Use WORKDIR instead of `RUN mkdir` + `cd` — cleaner and explicit.
10. One process per container — each container should do ONE thing well.

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your Docker image is 2.1 GB and takes 15 minutes to build. How do you optimize it? Optimization strategies:

1. USE ALPINE/SLIM BASE: `FROM node:18` → 900MB; `FROM node:18-alpine` → 100MB. Use node:18-alpine unless you need glibc.
2. MULTI-STAGE BUILD: Build stage: `FROM maven:3.9` (600MB) — compile your app. Final stage: `FROM eclipse-temurin:17-jre-alpine` (80MB) — just the JRE + JAR. Final image: ~100MB instead of 700MB.
3. LAYER CACHING: Copy package.json FIRST, then `npm install`, THEN copy source code. `npm install` layer is cached — rebuild only when package.json changes.
4. REMOVE BUILD ARTIFACTS:

```dockerfile
RUN apt-get install -y build-essential && \
    make && \
    apt-get remove -y build-essential && \
    apt-get autoremove -y && \
    rm -rf /var/lib/apt/lists/*
```

5. .dockerignore file:

```text
.git
node_modules
*.log
```

Prevents unnecessary files in build context.

Result: 2.1 GB → 80-150 MB, build time: 15 min → 2 min (with caching)

Q2: Scenario: Your Dockerfile has "RUN apt-get install -y python3 nodejs java" in 3 separate RUN commands. What is wrong with this? Problem: Each RUN instruction creates a SEPARATE LAYER in the image. 3 separate RUN commands = 3 layers = larger image, slower pull.

Also problematic: apt-get update and apt-get install are in separate RUN commands — package lists may be stale (cache from previous build used for apt-get update layer).

Fix — combine into ONE RUN:

```dockerfile
RUN apt-get update && \
    apt-get install -y \
      python3 \
      python3-pip \
      nodejs \
      default-jdk && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*
```

Benefits:

- One layer instead of 3
- apt-get update and install always run together — no stale cache
- Cleanup in same layer removes apt cache before layer is committed
- Smaller image size

Q3: Scenario: You have a Dockerfile. Building it with "docker build" takes 10 minutes every time because npm install downloads all packages. How do you fix caching? Problem: Source code COPY invalidates the cache, forcing npm install every time.

Bad order (your current Dockerfile):

```dockerfile
COPY . .        # copies everything
RUN npm install # runs every time any file changes
```

Fixed order (cache-optimized):

```dockerfile
# Step 1: Copy ONLY package files first
COPY package.json package-lock.json ./

# Step 2: Install dependencies (THIS IS NOW CACHED)
RUN npm install

# Step 3: Copy source code AFTER dependencies
COPY . .
```

How it works:

- Layer 1: `FROM node:18-alpine`
- Layer 2: `WORKDIR /app`
- Layer 3: `COPY package*.json` → if package.json unchanged: CACHED
- Layer 4: `RUN npm install` → if layer 3 cached: THIS IS CACHED
- Layer 5: `COPY . .` → only this layer rebuilds when source changes

Result: 10 minutes → 30 seconds for code changes (npm install skipped).

---

6. Docker Image Registry & Push

A Docker Registry is a server that stores and distributes Docker images. The most popular is Docker Hub (hub.docker.com) — the public registry with millions of official and community images. For enterprise use, AWS ECR (Elastic Container Registry) is the standard choice for storing private images on AWS, with IAM-based access control and automatic scanning for vulnerabilities.

6.1 Docker Hub — Public & Private

BASH

```
# Step 1: Login to Docker Hub
docker login
# Enter: Username, Password (or use access token for better security)

# Step 2: Tag your image (must match: dockerhub-username/imagename:tag)
docker tag myapp veera/myapp:v1.0
docker tag myapp veera/myapp:latest

# Step 3: Push to Docker Hub
docker push veera/myapp:v1.0
docker push veera/myapp:latest

# Anyone can pull (if public repo):
docker pull veera/myapp:v1.0

# Logout
docker logout
```

6.2 AWS ECR — Private Registry

```bash
# Pre-requisites:
# 1. AWS CLI installed and configured (aws configure)
# 2. IAM user/role with ECR permissions: ecr:GetAuthorizationToken,
#    ecr:BatchCheckLayerAvailability, ecr:PutImage, etc.
# 3. Create ECR repository in AWS Console:
#    AWS -> ECR -> Create Repository -> myapp-repo

# Step 1: Authenticate Docker to ECR
aws ecr get-login-password --region ap-south-1 | \
    docker login --username AWS --password-stdin \
    992382358200.dkr.ecr.ap-south-1.amazonaws.com

# Step 2: Build your image (if not already built)
docker build -t myapp .

# Step 3: Tag for ECR (format: account.dkr.ecr.region.amazonaws.com/repo:tag)
docker tag myapp:latest \
    992382358200.dkr.ecr.ap-south-1.amazonaws.com/myapp:latest

# Step 4: Push to ECR
docker push 992382358200.dkr.ecr.ap-south-1.amazonaws.com/myapp:latest

# Pull from ECR (on any EC2 with ECR access):
docker pull 992382358200.dkr.ecr.ap-south-1.amazonaws.com/myapp:latest
```

6.3 Build from GitHub URL Directly

```bash
# Build Docker image directly from a GitHub repository
# Docker clones the repo and uses its Dockerfile
docker build -t myapp https://github.com/CloudTechDevOps/project.git

# Specify branch or subdirectory:
docker build -t myapp https://github.com/org/repo.git#main:subfolder
```

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your Docker images are stored on Docker Hub (public). Your security team says production images must be private and scanned for vulnerabilities. What do you do? Migrate to AWS ECR:

1. Create ECR repository: `aws ecr create-repository --repository-name myapp --region us-east-1`
2. Enable image scanning:

```bash
aws ecr put-image-scanning-configuration \
  --repository-name myapp \
  --image-scanning-configuration scanOnPush=true
```

3. Enable lifecycle policy (auto-delete old images):

```bash
aws ecr put-lifecycle-policy --repository-name myapp \
  --lifecycle-policy-text '{"rules":[{"priority":1,"description":"Keep last 10","selection":{"tagStatus":"any","countType":"imageCountMoreThan","countNumber":10},"action":{"type":"expire"}}]}'
```

4. Update CI/CD pipeline to push to ECR instead of Docker Hub
5. For EC2/ECS/EKS: attach IAM role with `ecr:GetAuthorizationToken` + `ecr:BatchGetImage` — no hardcoded credentials needed

Security benefits:

- Private — only your AWS account has access
- IAM-controlled — fine-grained permissions
- Auto-scanning — CVE scan on every push
- Region-local — no internet egress cost for same-region pulls

Q2: Scenario: "docker push" fails with "unauthorized: authentication required". How do you fix it? Causes and fixes:

1. Not logged in: `docker login` (for Docker Hub) or `aws ecr get-login-password | docker login ...` (for ECR)
2. Wrong image tag format: Docker Hub requires `username/imagename:tag`

```bash
docker tag myapp YOURNAME/myapp:v1
docker push YOURNAME/myapp:v1
```

3. ECR token expired (tokens expire after 12 hours): re-run `aws ecr get-login-password ... | docker login ...` — add to CI/CD: run this before every push
4. IAM permissions missing: error "Not authorized to perform ecr:InitiateLayerUpload" — fix: add ECR permissions to IAM role/user: `ecr:GetAuthorizationToken`, `ecr:BatchCheckLayerAvailability`, `ecr:PutImage`, `ecr:InitiateLayerUpload`, `ecr:UploadLayerPart`, `ecr:CompleteLayerUpload`
5. Docker Hub rate limit (free tier: 100 pulls/6 hours): `docker login` with paid account OR migrate to ECR/GitHub Container Registry

---

7. Docker Networking

Docker networking allows containers to communicate with each other, with the host system, and with the outside world. Docker provides several network drivers, each with different behavior. Understanding networking is essential for multi-container applications where services (web app, database, cache) need to discover and communicate with each other.

7.1 Network Types

NETWORK TYPE	DESCRIPTION
bridge (DEFAULT)	Containers get their own IP (172.17.0.x). Can ping each other by IP. Isolated from host. Port mapping needed for external access.
host	Container shares host's network stack. Container uses host IP. No port mapping needed. No network isolation. Best performance.
none	Container has NO network — completely isolated. Only loopback interface. Used for security-sensitive workloads.
custom bridge	User-defined bridge: containers on same custom network can ping by CONTAINER NAME (DNS). Best for multi-container apps.
overlay	Multi-host networking for Docker Swarm/Kubernetes. Containers on different physical machines can communicate.
macvlan	Container gets its own MAC address on the physical network. Appears as a physical device on LAN.

7.2 Network Commands

BASH

```
# List all networks
docker network ls
# NETWORK ID    NAME      DRIVER    SCOPE
# abc123def     bridge    bridge    local
# def456abc     host      host      local
# ghi789jkl     none      null      local

# Inspect a network (see connected containers, IP ranges)
docker network inspect bridge

# Create a custom bridge network
docker network create myapp-network
docker network create --subnet=192.168.1.0/24 myapp-network

# Run containers on a custom network
docker run -dt --name webapp --network myapp-network nginx
docker run -dt --name database --network myapp-network mysql

# On custom network: containers can ping each other by NAME
docker exec -it webapp ping database   # works! DNS resolution by container name

# Connect running container to a network
docker network connect myapp-network existing-container

# Disconnect container from network
docker network disconnect myapp-network container-name

# Delete a network
docker network rm myapp-network
docker network prune   # remove all unused networks
```

7.3 Container-to-Container Communication

🏛️ **Architecture:**

BRIDGE NETWORK (default): Containers communicate by IP address.

```text
container1 (172.17.0.2) --ping 172.17.0.3--> container2
```

Problem: IPs can change when containers restart.

CUSTOM BRIDGE NETWORK (recommended for apps):

- Create: `docker network create app-network`
- Run: `docker run --network app-network --name web nginx`
- Run: `docker run --network app-network --name db mysql`
- Result: web container can reach: `ping db` — works! (Docker built-in DNS)

This is how Docker Compose works internally — creates a custom network and all services in the compose file can find each other by service name.

🎯 Scenario-Based Interview Questions

Q1: Scenario: You have two containers: "webapp" and "database". webapp needs to connect to database using hostname "database". How do you set this up? Use a custom Docker network (enables DNS resolution by container name):

BASH

```
# Step 1: Create a custom bridge network
docker network create app-network

# Step 2: Run database container on the network
docker run -dt \
  --name database \
  --network app-network \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0

# Step 3: Run webapp container on the same network
docker run -dt \
  --name webapp \
  --network app-network \
  -e DB_HOST=database \
  -e DB_PORT=3306 \
  mywebapp:v1

# webapp can now connect to MySQL using hostname 'database'
# No IP addresses needed -- Docker's internal DNS resolves 'database' to the
container's IP

# Verify:
docker exec -it webapp ping database       # should work
docker exec -it webapp nslookup database   # shows container IP
```

This is the foundation of how Docker Compose works — all services on same compose network.

Q2: Scenario: A security audit says your Docker containers can communicate with each other freely, creating a lateral movement risk. How do you address this? Default Docker bridge

network: ALL containers can reach ALL other containers. This violates the principle of least privilege.

Solution: Network segmentation with custom networks:

```bash
# Frontend network: webapp + nginx
docker network create frontend-net

# Backend network: webapp + database (NOT exposed to nginx directly)
docker network create backend-net

# nginx: only on frontend
docker run -dt --name nginx --network frontend-net nginx

# webapp: on BOTH networks (bridges frontend and backend)
docker run -dt --name webapp myapp
docker network connect frontend-net webapp
docker network connect backend-net webapp

# database: only on backend (nginx CANNOT reach database directly)
docker run -dt --name db --network backend-net mysql
```

Result:

- nginx → webapp: YES
- webapp → db: YES
- nginx → db: NO (different network — isolated)

If nginx is compromised, attacker cannot reach the database directly.

---

8. Docker Volumes — Persistent Data

By default, when a container is deleted, all data inside it is lost. Containers are ephemeral. For any data that must persist — database files, uploaded files, logs, configuration — you need Docker

Volumes. Volumes are managed by Docker and stored outside the container's filesystem, at `/var/lib/docker/volumes/` on the host.

Docker Concept:

Three ways to persist data in Docker:

1. **VOLUMES (recommended):** Docker manages the storage location. `docker run -v myvolume:/app/data myimage` Data in: `/var/lib/docker/volumes/myvolume/_data` Survives container deletion. Shareable between containers.
2. **BIND MOUNTS:** Mount a specific HOST directory into the container. `docker run -v /home/user/data:/app/data myimage` Container sees and can modify host files directly. Good for development (live code updates).
3. **tmpfs MOUNTS:** Data stored in HOST MEMORY only. Not persistent. `docker run --tmpfs /app/temp myimage` Data lost on container stop. Used for sensitive temporary data.

8.1 Volume Commands

BASH

```
# Create a named volume
docker volume create myvolume

# List all volumes
docker volume ls

# Inspect volume (shows mountpoint, creation date)
docker volume inspect myvolume
# Output: Mountpoint: /var/lib/docker/volumes/myvolume/_data

# Run container with volume mounted
docker run -dt --name app1 -v myvolume:/data ubuntu

# Create a file in the volume
docker exec -it app1 bash -c 'echo hello > /data/test.txt'

# DELETE container -- volume persists
docker rm -f app1

# Run NEW container with same volume -- data is still there!
docker run -dt --name app2 -v myvolume:/data ubuntu
docker exec -it app2 cat /data/test.txt   # -> hello

# Share volume between multiple containers
docker run -dt --name reader -v myvolume:/data:ro ubuntu   # read-only
docker run -dt --name writer -v myvolume:/data:rw ubuntu   # read-write

# Remove volume
docker volume rm myvolume
docker volume prune   # remove all unused volumes

# Check volume data on host
ls /var/lib/docker/volumes/myvolume/_data
```

8.2 Bind Mounts — Development Workflow

BASH

```
# Mount current directory into container (live code update)
docker run -dt \
    --name devapp \
    -v $(pwd):/app \
    -p 3000:3000 \
    node:18

# Now: edit files on your laptop -- changes instantly visible inside container
# No rebuild needed during development

# Useful for database data directories
docker run -dt \
    --name mysql_dev \
    -v /data/mysql:/var/lib/mysql \
    -e MYSQL_ROOT_PASSWORD=dev123 \
    mysql:8.0
```

🎯 **Scenario-Based Interview Questions**

Q1: Scenario: Your MySQL container was accidentally deleted. The database had important data. Is it gone forever? ANSWER DEPENDS ON HOW THE CONTAINER WAS STARTED:

Case A — No volume configured: `docker run -dt --name mysql mysql:8.0`

- Data was inside the container's writable layer
- Container deleted → DATA PERMANENTLY LOST
- This is why you should ALWAYS use volumes for databases!

Case B — Volume configured: `docker run -dt --name mysql -v mysql-data:/var/lib/mysql mysql:8.0`

- Data is in `/var/lib/docker/volumes/mysql-data/_data` on the HOST
- Container deleted → VOLUME STILL EXISTS
- Recover: `docker run -dt --name mysql-new -v mysql-data:/var/lib/mysql mysql:8.0`
- All data is back!

Verify: `docker volume ls` — look for "mysql-data". If volume exists: you can recover completely.

Lesson: ALWAYS add -v for databases:

- `-v mysql-data:/var/lib/mysql` (MySQL)
- `-v postgres-data:/var/lib/postgresql/data` (PostgreSQL)
- `-v mongo-data:/data/db` (MongoDB)

Q2: Scenario: You need two containers to share files in real-time. Container 1 writes reports, Container 2 reads and processes them. How do you set this up? Use a shared Docker volume:

```bash
# Create shared volume
docker volume create shared-reports

# Container 1: Report writer
docker run -dt \
  --name report-writer \
  -v shared-reports:/reports \
  myreport-app

# Container 2: Report processor
docker run -dt \
  --name report-processor \
  -v shared-reports:/input:ro \
  myprocessor-app

# Writer creates files in /reports -- they instantly appear in processor's /input
```

Verify:

```bash
docker exec report-writer ls /reports
docker exec report-processor ls /input
```

Alternatives for real-time file sharing:

- S3 bucket: both containers read/write via AWS SDK (more scalable, cloud-native)
- NFS mount: network filesystem shared across multiple hosts
- Redis: if sharing data (not files) — much faster for small data exchange

9. Docker Compose — Multi-Container Applications

Docker Compose is a tool for defining and running multi-container Docker applications using a single YAML configuration file. Instead of running multiple docker run commands with all their flags, you define all services, networks, and volumes in a `docker-compose.yml` file and start everything with one command: `docker-compose up`.

> 💡 **Layman Explanation:**
>
> Docker Compose = the conductor of an orchestra of containers.
>
> - Instead of starting each musician (container) one by one with complex instructions...
> - ...write one score (docker-compose.yml) and say "play" (docker-compose up).
> - All containers start in the right order, connected on the same network, with volumes mounted.
> - Stop everything at once: `docker-compose down`.
> - Perfect for: local development, testing, CI/CD, small deployments.

9.1 Installation

```bash
# Install Docker Compose V2 (Linux)
sudo curl -L \
  "https://github.com/docker/compose/releases/latest/download/docker-compose-$(uname -s)-$(uname -m)" \
  -o /usr/local/bin/docker-compose

sudo chmod +x /usr/local/bin/docker-compose

docker-compose version
# Docker Compose version v2.x.x

# Note: Docker Desktop (Mac/Windows) includes Compose automatically
```

9.2 Example: WordPress + MySQL

YAML

```yaml
# docker-compose.yml
version: '3.8'

services:
  database:                          # MySQL service
    image: mysql:8.0
    container_name: wordpress-db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass123
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass456
    volumes:                         # persist MySQL data
      - db-data:/var/lib/mysql
    networks:
      - app-network

  wordpress:                         # WordPress service
    image: wordpress:latest
    container_name: wordpress-app
    restart: always
    ports:
      - '8080:80'                    # access at http://host:8080
    environment:
      WORDPRESS_DB_HOST: database:3306   # uses SERVICE NAME for host
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass456
      WORDPRESS_DB_NAME: wordpress
    depends_on:
      - database                     # wait for MySQL to start first
    networks:
      - app-network

volumes:                             # named volume for data persistence
  db-data:
```

```
networks:                          # custom network for service-name DNS
  app-network:
```

9.3 Three-Tier Application

YAML

```yaml
# Full 3-tier app: nginx (proxy) + app + database
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    ports:
      - '80:80'
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
    depends_on:
      - webapp
    networks: [frontend]

  webapp:                           # build from Dockerfile in current directory
    build: .
    environment:
      DB_HOST: db
      DB_PORT: 5432
    depends_on:
      - db
    networks: [frontend, backend]

  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - postgres-data:/var/lib/postgresql/data
    networks: [backend]

volumes:
  postgres-data:

networks:                           # nginx + webapp can talk
```

```
  frontend:                          # webapp + db can talk (nginx CANNOT reach db)
  backend:
```

9.4 Docker Compose Commands

COMMAND	WHAT IT DOES
`docker-compose up`	Start all services (foreground — blocks terminal, shows logs)
`docker-compose up -d`	Start all services in detached mode (background)
`docker-compose down`	Stop and remove containers, networks (volumes preserved)
`docker-compose down -v`	Stop + remove containers, networks, AND volumes
`docker-compose ps`	List all containers managed by this compose file
`docker-compose logs`	View logs from all services
`docker-compose logs -f webapp`	Follow/stream logs from specific service
`docker-compose build`	Build/rebuild images (when using `build:` in compose file)
`docker-compose pull`	Pull latest images for all services
`docker-compose restart`	Restart all services
`docker-compose stop`	Stop services without removing containers
`docker-compose exec webapp bash`	Open shell in running service container
`docker-compose config`	Validate and view final compose configuration
`docker-compose -f prod.yml up`	Use a specific compose file name
`docker-compose scale webapp=3`	Scale a service to 3 replicas (V1 command)

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your Docker Compose app starts but the webapp container fails because the database isn't ready yet. "depends_on" didn't help. Why? Root cause: `depends_on` only waits for the database CONTAINER to START, not for MySQL to be READY to accept connections. The MySQL container starts in ~1 second but MySQL itself takes 10-15 seconds to initialize.

Fix Option 1 — Use healthcheck + condition:

```yaml
services:
  database:
    image: mysql:8.0
    healthcheck:
      test: ['CMD', 'mysqladmin', 'ping', '-h', 'localhost']
      interval: 5s
      retries: 10

  webapp:
    depends_on:
      database:
        condition: service_healthy   # waits for DB to be TRULY ready
```

Fix Option 2 — Application-level retry:

```bash
# Add retry logic in webapp startup:
for i in {1..30}; do
  mysql -h db -u user -ppass -e 'SELECT 1' && break
  sleep 2
done
```

Fix Option 3 — wait-for-it script: `CMD ['./wait-for-it.sh', 'db:3306', '--', 'python', 'app.py']`

Q2: Scenario: You want to run the same Docker Compose app in dev and prod with different configurations (different DB passwords, different resource limits). How? Use multiple compose files with override:

YAML

```yaml
# docker-compose.yml (base -- common for all environments)
services:
  webapp:
    image: myapp:latest
    environment:
      APP_ENV: base

  db:
    image: mysql:8.0
```

YAML

```yaml
# docker-compose.dev.yml (development overrides)
services:
  webapp:
    build: .                    # build from source in dev
    volumes:
      - .:/app                  # live code mounting
    environment:
      APP_ENV: development
      DEBUG: 'true'

  db:
    environment:
      MYSQL_ROOT_PASSWORD: devpass
```

```yaml
# docker-compose.prod.yml (production overrides)
services:
  webapp:
    deploy:
      replicas: 3
      resources:
        limits: {memory: 512M}
    environment:
      APP_ENV: production

  db:
    environment:
      MYSQL_ROOT_PASSWORD: SuperSecureProdPass!
```

```bash
# Run in dev:
docker-compose -f docker-compose.yml -f docker-compose.dev.yml up

# Run in prod:
docker-compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

---

10. Docker Swarm vs Kubernetes

When you have many containers running across multiple servers, you need an orchestration tool to manage them: scheduling, scaling, health checking, load balancing, and rolling updates. Two major options are Docker Swarm (Docker's native clustering tool) and Kubernetes (Google's open-source container orchestration platform that is now the industry standard).

FEATURE	DOCKER SWARM	KUBERNETES
Complexity	Simple — built into Docker	Complex — steep learning curve
Setup	`docker swarm init` + `docker join`	kubeadm, eksctl, kops, or managed
Auto-scaling	NOT supported	Supported (HPA, VPA, KEDA)
Self-healing	Basic — restarts failed containers	Advanced — reschedules, replaces, rollbacks
Load Balancing	Built-in basic LB	Built-in + integrates with cloud LBs
Rolling Updates	Supported	Advanced — canary, blue/green, rollback
Storage	Limited volume support	Persistent Volumes (PV/PVC) system
Networking	Overlay networks	CNI plugins (Calico, Flannel, Cilium)
Community	Shrinking	Massive — CNCF, Google, RedHat, AWS
Cloud Support	Limited (Docker EE)	AWS EKS, GKE, AKS — fully managed
Monitoring	Basic	Prometheus, Grafana, Jaeger ecosystem
Industry Adoption	Declining	Standard — used by 80%+ of enterprises

Docker Concept:

Recommendation: Learn Docker fundamentals thoroughly, then go directly to Kubernetes. Docker Swarm is simpler but its limitations mean most teams outgrow it quickly. Kubernetes is the industry standard — AWS EKS, Google GKE, Azure AKS all offer managed K8s. Docker Swarm vs Kubernetes = like comparing a bicycle to a car. Both get you somewhere. But for production scale, Kubernetes is the industry choice.

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your company runs 50 Docker containers on a single EC2. When that EC2 goes down, all services are unavailable. How do you architect for High Availability? Move from single-node to multi-node orchestration:

Option 1 — Docker Swarm (quick to set up, limited features):

```bash
# 3 EC2 instances: 1 manager + 2 workers
docker swarm init --advertise-addr <manager-ip>          # on manager
docker swarm join --token <token> <manager-ip>:2377      # on workers
docker service create --replicas 3 --name webapp myapp:v1
# Swarm distributes 3 replicas across 3 nodes
# If one node goes down: containers rescheduled on remaining nodes
```

Option 2 — Kubernetes (recommended for production):

```bash
# AWS EKS: managed K8s control plane
eksctl create cluster --name prod-cluster --nodegroup-name workers --nodes 3 --
nodes-min 2 --nodes-max 10
kubectl apply -f deployment.yaml   # 3 replicas across 3 AZs
# K8s ensures: 2+ nodes always running (PodDisruptionBudget)
# Auto-healing: failed pod replaced automatically within 30 seconds
# Auto-scaling: more traffic -> more pods -> more EC2 nodes
```

HA Architecture:

```text
us-east-1a: Node 1 (replicas: webapp-pod-1, db-pod-1)
us-east-1b: Node 2 (replicas: webapp-pod-2)
us-east-1c: Node 3 (replicas: webapp-pod-3)
If AZ us-east-1a fails: webapp still serves from 1b and 1c
```

11. Docker Quick Command Reference

Images

COMMAND	ACTION
`docker pull nginx:latest`	Pull image from registry
`docker images`	List local images
`docker build -t myapp:v1 .`	Build image from Dockerfile
`docker build -f Dockerfile2 -t myapp .`	Build using specific Dockerfile name
`docker tag myapp user/myapp:v1`	Tag for push
`docker push user/myapp:v1`	Push to registry
`docker rmi myapp:v1`	Remove image
`docker image prune`	Remove dangling images
`docker system prune -a`	Remove ALL unused images/containers/networks

Containers

COMMAND	ACTION
`docker run -dt --name app nginx`	Create and run in background
`docker run -it ubuntu /bin/bash`	Interactive terminal
`docker run -p 8080:80 nginx`	Port mapping host:container
`docker run -v vol:/data myapp`	Mount volume
`docker run -e KEY=value myapp`	Environment variable
`docker ps` / `docker ps -a`	Running / all containers
`docker exec -it app bash`	Enter running container
`docker logs -f app`	Stream container logs
`docker stop app` / `docker kill app`	Graceful / immediate stop
`docker rm -f app`	Force remove container
`docker stats`	Real-time resource usage

Networking & Volumes

COMMAND	ACTION
`docker network ls`	List networks
`docker network create mynet`	Create custom bridge network
`docker network inspect bridge`	Inspect network details
`docker volume create myvol`	Create named volume
`docker volume ls`	List volumes
`docker volume inspect myvol`	Volume details and mountpoint
`docker volume rm myvol`	Remove volume

Docker Compose

COMMAND	ACTION
`docker-compose up -d`	Start all services (background)
`docker-compose down`	Stop and remove containers/networks
`docker-compose down -v`	Remove containers, networks, volumes
`docker-compose ps`	List compose services
`docker-compose logs -f`	Stream all service logs
`docker-compose build`	Rebuild service images
`docker-compose exec webapp bash`	Shell into service container

Source document: "MultiCloud DevOps — Docker Complete Notes — by Veera Sir" (Containers + Dockerfile + Networking + Volumes + Compose + Scenario-Based Interview Q&A — Version 1.0)

K8S

PART 07 OF 08

Kubernetes

Container orchestration at scale — workloads, networking, scaling, and GitOps.

Architecture • Workloads • Services • RBAC • Volumes • Probes • Helm • ArgoCD • Monitoring • Interview Q&A

Document Legend: 💡 Blue = Layman | 📝 Green = Theory | ☸️ K8s Blue = Concept | 🏗️ Purple = Architecture | 🎯 Yellow = Interview Q&A | ⚠️ Warning | Dark Background = YAML/Commands

1. What is Kubernetes?

Kubernetes (often abbreviated **K8s** — 'K', 8 letters, 's') is an open-source container orchestration platform originally designed by Google, based on their internal system called Borg, which had been running Google's production workloads for over a decade. Kubernetes was donated to the Cloud Native Computing Foundation (CNCF) in 2015 and has since become the universal standard for running containerized applications at scale.

The problem Kubernetes solves: Docker lets you run one container easily. But running hundreds of containers across multiple servers reliably — handling failures, scaling up/down, load balancing, rolling updates, and self-healing — is far too complex to do manually. Kubernetes automates ALL of this.

💡 **Layman Explanation:**

- Docker = a single shipping container. Kubernetes = the entire port, ships, and cranes managing thousands of containers.
- If a container (ship) sinks, Kubernetes automatically builds a replacement and puts it back in the water — 'self-healing'.
- If traffic increases, Kubernetes automatically adds more containers — 'auto-scaling'.
- If a whole server (port) goes down, Kubernetes moves all the containers to a different healthy server.
- You just tell Kubernetes 'I want 5 copies of this app always running' — it handles the HOW, forever.

Fig 1: Kubernetes Overview diagram — shows the evolution from Virtualization (EC2+ASG) to Containerization+Orchestration (Docker+Kubernetes). Depicts a Control Node (API Server, ETCD, Scheduler, Controller) connected to Worker Node-1 and Worker Node-2 (each with kubelet, kube-proxy, pods, container runtime engine), fed by a CI/CD toolchain (GitHub, Jenkins, Docker, AWS ECR) via `kubectl apply -f`*, with users hitting a Load Balancer. Also shows a simplified "Kubernetes architecture" box: Control Plane (API Server, Scheduler, Controller Manager, etcd) talking to Node 1 and Node 2 (each running Pods behind a Load Balancer serving End Users). Notes: "Kubernetes is open source orchestration tool. Alternate for Kubernetes: OpenShift." "Kubernetes responsible to orchestrate the pods and nodes — if any pod or node delete, Kubernetes will create by using auto healing concept ensure always High available of the system." "KOPS → Control node and worker node our responsibility — we can integrate KOPS process on-prem and cloud as well." "Cloud managed Kubernetes: EKS - Elastic Kubernetes service, AKS - Azure Kubernetes service, GKE - Google Kubernetes engine. Control node taking care by Cloud (AWS, Azure, GCP). Worker node - We have to take care."*

1.1 Why Kubernetes? — The Evolution

ERA	APPROACH	LIMITATION
Traditional	Physical servers, one app per server	Wasteful, slow to provision, expensive
Virtualization	VMs (EC2+ASG) sharing hardware	Still heavy — full OS per VM, slow scaling
Containerization	Docker containers, lightweight	No orchestration — manual container management doesn't scale
Container Orchestration	Docker + Kubernetes	Solved! Automated scaling, healing, scheduling, networking

Kubernetes Concept:

- Kubernetes is responsible for orchestrating PODS and NODES.
- If any pod or node is deleted, Kubernetes immediately recreates it using AUTO-HEALING — ensuring High Availability.
- Kubernetes was developed by Google and released as an open-source tool (donated to CNCF in 2015).
- Alternative to Kubernetes: OpenShift (Red Hat's enterprise distribution built on top of Kubernetes).
- Cloud-managed Kubernetes services: EKS (AWS), AKS (Azure), GKE (Google Cloud).
- KOPS: lets you manage BOTH control plane and worker nodes yourself — works on-prem and cloud.
- With managed services (EKS/AKS/GKE): the CLOUD takes care of the Control Plane. YOU take care of Worker Nodes.

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your company runs 50 microservices manually with Docker on individual EC2 instances. Deployments are error-prone, scaling is manual, and a server crash takes down services until someone notices. How does Kubernetes solve this? Kubernetes provides automated solutions for each pain point:

1. DEPLOYMENT AUTOMATION: Define each microservice as a Deployment YAML — `kubectl apply -f` deploys it consistently, every time, with rolling updates and zero downtime.
2. AUTO-SCALING: HorizontalPodAutoscaler automatically adds/removes pod replicas based on CPU/memory load — no manual intervention.
3. SELF-HEALING: If a pod crashes or a node fails, Kubernetes' Controller Manager detects the deviation from desired state and automatically reschedules the pod onto a healthy node — often within seconds, before anyone notices.
4. SERVICE DISCOVERY: Kubernetes Services give each microservice a stable DNS name — no more hardcoded IPs that break when containers restart.
5. ROLLING UPDATES & ROLLBACKS: Deployments update one pod at a time, automatically rolling back if health checks fail.

Result: Manual error-prone operations become declarative, automated, and self-correcting.

Q2: Scenario: What is the difference between Docker Swarm and Kubernetes for orchestration? Why did your organization choose Kubernetes? Docker Swarm: Simple, built into Docker, easy to learn — but limited features: no auto-scaling, basic self-healing, smaller ecosystem.

Kubernetes: More complex but vastly more capable:

- Advanced scheduling (node affinity, taints/tolerations, resource requests/limits)
- Built-in auto-scaling (HPA, VPA, Cluster Autoscaler)
- Huge ecosystem: Helm, ArgoCD, Prometheus, Istio, hundreds of CNCF projects
- Industry standard — every cloud provider has managed Kubernetes (EKS/AKS/GKE)
- Massive community and job market — hiring is easier

Organizations choose Kubernetes because: it scales to enterprise complexity, has long-term vendor support, and the talent pool/tooling ecosystem is unmatched. Docker Swarm usage has been steadily declining in production environments.

2. Kubernetes Architecture

A Kubernetes cluster is composed of two distinct planes: the **Control Plane** (the 'brain' that makes all cluster-wide decisions) and **Worker Nodes** (the 'muscles' that actually run your application containers). Understanding the role of every component in this architecture is fundamental to mastering Kubernetes — it's also the single most commonly asked interview topic.

Fig 2: Kubernetes Architecture diagram — shows a toolchain (Jenkins, GitHub, Docker, AWS ECR) feeding `kubectl apply` into the Control Node's API Server, which connects to ETCD, Controller, and Scheduler. The Control Node connects to Worker Node-1 and Worker Node-2, each containing kubelet, kube-proxy, pods, and a container runtime engine, with an LB serving end users. Below it is a compact flow notation and a simplified "Kubernetes architecture" summary box (Control Plane: API Server, Scheduler, Controller Manager, etcd → Worker node 1/2 with pods).

2.1 Control Plane Components (Master Node)

The Control Plane manages the cluster and makes all global decisions — but does NOT run your application containers directly (in most production setups).

COMPONENT	ROLE
API Server	The FRONT DOOR of the entire cluster. Every single request — from kubectl, dashboards, or other components — passes through the API Server. It validates requests, processes authentication/authorization, and is the ONLY component that talks directly to etcd.
etcd	A distributed, fault-tolerant key-value database that stores the COMPLETE state of the cluster — every pod, deployment, secret, configmap, and their configuration. This is the 'single source of truth'.
Scheduler	Watches for newly created Pods with no assigned Node, and decides WHICH node is best suited to run them — based on resource availability, affinity rules, taints/tolerations, and other constraints.
Controller Manager	Runs multiple controller processes that continuously watch cluster state and work to drive the ACTUAL state toward the DESIRED state. Includes Deployment Controller, ReplicaSet Controller, Node Controller, etc.

2.2 Worker Node Components

COMPONENT	ROLE
kubelet	An agent running on EVERY worker node. It receives Pod specifications from the API Server and ensures the specified containers are actually running and healthy on that node. Talks directly to the container runtime.
kube-proxy	A network proxy on every node. Maintains network rules (iptables/IPVS) that allow communication to Pods from inside or outside the cluster. Implements the Service abstraction.
Container Runtime	The software that actually runs containers — pulls images, starts/stops containers. Examples: containerd, CRI-O. (Docker Engine directly is no longer used since Kubernetes 1.24+.)

Fig 3: Control Plane Internal Flow diagram — a numbered sequence diagram showing the Control Plane (API Server, ETCD, Scheduler, Controller) and Worker Node (Kubelet, KubeProxy, Pod-1, Pod-2, CRI) with 10 numbered steps of communication, described in full below in section 2.3.

2.3 The Complete Pod Creation Flow — Step by Step

🏗️ **Architecture:**

Numbered Flow (matches Fig 3):

1. Developer runs `kubectl apply -f pod.yaml` → request goes to API Server
2. API Server validates the request, authenticates/authorizes, then STORES the pod spec in etcd
3. Scheduler is WATCHING the API Server for unscheduled pods — it notices the new pod
4. Scheduler evaluates all nodes (CPU, memory, node selectors, taints) and picks the BEST node
5. Scheduler updates the pod spec with the chosen nodeName, sends it back to API Server
6. API Server updates etcd with the scheduling decision
7. Kubelet on the CHOSEN node is continuously watching the API Server — sees the new pod assigned to it
8. Kubelet instructs the Container Runtime (via CRI) to pull the image and start the container
9. Container Runtime creates the container, kube-proxy sets up networking rules
10. Kubelet reports SUCCESS back to the API Server
11. API Server updates etcd — the pod is now RUNNING. Controller continuously monitors for drift.

```text
Compact flow notation (from architecture diagram):

API --> etcd --> API --> controller --> API --> scheduler --> API -->
etcd --> API --> Kubelet --> CRI --> pod created ---> CRI --> Kubelet -->
API --> API --> etcd --> API --> developer (response)

KEY INSIGHT: The API Server is the CENTRAL HUB.
Every component talks TO the API Server — components never talk directly to
each other.
etcd is the ONLY persistent storage — if etcd is lost, the cluster state is
lost.
```

📝 **Theory & Key Points:**

- API Server is STATELESS — it doesn't store anything itself, it reads/writes via etcd.
- etcd uses the RAFT consensus algorithm for fault tolerance — typically run as a cluster of 3 or 5 nodes (odd numbers for quorum).
- Scheduler does NOT actually start the pod — it only DECIDES which node should run it. Kubelet does the actual starting.
- Controller Manager runs many controllers in a single binary: Node Controller, Replication Controller, Endpoints Controller, Service Account Controller.
- kube-proxy historically used iptables; modern clusters increasingly use eBPF (via Cilium) for better performance.
- A cluster can scale up to 5000 nodes (Kubernetes official scalability limit) — add more nodes to scale capacity.

🎯 Scenario-Based Interview Questions

Q1: Scenario: You run `kubectl apply -f deployment.yaml` but the pods stay in 'Pending' state forever. Walk through how you'd diagnose using your knowledge of the architecture. Pending state means the SCHEDULER could not find a suitable node. Diagnosis using architecture knowledge:

1. `kubectl describe pod <pod-name>` → check Events section for scheduling failure reason
2. Common causes (testing each component in the flow):
   - a. Insufficient resources: `kubectl describe nodes` → check if any node has enough CPU/memory
   - b. Node Selector/Affinity mismatch: pod requires a label no node has
   - c. Taints without matching Tolerations: all nodes are tainted, pod doesn't tolerate
   - d. PVC pending: if pod needs a PersistentVolumeClaim that hasn't been bound yet
3. Check Scheduler logs: `kubectl logs -n kube-system <scheduler-pod>`
4. Check API Server connectivity: `kubectl get componentstatuses` (deprecated but useful in some versions) or `kubectl get --raw='/healthz'`

The scheduler is the FIRST point of failure for Pending pods — it never even reaches kubelet.

Q2: Scenario: Your etcd cluster (the key-value store) crashes and all 3 etcd nodes are lost simultaneously. What happens to your running applications, and can you recover? Immediate impact:

- Existing PODS KEEP RUNNING — kubelet on each worker node continues managing already-scheduled containers independently, even without API Server/etcd access (for a while)
- But you CANNOT create new pods, scale deployments, or make ANY cluster changes — the API Server cannot read/write state
- kubectl commands will fail or hang
- If a pod crashes during this outage, it CANNOT be rescheduled (no scheduler decision possible)

Recovery:

1. etcd should ALWAYS be backed up regularly: `etcdctl snapshot save backup.db`
2. Restore from snapshot: `etcdctl snapshot restore backup.db`
3. Restart etcd cluster with restored data
4. API Server reconnects, Controller Manager reconciles actual vs desired state

Lesson: etcd is the SINGLE MOST CRITICAL component — always run odd-numbered etcd clusters (3 or 5) across different availability zones, with automated backups.

Q3: Scenario: A developer asks 'Why can't kubelet talk directly to the Scheduler to get pod assignments faster?' How do you explain Kubernetes' architectural design? Kubernetes deliberately uses a 'hub and spoke' model where the API Server is the ONLY communication point — no component talks directly to another.

Why this design:

1. SINGLE SOURCE OF TRUTH: etcd (accessed only via API Server) is the one place all state lives. Direct component-to-component communication would create inconsistent views of state.
2. LOOSE COUPLING: Components don't need to know about each other's existence or location — they all just watch/poll the API Server.
3. SECURITY: API Server enforces authentication/authorization for EVERY action — bypassing it would create security holes.
4. EXTENSIBILITY: New controllers/components can be added by simply watching the API Server — no need to modify existing components.
5. RESILIENCE: If the Scheduler crashes, kubelet keeps running existing pods fine — kubelet doesn't depend on Scheduler being alive.

This is why Kubernetes uses a 'declarative, watch-based' architecture instead of direct RPC calls between components.

---

3. Tooling — Minikube, kubectl, eksctl

Before working with Kubernetes, you need the right tools installed. **Minikube** gives you a local single-node cluster for learning/testing. **kubectl** is the command-line tool to interact with ANY Kubernetes cluster. **eksctl** simplifies creating and managing AWS EKS clusters specifically.

3.1 Minikube — Local Kubernetes for Learning

💡 **Layman Explanation:**

- Minikube = a 'Kubernetes in a box' that runs entirely on your laptop.
- Perfect for learning, testing YAML files, and local development — without needing a real cloud cluster.
- It creates a single-node cluster (one VM acts as BOTH control plane and worker node).

BASH

```
# Install Minikube (Linux)
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# Start a local cluster
minikube start

# Check cluster status
minikube status

# Update kube context (point kubectl at minikube)
minikube update-context

# Stop / Delete cluster
minikube stop
minikube delete
```

3.2 kubectl — The Kubernetes CLI

kubectl ('kube control' or 'kube-cuddle') is THE essential command-line tool for managing any Kubernetes cluster — local or cloud. Every action — creating resources, checking status, debugging, deleting — goes through kubectl.

```bash
# Install kubectl (Linux)
curl -LO https://storage.googleapis.com/kubernetes-release/release/$(curl -s
https://storage.googleapis.com/kubernetes-
release/release/stable.txt)/bin/linux/amd64/kubectl
chmod +x ./kubectl
sudo mv ./kubectl /usr/local/bin/kubectl

# Verify
kubectl version --client

# Most commonly used commands
kubectl get nodes                    # list all nodes in cluster
kubectl get pods                     # list pods in current namespace
kubectl get pods -A                  # list pods in ALL namespaces
kubectl get pods -o wide             # more details: node, IP, etc.
kubectl describe pod <name>          # detailed info + recent events
kubectl logs <pod-name>              # view container logs
kubectl exec -it <pod> -- /bin/sh    # open shell inside running pod
kubectl apply -f file.yaml           # create/update resources from YAML
kubectl delete -f file.yaml          # delete resources defined in YAML
kubectl delete pod <name>            # delete specific pod
```

3.3 eksctl — AWS EKS Cluster Management

eksctl is the official AWS-recommended CLI tool for creating and managing EKS (Elastic Kubernetes Service) clusters. It handles the complex AWS infrastructure (VPC, subnets, IAM roles, node groups) automatically — turning a multi-hour manual EKS setup into a single command.

BASH

```bash
# Install eksctl
curl --silent --location \
"https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_$(uname -
s)_amd64.tar.gz" | tar xz -C /tmp
sudo mv /tmp/eksctl /usr/local/bin
eksctl version

# PREREQUISITE: Create an IAM Role for EC2 (if bootstrapping from EC2)
# Or IAM user with programmatic access (if running from outside AWS)
# Required permissions: IAM, EC2, VPC, CloudFormation

# Create an EKS cluster + node group
eksctl create cluster --name cluster-name \
  --region region-name \
  --node-type instance-type \
  --nodes-min 2 \
  --nodes-max 2 \
  --zones <AZ-1>,<AZ-2>

# Real example:
eksctl create cluster --name test \
  --region us-east-1 \
  --node-type t2.medium

# Update local kubeconfig to point at the new cluster
aws eks update-kubeconfig --region <region> --name <cluster-name>

# Example:
aws eks update-kubeconfig --region ap-south-1 --name naresh

# Delete the entire cluster (removes VPC, nodes, everything)
eksctl delete cluster naresh --region ap-south-1
```

🎯 **Scenario-Based Interview Questions**

Q1: Scenario: You created an EKS cluster using eksctl, but `kubectl get nodes` returns 'Unable to connect to the server'. What's wrong? Most likely cause: kubeconfig is not pointing to the new cluster, or AWS credentials are misconfigured.

Diagnosis:

1. Check current kubeconfig context: `kubectl config current-context`
2. Update kubeconfig explicitly: `aws eks update-kubeconfig --region <region> --name <cluster-name>`
3. Verify AWS CLI credentials work: `aws sts get-caller-identity`
4. Check if your IAM user/role is in the aws-auth ConfigMap (cluster creator gets automatic access, but others need explicit mapping)
5. Check security groups: EKS control plane endpoint security group must allow your IP if using public endpoint access

If you're not the cluster creator: ask the creator to add your IAM ARN to the aws-auth ConfigMap with appropriate RBAC permissions (covered in the RBAC section).

Q2: Scenario: Compare Minikube vs EKS for a team transitioning from local development to production deployment. Minikube: Single-node, runs on YOUR laptop, free, instant start/stop. PERFECT for: learning YAML syntax, testing manifests, local development before pushing to a real cluster. NOT suitable for: production, team collaboration, real traffic.

EKS (AWS managed Kubernetes): Multi-node, runs in AWS, costs money (control plane + EC2 nodes), production-grade HA, integrates with AWS IAM/VPC/ELB/EBS. PERFECT for: production workloads, team-shared environments, anything serving real users.

Typical workflow: Developer writes/tests YAML on Minikube locally → once validated, the SAME YAML is applied to EKS via CI/CD pipeline. Kubernetes' portability means manifests written for Minikube work unchanged on EKS (assuming you're not using cloud-specific features like LoadBalancer service type, which behaves differently).

---

4. Kubernetes Workloads — Pod, ReplicaSet, Deployment

Kubernetes workload resources form a hierarchy of increasing capability: a bare Pod has zero resilience, a ReplicaSet adds self-healing, and a Deployment adds rolling updates on top of that.

Understanding WHY each layer exists — and why you should almost NEVER use a bare Pod directly in production — is essential.

Fig 4: Kubernetes Workloads diagram — shows Pod → ReplicaSet Controller → Deployment YAML boxes side by side (with notes: "POD create without control manager, it won't give any replicas, no high availability"; "RS only support Delete first ol versions and create V2 versions not able to handle upgrade the images version on live down time will be there — not recommended for production one"; "Deployment supports rolling updates — we can be able to update version without able to deleting all pods before it — can handle one by one pod create and delete without any downtime"). Below: a "ReplicaSet - Self healing" box showing 3 PODs. Then a "Kubernetes service" section stating: "Kubernetes service enables internal and external communications: 1. NodePort – External communications, 2. LoadBalancer – External communications, 3. ClusterIp – Internal communications, 4. Headless Service – Internal communications (stateful applications Database)". Diagrams show: (a) Frontend Deployment (LB/SVC) → ClusterIP → Backend Deployment (app) → headless SVC → StatefulSet database (mysql-0, mysql-1, mysql-2, each with separate EBS volumes) — labeled "allow to create pod with unique order even any pod delete it creates with same name, replica=3, Separate EBS Volumes manage to store database data". (b) Same Frontend/Backend pattern but Backend connects to AWS RDS instead of a StatefulSet — labeled "stateless app". (c) A NodePort service diagram: client hits 192.1.3.4:30008 (node-1) → SVC → IP tables → target port → routes to pod-2 (IRCTC) on node-2 at 192.1.3.5:30008, port 80, noting "By default node port range 30000-32767". (d) A terminal screenshot showing a Service-NodePort YAML/kubectl output. Bottom flow: "Request → kube-proxy → iptables rule match → Select Pod IP → Forward traffic".

4.1 Pod — The Smallest Deployable Unit

A **Pod** is the smallest deployable unit in Kubernetes. It represents one or more tightly-coupled containers that share networking (same IP) and storage. Most Pods run a single container, but multi-container Pods (sidecar pattern) are common for logging agents, service mesh proxies, etc.

```bash
# Imperative way (quick, not recommended for production)
kubectl run pod --image nginx
```

YAML

```yaml
# Declarative way — pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp
  labels:
    app: webapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

BASH

```bash
# Commands
kubectl apply -f pod.yaml
kubectl get pods
kubectl get pods -o wide              # shows node, IP
kubectl delete pod myapp
kubectl describe pod myapp            # full details + events
kubectl exec myapp -it -- /bin/sh     # open shell inside
```

⚠️ **Important Warning:**

A bare Pod created without a controller (Deployment/ReplicaSet) has NO replicas and NO high availability. If a bare Pod's node dies, the Pod is GONE forever — nothing recreates it. NEVER use bare Pods directly in production — always wrap them in a Deployment or StatefulSet.

4.2 ReplicaSet — Self-Healing

A **ReplicaSet** ensures a specified number of identical Pod replicas are always running. If a Pod crashes or is deleted, the ReplicaSet's controller immediately creates a replacement to maintain the desired count. This is the foundation of self-healing.

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-replicaset
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:                    # this is the Pod template
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx-container
        image: nginx:latest
        ports:
        - containerPort: 80
```

⚠️ **Important Warning:**

ReplicaSet LIMITATION: It does NOT support rolling updates of the image version. To upgrade the image, ReplicaSet must DELETE all old pods FIRST, then create new ones — causing DOWNTIME. This is why ReplicaSets are rarely used directly — Deployments (which manage ReplicaSets internally) are preferred.

4.3 Deployment — Rolling Updates + Self-Healing

A **Deployment** is the most commonly used workload resource. It manages ReplicaSets automatically, and adds critical capabilities ReplicaSets lack — most importantly **rolling updates** (upgrade pods one at a time, with zero downtime) and **rollback** (revert to a previous version if something breaks).

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx-container
        image: nginx:latest
        ports:
        - containerPort: 80
```

BASH

```bash
# Deployment commands
kubectl apply -f deployment.yaml
kubectl get deployments
kubectl get rs                                                  # see the ReplicaSet
it created
kubectl set image deployment/nginx-deployment nginx-container=nginx:1.21   # rolling
update
kubectl rollout status deployment/nginx-deployment
kubectl rollout history deployment/nginx-deployment
kubectl rollout undo deployment/nginx-deployment                # rollback to
previous version
kubectl scale deployment/nginx-deployment --replicas=5          # manual scaling
```

4.4 The Complete Hierarchy

🏗️ **Architecture:**

```
Deployment (manages rolling updates)
  └── ReplicaSet (ensures self-healing — desired replica count)
        └── Pods (the actual running containers)
```

When you update a Deployment's image:

1. Deployment creates a NEW ReplicaSet with the new image
2. Gradually scales UP the new ReplicaSet, scales DOWN the old one
3. Old ReplicaSet is kept (scaled to 0) — enables instant rollback
4. This is called a 'Rolling Update' — zero downtime, one pod replaced at a time

RESOURCE	SELF-HEALING	ROLLING UPDATES	RECOMMENDED FOR PRODUCTION
Pod	No	No	Never (use only for quick tests)
ReplicaSet	Yes	No (causes downtime)	Rarely (managed automatically by Deployment instead)
Deployment	Yes	Yes (zero downtime)	YES — standard choice for stateless apps

🎯 Scenario-Based Interview Questions

Q1: Scenario: You created a bare Pod (not via Deployment) for a quick test. The node it was running on terminated unexpectedly (spot instance reclaimed). What happens to the Pod? The Pod is GONE PERMANENTLY. There is no controller watching it, so nothing recreates it. Verify: `kubectl get pods` → the pod simply disappears from the list (it's not even shown as 'Failed' — it's just gone since the node it lived on is gone)

This demonstrates exactly why bare Pods should never be used in production:

- Spot instances, node failures, maintenance, and scaling events all terminate nodes
- Without a Deployment/ReplicaSet, there's no automatic recovery

Fix going forward: ALWAYS wrap workloads in a Deployment: `kubectl create deployment myapp --image=nginx --replicas=3`

Now if a node dies, the ReplicaSet controller (managed by the Deployment) detects the missing pod count and creates a replacement on a healthy node automatically.

Q2: Scenario: You update a Deployment's image from nginx:1.20 to nginx:1.21, but the new version has a critical bug causing crashes. How do you respond using Deployment features? Immediate rollback using Deployment's built-in revision history:

1. Check rollout history: `kubectl rollout history deployment/nginx-deployment` → Shows revision 1 (1.20), revision 2 (1.21 - current, broken)
2. Rollback to previous revision: `kubectl rollout undo deployment/nginx-deployment` → Instantly reverts to nginx:1.20
3. Verify rollback completed: `kubectl rollout status deployment/nginx-deployment`
4. If you want to rollback to a SPECIFIC older revision (not just previous): `kubectl rollout undo deployment/nginx-deployment --to-revision=1`

This is only possible because Deployment maintains ReplicaSet history — this is one of the MOST important production-saving features of Deployments over bare ReplicaSets.

Q3: Scenario: Explain why a ReplicaSet alone cannot safely update container image versions, but a Deployment can. ReplicaSet behavior when you change the image in its spec:

- ReplicaSet detects the template changed
- But ReplicaSet has NO 'rolling update' logic built in
- It would need to DELETE existing pods (old image) and CREATE new ones (new image)
- If done all at once: ALL pods go down simultaneously = DOWNTIME

Deployment behavior:

- Deployment CREATES A NEW ReplicaSet (with the new image) alongside the old one
- Gradually scales the NEW ReplicaSet UP (1, 2, 3...) while scaling the OLD ReplicaSet DOWN (3, 2, 1...)
- At every point during the rollout, SOME pods are serving traffic — ZERO DOWNTIME
- Uses maxSurge and maxUnavailable settings to control the pace

This is why Deployment is essentially 'ReplicaSet + a rolling-update controller layered on top' — it manages MULTIPLE ReplicaSets over time to achieve safe upgrades.

---

5. Container Runtime Interface (CRI)

CRI (Container Runtime Interface) is an API layer/plugin interface that allows kubelet to communicate with ANY container runtime in a standardized way. It defines exactly how kubelet should start, stop, and manage containers — without needing to know the implementation details of the specific runtime being used (containerd, CRI-O, etc.).

💡 **Layman Explanation:**

- CRI is like a universal remote control standard — any TV brand can work with any remote, as long as both follow the standard.
- Before CRI: kubelet was hardcoded to talk to Docker specifically.
- After CRI: kubelet talks through a standardized API — Docker, containerd, CRI-O can all 'plug in' as long as they implement the CRI standard.

⚠️ **Important Warning:**

IMPORTANT: Since Kubernetes v1.24, Docker support (dockershim) was REMOVED from kubelet directly. `docker ps` will NOT show your Kubernetes containers unless you're explicitly using cri-dockerd as the runtime. Most modern clusters use containerd or CRI-O directly — NOT Docker Engine.

5.1 Before vs After Kubernetes 1.24

```text
BEFORE v1.24 (used Docker via a translator called dockershim):

Kubernetes (kubelet)
        ↓
dockershim (translator layer — built into kubelet)
        ↓
Docker Engine
        ↓
containerd → runc (actually runs the container)

AFTER v1.24 (native CRI — dockershim removed):

Kubernetes (kubelet)
        ↓
CRI (native interface)
        ↓
containerd / CRI-O / cri-dockerd
```

5.2 Inspecting Containers — crictl

Since `docker ps` no longer works for inspecting Kubernetes containers (post v1.24), use **crictl** — the CRI-compatible equivalent.

```bash
# Install crictl
VERSION='v1.30.0'
curl -LO https://github.com/kubernetes-sigs/cri-tools/releases/download/$VERSION/crictl-$VERSION-linux-amd64.tar.gz
sudo tar zxvf crictl-$VERSION-linux-amd64.tar.gz -C /usr/local/bin

# Check container details at the RUNTIME level (lower level than pods)
sudo crictl ps

# Check POD-level details (Kubernetes abstraction level)
kubectl get pods
```

🎯 Scenario-Based Interview Questions

Q1: Scenario: A new engineer SSHs into a worker node and runs `docker ps` expecting to see the running application containers, but gets 'Cannot connect to the Docker daemon' or sees an empty list. What's happening? Since Kubernetes v1.24+, dockershim was removed. Most modern clusters use containerd directly as the CRI runtime — Docker Engine is simply NOT INSTALLED on the node at all (or if it is, it's unrelated to what Kubernetes is running).

Fix: Use the CRI-compatible tool instead:

```text
sudo crictl ps            # shows actual running containers via CRI
kubectl get pods -o wide  # shows pods + which node they're on
```

Verify the runtime in use: `kubectl get nodes -o wide` # CONTAINER-RUNTIME column shows containerd://1.6.x or similar

This is a common 'gotcha' for engineers transitioning from Docker-based workflows to native CRI-based Kubernetes.

6. Kubernetes Services

A **Service** is a stable network abstraction for exposing applications running as one or more Pods. Pods are ephemeral — they get new IP addresses every time they restart. A Service solves this by giving your application a permanent, stable network identity that doesn't change even as the underlying pods are created and destroyed.

> 💡 **Layman Explanation:**
>
> - Pods are like food delivery riders — they constantly change (new rider each time, different phone number/IP).
> - A Service is like the restaurant's PERMANENT phone number — you always call the SAME number.
> - Behind the scenes, the restaurant assigns whichever rider (pod) is available to handle your order.
> - You never need to know or care WHICH rider (pod) is currently working — the Service handles that routing.

6.1 Service Types

TYPE	DESCRIPTION
ClusterIP (default)	Internal-only IP address — reachable ONLY from within the cluster. Used for backend services, databases, internal APIs.
NodePort	Exposes a port on EVERY node (range 30000-32767). External traffic reaches the service via `<NodeIP>:<NodePort>`. Good for simple external access.
LoadBalancer	Creates a cloud provider load balancer (AWS ELB/ALB, GCP LB, Azure LB) automatically — gives you a public IP/DNS. The standard way to expose production services externally.
Headless (`clusterIP: None`)	No load balancing — used for StatefulSets where you need to address EACH pod individually (e.g., database replicas) via DNS.

6.2 Port Terminology

PORT FIELD	MEANING
port	The port the SERVICE itself exposes. Other pods inside the cluster talk to the service on this port.
targetPort	The actual port the CONTAINER is listening on. The service forwards traffic from 'port' to this 'targetPort' inside the pod.
nodePort	The port exposed on EVERY node's external IP (only for NodePort/LoadBalancer types). Default range: 30000-32767.

YAML

```yaml
# ClusterIP Service (default — internal only)
apiVersion: v1
kind: Service
metadata:
  name: myapp-clusterip
spec:
  type: ClusterIP
  selector:
    app: myapp
  ports:
  - port: 80
    targetPort: 8080
```

YAML

```yaml
# NodePort Service (external access via node IP)
apiVersion: v1
kind: Service
metadata:
  name: myapp-nodeport
spec:
  type: NodePort
  selector:
    app: myapp
  ports:
  - port: 80
    targetPort: 8080
    nodePort: 30008    # optional — auto-assigned if omitted
```

YAML

```yaml
# LoadBalancer Service (production external access — AWS creates ELB)
apiVersion: v1
kind: Service
metadata:
  name: myapp-lb
spec:
  type: LoadBalancer
  selector:
    app: myapp
  ports:
  - port: 80
    targetPort: 8080
```

6.3 NodePort Traffic Flow

🏗️ **Architecture:**

Request → kube-proxy → iptables rule match → Select Pod IP → Forward traffic

Example: User hits 192.1.3.4:30008

1. Request arrives at Node-1's external IP on port 30008
2. kube-proxy's iptables rules intercept it
3. iptables rules randomly select one healthy Pod IP behind the service
4. Traffic is forwarded (DNAT) to that Pod's IP:targetPort (e.g., 192.168.8.216:80)
5. Even though you hit Node-1, traffic might be forwarded to a Pod running on Node-2 — kube-proxy handles cross-node routing automatically

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your team needs to expose an internal microservice ONLY to other pods within the cluster (never to the internet). Which Service type do you use and why? Use ClusterIP (the default type):

```yaml
apiVersion: v1
kind: Service
metadata:
  name: backend-api
spec:
  type: ClusterIP   # or omit — this is default
  selector:
    app: backend
  ports:
  - port: 8080
    targetPort: 8080
```

Why ClusterIP:

- Only assigns an INTERNAL IP, never exposed outside the cluster
- Other pods reach it via DNS: `http://backend-api.namespace.svc.cluster.local:8080`
- No security risk of accidental internet exposure
- This is the correct choice for: databases, internal APIs, microservice-to-microservice communication

Using NodePort or LoadBalancer here would be a SECURITY MISTAKE — it would expose an internal-only service to external traffic unnecessarily.

Q2: Scenario: You created a LoadBalancer service for your frontend app, but `kubectl get svc` shows EXTERNAL-IP as 'pending' forever. What's wrong? Diagnosis:

1. Check if you're running on a CLOUD provider that supports LoadBalancer — Minikube/bare-metal clusters do NOT automatically provision load balancers (EXTERNAL-IP will stay pending unless MetalLB or similar is installed)
2. On EKS/AKS/GKE: check IAM permissions — the cluster's node IAM role needs permissions to create ELB/ALB resources
3. Check events: `kubectl describe svc myapp-lb` → look for error messages in Events

4. Check AWS console: Load Balancers section — sometimes the LB IS created but DNS propagation is just slow
5. Check Service Controller logs (cloud-controller-manager) for errors

On Minikube specifically: use `minikube tunnel` to simulate a LoadBalancer locally, since Minikube has no real cloud infrastructure to provision one.

Q3: Scenario: A stateful database pod's IP keeps changing every time it restarts, breaking client connections. How does a Headless Service solve this differently from a regular Service? Regular ClusterIP Service: Provides ONE stable IP that load-balances across ALL matching pods. Good for STATELESS apps, but BAD for databases where each replica is unique (one PRIMARY for writes, replicas for reads) — you can't randomly load-balance write requests across replicas.

Headless Service (`clusterIP: None`): NO load balancing. DNS returns the IP of EACH individual pod directly. Clients can address specific pods by name: `podname.servicename.namespace.svc.cluster.local`

Example: `mysql-0.mysql.default.svc.cluster.local` always resolves to the SAME pod (mysql-0, the primary), letting your app reliably send writes to mysql-0 and reads to mysql-1, mysql-2.

This is THE standard pattern for StatefulSets — covered in detail in the StatefulSet section.

---

7. Autoscaling — HPA, Cluster Autoscaler, Metrics Server

Kubernetes provides multiple layers of autoscaling. **HorizontalPodAutoscaler (HPA)** scales the NUMBER of pod replicas based on metrics like CPU/memory usage. **Cluster Autoscaler** scales the NUMBER of worker nodes when pods can't be scheduled due to insufficient capacity. Both rely on the **Metrics Server** to gather resource usage data.

7.1 Horizontal Pod Autoscaler (HPA)

HPA automatically adjusts the number of pod replicas in a Deployment/StatefulSet to match current demand. Horizontal scaling (more pods) is different from vertical scaling (more CPU/memory PER pod) — Kubernetes' HPA does the former; VPA (Vertical Pod Autoscaler) does the latter.

💡 **Layman Explanation:**

- Horizontal scaling = hiring MORE cashiers when the checkout line gets long.
- Vertical scaling = making ONE cashier work faster (give them more hands/resources).
- HPA watches the 'line length' (CPU/memory usage) and automatically adjusts how many cashiers (pods) are working.
- If load decreases, HPA sends cashiers home — scales back down to save resources.

7.2 Metrics Server — Prerequisite for HPA

HPA needs resource usage DATA to make scaling decisions. The Metrics Server aggregates CPU/memory usage across the cluster — it is NOT deployed by default in EKS and must be installed separately.

BASH

```
# Deploy Metrics Server
kubectl apply -f https://github.com/kubernetes-sigs/metrics-
server/releases/latest/download/components.yaml

# Verify it's running
kubectl get deployment metrics-server -n kube-system

# View current resource usage
kubectl top pod     # CPU/memory usage per pod
kubectl top node    # CPU/memory usage per node
```

7.3 Creating an HPA

BASH

```
# Imperative HPA creation
kubectl autoscale deployment myapp --cpu-percent=50 --min=2 --max=10
```

```yaml
# Declarative HPA YAML
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: myapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: myapp
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50    # scale up if avg CPU > 50%
```

```bash
# Check HPA status
kubectl get hpa
kubectl describe hpa myapp-hpa
```

7.4 Cluster Autoscaler — Scaling NODES

While HPA scales PODS, the **Cluster Autoscaler** scales the underlying WORKER NODES. If HPA wants to create more pods but there's no node capacity, Cluster Autoscaler automatically requests AWS to launch additional EC2 instances in the node group.

BASH

```
# Deploy Cluster Autoscaler
kubectl apply -f \
https://raw.githubusercontent.com/kubernetes/autoscaler/cluster-autoscaler-
1.29.0/cluster-autoscaler/cloudprovider/aws/examples/cluster-autoscaler-
autodiscover.yaml

# Verify pod is running
kubectl -n kube-system get pods -l app=cluster-autoscaler

# Edit deployment to add your cluster name (REQUIRED step)
kubectl -n kube-system edit deployment.apps/cluster-autoscaler
# Add --node-group-auto-discovery flag with your cluster name

# IMPORTANT: Node group IAM role needs autoscaling permissions
# AWS Console → Node Group → IAM Role → Attach AutoScalingFullAccess (or
# custom least-privilege policy)

# Configure node group scaling limits
aws eks update-nodegroup-config \
  --cluster-name naresh \
  --nodegroup-name ng-af5ac006 \
  --scaling-config minSize=2,maxSize=6,desiredSize=3

# Watch Cluster Autoscaler logs
kubectl -n kube-system logs -f deployment/cluster-autoscaler
```

AUTOSCALER	SCALES WHAT	TRIGGER
HPA (Horizontal Pod Autoscaler)	Number of POD replicas	CPU/memory usage exceeds threshold
VPA (Vertical Pod Autoscaler)	CPU/memory PER pod	Pod consistently hits resource limits
Cluster Autoscaler	Number of WORKER NODES	Pods stuck Pending due to insufficient node capacity

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your e-commerce app gets 10x traffic during a flash sale. HPA scales pods from 3 to 20, but new pods stay 'Pending' because there's no node capacity. How do you fix this end-to-end? This requires BOTH HPA (pod scaling) AND Cluster Autoscaler (node scaling) working together:

1. HPA correctly scaled pods 3→20 based on CPU/memory metrics — this part worked
2. But the EXISTING nodes don't have enough capacity for 20 pods — they're stuck Pending
3. THIS is exactly what Cluster Autoscaler solves:
    - It watches for Pending pods caused by insufficient resources
    - Automatically requests AWS to launch NEW EC2 instances in the node group
    - New nodes join the cluster, scheduler places the Pending pods on them

Fix: Ensure Cluster Autoscaler is deployed and node group scaling config allows growth: `aws eks update-nodegroup-config --cluster-name prod --nodegroup-name ng-1 --scaling-config minSize=3,maxSize=20,desiredSize=5`

Without Cluster Autoscaler, HPA can request more pods all day, but they'll never schedule if there's no node room — both layers are required for true elastic scaling.

Q2: Scenario: HPA shows 'unknown' for current CPU utilization and never scales, even under heavy load. What's the root cause? HPA relies on Metrics Server to get resource usage data. If Metrics Server is NOT installed or not functioning, HPA has no data to make decisions.

Diagnosis:

```text
kubectl get deployment metrics-server -n kube-system   # check if it exists and is running
kubectl top pod                                        # if this fails, Metrics Server is broken
```

Common causes:

1. Metrics Server not deployed at all (EKS doesn't include it by default!) Fix: `kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml`
2. Metrics Server pod is CrashLooping due to certificate validation issues (common on some clusters) Fix: add `--kubelet-insecure-tls` flag to metrics-server args (workaround for

self-signed certs)

3. Pod resource REQUESTS not defined — HPA percentage calculations require `requests.cpu` to be set in the pod spec; without it, percentage-based scaling cannot compute a baseline.

---

8. Ingress

An **Ingress** is a Kubernetes object that manages external HTTP/HTTPS access to services inside the cluster, based on URL paths or hostnames. While a LoadBalancer Service exposes ONE service with ONE external IP, Ingress lets you expose MANY services through a SINGLE load balancer — routing traffic based on the URL path or domain name. This dramatically reduces cloud costs (one ALB instead of many) and centralizes traffic management.

💡 **Layman Explanation:**

- Without Ingress: each microservice needs its OWN LoadBalancer = expensive (AWS charges per LB).
- With Ingress: ONE LoadBalancer routes to MANY services based on the URL path.
- Example: mysite.com/api → backend-service. mysite.com/images → image-service. mysite.com/ → frontend-service.
- Ingress is like a building receptionist directing visitors to the right department based on what they're asking for.

8.1 Installing the NGINX Ingress Controller

Ingress RESOURCES (rules) require an **Ingress Controller** to actually implement them. NGINX Ingress Controller is the most popular choice.

BASH

```bash
# Create namespace for ingress controller
kubectl create namespace ingress-nginx

# Install the NGINX Ingress Controller
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-
nginx/controller-v1.2.1/deploy/static/provider/cloud/deploy.yaml

# Verify controller is running
kubectl get pods -n ingress-nginx
```

8.2 Example: Path-Based Routing

YAML

```yaml
# deployment-path1.yaml — first app
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app1-deployment
spec:
  replicas: 2
  selector:
    matchLabels: { app: app1 }
  template:
    metadata:
      labels: { app: app1 }
    spec:
      containers:
      - name: app1
        image: hashicorp/http-echo
        args: ['-text=Hello from App 1']
---
apiVersion: v1
kind: Service
metadata:
  name: app1-service
spec:
  selector: { app: app1 }
  ports: [{ port: 80, targetPort: 5678 }]
```

YAML

```yaml
# ingress-resource.yaml — routing rules
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - http:
      paths:
      - path: /app1
        pathType: Prefix
        backend:
          service:
            name: app1-service
            port: { number: 80 }
      - path: /app2
        pathType: Prefix
        backend:
          service:
            name: app2-service
            port: { number: 80 }
```

BASH

```bash
# After applying all files:
kubectl get ingress    # shows the LoadBalancer link
# Access: http://<lb-address>/app1 and http://<lb-address>/app2
```

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your company has 15 microservices. Using LoadBalancer service type for each would cost ~$240/month (15 × $16/month per AWS ALB). How does Ingress reduce this cost? With LoadBalancer services: 15 separate AWS Load Balancers are created — 15 × $16-20/month = $240-300/month, PLUS data transfer costs per LB.

With Ingress:

1. Deploy ONE NGINX Ingress Controller (itself exposed via ONE LoadBalancer service)
2. Define 15 Ingress rules routing different paths/hosts to different internal ClusterIP services
3. Total cost: ONE Load Balancer (~$16-20/month) regardless of how many backend services you route to

Example savings: $240/month → $20/month = 92% reduction

Additional benefits beyond cost:

- Centralized SSL/TLS termination (one certificate config point)
- Centralized routing rules and observability
- Path-based and host-based routing in one place
- Can add rate limiting, auth, rewrite rules centrally via annotations

Q2: Scenario: After applying your Ingress resource, `kubectl get ingress` shows the LoadBalancer address, but visiting the URL returns 404 Not Found. How do you debug? Step-by-step debugging:

1. Verify Ingress Controller is running: `kubectl get pods -n ingress-nginx`
2. Check Ingress resource details: `kubectl describe ingress myapp-ingress` → look for backend service errors
3. Verify the BACKEND SERVICE exists and has endpoints:

```text
kubectl get svc app1-service
kubectl get endpoints app1-service   # if EMPTY, no pods match the service selector!
```

4. Check pathType: 'Prefix' vs 'Exact' — 'Exact' requires the EXACT path match, 'Prefix' matches anything starting with that path
5. Check ingressClassName matches your installed controller (nginx)

6. Check Ingress Controller logs: `kubectl logs -n ingress-nginx <controller-pod>`
7. Common mistake: forgetting the rewrite-target annotation when backend app expects root path '/' but Ingress passes '/app1/' literally

---

9. RBAC — Role-Based Access Control

RBAC (Role-Based Access Control) is the mechanism Kubernetes uses to control WHO can do WHAT within a cluster. It is essential for any multi-user or multi-team Kubernetes environment — without RBAC, anyone with cluster access could do ANYTHING, including deleting production resources. RBAC works through 4 core objects: Role, ClusterRole, RoleBinding, and ClusterRoleBinding.

💡 **Layman Explanation:**

- RBAC is like an office building's keycard system.
- Role/ClusterRole = what doors a keycard CAN unlock (define the permissions).
- RoleBinding/ClusterRoleBinding = which EMPLOYEE gets which keycard (assign the permissions to a person).
- Without binding a Role to someone, the Role exists but does nothing — like a keycard sitting in a drawer, unassigned.

9.1 The Four RBAC Objects

OBJECT	PURPOSE
Role	Defines WHAT actions are allowed, scoped to ONE namespace. Example: 'Can read pods in the dev namespace'.
ClusterRole	Like Role, but works across the WHOLE cluster (all namespaces) or for cluster-scoped resources (nodes, namespaces themselves).
RoleBinding	Connects a Role to a User/Group/ServiceAccount — WITHIN one namespace. 'Give Alice the pod-reader role in the dev namespace'.
ClusterRoleBinding	Connects a ClusterRole to a User/Group — across the WHOLE cluster. 'Give Bob admin access everywhere'.

9.2 Role vs ClusterRole — Detailed Comparison

FEATURE	ROLE	CLUSTERROLE
Scope	Only ONE namespace	Entire cluster (all namespaces)
Namespaced resources	Yes	Yes
Cluster-wide resources (nodes, namespaces)	No	Yes
Binding type used	RoleBinding	ClusterRoleBinding
Example use case	Developer access to pods in 'dev' namespace only	Admin access to ALL namespaces

9.3 EKS-Specific: Mapping IAM to Kubernetes RBAC

On AWS EKS, IAM users/roles don't automatically have any Kubernetes permissions. You must explicitly map them to Kubernetes identities using the **aws-auth ConfigMap** in the kube-system namespace, and THEN assign Kubernetes RBAC permissions to those identities.

🏗️ **Architecture:**

Authentication Flow (IAM → Kubernetes RBAC):

```text
User runs 'kubectl get nodes'
        ↓
kubeconfig (points to EKS cluster + uses AWS credentials)
        ↓
aws eks get-token (generates a short-lived authentication token)
        ↓
IAM Authentication (AWS verifies the IAM user/role is valid)
        ↓
aws-auth ConfigMap (maps the IAM ARN to a Kubernetes username/group)
        ↓
Kubernetes User / Group (now Kubernetes knows WHO this is)
        ↓
RBAC Authorization (checks if this user/group has permission for THIS action)
        ↓
API Server Response (allowed or denied)
```

9.4 Complete RBAC Setup — Step by Step

```bash
# STEP 1: Create IAM user with EKS cluster permissions
# (Done via AWS Console or CLI — outside Kubernetes)

# STEP 2: Configure AWS CLI profile for that user
aws configure --profile IAMuser
```

YAML

```yaml
# STEP 3: Create a Kubernetes Role
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: developer-role
rules:
  - apiGroups: ['']          # '' = core API group
    resources: ['pods']
    verbs: ['get', 'list']
  - apiGroups: ['apps']
    resources: ['deployments']
    verbs: ['get', 'list']
```

YAML

```yaml
# STEP 4: Create a RoleBinding (connect Role to a Group)
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default
subjects:
  - kind: Group
    name: 'developer'
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: developer-role
  apiGroup: rbac.authorization.k8s.io
```

BASH

```bash
# STEP 5: Add the IAM user to the aws-auth ConfigMap, mapping to the
# 'developer' group
kubectl edit cm aws-auth -n kube-system
```

```yaml
# Add this under mapUsers:
mapUsers: |
  - userarn: arn:aws:iam::730335657713:user/nareshit
    username: nareshit
    groups:
    - developer
```

```bash
# STEP 6: Update local kubeconfig for the new user to test
aws eks update-kubeconfig --name test --profile devops
```

9.5 Granting Specific Resource Access

```yaml
# Restrict access to ONE specific pod (not all pods)
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-read-specific
  namespace: default
rules:
  - apiGroups: ['']
    resources: ['pods']
    resourceNames:        # restricts to SPECIFIC named resource(s)
      - my-pod
    verbs:
      - get
      - describe
```

```yaml
# Bind a Role DIRECTLY to a User (without using a Group)
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default
subjects:
  - kind: User
    name: developer1
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: developer-role
  apiGroup: rbac.authorization.k8s.io
```

```yaml
# aws-auth mapping for the individual user:
mapUsers: |
  - userarn: arn:aws:iam::730335657713:user/developer1
    username: developer1
```

9.6 ClusterRole — Full Admin Permissions

```yaml
# ClusterRole granting ALL permissions on ALL resources cluster-wide
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cluster-admin-custom
rules:
  - apiGroups: ['*']    # ALL API groups
    resources: ['*']    # ALL resources (pods, services, deployments, etc.)
    verbs: ['*']        # ALL actions (get, list, create, update, delete, etc.)
```

```yaml
# Bind ClusterRole to a GROUP across the entire cluster
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: cluster-admin-custom-binding
subjects:
- kind: Group
  name: admin-team    # mapped via aws-auth
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: cluster-admin-custom
  apiGroup: rbac.authorization.k8s.io
```

9.7 system:masters — Built-in Super Admin

⚠️ **Important Warning:**

`system:masters` is a BUILT-IN Kubernetes group providing FULL CLUSTER-ADMIN access — bypasses all RBAC checks entirely. Use this group with EXTREME caution — it is equivalent to root access on the entire cluster. Only grant `system:masters` to a tiny number of trusted administrators.

```yaml
# Grant a USER full admin access via system:masters
mapUsers: |
  - userarn: arn:aws:iam::381491944316:user/user-1
    username: user-1
    groups:
    - system:masters
```

```yaml
# Grant a ROLE (e.g., an EC2 instance role) full admin access
mapRoles: |
  - groups:
    - system:masters
    rolearn: arn:aws:iam::545009827818:role/ec2-admin2
    username: ec2-admin2
```

```yaml
# Complete aws-auth ConfigMap example with node group + admin user
apiVersion: v1
data:
  mapRoles: |
    - groups:
      - system:bootstrappers
      - system:nodes
      rolearn: arn:aws:iam::992382358200:role/eksctl-naresh-nodegroup-
NodeInstanceRole-9GWNpfucPXRt
      username: system:node:{{EC2PrivateDNSName}}
  mapUsers: |
    - userarn: arn:aws:iam::483216680875:user/devops
      username: devops
      groups:
      - developer
kind: ConfigMap
metadata:
  name: aws-auth
  namespace: kube-system
```

9.8 Service Accounts for Pod-to-API Communication

A **ServiceAccount** is an identity used by PODS (not humans) to authenticate with the Kubernetes API. By default, every pod uses the 'default' ServiceAccount in its namespace — but for security, you should create DEDICATED ServiceAccounts with minimal required permissions for each application.

YAML

```yaml
# 1. Create a Service Account
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-service-account
  namespace: default
```

YAML

```yaml
# 2. Create a Role for it (e.g., read-only pod access)
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: default
rules:
  - apiGroups: ['']
    resources: ['pods']
    verbs: ['get', 'list']
```

YAML

```yaml
# 3. Bind the Role to the Service Account
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: default
subjects:
  - kind: ServiceAccount
    name: my-service-account
    namespace: default
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

```yaml
# 4. Attach the Service Account to a Pod
apiVersion: v1
kind: Pod
metadata:
  name: myapp
spec:
  serviceAccountName: my-service-account    # attach it here
  containers:
  - name: nginx-container
    image: nginx
```

```bash
# Verify: exec into the pod, try listing pods
kubectl exec -it myapp -- /bin/bash
kubectl get pods    # should succeed — has pod-reader role
```

📝 **Theory & Key Points:**

- ServiceAccount → an identity used BY pods to call the API.
- Role & RoleBinding → grant permissions TO that ServiceAccount.
- Attach to Pod → via `serviceAccountName` field in the pod spec.
- `kubectl get rb` / `kubectl get rolebinding` / `kubectl api-resources` — useful commands to audit existing RBAC.
- After ANY change to aws-auth, run `aws eks update-kubeconfig` for the affected user to refresh their access.

🎯 Scenario-Based Interview Questions

Q1: Scenario: A developer can run `kubectl get pods` successfully but gets 'Forbidden' when running `kubectl delete pod mypod`. Explain exactly why, using RBAC concepts. This means their Role grants 'get' and 'list' verbs on pods, but NOT the 'delete' verb.

RBAC checks EVERY action against the exact list of allowed verbs:

```yaml
rules:
  - apiGroups: ['']
    resources: ['pods']
    verbs: ['get', 'list']    # delete is NOT here
```

Fix: add 'delete' to the verbs list if this user should be allowed to delete pods: `verbs: ['get', 'list', 'delete']`

This demonstrates RBAC's PRINCIPLE OF LEAST PRIVILEGE — by default, NOTHING is allowed; you must explicitly grant each specific action. 'Forbidden' errors are RBAC working correctly, not a bug.

Q2: Scenario: Your platform team needs Team A to have full access to namespace 'team-a-ns' but ZERO visibility into namespace 'team-b-ns'. How do you architect this with RBAC? Use NAMESPACE-SCOPED Roles + RoleBindings (NOT ClusterRole, which would span all namespaces):

1. Create a Role scoped to team-a-ns:

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: team-a-full-access
  namespace: team-a-ns    # SCOPED to this namespace only
rules:
  - apiGroups: ['*']
    resources: ['*']
    verbs: ['*']
```

2. Create a RoleBinding in team-a-ns:

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: team-a-binding
  namespace: team-a-ns
subjects:
  - kind: Group
    name: team-a-developers
roleRef:
  kind: Role
  name: team-a-full-access
```

3. Map Team A's IAM users to the 'team-a-developers' group in aws-auth

Result: Team A has FULL control inside team-a-ns, but since NO Role/RoleBinding exists for them in team-b-ns, they get 'Forbidden' for ANY action there — including even `kubectl get pods -n team-b-ns`.

Q3: Scenario: An engineer accidentally added their own IAM user to the system:masters group in aws-auth while testing. What is the security risk, and what should you do? CRITICAL SECURITY RISK: `system:masters` grants COMPLETE CLUSTER-ADMIN access, bypassing ALL RBAC restrictions entirely — this user can now delete ANY resource in ANY namespace, modify RBAC itself, read ALL secrets, and even delete the entire cluster's workloads.

Immediate remediation:

1. Edit aws-auth ConfigMap immediately: `kubectl edit cm aws-auth -n kube-system`
2. Remove the system:masters group assignment for that user
3. Replace with an appropriately scoped Role/RoleBinding for their ACTUAL job function
4. Audit CloudTrail/Kubernetes audit logs for any actions taken while they had system:masters access
5. Review the process that allowed unreviewed changes to aws-auth — implement PR-based GitOps for aws-auth changes (e.g., managed via Terraform with mandatory code review)

Best practice: NEVER directly edit aws-auth manually in production — manage it via Infrastructure as Code (Terraform aws-auth resource or eksctl) with PR review enforced.

Q4: Scenario: A pod running in your cluster needs to call the Kubernetes API to list other pods (for a custom monitoring tool), but should NOT be able to delete or modify anything. How do you set this up securely? Use a dedicated, minimally-scoped ServiceAccount — NEVER use the default ServiceAccount or grant cluster-admin:

1. Create ServiceAccount:

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: monitoring-readonly
  namespace: monitoring
```

2. Create READ-ONLY Role:

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole    # use ClusterRole if monitoring needs to see pods across ALL
namespaces
metadata:
  name: pod-watcher
rules:
  - apiGroups: ['']
    resources: ['pods']
    verbs: ['get', 'list', 'watch']    # READ-ONLY — no create/update/delete
```

3. Bind with ClusterRoleBinding (since it needs cross-namespace visibility):

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: monitoring-binding
subjects:
  - kind: ServiceAccount
    name: monitoring-readonly
    namespace: monitoring
roleRef:
  kind: ClusterRole
  name: pod-watcher
```

4. Attach to pod: `serviceAccountName: monitoring-readonly`

This follows least-privilege: the monitoring tool can WATCH pods cluster-wide but cannot delete, create, or modify ANYTHING.

---

10. EKS Pod-to-AWS Service Communication (IRSA)

Applications running inside EKS pods often need to access AWS services like S3, DynamoDB, or SQS. The secure, AWS-recommended way to do this is **IAM Roles for Service Accounts (IRSA)** — it allows EACH POD to assume its own IAM role with exactly the permissions it needs, without ever storing AWS credentials anywhere.

💡 **Layman Explanation:**

- IRSA = giving each employee (pod) their OWN keycard (IAM role) instead of sharing one master key (Node IAM Role) among everyone.
- Without IRSA: ALL pods on a node share the SAME permissions — if one pod is compromised, attacker gets ALL the node's AWS access.
- With IRSA: Each pod gets temporary, auto-rotating credentials scoped to EXACTLY what that pod needs — nothing more.

10.1 How IRSA Works — Architecture

🏗️ **Architecture:**

```text
EKS Pod
   ↓
ServiceAccount (annotated with an IAM Role ARN)
   ↓
OIDC Provider (validates the pod's identity token)
   ↓
IAM Role (IRSA) — temporary credentials issued, auto-rotated
   ↓
Amazon S3 (or any AWS service the IAM Role permits)
```

10.2 Complete IRSA Setup — S3 Access Example

```json
// STEP 1: Create IAM policy JSON for S3 access
// s3-access-policy.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListAllMyBuckets", "s3:ListBucket"],
      "Resource": "*"
    }
  ]
}
```

BASH

```bash
# STEP 2: Create the IAM policy in AWS
aws iam create-policy \
  --policy-name EKS_S3_Access_Policy \
  --policy-document file://s3-access-policy.json
# Output: arn:aws:iam::664418968609:policy/EKS_S3_Access_Policy

# STEP 3: Associate OIDC provider with your EKS cluster (one-time per cluster)
# This allows AWS IAM to TRUST tokens issued by your EKS cluster
eksctl utils associate-iam-oidc-provider \
  --region=us-east-1 \
  --cluster=project-eks \
  --approve

# STEP 4: Create an IAM Role bound to a ServiceAccount (using eksctl)
eksctl create iamserviceaccount \
  --name s3-access-sa \
  --namespace default \
  --cluster project-eks \
  --attach-policy-arn arn:aws:iam::975050030406:policy/EKS_S3_Access_Policy \
  --approve
```

YAML

```yaml
# STEP 5: Create a pod that uses this ServiceAccount
apiVersion: v1
kind: Pod
metadata:
  name: aws-cli-pod
spec:
  serviceAccountName: s3-access-sa    # this gives the pod IAM permissions!
  containers:
  - name: aws-cli
    image: amazonlinux:2
    command: ['sleep', '3600']
    tty: true
```

```bash
# STEP 6: Apply and test
kubectl apply -f aws-cli-pod.yaml
kubectl exec -it aws-cli-pod -- /bin/bash

# STEP 7: Install AWS CLI inside the pod (for testing)
yum install -y unzip curl
curl 'https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip' -o 'awscliv2.zip'
unzip awscliv2.zip && ./aws/install

# STEP 8: Test access — NO hardcoded credentials needed!
aws s3 ls
# Output:
# 2025-03-12 14:02:28 mysmh.co.in
# 2025-02-16 16:50:28 syedmujtaba
```

10.3 Security Comparison — Three Approaches

METHOD	HOW IT WORKS	SECURITY RATING
IRSA (Recommended)	Pod → ServiceAccount → IAM Role → AWS Service. Temporary, auto-rotated.	★★★★★ Production standard
Node IAM Role	Pod → Node IAM Role → AWS Service. ALL pods on node share same access.	★★ Limited — multi-tenant risk
Access Keys in Pod	Hardcoded AWS_ACCESS_KEY_ID in env vars/secrets.	★ NOT recommended — leakage risk

📝 **Theory & Key Points:**

- IRSA Benefits: No hardcoded AWS keys, least privilege per pod, temporary auto-rotated credentials, AWS-recommended for production.
- Node IAM Role problem: ALL pods on that EC2 node inherit the SAME permissions — a compromised pod gets the full node's AWS access.
- Access Keys problem: Stored in env vars or Kubernetes Secrets — risk of leakage, requires manual rotation, audit trail is weaker.
- OIDC (OpenID Connect) provider is the TRUST mechanism that lets AWS IAM verify a Kubernetes ServiceAccount token is legitimate.

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your security team flagged that 'all pods in the cluster can access the production S3 bucket' during an audit. Investigation shows pods are using the Node IAM Role. How do you fix this with IRSA? Migrate from Node IAM Role to IRSA for least-privilege access:

1. Identify which SPECIFIC pods actually need S3 access (e.g., only the 'report-generator' app)
2. Remove broad S3 permissions from the Node IAM Role (the role attached to EC2 instances in the node group)
3. Create a scoped IAM policy with ONLY the specific S3 actions/resources needed: `{ "Effect": "Allow", "Action": ["s3:GetObject", "s3:PutObject"], "Resource": "arn:aws:s3:::prod-reports-bucket/*" }`
4. Create an IRSA ServiceAccount: `eksctl create iamserviceaccount --name report-generator-sa --attach-policy-arn <scoped-policy-arn> --approve`
5. Update ONLY the report-generator Deployment to use `serviceAccountName: report-generator-sa`
6. All OTHER pods now have ZERO S3 access (since Node Role no longer grants it)

Result: Only the specific application that needs S3 access gets it — and ONLY for the exact actions/bucket it needs. This satisfies the audit's least-privilege requirement.

Q2: Scenario: A pod with IRSA configured still gets 'Access Denied' when calling `aws s3 ls`. Walk through the troubleshooting steps. Systematic IRSA troubleshooting:

1. Verify OIDC provider is associated with the cluster: `eksctl utils associate-iam-oidc-provider --cluster=<name> --approve` (idempotent — safe to re-run)
2. Check the ServiceAccount has the correct IAM role annotation: `kubectl get sa s3-access-sa -o yaml` → Look for: `eks.amazonaws.com/role-arn: arn:aws:iam::ACCOUNT:role/...`
3. Verify the pod is actually using that ServiceAccount: `kubectl get pod aws-cli-pod -o jsonpath='{.spec.serviceAccountName}'`
4. Check inside the pod that AWS SDK credentials are being picked up: `env | grep AWS` → should show AWS_ROLE_ARN and AWS_WEB_IDENTITY_TOKEN_FILE
5. Verify the IAM policy attached to the role actually grants the needed permission (check exact Action and Resource match)
6. Check IAM role's TRUST POLICY references the correct OIDC provider and `namespace:serviceaccount` condition

7. If the pod was created BEFORE the ServiceAccount existed/was updated: restart the pod — environment variables are injected at pod CREATION time, not dynamically.

---

11. Resource Requests/Limits & Image Pull Policy

Two often-overlooked but CRITICAL pod configuration settings: **resource requests/limits** (controlling how much CPU/memory a container can use) and **imagePullPolicy** (controlling WHEN Kubernetes downloads container images). Misconfiguring either causes real production problems — from resource starvation to stale image deployments.

11.1 Resource Requests and Limits

Requests = **minimum** guaranteed resources for scheduling. Limits = **maximum** resources the container is ALLOWED to consume. The scheduler uses REQUESTS to decide which node has room for the pod; the kubelet enforces LIMITS at runtime.

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: example-pod
spec:
  containers:
  - name: example-container
    image: nginx:latest
    resources:
      requests:
        cpu: '50m'        # 50 millicores = 0.05 of one CPU core
        memory: '100Mi'   # 100 Mebibytes
      limits:
        cpu: '100m'       # max 0.1 of one CPU core
        memory: '200Mi'   # max 200 Mebibytes
```

SETTING	MEANING
requests.cpu / requests.memory	MINIMUM resources guaranteed for scheduling. Scheduler only places pod on a node with this much SPARE capacity.
limits.cpu / limits.memory	MAXIMUM resources the container can use. Exceeding memory limit → OOMKilled. Exceeding CPU limit → throttled (NOT killed).
CPU units	Measured in 'millicores' (m). 1000m = 1 full CPU core. 500m = half a core.
Memory units	Mi (Mebibyte, 2^20 bytes) or Gi (Gibibyte). Different from M/G (decimal — used by marketing, not K8s).

⚠️ **Important Warning:**

Exceeding MEMORY limit → container is immediately OOMKilled (process terminated) and restarted. Exceeding CPU limit → container is THROTTLED (slowed down), NOT killed. CPU is 'compressible'; memory is not. If you don't set requests/limits at all → pod can consume UNLIMITED resources, potentially starving other pods on the same node ('noisy neighbor' problem). Always set BOTH requests and limits in production — never leave them unset.

11.2 imagePullPolicy

Controls WHEN kubelet downloads (pulls) a container image — every time, only if missing, or never.

POLICY	BEHAVIOR
Always	Image is ALWAYS pulled from the registry, even if a local copy exists. Ensures you get the latest version of a mutable tag (like `:latest`).
IfNotPresent	Image is pulled ONLY if not already cached on the node. Faster restarts — but risk of stale images if the tag was updated without changing the tag name.
Never	Image is NEVER pulled — must already exist locally on the node, or the pod fails with ImagePullBackOff.

11.3 Default Behavior Rules

YAML

```yaml
# RULE: If imagePullPolicy is NOT explicitly set, the DEFAULT depends on the
# image tag:

image: nginx           # implicitly 'nginx:latest' → default policy = Always
image: nginx:latest    # explicitly latest         → default policy = Always
image: nginx:1.19      # specific version tag       → default policy = IfNotPresent

# Explicitly setting the policy (always recommended for clarity):
apiVersion: v1
kind: Pod
metadata:
  name: myapp
spec:
  containers:
  - name: nginx-container
    image: nginx:1.19
    imagePullPolicy: IfNotPresent    # explicit is better than implicit
```

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your production cluster experiences a 'noisy neighbor' problem — one misbehaving pod consumes all CPU on a node, starving 5 other pods that become unresponsive. How does setting resource limits prevent this? Without limits, ANY pod can consume unlimited CPU/memory on its node — a memory leak or infinite loop in one application can starve ALL other pods sharing that node.

Fix: Set resource requests AND limits on every container:

```yaml
resources:
  requests:
    cpu: '250m'
    memory: '256Mi'
  limits:
    cpu: '500m'       # hard ceiling — Kubernetes throttles CPU beyond this
    memory: '512Mi'   # hard ceiling — exceeding this OOMKills the container
```

With limits in place:

- The misbehaving pod is CPU-throttled at 500m — it cannot consume more, regardless of how much it tries
- If it leaks memory past 512Mi, it gets OOMKilled and restarted — contained, not affecting neighbors
- Other pods on the node continue running normally with their guaranteed requests

Additionally: implement ResourceQuota at the namespace level and LimitRange to enforce default requests/limits cluster-wide, preventing any pod from being deployed WITHOUT them.

Q2: Scenario: You pushed a new version of your Docker image tagged 'myapp:v1' (same tag, updated content) to ECR, but your Kubernetes pods are still running the OLD code after redeployment. What's the root cause? Root cause: imagePullPolicy is set to 'IfNotPresent' (or defaulted to it because you used a specific tag, not 'latest'). Since 'myapp:v1' was already cached on the node from a previous pull, Kubernetes assumes it doesn't need to re-pull — even though the IMAGE CONTENT behind that tag changed in the registry.

Fix Option 1 (immediate): Force a re-pull by deleting the pod (forces a fresh schedule + pull if policy allows, but IfNotPresent still won't help if cached): Better: `kubectl rollout restart deployment/myapp` (combined with `imagePullPolicy: Always`)

Fix Option 2 (correct long-term practice): NEVER reuse mutable tags like 'v1' or 'latest' for actual deployments. Use IMMUTABLE, UNIQUE tags per build: `myapp:v1.2.3-build456` or `myapp:git-sha-abc123` This way, EVERY deployment uses a brand-new, never-before-seen tag → kubelet is FORCED to pull it (cache miss), regardless of imagePullPolicy.

This is industry best practice: tag images with git commit SHA or semantic version + build number — never overwrite an existing tag's content.

---

12. Pod Scheduling — NodeSelector, Affinity, DaemonSet, Taints

Kubernetes' Scheduler decides which node a pod should run on. By default, it considers resource availability — but you can add constraints to control placement more precisely: forcing pods onto specific hardware, spreading pods across all nodes, or repelling pods from certain nodes entirely. These mechanisms are essential for production workloads with hardware requirements, compliance needs, or specialized node pools (GPU nodes, spot instances, etc.).

12.1 NodeSelector — Simple Node Constraints

The simplest scheduling constraint. You label nodes, then specify `nodeSelector` in the pod spec — the scheduler ONLY places the pod on nodes matching ALL specified labels.

BASH

```
# Label a node
kubectl label nodes ip-192-168-60-75.ec2.internal size=large
kubectl label nodes ip-192-168-4-19.ec2.internal size=small

# Remove a label (note the trailing dash)
kubectl label nodes ip-192-168-24-53.us-west-2.compute.internal size-

# List all nodes with their labels
kubectl get nodes --show-labels
```

```yaml
# Pod using nodeSelector
apiVersion: v1
kind: Pod
metadata:
  name: myapp
spec:
  containers:
  - name: nginx-container
    image: nginx
  nodeSelector:
    size: large    # ONLY schedules on nodes labeled size=large
```

📝 **Theory & Key Points:**

- Labels are CASE-SENSITIVE: 'Large' ≠ 'large'.
- An UNLABELED pod (no nodeSelector) CAN schedule on a LABELED node — labels don't restrict pods without a selector.
- A LABELED pod (with nodeSelector) CANNOT schedule on an UNLABELED node — must find a matching label.
- If you REMOVE a node's label AFTER a pod is already scheduled there: NO IMPACT on the already-running pod — it keeps running. The label only matters at SCHEDULING time, not afterward.
- If NO node matches the required label, the pod stays in 'Pending' state forever.

12.2 Node Affinity — More Expressive Constraints

Node Affinity is conceptually similar to nodeSelector but with more expressive syntax (operators like `In`, `NotIn`, `Exists`) and two distinct behaviors: strict requirement vs soft preference.

TYPE	BEHAVIOR
requiredDuringSchedulingIgnoredDuringExecution	HARD requirement — scheduler CANNOT place pod unless the rule matches. Functions like nodeSelector, but with richer syntax (operators).
preferredDuringSchedulingIgnoredDuringExecution	SOFT preference — scheduler TRIES to find a matching node, but schedules on ANY available node if no match is found.

YAML

```yaml
# a) REQUIRED node affinity — strict, like nodeSelector but more expressive
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: disktype
            operator: In
            values:
            - ssd
  containers:
  - name: nginx
    image: nginx
    imagePullPolicy: IfNotPresent
```

```yaml
# b) PREFERRED node affinity — soft, falls back to ANY node if no match
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 1
        preference:
          matchExpressions:
          - key: disktype
            operator: In
            values:
            - ssd
  containers:
  - name: nginx
    image: nginx
    imagePullPolicy: IfNotPresent
```

12.3 DaemonSet — One Pod Per Node

A **DaemonSet** ensures a copy of a pod runs on EVERY node in the cluster (or a filtered subset). Unlike Deployments (which scale to N replicas distributed across SOME nodes), DaemonSets specifically guarantee node-level coverage — essential for log collectors, monitoring agents, and network plugins.

💡 **Layman Explanation:**

- Deployment: 'I want 3 copies of my app, somewhere in the cluster' — Kubernetes decides where.
- DaemonSet: 'I want EXACTLY ONE copy running on EVERY SINGLE node' — like a security guard stationed at every building entrance.
- Example: if you have 3 nodes, a DaemonSet creates exactly 3 pods — one per node, automatically.
- Add a 4th node → DaemonSet automatically creates a 4th pod there too.

YAML

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: nginx
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: test-nginx
        image: nginx
        ports:
        - containerPort: 8080
        resources:
          limits:
            cpu: 100m
            memory: 200Mi
          requests:
            cpu: 50m
            memory: 100Mi
```

📝 **Theory & Key Points:**

- Real-world DaemonSet use cases: Fluentd/Fluent Bit (log collection), Datadog/Prometheus Node Exporter (monitoring), Calico/Cilium (CNI networking plugins).
- If you have 3 worker nodes, the SAME pod definition automatically creates 3 pods — one guaranteed per node.

12.4 Taints and Tolerations — Repelling Pods

While NodeSelector/Affinity work from the Pod's perspective ('I want THIS node'), Taints and Tolerations work from the Node's perspective ('I REJECT pods unless they specifically tolerate me'). A **Taint** is applied to a node to mark it as special — the scheduler will NOT place ANY pod there UNLESS that pod has a matching **Toleration**.

TAINT EFFECT	BEHAVIOR
NoSchedule	NEW pods WITHOUT matching toleration will NOT be scheduled on this node. EXISTING pods already running are NOT evicted.
NoExecute	NEW pods WITHOUT matching toleration will NOT be scheduled. EXISTING pods WITHOUT matching toleration are IMMEDIATELY EVICTED.

BASH

```
# Apply a taint to a node
kubectl taint nodes ip-192-168-3-253.ap-south-1.compute.internal app=blue:NoSchedule

# Apply NoExecute taint (evicts existing non-tolerating pods immediately)
kubectl taint nodes ip-192-168-40-106.ec2.internal app=blue:NoExecute

# Remove a taint (note the trailing dash)
kubectl taint node ip-192-168-36-40.ec2.internal app=blue:NoSchedule-

# Check which nodes are tainted
kubectl describe nodes <node-name> | grep Taints
kubectl get nodes -o custom-columns=NAME:.metadata.name,TAINTS:.spec.taints
```

YAML

```yaml
# Pod with a TOLERATION (allows scheduling on the tainted node)
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    env: test
spec:
  containers:
  - name: nginx
    image: nginx
    imagePullPolicy: IfNotPresent
  tolerations:
  - key: 'app'
    operator: 'Equal'
    value: 'blue'
    effect: 'NoSchedule'
```

⚠️ **Important Warning:**

Taint + Toleration only ALLOWS the pod to be scheduled on the tainted node — it does NOT FORCE it there. A tolerating pod CAN still be scheduled on OTHER (non-tainted) nodes too — toleration doesn't restrict to ONLY the tainted node. A pod WITHOUT a matching toleration can NEVER be scheduled on a tainted node, even if every other node is full. Common mistake: confusing 'Toleration' with 'Affinity' — Toleration only REMOVES the repulsion; it doesn't ATTRACT the pod. Combine with nodeSelector/Affinity if you want pods to specifically PREFER the tainted node.

12.5 Draining a Node — Maintenance Operations

The `kubectl drain` command safely evicts all pods from a node so it can be taken down for maintenance or removed from the cluster — without disrupting your applications (since their ReplicaSets/Deployments automatically recreate them on other healthy nodes).

BASH

```
# Drain a node (evicts pods, prevents new scheduling)
kubectl drain ip-192-168-56-36.ap-south-1.compute.internal \
  --ignore-daemonsets \
  --delete-emptydir-data

# --ignore-daemonsets: allows draining even with DaemonSet pods present
#                      (DaemonSet pods are designed to run on every node —
#                       can't 'evict' them elsewhere)
# --delete-emptydir-data: allows deletion of emptyDir volume data (ephemeral,
#                         tied to node)

# Verify drain completed
kubectl get nodes
# Node shows 'SchedulingDisabled' status — drained successfully

# Make the node schedulable again after maintenance
kubectl uncordon ip-192-168-56-36.ap-south-1.compute.internal
```

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your company has a GPU node pool for ML workloads (expensive) and a regular CPU node pool. How do you ensure ONLY ML pods run on the GPU nodes, while preventing regular pods from accidentally scheduling there and wasting GPU resources? Use a COMBINATION of Taints (to REPEL non-ML pods) AND Node Affinity (to ATTRACT ML pods):

1. Taint the GPU nodes: `kubectl taint nodes gpu-node-1 workload=gpu:NoSchedule` → Now NO pod can schedule here UNLESS it tolerates this taint
2. Add toleration + node affinity to ONLY the ML pod spec:

```yaml
tolerations:
- key: 'workload'
  operator: 'Equal'
  value: 'gpu'
  effect: 'NoSchedule'
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - key: workload
          operator: In
          values: ['gpu']
```

Result:

- Regular pods (no toleration) CANNOT schedule on GPU nodes (repelled by taint)
- ML pods (toleration + affinity) WILL be scheduled on GPU nodes specifically (affinity attracts them there, toleration allows it)
- This combination is the standard pattern for dedicated/specialized node pools.

Q2: Scenario: You need to perform OS-level security patching on a worker node running 15 production pods. How do you do this with ZERO downtime to your application? Use `kubectl drain`, relying on Kubernetes' self-healing to reschedule pods elsewhere FIRST:

1. Verify other nodes have spare capacity for the 15 pods: `kubectl describe nodes` (check available CPU/memory across other nodes)

2. Drain the target node: `kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data`
    - Kubernetes gracefully terminates pods on this node
    - Since they're managed by Deployments/ReplicaSets, NEW pods are immediately created on OTHER healthy nodes
    - If using PodDisruptionBudgets, drain respects minimum-available constraints (won't take down too many replicas simultaneously)
3. Verify pods are healthy on other nodes: `kubectl get pods -o wide`
4. Perform OS patching/maintenance on the now-empty node
5. Bring the node back into service: `kubectl uncordon <node-name>`

This achieves zero-downtime maintenance — IF your application has multiple replicas (≥2) and a PodDisruptionBudget configured to prevent over-aggressive eviction.

Q3: Scenario: You applied a NoExecute taint to a node while debugging a stateful database pod that was running there. The pod was immediately evicted and its data was lost. What went wrong and how do you prevent this? Root cause: NoExecute taints IMMEDIATELY EVICT any existing pod WITHOUT a matching toleration — unlike NoSchedule, which only blocks NEW scheduling but leaves running pods alone.

If the database pod used local node storage (hostPath or emptyDir) without proper PersistentVolume backing, data was permanently lost upon eviction.

Prevention going forward:

1. NEVER use NoExecute taints on nodes running stateful workloads without confirming proper persistent storage (EBS-backed PV, not emptyDir/hostPath)
2. For stateful pods, add a toleration with `tolerationSeconds` to allow GRACEFUL handling:

```yaml
tolerations:
- key: 'app'
  operator: 'Equal'
  value: 'blue'
  effect: 'NoExecute'
  tolerationSeconds: 300    # gives 5 minutes before eviction — time to react/backup
```

3. Use proper StatefulSet + PersistentVolumeClaim with cloud-backed storage (EBS) instead of node-local storage — covered in the Volumes section. EBS volumes persist independently of pod/node lifecycle.

---

13. Kubernetes Volumes — Persistent Storage

Containers are ephemeral by nature — when a container restarts, ANY data written inside it is lost. A **Kubernetes Volume** is a directory accessible to containers in a pod that persists data beyond a single container's lifecycle. However, a volume's lifecycle is still tied to its POD — when the pod is deleted, the volume is destroyed (unless using cloud-backed persistent storage).

💡 **Layman Explanation:**

- Container's own filesystem = a whiteboard that gets WIPED CLEAN every time the container restarts.
- Volume = a separate notebook that survives even if the container restarts, AS LONG AS the pod itself isn't deleted.
- Persistent Volume (cloud-backed, e.g., EBS) = a notebook stored in a SAFE outside the building — survives even if the building (node) is demolished.

13.1 Three Volume Provisioning Approaches

APPROACH	HOW IT WORKS	RECOMMENDATION
1. Static hostPath	Volume created on the node's local disk directly. PV+PVC+Deployment.	NOT RECOMMENDED — data lost if node is deleted
2. Static Cloud EBS	Manually create an EBS volume in AWS, then bind PV+PVC to it.	Works but EBS must be created MANUALLY each time
3. Dynamic Provisioning (StorageClass)	StorageClass automatically creates EBS + PV when a PVC requests storage.	RECOMMENDED — fully automated

⚠️ **Important Warning:**

hostPath LIMITATION: The volume is LOCAL to the specific node where it was created. If the pod is rescheduled to a DIFFERENT node (common during failures/maintenance), it tries to use the same path on the NEW node — but that directory has NO data from the original node, causing DATA LOSS.

13.2 Setting Up Dynamic Provisioning with EBS CSI Driver

BASH

```
# Pre-requisite: node group IAM role needs EBS full access (or scoped EBS CSI
# permissions)

# Install Helm (required for CSI driver installation)
wget https://get.helm.sh/helm-v3.14.0-linux-amd64.tar.gz
tar -zxvf helm-v3.14.0-linux-amd64.tar.gz
mv linux-amd64/helm /usr/local/bin/helm
chmod 777 /usr/local/bin/helm
helm version

# Install AWS EBS CSI Driver via Helm
helm repo add aws-ebs-csi-driver https://kubernetes-sigs.github.io/aws-ebs-csi-driver
helm repo update
helm upgrade --install aws-ebs-csi-driver \
  --namespace kube-system \
  aws-ebs-csi-driver/aws-ebs-csi-driver
```

13.3 Access Modes

ACCESS MODE	USE CASE
ReadWriteOnce (RWO)	Volume can be mounted READ-WRITE by a SINGLE node. Most common for databases.
ReadWriteMany (RWX)	Volume can be mounted READ-WRITE by MULTIPLE nodes simultaneously. Needed for shared file storage (e.g., NFS, EFS).
ReadOnlyMany (ROX)	Volume can be mounted READ-ONLY by MULTIPLE nodes. Good for shared config/reference data.

AWS EBS volumes only support ReadWriteOnce. For ReadWriteMany, use EFS (Elastic File System) instead.

13.4 PVC Reclaim Policies & Binding Modes

SETTING	BEHAVIOR
reclaimPolicy: Delete (default)	When the PVC is deleted, the PV is automatically deleted too. BUT the underlying EBS volume is NOT auto-deleted — you must delete it manually.
reclaimPolicy: Retain	The PV (and underlying storage) PERSISTS even after the PVC is deleted — protects against accidental data loss.
volumeBindingMode: Immediate (default)	The PV is created IMMEDIATELY when the PVC is created, before any pod claims it.
volumeBindingMode: WaitForFirstConsumer	The PV is created ONLY when a pod actually CLAIMS the storage — ensures correct AZ placement (volume created in same AZ as the pod's node).

13.5 Complete Dynamic Provisioning Example

YAML

```yaml
# StorageClass — defines HOW to dynamically provision storage
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ebs-sc
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer    # wait for pod to determine AZ
reclaimPolicy: Retain                       # protect data even if PVC deleted
parameters:
  type: gp3
```

YAML

```yaml
# PersistentVolumeClaim — requests storage from the StorageClass
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myapp-pvc
spec:
  accessModes:
  - ReadWriteOnce
  storageClassName: ebs-sc
  resources:
    requests:
      storage: 10Gi
```

YAML

```yaml
# Deployment using the PVC
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 1
  selector:
    matchLabels: { app: myapp }
  template:
    metadata:
      labels: { app: myapp }
    spec:
      containers:
      - name: myapp
        image: nginx
        volumeMounts:
        - name: data
          mountPath: /usr/share/nginx/html
      volumes:
      - name: data
        persistentVolumeClaim:
          claimName: myapp-pvc
```

🎯 **Scenario-Based Interview Questions**

Q1: Scenario: A pod using hostPath volume for a database was rescheduled to a different node after a deployment update. All historical data disappeared. What went wrong, and what's the correct fix? Root cause: hostPath volumes are LOCAL to the specific node they were created on. When the scheduler placed the pod on a NEW node (during rescheduling), Kubernetes mounted the SAME path (e.g., `/mnt/data`) — but on a DIFFERENT physical disk that has never seen this data before. The 'volume' appears empty because it literally IS a different, unrelated directory.

Fix: Migrate to cloud-backed dynamic provisioning (EBS via StorageClass):

1. Create a StorageClass with `provisioner: ebs.csi.aws.com`
2. Create a PVC requesting storage from that StorageClass
3. Mount the PVC in the pod instead of hostPath

With EBS-backed storage: the volume is a SEPARATE AWS resource, independent of any specific node. When the pod is rescheduled, Kubernetes automatically DETACHES the EBS volume from the old node and RE-ATTACHES it to the new node — the data follows the pod, not the node.

Q2: Scenario: You set `reclaimPolicy: Delete` (the default) on your StorageClass. A developer accidentally ran `kubectl delete pvc database-pvc`, and now you discover the EBS volume containing 6 months of customer data still exists in AWS but is 'released' and orphaned. Why didn't it auto-delete, and how do you recover? With `reclaimPolicy: Delete`, when a PVC is deleted, Kubernetes deletes the PV object AND typically triggers EBS volume deletion via the CSI driver. However, in some configurations or race conditions, the underlying EBS volume can be left in a 'released'/orphaned state rather than fully deleted, OR the developer caught it before final deletion completed.

Recovery:

1. Check AWS Console → EC2 → Volumes → find the orphaned volume (look for tags referencing the old PVC name)
2. If still present, immediately create a snapshot to prevent any further risk: `aws ec2 create-snapshot --volume-id vol-xxx`
3. To reuse the data: create a NEW PV manually pointing to that EBS volume ID, then a NEW PVC binding to it

Prevention for the future:

1. ALWAYS use `reclaimPolicy: Retain` for critical/production data PVCs — this PREVENTS the PV (and EBS volume) from being deleted even if the PVC is accidentally removed
2. Restrict `kubectl delete pvc` via RBAC for non-admin users on production namespaces

3. Enable EBS volume deletion protection at the AWS level

Q3: Scenario: Your StatefulSet's pods keep getting scheduled in different Availability Zones than their EBS volumes, causing 'volume node affinity conflict' errors. How does volumeBindingMode fix this? Root cause: With `volumeBindingMode: Immediate` (the default), the EBS volume is created IMMEDIATELY when the PVC is created — BEFORE Kubernetes knows which node/AZ the pod will actually be scheduled to. If the volume gets created in us-east-1a but the pod is later scheduled to us-east-1b, EBS volumes (which are AZ-specific) cannot attach cross-AZ — causing the error.

Fix: Use `volumeBindingMode: WaitForFirstConsumer` in the StorageClass:

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ebs-sc
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer    # KEY FIX
```

With this setting: Kubernetes WAITS to create the actual EBS volume until a POD claims the PVC and is scheduled. At that point, Kubernetes KNOWS exactly which node/AZ the pod landed on, and creates the EBS volume in that SAME AZ — eliminating the affinity conflict entirely.

This is why WaitForFirstConsumer is the RECOMMENDED setting for any multi-AZ cluster.

---

14. StatefulSet & Headless Service

A **StatefulSet** is the workload API object for managing stateful applications — databases, message queues, distributed systems — where EACH pod needs a stable, unique, persistent identity that survives rescheduling. Unlike Deployments (where pods are interchangeable, like cattle), StatefulSet pods are like 'pets' — each one is unique and important.

💡 **Layman Explanation:**

- Deployment pods = identical twins wearing name tags that get randomly reassigned — any twin can do any job.
- StatefulSet pods = employees with PERMANENT name badges — pod-0 is ALWAYS pod-0, even after restarting.
- Database example: pod mysql-0 is the PRIMARY (handles writes). mysql-1 and mysql-2 are REPLICAS (handle reads).
- If mysql-0 restarts, it comes BACK as mysql-0 (not a randomly-named new pod) — clients can keep relying on that identity.

14.1 When to Use StatefulSets

STATEFULSET GUARANTEE	WHY IT MATTERS
Stable, unique network identifiers	Each pod gets a predictable hostname: `<statefulset-name>-0, -1, -2...` that persists across restarts
Stable, persistent storage	Each pod gets its OWN PersistentVolumeClaim that follows it across rescheduling — pod-0 always reattaches to its own EBS volume
Ordered, graceful deployment and scaling	Pods are created/deleted in ORDER: pod-0 first, then pod-1, then pod-2 (and reverse order for scale-down)
Ordered, automated rolling updates	Updates happen in REVERSE ordinal order (highest number first) — preserving consistency for the primary instance

14.2 Headless Service — Direct Pod Addressing

A **Headless Service** (`clusterIP: None`) does NOT load-balance traffic. Instead, DNS queries return the IPs of EACH individual pod directly. This is essential for StatefulSets — you need to address a SPECIFIC pod (the primary for writes) rather than a randomly-load-balanced one.

YAML

```yaml
# Two services + StatefulSet in the same file
apiVersion: v1
kind: Service
metadata:
  name: nginx-headless
  labels: { app: nginx }
spec:
  ports:
  - port: 80
    name: web
  clusterIP: None    # THIS makes it headless
  selector: { app: nginx }
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-normal
  labels: { app: nginx }
spec:
  ports:
  - port: 80
    name: web
  selector: { app: nginx }
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: 'nginx-headless'    # MUST reference the headless service
  replicas: 2
  selector:
    matchLabels: { app: nginx }
  template:
    metadata:
      labels: { app: nginx }
    spec:
```

```
      containers:
      - name: nginx
        image: k8s.gcr.io/nginx-slim:0.8
        ports:
        - containerPort: 80
          name: web
  volumeClaimTemplates:    # EACH pod gets its OWN PVC
  - metadata:
      name: www
    spec:
      accessModes: ['ReadWriteOnce']
      resources:
        requests:
          storage: 10Mi
```

14.3 DNS Resolution — Normal vs Headless

BASH

```
# Test from a temporary pod:
kubectl run -i --tty --image nginx:alpine dns-test

# nslookup the NORMAL service (load-balanced)
nslookup nginx-normal
# Returns: ONE cluster IP — acts like an internal load balancer
# Result is unpredictable which pod responds — NOT recommended for read/write-
# specific DB operations

# nslookup the HEADLESS service (direct pod addressing)
nslookup nginx-headless
# Returns MULTIPLE IPs — one per pod:
# Name:      mysql
# Address 1: 192.168.11.161 mysql-0.mysql.default.svc.cluster.local
# Address 2: 192.168.61.95  mysql-1.mysql.default.svc.cluster.local

# Curl a SPECIFIC pod directly (important for write vs read operations):
curl web-0.nginx-headless    # ALWAYS hits pod web-0 (e.g., primary/write)
curl web-1.nginx-headless    # ALWAYS hits pod web-1 (e.g., replica/read)
```

📝 **Theory & Key Points:**

- DNS pattern for headless service pods: `<pod-name>.<service-name>.<namespace>.svc.cluster.local`
- Example: `mysql-0.mysql.default.svc.cluster.local` always resolves to the SAME specific pod.
- Use case: write operations ALWAYS go to pod-0 (primary); read operations can go to pod-1, pod-2 (replicas).
- If there's only ONE database pod (no read/write split needed): a regular Deployment + ClusterIP Service is simpler and sufficient — no need for the StatefulSet complexity.

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your application connects to a MySQL StatefulSet using the NORMAL (load-balanced) service instead of the headless service. Sometimes writes succeed, sometimes they fail with 'read-only replica' errors. What's the root cause? Root cause: The normal/regular Service load-balances requests RANDOMLY across ALL matching pods — including BOTH the primary (mysql-0, accepts writes) AND replicas (mysql-1, mysql-2, READ-ONLY). When a write request happens to land on a replica pod, it fails because replicas reject write operations.

Fix: Use the HEADLESS service and address the PRIMARY pod SPECIFICALLY for writes:

```text
# Application config for WRITES:
DB_WRITE_HOST = 'mysql-0.mysql-headless.default.svc.cluster.local'

# Application config for READS (can use any replica, or even round-robin manually):
DB_READ_HOSTS = ['mysql-1.mysql-headless...', 'mysql-2.mysql-headless...']
```

This explicit read/write split via headless service DNS is the standard pattern for StatefulSet-based databases — it's the application's responsibility to route writes to the correct (primary) pod since Kubernetes itself doesn't understand database replication roles.

Q2: Scenario: You scale your StatefulSet from 3 to 5 replicas. In what order are the new pods created, and why does this ordering matter for a database cluster? StatefulSet GUARANTEES ORDERED, SEQUENTIAL pod creation: pod-3 is created and becomes READY BEFORE pod-4 starts creating.

Why this matters for databases:

- New replicas (pod-3, pod-4) typically need to SYNC DATA from the existing primary (pod-0) before becoming fully operational
- If pods were created in parallel (like a Deployment does), MULTIPLE replicas might try to sync simultaneously, overwhelming the primary or causing replication conflicts
- Sequential creation ensures pod-3 fully joins and stabilizes the cluster BEFORE pod-4 even starts — a controlled, safe expansion

Scale-DOWN order is the REVERSE: highest ordinal removed FIRST (pod-4 removed before pod-3) — ensures you don't accidentally remove the primary (pod-0) while replicas still depend on it.

This ordered behavior is THE defining characteristic that distinguishes StatefulSet from Deployment, where pod creation/deletion order is NOT guaranteed.

---

15. Health Probes — Startup, Readiness, Liveness

Kubernetes **Probes** are health-check mechanisms that continuously monitor whether your application is alive, ready, and properly started. Misunderstanding probe behavior is one of the most common causes of mysterious production incidents — CrashLoopBackOff, pods stuck unready, or premature restarts during slow startups.

15.1 Reading Pod Status — The READY Column

READY COLUMN	MEANING
1/1	All containers running AND ready — HEALTHY pod
0/1	Container running but FAILING readiness or startup probe — not yet serving traffic
0/1 + CrashLoopBackOff	Container keeps CRASHING and restarting repeatedly
2/2	Multi-container pod (e.g., app + sidecar) — both containers are Ready

Kubernetes Concept:

Quick Decision Rule:

- Startup probe failure → endless restart loop (container never gets past startup phase)
- Readiness probe failure → Pod KEEPS RUNNING but is EXCLUDED from Service traffic (no restarts)
- Liveness probe failure → Pod RESTARTS — but only AFTER the startup probe has succeeded once

15.2 Startup Probe

Designed for slow-starting applications. It tells Kubernetes: 'I'm still starting up, don't kill me yet!' Without a startup probe, a slow-starting app might be killed by the liveness probe BEFORE it even finishes initializing.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-startup-only
spec:
  containers:
  - name: nginx
    image: nginx:latest
    command: ['sh', '-c', 'sleep 40 && nginx -g "daemon off;"']
    ports:
    - containerPort: 80
    startupProbe:
      httpGet:
        path: /test
        port: 80
      failureThreshold: 12    # allow 12 consecutive failures
      periodSeconds: 5        # check every 5s (12 × 5 = 60s total grace time)
```

```text
# Behavior:
# K8s GETs http://<pod-ip>:80/test every 5 seconds
# Allows up to 12 consecutive failures = 60 seconds total grace period
# If probe doesn't succeed within that window, kubelet KILLS and RESTARTS the
# container
# After repeated failures: pod enters CrashLoopBackOff
```

15.3 Readiness Probe

Determines whether a container is ready to serve traffic. CRITICALLY: failing readiness probe does NOT restart the container — it only removes the pod from the Service's load-balancing endpoints.

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-readiness-test
  labels:
    app: nginx-test
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
    readinessProbe:
      httpGet:
        path: /path
        port: 80
      initialDelaySeconds: 5    # wait 5s after container starts before first check
      periodSeconds: 5          # check every 5s
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-readiness-svc
spec:
  selector:
    app: nginx-test
  ports:
  - port: 80
    targetPort: 80
  type: LoadBalancer
```

📝 **Theory & Key Points:**

- Purpose: 'Am I ready to serve traffic right now?'
- Failing readiness → Pod marked NotReady → REMOVED from Service endpoints list → no new traffic routed to it
- Pod KEEPS RUNNING — no restarts happen due to readiness failure alone
- Use case: a pod temporarily overwhelmed (e.g., during a cache warm-up) can be pulled from rotation WITHOUT being killed — it rejoins automatically once healthy again

15.4 Liveness Probe

Checks whether a container is still alive and functioning correctly. If it fails REPEATEDLY (per failureThreshold), Kubernetes assumes the container is unrecoverable and restarts it.

YAML

```yaml
# COMBINED: Startup + Readiness + Liveness probes together
apiVersion: v1
kind: Pod
metadata:
  name: nginx-full-probes
  labels:
    app: nginx-full-probes
spec:
  containers:
  - name: nginx
    image: nginx:latest
    command: ['sh', '-c', 'sleep 40 && nginx -g "daemon off;"']
    ports:
    - containerPort: 80

    startupProbe:
      httpGet:
        path: /te
        port: 80
      failureThreshold: 12
      periodSeconds: 5

    readinessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 5

    livenessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 10
      periodSeconds: 10
      failureThreshold: 3
```

```text
# Liveness probe parameters explained:
# initialDelaySeconds = 10   → wait 10s after container starts before FIRST check
# periodSeconds = 10         → check every 10 seconds
# failureThreshold = 3       → if 3 CONSECUTIVE checks fail, kubelet RESTARTS
#                              the container
```

```bash
# Manual testing — break the health endpoint to trigger failures:
kubectl exec -it <pod-name> -- sh
mv /usr/share/nginx/html/healthz /usr/share/nginx/html/healthz.bak
```

15.5 Probe Comparison Table

PROBE	QUESTION ASKED	ON FAILURE
Startup	Has the app finished STARTING UP?	Restarts container (after grace period exhausted)
Readiness	Is the app READY to serve traffic RIGHT NOW?	Removes from Service endpoints — NO restart
Liveness	Is the app STILL ALIVE and functioning?	Restarts container after threshold failures

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your application takes 90 seconds to fully initialize (loading large ML models into memory). Without a startup probe, the pod enters CrashLoopBackOff repeatedly. Explain why, and how a startup probe fixes it. Without a startup probe, ONLY the liveness probe runs from the moment the container starts (after its own initialDelaySeconds). If liveness checks begin failing because the app hasn't finished loading models yet (e.g., liveness periodSeconds=10, failureThreshold=3 → fails at 30 seconds, but app needs 90), Kubernetes assumes the app is DEAD and restarts it — repeating this cycle forever (CrashLoopBackOff) since the app NEVER gets enough uninterrupted time to finish loading.

Fix: Add a startupProbe with enough grace period:

```yaml
startupProbe:
  httpGet: { path: /healthz, port: 8080 }
  failureThreshold: 20
  periodSeconds: 5
  # Total grace: 20 × 5 = 100 seconds — enough for the 90-second startup
```

While the startupProbe is active, liveness and readiness probes are COMPLETELY DISABLED — kubelet trusts the app is still starting and won't kill it. Only once the startupProbe SUCCEEDS does kubelet begin running liveness/readiness checks normally.

Q2: Scenario: During a traffic spike, one of your pods becomes temporarily slow (high latency) but is NOT crashed. You want it removed from load balancing until it recovers, WITHOUT restarting it (since restarting would lose its warm cache). Which probe do you configure, and how? Use a READINESS probe with appropriate latency-sensitive checks — NOT a liveness probe.

```yaml
readinessProbe:
  httpGet:
    path: /health/ready
    port: 8080
  periodSeconds: 5
  failureThreshold: 2     # quick to detect degradation
  timeoutSeconds: 2       # short timeout catches slow responses
```

Behavior: If the pod becomes slow and fails the readiness check (e.g., the health endpoint takes too long to respond), Kubernetes marks the pod 'NotReady' and REMOVES it from the Service's endpoint list — traffic stops being routed to it. The pod CONTINUES RUNNING (cache stays warm), and once it recovers and passes readiness checks again, it's AUTOMATICALLY ADDED BACK to receive traffic.

This is precisely the readiness probe's purpose: temporary unavailability handling WITHOUT destructive restarts. Using a liveness probe here would be WRONG — it would restart the pod and lose the warm cache unnecessarily.

Q3: Scenario: A teammate configured ONLY a liveness probe (no readiness probe) on a Deployment. After deploying a new version, users briefly see 502 errors during the rollout. Why, and how does adding a readiness probe fix this? Without a readiness probe, Kubernetes assumes a pod is READY the MOMENT it starts (container status = Running), even if the application inside hasn't finished initializing (e.g., still connecting to the database, warming caches, loading config). During a rolling update, NEW pods are added to the Service endpoints IMMEDIATELY upon starting — but if they're not actually ready to handle requests yet, users hit them and get errors (502 Bad Gateway) until the app finishes initializing.

Fix: Add a readinessProbe that accurately reflects true application readiness:

```yaml
readinessProbe:
  httpGet: { path: /health/ready, port: 8080 }
  initialDelaySeconds: 5
  periodSeconds: 3
```

Now during rolling updates: new pods are added to the Service ONLY AFTER passing the readiness check — meaning only TRULY ready pods receive traffic. This eliminates the 502 errors during deployments, because Kubernetes won't route traffic to a pod still warming up.

---

16. Helm — The Kubernetes Package Manager

Helm is often called 'the apt/yum/npm for Kubernetes'. It automates the creation, packaging, configuration, and deployment of Kubernetes applications by bundling multiple YAML files

(Deployment, Service, ConfigMap, Ingress, etc.) into a single, reusable, parameterized package called a **Chart**.

> 💡 **Layman Explanation:**
>
> - Without Helm: you maintain 10 separate YAML files per application, manually editing each for every environment.
> - With Helm: ONE chart + different values.yaml files per environment (dev/staging/prod) — same template, different inputs.
> - Think of a Helm Chart like a HOUSE BLUEPRINT — the same blueprint builds a 3-bedroom house, but you customize paint color, flooring, etc. via 'values'.

16.1 Installing Helm

```bash
wget https://get.helm.sh/helm-v3.14.0-linux-amd64.tar.gz
tar -zxvf helm-v3.14.0-linux-amd64.tar.gz
mv linux-amd64/helm /usr/local/bin/helm
chmod 777 /usr/local/bin/helm
helm version
```

16.2 Chart Structure

```bash
# Create a sample chart
helm create helloworld
```

TEXT

```
# Resulting structure:
helloworld
├── charts/               # dependency charts (sub-charts)
├── Chart.yaml            # chart metadata: name, version, description
├── templates/             # Kubernetes YAML templates (parameterized)
│   ├── deployment.yaml
│   ├── _helpers.tpl       # reusable template snippets
│   ├── hpa.yaml
│   ├── ingress.yaml
│   ├── NOTES.txt          # shown after successful install
│   ├── serviceaccount.yaml
│   ├── service.yaml
│   └── tests/
│       └── test-connection.yaml
└── values.yaml            # DEFAULT configuration values
```

16.3 Helm Commands

BASH

```bash
# Install (RELEASE_NAME = your instance name, CHART_NAME = the chart)
helm install firstproject helloworld

# Install with CUSTOM values (override defaults)
helm install firstproject helloworld --set replicaCount=3 --set image.tag=v2
helm install firstproject helloworld -f custom-values.yaml

# List all releases
helm list -a

# Upgrade an existing release
helm upgrade firstproject helloworld

# Rollback to a previous release version
helm rollback firstproject 1

# Delete (uninstall) a release
helm delete firstproject

# Install from a public repository (e.g., Prometheus/Grafana)
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install monitoring prometheus-community/kube-prometheus-stack
```

🎯 **Scenario-Based Interview Questions**

Q1: Scenario: Your team manages 3 environments (dev/staging/prod) for the same application. Without Helm, you maintain 3 nearly-identical sets of YAML files, and updates require editing all 3 separately. How does Helm solve this? Use ONE Helm chart with environment-specific values files:

```text
helloworld/
├── templates/            (SHARED across all environments — written once)
├── values.yaml           (default/base values)
├── values-dev.yaml       (dev overrides: replicaCount=1, small resources)
├── values-staging.yaml   (staging overrides: replicaCount=2)
└── values-prod.yaml      (prod overrides: replicaCount=5, larger resources,
autoscaling enabled)
```

Deploy to each environment with the same chart:

```text
helm install myapp-dev ./helloworld -f values-dev.yaml --namespace dev
helm install myapp-staging ./helloworld -f values-staging.yaml --namespace
staging
helm install myapp-prod ./helloworld -f values-prod.yaml --namespace prod
```

Benefit: Template logic is written ONCE. Any bug fix or structural change to `templates/deployment.yaml` automatically applies to all 3 environments on next `helm upgrade` — no manual duplication or drift between environment configs.

---

17. ArgoCD — GitOps Continuous Delivery

ArgoCD is a declarative, GitOps continuous delivery tool for Kubernetes. The core idea of GitOps is: **Git is the single source of truth** for what should be running in your cluster. ArgoCD continuously monitors your Git repository and automatically synchronizes the cluster state to match — if someone manually changes something in the cluster, ArgoCD detects the drift and can automatically revert it.

💡 **Layman Explanation:**

- Traditional CI/CD (e.g., Jenkins): 'PUSH' model — pipeline actively pushes changes INTO the cluster.
- GitOps (ArgoCD): 'PULL' model — ArgoCD inside the cluster actively WATCHES Git and pulls changes when detected.
- Benefit: no cluster credentials need to leave the cluster — more secure, since ArgoCD has access TO Git, not the reverse.
- Git commit history = complete audit trail of every infrastructure change ever made.

17.1 Installing ArgoCD

BASH

```
# Create namespace and install
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-
cd/stable/manifests/install.yaml

# Verify pods are running
kubectl get pods -n argocd
kubectl get svc -n argocd

# Change the argocd-server service to LoadBalancer (or NodePort) for external
# access
kubectl edit svc argocd-server -n argocd
kubectl get svc -n argocd    # note the EXTERNAL-IP and port

# Get the initial admin password
kubectl edit secret argocd-initial-admin-secret -n argocd
# Decode the base64-encoded password:
echo bnFabGx3emtCNjB5dFZQSA== | base64 --decode

# Access: http://<node-ip>:<node-port>
# username: admin
```

17.2 GitOps Workflow

🏗️ **Architecture:**

```text
Developer pushes code change → CI pipeline builds + pushes new image to registry
         ↓
CI updates the IMAGE TAG in a Git repository (the 'manifests repo' — separate from app code repo)
         ↓
ArgoCD CONTINUOUSLY WATCHES this manifests repo for changes
         ↓
ArgoCD detects the new commit → compares Desired State (Git) vs Actual State (cluster)
         ↓
ArgoCD automatically SYNCS the cluster to match Git (pulls and applies the new manifests)
         ↓
Application is updated — NO direct kubectl apply or Jenkins deploy step needed!
```

📝 **Theory & Key Points:**

- ArgoCD ensures the cluster ALWAYS matches Git — even reverting MANUAL kubectl changes if drift is detected.
- This eliminates 'configuration drift' — a common production problem where the running cluster slowly diverges from documented/version-controlled configuration.
- ArgoCD provides a visual dashboard showing sync status, diffs, and application health — much better visibility than CLI-only deployments.
- Self-healing option: ArgoCD can be configured to AUTOMATICALLY fix any drift it detects, with NO human intervention.

🎯 **Scenario-Based Interview Questions**

Q1: Scenario: An engineer manually ran `kubectl scale deployment myapp --replicas=10` directly on the cluster to handle an emergency traffic spike, bypassing Git. A few minutes later, the replica count reverted back to 3. Why did ArgoCD do this, and was it the right behavior?
This is ArgoCD's GitOps reconciliation working AS DESIGNED. ArgoCD continuously compares the cluster's ACTUAL state against the DESIRED state defined in Git. The Git manifest still says `replicas: 3` — so when ArgoCD's reconciliation loop ran (typically every few minutes), it detected the cluster had DRIFTED from Git (10 replicas vs the documented 3) and AUTOMATICALLY REVERTED it back to match Git.

Was this correct behavior? YES, by GitOps principles — but it highlights an important operational lesson:

Correct approach during emergencies:

1. Update the Git repository FIRST (e.g., edit deployment.yaml: `replicas: 10`, commit, push)
2. ArgoCD detects the Git change and syncs the cluster automatically within seconds
3. NEVER bypass Git with direct kubectl commands on ArgoCD-managed resources — it creates a false sense of change that gets silently undone

For TRUE emergencies needing instant action: temporarily disable ArgoCD auto-sync for that application, make the kubectl change, THEN update Git to match and re-enable sync — to avoid the 'fight' between manual changes and GitOps reconciliation.

---

18. Monitoring — Prometheus & Grafana

Production Kubernetes clusters require robust observability. **Prometheus** collects and stores metrics as time-series data (CPU, memory, request rates, error rates over time). **Grafana** is the visualization layer — an analytics web application that ingests data from Prometheus (and other sources) and displays it in customizable dashboards and charts.

Fig 5: Prometheus + Grafana Monitoring Architecture diagram — shows Short-lived Jobs pushing metrics at exit → Pushgateway → Prometheus Server (inside the Kubernetes Cluster, alongside Service Discovery via Kubernetes API Server / file_sd static config, and a Node with HDD/SSD). The Prometheus Server contains Retrieval, TSDB, and HTTP Server components. Jobs/Exporters are also shown as Prometheus Targets feeding the Retrieval component. Prometheus pushes alerts to Alertmanager, which notifies PagerDuty, Email, and other channels (webhook/etc). The HTTP Server

is queried via PromQL by the Prometheus Web UI, Grafana, and API Clients. A legend at the bottom maps arrow styles to Metrics Flow / Alert Flow / Data Flow, and box colors to Metric Sources / Prometheus Components / Storage / Alerting / Visualization-Clients / Notifications / Service Discovery. Technologies used: Kubernetes, Prometheus, Alertmanager, Grafana, Pushgateway, PagerDuty, Email, Webhook.

18.1 Architecture Components

COMPONENT	ROLE
Prometheus Server	Core component: Retrieval (scrapes metrics), TSDB (Time-Series Database for storage), HTTP Server (exposes data via PromQL queries)
Service Discovery	Kubernetes API Server integration — Prometheus automatically discovers new pods/services to scrape, no manual config needed for new deployments
Pushgateway	For SHORT-LIVED jobs (batch jobs, cron jobs) that finish BEFORE Prometheus would normally scrape them — they PUSH metrics instead of waiting to be pulled
Alertmanager	Receives alerts FROM Prometheus when metric thresholds are breached, and routes notifications to PagerDuty, Email, Slack, webhooks
Grafana	Visualization layer — queries Prometheus (via PromQL) and renders dashboards, graphs, and alerts in a user-friendly web UI
Node Exporter	DaemonSet that exposes HARDWARE/OS-level metrics (CPU, disk, network) from each node for Prometheus to scrape

18.2 Installing via Helm

```bash
# Add Prometheus community Helm repository
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

# Install the full kube-prometheus-stack (includes Prometheus + Grafana +
# Alertmanager)
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace

# Verify pods are running
kubectl get pods -n monitoring

# Access Grafana dashboard (port-forward for local testing)
kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring
# Default credentials: admin / prom-operator (check values.yaml for exact default)
```

📝 **Theory & Key Points:**

- PromQL (Prometheus Query Language) is used to query metrics: e.g., `rate(http_requests_total[5m])` for request rate over 5 minutes.
- Metric Flow: Pods expose metrics → Prometheus SCRAPES (pulls) them at intervals → stores in TSDB → Grafana QUERIES Prometheus → displays graphs.
- PagerDuty/Email/Slack integration via Alertmanager enables on-call engineers to be notified instantly when thresholds breach (e.g., CPU > 90% for 5 minutes).
- For detailed setup walkthrough, refer to the author's blog: medium.com/@veerababu.narni232 — Deployment of Prometheus and Grafana using Helm in EKS cluster.

🎯 Scenario-Based Interview Questions

Q1: Scenario: Your team needs to be alerted via Slack within 2 minutes whenever any pod's memory usage exceeds 90% for more than 3 minutes. How do you configure this end-to-end with Prometheus + Alertmanager?

1. Define a PrometheusRule (alerting rule) using PromQL:

```yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: high-memory-alert
spec:
  groups:
  - name: memory-alerts
    rules:
    - alert: HighMemoryUsage
      expr: (container_memory_usage_bytes / container_spec_memory_limit_bytes) >
0.9
      for: 3m    # must be true continuously for 3 minutes before firing
      labels:
        severity: warning
      annotations:
        summary: 'Pod {{ $labels.pod }} memory usage above 90%'
```

2. Configure Alertmanager to route this alert to Slack:

```yaml
receivers:
- name: 'slack-notifications'
  slack_configs:
  - api_url: '<slack-webhook-url>'
    channel: '#k8s-alerts'
route:
  receiver: 'slack-notifications'
  group_wait: 30s    # batches alerts together briefly before sending
```

3. Result: Prometheus continuously evaluates the rule every scrape interval. Once the condition holds true for 3+ minutes, Alertmanager fires and posts to Slack — typically within 1-2 minutes of the alert condition triggering.

---

19. EFK Stack — Centralized Log Management

In a Kubernetes cluster with hundreds of pods constantly being created and destroyed, logs are ephemeral — when a pod is deleted, its logs are GONE forever (unless captured elsewhere). The **EFK Stack** (Elasticsearch, Fluent Bit, Kibana) solves this by collecting, centralizing, and making ALL cluster logs searchable — even after the originating pod no longer exists.

💡 **Layman Explanation:**

- Without EFK: each pod's logs are like a sticky note that gets thrown away when the desk (pod) is cleared.
- With EFK: every sticky note is photographed and filed in a searchable archive BEFORE the desk gets cleared.
- Fluent Bit = the photographer (collects logs from every pod). Elasticsearch = the archive (stores and indexes everything). Kibana = the search engine UI (lets you search/visualize logs).

19.1 EFK Components

COMPONENT	ROLE
Elasticsearch	A distributed search and analytics engine — stores ALL collected logs in a searchable, indexed format
Fluent Bit	A lightweight log processor and forwarder, deployed as a DaemonSet (one per node) — collects logs from every container and ships them to Elasticsearch
Kibana	The visualization/search UI — lets engineers search across ALL cluster logs, build dashboards, and set up log-based alerts

📝 **Theory & Key Points:**

- Fluent Bit (lightweight) is preferred over the heavier Fluentd in modern Kubernetes setups — lower resource footprint, ideal for DaemonSet deployment on every node.
- EFK is essential for DEBUGGING distributed microservices — a single user request might pass through 5 different pods; EFK lets you trace the FULL request path by searching logs by request ID/trace ID across ALL services.
- For a complete EFK setup walkthrough, refer to the author's blog: medium.com/@veerababu.narni232 — Setting up the EFK Stack.

🎯 **Scenario-Based Interview Questions**

Q1: Scenario: A customer reports an error that happened 'sometime yesterday afternoon' in your microservices app. The pod that handled their request has long since been replaced (rolling deployment). How does EFK help you debug this? Without centralized logging: the original pod's logs are COMPLETELY GONE — that pod no longer exists, and `kubectl logs` only works for currently-existing pods (or very recently terminated ones, with `--previous` flag, but only briefly).

With EFK stack already deployed:

1. Open Kibana dashboard
2. Search by time range: 'yesterday afternoon' + filter by customer ID, request ID, or error keywords
3. Fluent Bit had already shipped that pod's logs to Elasticsearch in REAL-TIME as they were generated — independent of whether the pod still exists
4. Find the exact error stack trace, timestamp, and even trace the request across MULTIPLE microservices if using distributed tracing (correlation IDs)
5. Build a saved Kibana search/dashboard to quickly find similar issues in the future

This demonstrates why centralized logging (EFK) is NON-NEGOTIABLE for production Kubernetes — without it, any historical debugging beyond the pod's lifetime is essentially impossible.

---

20. Real-World Architecture Examples

Bringing together everything covered so far, this section walks through complete, production-style architectures — showing how Pods, Deployments, Services, ConfigMaps, and AWS managed services (RDS) all combine into a working microservices platform.

20.1 EKS Microservices Demo — E-Commerce Platform

Fig 6: EKS Microservices Demo Project Architecture — "Commerce Platform - Microservices on Amazon EKS". Shows Users → Internet → AWS Load Balancer (ELB, public URL/External-IP) → Amazon EKS Cluster (Namespace: commerce), which contains 4 parallel microservice stacks, each following the pattern Service (LoadBalancer, exposed publicly) → Deployment (Replicas: 2, manages pods/rolling updates) → Pods (2× NGINX, running containers) → ConfigMap (stores HTML, mounted to /usr/share/nginx/html): - commerce-service → commerce-deployment → Pods → ConfigMap commerce-html ("Main entry page") - cart-service → cart-deployment → Pods → ConfigMap cart-html ("Cart application") - checkout-service → checkout-deployment → Pods → ConfigMap checkout-html ("Checkout application") - payment-service → payment-deployment → Pods → ConfigMap payment-html ("Payment application") A side panel shows "How Traffic Flows (End-to-End)": Users → Internet → AWS Load Balancer → Kubernetes Service → Deployment → Pods (NGINX) → HTML Content. Key Technologies listed: Amazon EKS, Kubernetes, NGINX, ConfigMap, Service (LoadBalancer), Deployment, Pods, Namespace, AWS ELB. A "What Each Component Does" legend explains Service/Deployment/Pods/ConfigMap/Namespace/EKS Cluster responsibilities, and a color legend maps Commerce/Cart/Checkout/Payment Service colors.

This architecture demonstrates a realistic microservices e-commerce platform with 4 independent services, each following the SAME pattern: Service (LoadBalancer) → Deployment → Pods → ConfigMap.

COMPONENT	PURPOSE
Namespace: commerce	Logically isolates all e-commerce-related resources — improves security and organization
Service (LoadBalancer)	Each microservice (commerce, cart, checkout, payment) gets its OWN AWS Load Balancer for public access
Deployment (Replicas: 2)	Manages pod lifecycle — handles rolling updates and self-healing for EACH service independently
Pods (2× NGINX)	Each service runs 2 replica pods for high availability — if one fails, the other keeps serving
ConfigMap	Stores HTML/CSS content SEPARATELY from the container image — mounted into NGINX at runtime

20.2 How Traffic Flows (End-to-End)

🏗️ **Architecture:**

Users → Internet → AWS Load Balancer (ELB) → Kubernetes Service → Deployment → Pods (NGINX) → HTML Content

Key insight: This pattern REPEATS for EACH microservice (commerce, cart, checkout, payment) — each one independently exposed, independently scalable, independently deployable.

20.3 Three-Tier Architecture with AWS RDS

Fig 7: "Real-Time Application Architecture on Kubernetes with AWS RDS" — shows Users/Clients → Internet → LoadBalancer Service → Frontend (Kubernetes): Frontend Pod 1 and Frontend Pod 2 (ReplicaSet/Deployment), each running an App container. Frontend communicates internally via ClusterIP Service to Backend (Kubernetes): Backend Pod 1 and Backend Pod 2 (ReplicaSet/Deployment), each running an App container. Backend connects to Database (AWS RDS) — Amazon RDS for MySQL/PostgreSQL/etc (Managed Service, "No Headless Service Required") — via a secure database connection using JDBC/SSL over VPC, to an RDS Primary (Master, Write) with Synchronous Replication to RDS Read Replica 1 and RDS Read Replica 2 (Reader, Read/Reporting), noting Multi-AZ High Availability, Automated Backups, Patching & Maintenance, Encryption & Security, Monitoring & Alarms. Four numbered communication layers at top: 1. NodePort (External communications), 2. LoadBalancer (External communications), 3. ClusterIP (Internal communications), 4. AWS RDS (Managed Database Service, No Headless Service Required). Below, a

"How Communication Works" panel: ① LoadBalancer (External) — Users access the application from the internet via LoadBalancer service. ② ClusterIP (Internal) — Frontend communicates with Backend using ClusterIP service; traffic is internal and not exposed outside the cluster. ③ Backend to RDS (Database) — Backend connects to AWS RDS using the RDS endpoint over a secure connection within the VPC; no Headless Service is required. ④ AWS RDS (Managed Database) — RDS handles replication, failover, backups, patching, encryption, and monitoring. A Legend defines LB/CI/App/Pod/M(Primary)/R(Reader) icons. "Key Benefits" panel: external access secure and simplified via LoadBalancer service; internal services communicate securely within the cluster; AWS RDS provides high availability with Multi-AZ deployment; read replicas improve read performance and offload reporting queries; automated backups, patching and failover handled by AWS; scalable, secure and production-ready architecture. "RDS Components" panel: RDS Primary (Master/Writer) handles all write operations; RDS Read Replicas handle read traffic (scale-out reads); Synchronous Replication ensures durability and minimal data loss; Multi-AZ automatic failover to standby in another AZ; managed by AWS — no infrastructure management required. Abbreviations: LB=LoadBalancer, CI=ClusterIP, RDS=Amazon Relational Database Service, AWS=Amazon Web Services, AZ=Availability Zone, VPC=Virtual Private Cloud, SSL=Secure Sockets Layer, JDBC=Java Database Connectivity. Badges at bottom: Secure, Scalable, Highly Available, Production Ready, Observable.

This pattern demonstrates the standard three-tier production architecture: stateless Frontend and Backend tiers running in Kubernetes, connected to a managed AWS RDS database OUTSIDE the cluster — combining Kubernetes' container orchestration strengths with AWS's mature, fully-managed database service.

COMMUNICATION LAYER	PURPOSE
1. LoadBalancer (Frontend)	External communication — Users access the application from anywhere via the internet
2. ClusterIP (Internal)	Frontend ↔ Backend communication stays INSIDE the cluster — never exposed externally
3. AWS RDS Connection	Backend connects to RDS using a secure JDBC/SSL connection OVER the VPC — RDS itself needs NO Headless Service since it's managed entirely by AWS

📝 **Theory & Key Points:**

- Why use RDS instead of running MySQL/PostgreSQL as a StatefulSet in Kubernetes? AWS RDS provides: Multi-AZ automatic failover, automated backups, automated patching, encryption at rest, built-in monitoring/alarms — all WITHOUT you managing any of it.
- RDS Primary (Master) handles ALL WRITE operations. RDS Read Replicas handle READ traffic — offloading reporting/analytics queries from the primary.
- Synchronous replication between RDS Primary and replicas ensures high durability and minimal data loss in failover scenarios.
- This hybrid pattern (Kubernetes for compute + RDS for data) is EXTREMELY common in production AWS environments — let each tool do what it's best at.

20.4 How Kubernetes Works — End to End (Complete Lifecycle)

Fig 8: "HOW KUBERNETES WORKS (END-TO-END) — From Your Request to a Running, Healthy Application" — a 17-step numbered infographic: ① Cluster Setup — A Kubernetes cluster has Control Plane components that manage the cluster and Worker Nodes where your applications run (shows Control Plane: API Server, Scheduler, Controller Manager, etcd; Worker Nodes: boxes for Deployment, Service, ConfigMap, Secret). ② Define Desired State — You define your application using YAML manifests (shows a sample Deployment YAML: apiVersion apps/v1, kind Deployment, name my-app, replicas 3, selector matchLabels app my-app, template with container app image my-app:1.0). ③ Submit Request — You run the command to apply your manifest: `$ kubectl apply -f app.yaml` *(kubectl icon → API Server icon). ④ API Server Validates & Stores State — Request is validated against schema, stored in etcd (cluster's key-value database); this becomes the Source of Truth. Shows Authentication (Who are you?), Authorization (What can you do?), Admission Controllers (Validate the request). ⑤ Controllers Watch for Changes — Controllers continuously watch the API Server for changes in desired state. Examples: Deployment Controller, ReplicaSet Controller, StatefulSet Controller, Node Controller, Endpoint Controller, Service Controller. ⑥ Reconciliation Loop (Actual vs Desired State) — Controllers compare the Desired State (from etcd) with the Actual State (from the cluster). If there is a difference, they take action to make the actual state match the desired state. Example Flow: Deployment → ReplicaSet → Pods (shown as Desired State vs Actual State comparison feeding a Reconciliation Loop icon). ⑦ Scheduler Assigns Pods to Nodes — Scheduler watches for unscheduled Pods and selects the best node based on: CPU/Memory availability, Node Affinity/Anti-Affinity, Taints & Tolerations, Topology rules, Other scheduling policies. ⑧ Pod Assigned → Kubelet Takes Over — The Pod spec is sent to the selected node. Kubelet on the node: Receives the Pod spec, Ensures containers should be running, Talks to the container runtime. ⑨ Container Runtime Runs Containers — Container runtime (containerd / CRI-O): Pulls image from registry, Creates containers, Starts containers inside the Pod. ⑩ Networking (CNI Plugin) — CNI plugin sets up*

networking for the Pod: Assign Pod IP, Configures routing, Enables communication within the cluster (shows example Pod IP 10.244.1.5 and CNI plugins e.g. Calico, Cilium). ⑪ Service Networking (kube-proxy or eBPF) — kube-proxy (or eBPF data plane) sets up rules: Load balance traffic, Route to healthy Pods, Provides stable virtual IP for Service. ⑫ DNS & Service Discovery — CoreDNS provides DNS resolution for Services and Pods (service-name.namespace.svc.cluster.local, pod-ip.namespace.pod.cluster.local). Enables seamless communication between applications. ⑬ Health Checks & Status Updates — Kubelet performs health checks and reports status back to API Server: Liveness Probe, Readiness Probe, Startup Probe. Status is stored in etcd and visible via `kubectl get pods`*. ⑭ Self-Healing — If something goes wrong: Pod crashes → Restarted, Node fails → Pod rescheduled, Health check fails → Removed from service. Kubernetes automatically detects and heals. ⑮ Scaling — Scale your application as needed: Manual Scaling (replicas), Horizontal Pod Autoscaler (HPA), Vertical Pod Autoscaler (VPA), Cluster Autoscaler (adds/removes nodes). ⑯ Rolling Updates & Rollbacks — Kubernetes updates your application with zero downtime: Rolling updates (new version gradually replaces old), Monitors health, Automatic rollback if failure detected. ⑰ Continuous Reconciliation Loop — Kubernetes never stops! It constantly watches the cluster, detects drift, and reconciles the actual state to match the desired state (Watch → Detect Drift → Reconcile cycle icon). A final "Key Takeaway" box: Kubernetes is a declarative, self-healing, and automated system that manages containers at scale. You describe the desired state → Kubernetes handles the rest. Alongside a "THE BIG PICTURE" callout: You declare the desired state (You define WHAT you want) → Kubernetes figures out HOW to make it happen → Your app runs, scales, heals, and stays available.*

This comprehensive diagram ties together EVERY concept covered in this document into one unified flow — from writing a YAML manifest to a fully running, self-healing, auto-scaling application. Use this as your master mental model for how Kubernetes operates as a complete system.

🏗️ **Architecture:**

THE BIG PICTURE — Kubernetes' Core Philosophy:

1. You DECLARE the desired state (via YAML) — 'I want 3 replicas of this app running'
2. Kubernetes FIGURES OUT how to make it happen — scheduling, networking, storage, all automated
3. Kubernetes CONTINUOUSLY RECONCILES — constantly watching, detecting drift, and self-correcting
4. Your application runs, scales, heals, and stays available — with minimal ongoing human intervention

This DECLARATIVE, SELF-HEALING, AUTOMATED model is THE fundamental reason Kubernetes has become the universal standard for running containerized applications at scale.

21. Quick Command Reference

Cluster & Node Management

COMMAND	DESCRIPTION
`kubectl get nodes`	List all nodes
`kubectl describe node <name>`	Detailed node info
`kubectl label nodes <name> key=value`	Label a node
`kubectl taint nodes <name> key=value:NoSchedule`	Taint a node
`kubectl drain <name> --ignore-daemonsets`	Safely evict pods for maintenance
`kubectl uncordon <name>`	Make node schedulable again
`kubectl top node`	Resource usage per node

Pods & Workloads

COMMAND	DESCRIPTION
`kubectl get pods -o wide`	List pods with node/IP details
`kubectl describe pod <name>`	Full pod details + events
`kubectl logs -f <pod>`	Stream pod logs
`kubectl exec -it <pod> -- /bin/sh`	Open shell in pod
`kubectl apply -f file.yaml`	Create/update from YAML
`kubectl delete -f file.yaml`	Delete resources in YAML
`kubectl rollout status deployment/<name>`	Check rollout progress
`kubectl rollout undo deployment/<name>`	Rollback to previous version
`kubectl scale deployment/<name> --replicas=5`	Manual scaling
`kubectl top pod`	Resource usage per pod

Services & Networking

COMMAND	DESCRIPTION
`kubectl get svc`	List services
`kubectl get ingress`	List ingress resources + LB address
`kubectl get endpoints <svc>`	Show pod IPs behind a service
`kubectl port-forward svc/<name> 8080:80`	Local port forwarding for testing

RBAC & Security

COMMAND	DESCRIPTION
`kubectl get roles, rolebindings`	List namespace-scoped RBAC
`kubectl get clusterroles, clusterrolebindings`	List cluster-wide RBAC
`kubectl auth can-i <verb> <resource>`	Check if current user can perform an action
`kubectl edit cm aws-auth -n kube-system`	Edit IAM-to-RBAC mapping (EKS)

Storage

COMMAND	DESCRIPTION
`kubectl get pv, pvc`	List PersistentVolumes / Claims
`kubectl get storageclass`	List available StorageClasses
`kubectl describe pvc <name>`	Check PVC binding status

Helm & ArgoCD

COMMAND	DESCRIPTION
`helm install <release> <chart>`	Install a Helm chart
`helm upgrade <release> <chart>`	Upgrade a release
`helm rollback <release> <revision>`	Rollback to previous version
`helm list -a`	List all releases
`argocd app sync <app-name>`	Manually trigger ArgoCD sync
`argocd app get <app-name>`	Check ArgoCD application status

ANS

PART 08 OF 08

Ansible

Agentless configuration management, provisioning, and automation.

Topics covered: Architecture • Inventory • Ad-hoc Commands • Modules • Playbooks • Variables • Loops • Handlers • Templates • Vault • Roles • Interview Q&A

Document Legend: 💡 Blue = Layman Explanation · 📝 Green = Theory · 🔴 Red = Ansible Concept · 🏗️ Purple = Architecture · 🎯 Yellow = Scenario Interview Q&A · ⚠️ Orange = Warning · Dark background = YAML / Shell Commands (copy-paste ready)

10 Diagrams • 22 Sections • 21 Scenario Interview Q&A Sets • Corrected & Expanded

1. What is Ansible?

Ansible is an **open-source, agentless IT automation engine** written in Python. It is used to **configure systems, install software, and orchestrate complex multi-step workflows** such as rolling application deployments, zero-downtime updates, and continuous delivery pipelines. Ansible communicates with remote machines over standard **SSH** (or WinRM for Windows) — it does not require any special agent software to be pre-installed on the machines it manages.

Ansible was created by Michael DeHaan in 2012, and the project was acquired by **Red Hat** in 2015. Its biggest strengths are **simplicity, readability (YAML syntax), and a very low learning curve** compared to older configuration management tools.

💡 **Layman Explanation:**

Think of Ansible as a remote control for a large number of computers. Instead of logging in to 100 servers one by one and typing the same commands, you write the steps ONCE in a simple text file (a 'playbook') and Ansible runs those exact steps on all 100 servers at the same time. It works over plain SSH — the same way you normally log in to a Linux server — so there is nothing extra to install on the servers being managed.

1.1 Why Automation? — The Problem Ansible Solves

Before configuration management tools existed, engineers manually SSH'd into every server to install packages, edit config files, and restart services. This does not scale and is highly error-prone.

📝 **Theory & Key Points:**

Manual server management fails at scale because of four core problems:

- **Inconsistency** — one server gets configured slightly differently from another ('configuration drift').
- **No audit trail** — nobody knows what was changed, when, or by whom.
- **Slow & repetitive** — the same 10 commands typed by hand on 50 servers.
- **Human error** — a single typo during a 2 AM deployment can cause an outage. Ansible fixes all four by making infrastructure 'Infrastructure as Code' — the desired state of every server is written down in version-controlled YAML files, and Ansible enforces that state consistently, every single time.

1.2 Agentless Push Model

🔴 **Ansible Concept**

Ansible is **AGENTLESS** — no daemon or client software runs permanently on managed nodes. It uses a **PUSH** model: the control node initiates the connection to the managed nodes (over SSH) and pushes out instructions. (Compare with Puppet/Chef, which usually **PULL** configuration from a central server on a schedule.) When a module runs, Ansible copies a small, temporary Python script to the remote host, executes it, captures the JSON result, and then deletes the script — nothing is left running in the background. This is why Ansible is described as lightweight and easy to bootstrap: if a machine has Python and SSH access, Ansible can manage it.

Fig A: Push model (Ansible) vs Pull model (Puppet/Chef) — diagram showing the Control Node (Ansible) with arrows pointing OUT to Node A and Node B (labelled "PUSH — control node initiates"), contrasted with a Puppet/Chef Master Server with an arrow pointing DOWN to an Agent Node (labelled "PULL — agent checks in on a schedule"). The control node initiates every connection instead of waiting for agents to check in.

1.3 Ansible vs Other Configuration Management Tools

TOOL	AGENT REQUIRED?	MODEL	LANGUAGE	LEARNING CURVE
Ansible	No (agentless)	Push	YAML	Low
Puppet	Yes (puppet agent)	Pull	Puppet DSL (Ruby-like)	High
Chef	Yes (chef-client)	Pull	Ruby	High
SaltStack	Optional (minion)	Push/Pull	YAML + Python	Medium

🎯 **Scenario-Based Interview Questions**

Q1: Your company runs 60 EC2 servers configured manually. Deployments are inconsistent and a junior engineer once ran the wrong command on production. How would Ansible fix this?

1. CONSISTENCY: The exact same playbook is applied to every server — no manual typing means no typos.
2. VERSION CONTROL: Playbooks live in Git, so every change is reviewed, tracked, and reversible.
3. DRY-RUN SAFETY: `--check` mode lets engineers preview changes before applying them for real.
4. LEAST PRIVILEGE: Only the control node needs SSH access with sudo — junior engineers run playbooks, not raw destructive commands. Result: deployments become repeatable, reviewable, and safe.

Q2: Why would a team pick Ansible over Puppet or Chef for a fast-moving startup with 15 servers? → No agent to install/maintain on every node — faster onboarding of new servers. → YAML is far easier to read than Puppet DSL or Ruby — new engineers can contribute within a day. → Push model means changes apply immediately, on demand — no waiting for the next pull cycle. → Lower operational overhead is ideal for small teams without a dedicated infra team; Puppet/Chef shine more at very large, complex enterprise estates with dedicated CM engineers.

2. Ansible Architecture

An Ansible setup is made of two kinds of machines: the **Control Node** and one or more **Managed Nodes**. There is no 'cluster' in the Kubernetes sense — Ansible's architecture is intentionally simple.

COMPONENT	ROLE
Control Node	The machine where Ansible is installed and playbooks/commands are run FROM. Only Linux/Unix-like OS is supported as a control node (not Windows). There can be one primary control node, and optionally a backup.
Managed Node	Any server being configured/automated. It needs only Python and SSH access — NO Ansible installation or agent is required here.
Inventory	A file that lists all managed node IP addresses/hostnames, optionally organized into groups (e.g. `[web]`, `[db]`).
Modules	Small, reusable units of Python code (`ping`, `yum`, `copy`, `service`, etc.) that Ansible ships to a managed node to perform one specific task.
Playbook	A YAML file describing an ordered list of tasks (built from modules) to run against a group of hosts.

Fig 1: Ansible Architecture (user-drawn reference diagram) — shows a Control Node with Ansible installed, connected via SSH to multiple target EC2 instances. The diagram notes: "configuration management tool" (Ansible, with Puppet/Chef as alternate tools), a set of target servers reached over SSH, a playbook and inventory file feeding the control node, and callouts: "What configuration = Playbook.yml", "How ansible connect to server = SSH", "which servers = Inventory file". It also shows a second panel with an "Ansible Management Node" containing a playbook and inventory connecting to Host 1 (group A), Host 2 (group A), and Host N (group B). A note states: "I can configure any configuration through launch template script also so when app creates a server script will execute directly — why ansible? — configuration automation can handle through it correctly before we update anything, so we can see clearly what is changing." Below that: "By using ansible we can run ad-hoc commands ex: `ansible all -a 'yum install git -y' -b`" with labels pointing to "Inventory / all hosts", "argument", and "become root".

2.1 How Ansible Connects to Managed Nodes

🏗️ **Architecture Note**

Connection transport = SSH (default) for Linux, WinRM for Windows targets. Authentication is normally **KEY-BASED (passwordless)** using an SSH key pair — this is what makes automation possible without typing a password on every run. Flow: Control Node reads Inventory → opens SSH session to each target → copies a small Python module → executes it → gathers JSON result → removes the temp file → reports back changed / ok / failed. Because everything happens over SSH, Ansible can manage on-prem servers, cloud VMs (EC2, Azure VM, GCE), and even network devices — anything reachable over SSH/WinRM.

💡 **Layman Explanation:**

Imagine Ansible as a delivery courier with a master key (the SSH key). It walks up to every target server's door (SSH port 22), lets itself in with the key (no need to ring the doorbell / type a password), drops off a small instruction note (the module), waits for the job to be done, picks up the note again, and leaves. Nothing stays behind on the server — that's the 'agentless' part.

🎯 **Scenario-Based Interview Questions**

Q1: A new engineer asks: 'Do I need to install Ansible on all 200 servers we manage?' What do you tell them? No — Ansible only needs to be installed on the Control Node. Managed nodes only need: (1) Python installed, and (2) SSH access from the control node (ideally key-based). This is the core advantage of the agentless architecture — onboarding a new server takes minutes, not a software rollout.

Q2: Your control node can SSH manually into a target server, but `ansible all -m ping` fails with a Python interpreter error. What's the likely cause and fix? Likely cause: Python is missing, or Ansible is targeting the wrong Python interpreter path on that managed node (common on minimal/Alpine images). Fix 1: Install Python3 on the managed node. Fix 2: Set `ansible_python_interpreter` as a host/group variable pointing to the correct Python binary path.

---

3. Installation

Ansible is installed only on the Control Node. It is available in most Linux distribution package repositories.

```bash
# RHEL / CentOS / Amazon Linux / Fedora
sudo dnf install ansible -y

# Ubuntu / Debian
sudo apt update && sudo apt install ansible -y

# Verify installation
ansible --version
```

Installing Ansible on the control node

📝 **Theory & Key Points:**

- `ansible --version` also shows useful diagnostic info: the config file being used, the default module search path, and the Python interpreter version — always check this first when debugging unexpected behaviour.

🎯 **Scenario-Based Interview Questions**

Q1: After installing Ansible, `ansible --version` shows an old version even though you just upgraded. What would you check? Check which `ansible` binary is on PATH — a system package manager version and a pip-installed version can coexist and shadow each other. Run `which ansible` and `pip show ansible` to confirm which install is being used, then remove the unwanted one or fix PATH ordering.

Q2: Should Ansible be installed on the managed (target) nodes too? No — only Python is required on managed nodes. Installing Ansible itself there is unnecessary and against the agentless design.

4. SSH Key-Based (Passwordless) Authentication

Ansible works best with key-based SSH authentication so that playbooks can run unattended, without a human typing a password for every host on every run.

4.1 Setup Steps (corrected)

```bash
# 1. On the control node, generate a key pair
ssh-keygen

# 2. Copy the PUBLIC key to the target server
ssh-copy-id <target-private-ip>

# (manual alternative: copy the contents of ~/.ssh/id_rsa.pub
# and append it into ~/.ssh/authorized_keys on the target server)

# 3. Test the connection — should log in without asking for a password
ssh <target-private-ip>
```

Passwordless SSH setup — the original notes had a typo: it is `authorized_keys`, not `authorizedkeys`

> 💡 **Layman Explanation:**
>
> The PUBLIC key is like a padlock you hand out — it's safe to give copies to every server. The PRIVATE key is the only key that opens that padlock — it never leaves the control node. Once a server has your padlock installed (in `authorized_keys`), only your private key can unlock SSH access to it — no password typing needed.

> ⚠️ **Important Warning:**
>
> Never copy your private key (`id_rsa`) to a server you do not fully trust. Anyone holding your private key can log in to every server that trusts the matching public key. Always set file permissions correctly: `~/.ssh` should be `700`, and the private key file should be `600`.

🎯 **Scenario-Based Interview Questions**

Q1: `ansible all -m ping` suddenly fails with 'Permission denied (publickey)' on one specific host. How do you troubleshoot?

1. Confirm the public key is actually present in that host's `~/.ssh/authorized_keys`.
2. Check file/folder permissions (700 on `.ssh`, 600 on `authorized_keys`) — SSH silently refuses keys if permissions are too open.
3. Run the command with `-vvv` for verbose SSH debug output to see exactly which key/user was tried.
4. Confirm the inventory is using the correct `remote_user` for that host.

Q2: Why is passwordless SSH preferred over typing a password interactively for Ansible automation? Playbooks often run unattended (from CI/CD pipelines, cron jobs, or against 50+ hosts) — nobody is present to type a password per host. Key-based auth is also more secure and auditable than shared passwords, and works cleanly with ssh-agent.

---

5. Inventory File

The Inventory file is the list of managed nodes Ansible automates against. It defines **WHICH** servers Ansible talks to, and can organize them into **groups** so a command or playbook can target 'all web servers' or 'all database servers' instead of typing individual IPs every time.

5.1 Default vs Custom Inventory Path

APPROACH	LOCATION / COMMAND
Default inventory path	`/etc/ansible/hosts` (used automatically — no `-i` flag needed)
Custom inventory file	Any file you create, e.g. `vi inventory`, referenced explicitly with `-i inventory`

```ini
# Example custom inventory file with groups
[web]
192.168.1.2
192.168.1.5

[db]
192.168.1.3

[all:children]
web
db
```

inventory — group names in [brackets]; [all:children] groups groups

🔴 Ansible Concept

`all` is a built-in group meaning every host in the inventory — you can target `ansible all ...` or a specific group like `ansible web ...`. Groups can be nested using the special `[groupname:children]` syntax. Per-group and per-host variables can be stored in `group_vars/<groupname>.yml` and `host_vars/<hostname>.yml` directories placed next to the inventory file — Ansible loads these automatically.

Fig B: Inventory groups — a diagram showing `[all]` at the top with arrows branching down to three groups: `[web]` (192.168.1.2, 192.168.1.5), `[db]` (192.168.1.3), and `[monitoring]` (192.168.1.9), with a caption noting that `group_vars/web.yml`, `group_vars/db.yml` etc. apply automatically per group. 'all' encompasses every group; each group can carry its own group_vars automatically.

🎯 **Scenario-Based Interview Questions**

Q1: You have 20 web servers and 5 database servers. A teammate keeps running yum installs against the wrong set of machines by copy-pasting raw IPs. How does the inventory file solve this? Organize hosts into named groups: `[web]` and `[db]`. Now commands target the group name directly: `ansible web -m yum -a "name=httpd state=latest" -b` — impossible to accidentally hit the database group. Groups also let group_vars apply different settings (e.g. different users/ports) automatically per environment.

Q2: Difference between running `ansible all -a "uptime"` with no `-i` flag, vs `ansible -i inventory all -a "uptime"`? Without `-i`, Ansible falls back to the default inventory path `/etc/ansible/hosts`. With `-i inventory`, Ansible reads your custom inventory file in the current directory instead — useful for keeping project-specific inventories in version control.

Q3: How would you run a command against ONLY one specific server instead of a whole group? Use the exact host or IP as the pattern instead of a group name, e.g.: `ansible 192.168.1.2 -m ping` You can also use patterns like `web[0]` (first host in a group) or wildcard patterns for more advanced targeting.

---

6. Ad-hoc Commands

Ad-hoc commands are **one-off, single-line commands** run directly from the terminal — no playbook file needed. They are ideal for quick checks and simple, one-time tasks (e.g. 'is this package installed on all servers?').

BASH

```
# General syntax
ansible <host-pattern> -i <inventory> -m <module> -a "<arguments>" -b

# Examples (corrected)
ansible all -m ping                                          # connectivity check
ansible -i inventory all -a "yum install maven -y" -b        # custom inventory
ansible all -a "yum install git -y" -b                       # default inventory path
ansible web -a "git --version" -b                            # target only the 'web'
group
ansible -i inventory all -a "touch file100" -b
```

Ad-hoc command examples

FLAG	MEANING
`-i`	Path to a custom inventory file (skip this to use the default `/etc/ansible/hosts`)
`-m`	Which module to use (defaults to the 'command' module if omitted)
`-a`	Arguments passed to the module
`-b`	'become' — run the task as root (like sudo)
`-u`	Connect as a specific remote user
`-k`	Prompt for SSH password instead of using keys

📝 **Theory & Key Points:**

- If `-m` is omitted, Ansible defaults to the 'command' module, so `ansible all -a "uptime"` works even without `-m command`.
- Ad-hoc commands are NOT idempotent by default when using the raw 'command' or 'shell' modules — running the same shell command twice may have side effects. Prefer proper modules (`yum`, `copy`, `file`, etc.) whenever possible, since those ARE idempotent.

🎯 **Scenario-Based Interview Questions**

Q1: You need to quickly check the disk usage on all 40 production servers right now, without writing a playbook. What do you run? `ansible all -a "df -h"` This is exactly the use-case ad-hoc commands are designed for: a fast, one-off check across many machines with zero setup.

Q2: A teammate ran `ansible all -a "rm -rf /tmp/olddata" -b` by mistake against the 'all' group instead of a specific host. What safeguard should be added going forward? Always target the narrowest possible group/host pattern, never 'all', for destructive ad-hoc commands. Use `--limit` to further restrict the target set as an extra safety net, e.g. `--limit web01`. For anything destructive or repeatable, move it into a reviewed playbook instead of a raw ad-hoc command.

Q3: What is the difference between the 'command' module and the 'shell' module in an ad-hoc command? `command` runs the program directly without invoking a shell — safer, but does NOT support pipes (`|`), redirects (`>`), or environment variable expansion. `shell` runs through `/bin/sh`, so pipes/redirects/variables work, but it's less safe and slightly slower. Use `command` unless you specifically need shell features.

---

7. The ansible.cfg Configuration File

`ansible.cfg` controls Ansible's default behaviour — things like whether SSH host keys are verified, which user to connect as, how many hosts run in parallel, and where the inventory file lives.

7.1 Config File Lookup Order (first found wins)

1. **ANSIBLE_CONFIG** environment variable (if set, always wins)
2. **./ansible.cfg** — in the current directory where the command is run
3. **~/.ansible.cfg** — in the user's home directory
4. **/etc/ansible/ansible.cfg** — the global, system-wide default

Fig C: ansible.cfg lookup order (top wins) — a flowchart with four stacked boxes, top to bottom: "1. ANSIBLE_CONFIG env var" → "2. ./ansible.cfg (current dir)" → "3. ~/.ansible.cfg (home dir)" → "4. /etc/ansible/ansible.cfg (global)". The first file found (top to bottom) wins; lower ones are ignored.

```ini
sudo vi /etc/ansible/ansible.cfg

[defaults]
host_key_checking = False
inventory = ./inventory
remote_user = ec2-user
private_key_file = ~/.ssh/id_rsa
forks = 10
```

Common ansible.cfg settings

🔴 Ansible Concept

`host_key_checking = False` disables the interactive 'yes/no, are you sure you want to continue connecting?' SSH prompt — essential for unattended automation, but only disable this on trusted internal networks. `forks` controls how many hosts Ansible talks to IN PARALLEL (default is 5) — raising it speeds up large fleets.

🎯 Scenario-Based Interview Questions

Q1: Every ad-hoc command hangs waiting for a manual 'yes' prompt the first time it connects to a new server. How do you fix this permanently? Set `host_key_checking = False` under `[defaults]` in ansible.cfg — this skips the interactive SSH fingerprint confirmation prompt so automation never hangs.

Q2: Running the same playbook against 200 servers is slow. What single ansible.cfg setting most directly improves this, and why? Increase `forks` (e.g. `forks = 20` or higher) — this raises the number of hosts Ansible connects to and configures simultaneously instead of the default 5, cutting total runtime roughly proportionally (up to your control node's CPU/network limits).

8. Two Approaches to Configure Host Nodes

Both approaches below assume: one Ansible control-node EC2 instance and two (or more) target node EC2 instances, with Ansible already installed on the control node and `host_key_checking` disabled.

8.1 Approach 1 — Generate a Fresh Key Pair

BASH

```
# 1. Add target node private IPs to the inventory
sudo vi /etc/ansible/hosts
  <private-ip-1>
  <private-ip-2>

# 2. Disable host key checking
sudo vi /etc/ansible/ansible.cfg
  [defaults]
  host_key_checking = False

# 3. Generate a new SSH key pair on the control node
ssh-keygen
cd ~/.ssh

# 4. Copy id_rsa.pub and paste it into each node's authorized_keys
sudo vi authorized_keys     # (on each target node)

# 5. Test connectivity
ansible all -m ping
```

Approach 1 — fresh key pair generated on the control node

8.2 Approach 2 — Reuse an Existing Local Private Key

Used when the target nodes were already launched with a known AWS key pair (a `.pem` file) and you don't want to generate/distribute a brand-new key.

```bash
# 1 & 2: same inventory + ansible.cfg steps as Approach 1

# 3. On the control node, create ~/.ssh/id_rsa and paste in
#    the EXISTING private key content used to launch the nodes
cd ~/.ssh
sudo vi id_rsa
chmod 600 id_rsa

# Do NOT run ssh-keygen in this approach — you are reusing a key,
# not creating one. No public key needs to be copied to the nodes,
# because the matching public key is already baked into the AMI/instance.

ansible all -m ping
```

Approach 2 — reuse the local/AWS private key directly

📝 **Theory & Key Points:**

- APPROACH 1 vs APPROACH 2 — key difference: in Approach 1 you distribute a NEW public key to every node; in Approach 2 you bring the control node the SAME private key that already matches a public key baked into the nodes (e.g. an AWS EC2 key pair). Both end with passwordless SSH from the control node.

8.3 AMI Backup of a Configured Node

1. Pick any existing Approach-2 node that is already trusted by the control node.
2. Create an AMI (Amazon Machine Image) from that node.
3. Launch a new instance from that AMI — with the same key pair or a different one.
4. Add the new node's private IP into the control node's inventory.
5. Run the ping module — the new node responds immediately.

🔴 **Ansible Concept**

This works WITHOUT uploading any private key to the control node again, because the AMI backup already contains the matching public key inside its baked-in `authorized_keys` file — the trust relationship is 'inherited' by every instance launched from that AMI.

🎯 **Scenario-Based Interview Questions**

Q1: You are onboarding 50 new EC2 nodes that were all launched from the same AWS key pair. Which approach is fastest, and why? Approach 2 — reuse the existing AWS private key on the control node. There is no need to individually copy a new public key to 50 servers; the trust already exists because they share the same original key pair. Just add all 50 IPs to the inventory and ping.

Q2: Your team wants every new server auto-provisioned by an Auto Scaling Group to be immediately Ansible-manageable with zero manual key copying. How would AMI backup help? Bake a 'golden AMI' from a node that already trusts the control node's key. Configure the ASG's Launch Template to use that AMI. Every new instance the ASG launches already has the correct `authorized_keys` entry, so the control node can manage it immediately — no manual SSH key distribution step at scale-out time.

Q3: What security risk does Approach 2 introduce that Approach 1 does not, and how do you mitigate it? Risk: the control node now holds a copy of a private key that may also be used elsewhere (e.g. by engineers logging in manually) — if the control node is compromised, that shared key is exposed. Mitigation: use a dedicated key pair exclusively for Ansible automation (closer to Approach 1), restrict file permissions (`chmod 600`), and rotate keys periodically.

---

9. Ansible Modules

Modules are **reusable, standalone units of code** that Ansible ships to the managed node to perform ONE specific job — installing a package, copying a file, managing a user, starting a service, and so on. Ansible includes thousands of built-in modules, and custom modules can also be written.

🔴 **Ansible Concept**

IDEMPOTENCY: nearly every official module is idempotent — running it twice produces the SAME end state and reports 'changed' only the first time. Example: the `yum` module with `state=present` will not reinstall a package that's already there; it simply reports 'ok'. This is what makes playbooks safe to re-run at any time.

Fig D: Idempotency in action — a flow diagram: "Run 1: httpd NOT installed" → "Module installs it, reports: CHANGED" → "Run 2 (re-run): httpd already there" → "reports: OK (no change)". The second run of the same task reports 'ok' instead of 'changed' because the desired state already exists.

9.1 Common Modules Reference

MODULE	PURPOSE
`ping`	Tests connectivity + confirms a usable Python interpreter exists
`stat`	Retrieves file/path status info (exists, size, permissions, etc.)
`user`	Creates, modifies, or removes user accounts
`setup`	Gathers 'facts' — detailed system information (IP, OS, memory, CPU, etc.)
`file`	Manages files/directories: create, touch, delete, set permissions
`copy`	Copies a file from the control node to managed nodes
`yum` / `apt`	Installs, updates, or removes packages (RHEL/Debian family)
`service` / `systemd`	Starts, stops, restarts, enables a system service
`get_url`	Downloads a file from a URL directly onto the managed node
`template`	Renders a Jinja2 template file onto the managed node (see Section 16)

```bash
ansible -i inventory all -m ping
ansible -i inventory all -m stat -a "path=/var/www/html"
ansible -i inventory all -m user -a "name=naresh" -b
ansible -i inventory all -m setup
ansible -i inventory all -m file -a "name=demo state=touch"
ansible -i inventory all -m copy -a "src=file1 dest=~"

# yum/apt states: latest | present | absent
ansible all -m yum -a "name=httpd state=latest" -b    # install/update
ansible all -m yum -a "name=httpd state=present" -b   # ensure installed
ansible all -m yum -a "name=httpd state=absent" -b    # uninstall

# service states: started | stopped | restarted
ansible all -m service -a "name=httpd state=started" -b
ansible all -m service -a "name=httpd state=stopped" -b
ansible all -m service -a "name=httpd state=restarted" -b
ansible all -m systemd -a "name=httpd" -b    # check detailed status

# download a file straight from the internet
ansible all -m get_url -a "url=https://example.com/app.tar.gz dest=/opt/app.tar.gz
mode=0644"
```

Module usage examples (corrected)

💡 **Layman Explanation:**

A module is like a specific tool in a toolbox — 'yum' is the wrench for packages, 'copy' is for moving files, 'service' is the switch for turning things on/off. You don't write the tool yourself; you just tell Ansible which tool to use and with what settings, and it hands that exact tool to every server on your list.

🎯 **Scenario-Based Interview Questions**

Q1: You ran the same playbook twice by accident. The second run shows 'changed=0' for the package install task. Is that a bug? No — that is idempotency working correctly. The yum module checks the CURRENT state first; since httpd was already installed from the first run, the second run correctly reports no change needed.

Q2: You need to know the OS version and total RAM of 100 servers before planning an upgrade. Which module do you reach for? The 'setup' module — it gathers detailed facts (`ansible_distribution`, `ansible_memtotal_mb`, etc.) from every host, which you can then filter/report on.

Q3: A task using the 'command' module to run a custom install script shows 'changed' every single time it runs, even when nothing actually changed. Why, and how do you fix it? `command` / `shell` modules are NOT idempotent by nature — they always report 'changed' because Ansible has no way to know if the underlying action was a no-op. Fix: use `creates` or `removes` arguments on the command/shell task (skip if a marker file already exists), or better — replace it with a proper idempotent module if one exists for that job.

---

10. Ansible Playbooks

A Playbook is a YAML file containing an ordered list of **Plays** — each Play maps a group of hosts to a set of **Tasks**. Tasks run top-to-bottom, in the exact order written. Playbooks are how you move from one-off ad-hoc commands to repeatable, version-controlled automation.

10.1 Basic Structure

```yaml
---
- name: first playbook
  hosts: all
  become: yes
  tasks:
    - name: install httpd software
      yum:
        name: httpd
        state: latest
    - name: start web server
      service:
        name: httpd
        state: started
```

test-playbook.yaml

```bash
# to execute a playbook
ansible-playbook -i inventory test-playbook.yaml
```

Execution

10.2 A Fuller Example — Deploy Through Copy

```yaml
---
- name: first playbook
  hosts: all
  become: yes
  tasks:
    - name: install httpd software
      yum:
        name: httpd
        state: latest
    - name: start web server
      service:
        name: httpd
        state: started
    - name: copying the files
      copy:
        src: index.html
        dest: /var/www/html/index.html
    - name: restart server
      service:
        name: httpd
        state: restarted
```

Fig 2: End-to-end playbook execution flow — a diagram showing Inventory File (hosts), Playbook (.yml), and ansible.cfg all feeding into the "ANSIBLE ENGINE (Control Node)" box, labelled "Parses YAML, reads inventory, runs modules". The engine connects via "SSH (Push, Agentless), Port 22, key-based auth" out to Node 1, Node 2, Node 3, and Node N. A caption notes: modules are copied as small Python scripts, executed, then removed (agentless, temporary execution).

📝 **Theory & Key Points:**

- YAML is indentation-sensitive — use SPACES, never TABS. A misaligned task under 'tasks:' is one of the most common beginner errors.
- `hosts:` decides WHICH group from the inventory this play targets (e.g. all, web, db).
- `become: yes` is the playbook-level equivalent of the `-b` flag in ad-hoc commands — it runs tasks with elevated (root) privileges.

💡 **Layman Explanation:**

A playbook is a recipe. 'hosts' says which kitchen to cook in. Each 'task' is one recipe step, always done in order — install ingredients, then start the oven, then plate the food, then serve. Run the same recipe on Monday and Friday — you get the exact same dish both times, because Ansible only does the steps that are still needed (idempotency).

🎯 **Scenario-Based Interview Questions**

Q1: A junior engineer's playbook run fails immediately with a YAML parsing error, but the file 'looks right' visually. What is the most common cause? Mixed tabs and spaces, or inconsistent indentation — YAML requires consistent SPACE-based indentation. Use a YAML linter (yamllint) or an editor with YAML syntax highlighting to catch this before running ansible-playbook.

Q2: You need to deploy a new index.html to 30 web servers and want the web service to pick up the change immediately, but ONLY if the file actually changed. How do handlers improve on the plain 'restart server' task shown above? Replace the unconditional 'restart server' task with a handler triggered by `notify` on the copy task. The handler only fires when the copy task reports 'changed' — if the file is already identical, no unnecessary restart (and no brief service blip) happens. (Full detail in Section 14 — Handlers.)

Q3: How would you preview exactly what a playbook WOULD change, without actually applying it? Run it in check mode: `ansible-playbook -i inventory test-playbook.yaml --check` Combine with `--diff` to also see line-by-line file differences before committing to the real run.

---

11. Variables

Variables let a single playbook be reused across different environments (dev/stage/prod) or different packages/values, instead of hardcoding values into every task.

```yaml
---
- name: first playbook
  hosts: all
  become: yes
  vars:
    a: httpd
    b: present
    c: started
    d: restarted
  tasks:
    - name: install httpd software
      yum:
        name: "{{ a }}"
        state: "{{ b }}"
    - name: start web server
      service:
        name: "{{ a }}"
        state: "{{ c }}"
    - name: copying the files
      copy:
        src: index.html
        dest: /var/www/html/index.html
    - name: restart server
      service:
        name: "{{ a }}"
        state: "{{ d }}"
```

Global (play-level) variables — corrected: 'vars' is a list of key: value pairs, referenced with `{{ }}`

11.1 Where Else Variables Can Live

LOCATION	SCOPE
`vars:` (in the playbook)	Applies only within that play
`group_vars/<group>.yml`	Applies to every host in that inventory group
`host_vars/<hostname>.yml`	Applies to one specific host only
`-e "key=value"` (extra-vars, CLI)	Highest precedence — overrides everything else, useful for one-off runs
`roles/<role>/defaults/main.yml`	Lowest precedence — role defaults, easily overridden

🔴 **Ansible Concept**

Variable names must be referenced using double curly braces: `"{{ variable_name }}"` — and when used as the FIRST thing in a YAML value, they should generally be quoted to avoid YAML parsing ambiguity. If the same variable is defined in multiple places, Ansible applies a well-defined PRECEDENCE order — command-line extra-vars (`-e`) always win, role defaults always lose, everything else falls in between.

Fig E: Variable precedence (top = highest) — a stack of boxes from top to bottom: "-e extra-vars (CLI)" → "Playbook vars: / vars_files" → "host_vars / group_vars" → "Role vars (roles//vars)" → "Role defaults (roles/*/defaults)". Higher rows silently override lower rows when the same variable name is defined in more than one place.*

🎯 **Scenario-Based Interview Questions**

Q1: You want to reuse ONE playbook to install either 'httpd' or 'nginx' depending on which team runs it, without editing the YAML file each time. Keep the package name as a variable, e.g. `"{{ web_package }}"`, and pass it at runtime: `ansible-playbook site.yml -e "web_package=nginx"` No playbook edit required — the same file now serves both teams.

Q2: A variable is defined both in `group_vars/web.yml` AND passed via `-e` on the command line with a different value. Which one wins, and why does that matter operationally? The `-e` (extra-vars) value always wins — it has the highest precedence in Ansible's variable resolution order. This matters because it lets operators safely override a default for a single emergency run (e.g. a hotfix version) without editing and committing a change to group_vars.

12. Tags

Tags let you run — or deliberately skip — specific tasks inside a large playbook, instead of always executing every single task from top to bottom.

```yaml
---
- name: first playbook
  hosts: all
  tasks:
    - name: installing git
      yum:
        name: git
        state: present
      tags: a
    - name: installing maven
      yum:
        name: maven
        state: present
      tags: b
    - name: create user
      user:
        name: test
        state: present
      tags: c
```

```bash
ansible-playbook name.yml --tags a          # run ONLY tag a
ansible-playbook name.yml --tags b,c        # run tags b AND c
ansible-playbook name.yml --skip-tags "c"   # run everything EXCEPT tag c
ansible-playbook name.yml --skip-tags "c,d" # skip multiple tags
```

Running/skipping tagged tasks

💡 **Layman Explanation:**

Think of tags as labelled sticky notes on individual recipe steps. Normally you cook the whole recipe. But if you only want the 'dessert' step today, you say 'just do the dessert-tagged steps' — everything else is skipped.

🎯 **Scenario-Based Interview Questions**

Q1: A 40-task playbook provisions an entire server (packages, users, firewall rules, app deploy). You only need to re-run the user-creation part after onboarding a new hire. What's the fastest safe option? Tag the user-creation task(s) (e.g. `tags: users`) and run: `ansible-playbook site.yml --tags users` This avoids re-running the entire 40-task provisioning flow just to add one user.

Q2: How would you run a full playbook but temporarily skip a known-slow, non-critical monitoring-agent install task? Tag that task (e.g. `tags: monitoring`) and run: `ansible-playbook site.yml --skip-tags monitoring` Everything else executes normally; only the tagged task is bypassed for this run.

---

13. Loops

Loops let a single task repeat over a list of items — installing five packages, or applying several name/state combinations — instead of writing a near-identical task five times.

YAML

```yaml
---
- name: loop playbook
  hosts: all
  become: yes
  tasks:
    - name: install multiple packages
      yum:
        name: "{{ item }}"
        state: latest
      with_items:
        - git
        - tree
```

Simple list loop (legacy with_items syntax)

YAML

```yaml
---
- name: loop playbook
  hosts: all
  become: yes
  tasks:
    - name: install multiple software with different states
      yum:
        name: "{{ item.x }}"
        state: "{{ item.y }}"
      with_items:
        - { x: httpd, y: latest }
        - { x: git,   y: absent }
        - { x: tree,  y: latest }
    - name: start httpd server
      service:
        name: "{{ item.x }}"
        state: "{{ item.z }}"
      with_items:
        - { x: httpd, z: started }
```

Looping over a list of dictionaries

🔴 **Ansible Concept**

MODERN SYNTAX: `with_items` still works but is considered LEGACY. Current best practice is the generic `loop:` keyword, which behaves the same way for simple lists: `loop: [git, tree]` instead of `with_items: [git, tree]` `loop` is preferred going forward because it is more predictable and is the direction Ansible's own documentation now recommends; `with_items` remains fully supported for backward compatibility.

🎯 **Scenario-Based Interview Questions**

Q1: You need to install git, tree, and htop on every server. Writing three separate yum tasks feels repetitive. How do you simplify it? Use a single yum task with a loop over the three package names — either `loop: [git, tree, htop]` or the legacy `with_items:` list form — instead of three near-duplicate tasks.

Q2: You need to create three users, each with a different shell (bash, sh, zsh). How would a loop over dictionaries help here? Loop over a list of dictionaries, e.g. `{name: alice, shell: /bin/bash}`, `{name: bob, shell: /bin/zsh}`, referencing `item.name` and `item.shell` inside the 'user' module task — one task handles all three users cleanly.

---

14. Handlers

Handlers are special tasks that only run when explicitly **notified** by another task — AND only if that task reports a status of '**changed**'. They are most commonly used to restart or reload a service only when its configuration actually changed.

YAML

```yaml
---
- name: first playbook
  hosts: all
  become: yes
  tasks:
    - name: install httpd software
      yum:
        name: httpd
        state: latest
    - name: start web server
      service:
        name: httpd
        state: started
    - name: copying the files
      copy:
        src: index.html
        dest: /var/www/html/index.html
      notify: Restart web server
  handlers:
    - name: Restart web server
      service:
        name: httpd
        state: restarted
```

Corrected: the notify name must EXACTLY match the handler's name (original notes had a typo — 'Restart web serve')

🔴 **Ansible Concept**

The value passed to `notify:` must match a handler's `name:` EXACTLY, character for character — a common bug is a typo or trailing-space mismatch, which silently means the handler never fires (with no visible error). Handlers, by default, only run ONCE at the very END of the play — even if notified by several different tasks — and only if at least one of those tasks actually reported 'changed'. If a playbook run fails partway through, handlers notified earlier in that run are NOT executed — this can leave a service un-restarted; `meta: flush_handlers` can force handlers to run immediately if needed.

Fig F: Handler notify flow — a diagram: "Task: copy index.html" → "changed = true?" → branches to "YES → notify fires, Handler queued" or "NO → handler skipped entirely" → both paths converge at "End of play: queued handler (Restart web server) runs ONCE". The handler only fires when the notifying task actually reports 'changed', and runs once at the end of the play.

💡 **Layman Explanation:**

A handler is like a 'only ring this bell if something actually changed' rule. If you copy an IDENTICAL file that was already there, nothing changed, so the bell (handler) never rings, and the service is not needlessly restarted — avoiding pointless downtime.

🎯 **Scenario-Based Interview Questions**

Q1: You notify a handler named 'Restart web server' but it never seems to run, even after the copy task reports 'changed'. What is the most likely bug? A mismatch between the `notify:` string and the handler's `name:` string — even a small typo, extra space, or case difference breaks the match silently. Fix: make the notify value and the handler name character-for-character identical.

Q2: A playbook updates an nginx config file and restarts nginx via a handler, but you want the restart to happen immediately after the config task instead of waiting until the end of the whole play. How? Insert `meta: flush_handlers` as a task right after the config-copy task — this forces any pending notified handlers to run immediately instead of waiting for the natural end-of-play trigger.

Q3: Why use a handler instead of simply adding an unconditional 'service: state=restarted' task after every config change? An unconditional restart task runs EVERY single time the playbook runs, even when nothing changed — causing unnecessary brief downtime/blips on every deployment. A handler only restarts when the underlying task reports a real change, keeping restarts minimal and safe.

---

15. Conditionals — the 'when' Statement

Conditionals let a task run only if a certain condition is true — based on a variable, a fact about the host (OS, memory), or the result of a previous task.

YAML

```yaml
---
- name: first playbook
  hosts: all
  become: yes
  vars:
    a: 5
  tasks:
    - name: install httpd software
      yum:
        name: httpd
        state: latest
      when: a == 5
    - name: start web server
      service:
        name: httpd
        state: started
    - name: copying the files
      copy:
        src: index.html
        dest: /var/www/html/index.html
    - name: restart server
      service:
        name: httpd
        state: restarted
```

Corrected: 'when:' takes the condition directly on the same/next line (no extra colon needed on a separate line)

```yaml
# Conditional using a gathered fact
- name: install httpd only on RedHat family OS
  yum:
    name: httpd
    state: latest
  when: ansible_facts['os_family'] == "RedHat"

# Multiple conditions (AND)
  when:
    - ansible_facts['distribution'] == "Ubuntu"
    - ansible_facts['distribution_major_version'] == "22"

# OR condition
  when: a == 5 or a == 10
```

Real-world conditional patterns

🔴 **Ansible Concept**

Facts (gathered automatically by the `setup` module at the start of every play, unless `gather_facts: false`) are the most common source of real conditionals — e.g. only install a package on the correct OS family. A list under `when:` is implicitly AND-ed together — every condition in the list must be true for the task to run.

Fig G: Conditional task execution (when:) — a diagram: "Task reaches 'when:' check" branches "yes" to "Condition TRUE → task runs" (green) and "no" to "Condition FALSE → task SKIPPED" (red). The when: check acts as a gate; a false condition simply skips the task (not a failure).

🎯 **Scenario-Based Interview Questions**

Q1: One playbook needs to target a mixed fleet of RHEL and Ubuntu servers, but the package manager task differs (yum vs apt). How do conditionals solve this cleanly? Write two tasks — one using the yum module `when: ansible_facts["os_family"] == "RedHat"`, and one using the apt module `when: ansible_facts["os_family"] == "Debian"`. Only the task matching the actual host's OS family executes on each host, in a single unified playbook.

Q2: You only want a task to run on hosts with less than 2GB of RAM (to install a lightweight variant of an app). How would you express that? `when: ansible_facts['memtotal_mb'] < 2048` This relies on the 'setup' module's automatically gathered `memtotal_mb` fact — no manual input needed per host.

---

16. Jinja2 Templates (template module)

The `copy` module moves a file byte-for-byte, unchanged. The `template` module is different: it takes a Jinja2 (.j2) file, substitutes in live variable values, and writes the RESULT onto the managed node. This is essential for config files that differ per server (ports, hostnames, memory limits, etc.).

JINJA2

```
# templates/httpd.conf.j2
ServerName {{ ansible_facts['hostname'] }}
Listen {{ http_port }}

{% if enable_ssl %}
SSLEngine on
{% endif %}
```

A simple Jinja2 template file

```yaml
- name: deploy httpd config from template
  template:
    src: httpd.conf.j2
    dest: /etc/httpd/conf/httpd.conf
  vars:
    http_port: 8080
    enable_ssl: true
  notify: Restart web server
```

Using the template module — same 'src/dest' shape as copy, but renders variables & logic first

💡 **Layman Explanation:**

'copy' is like photocopying a fixed page — every server gets an identical copy. 'template' is like a mail-merge letter — the same base document, but each server's copy gets its own hostname, port number, or settings automatically filled in.

📝 **Theory & Key Points:**

- Jinja2 templates support `{{ variables }}`, `{% if %}` / `{% for %}` logic blocks, and filters (e.g. `{{ name | upper }}`) — powerful enough to generate entire config files dynamically from a single template plus per-host variables.

🎯 **Scenario-Based Interview Questions**

Q1: 50 web servers each need a slightly different 'Listen' port in their httpd config, but the rest of the config is identical. copy vs template — which do you use? template — define `http_port` as a `host_vars` value per server, reference `{{ http_port }}` inside a single shared `httpd.conf.j2`, and Ansible renders a correctly customised file for each host from ONE template.

Q2: How would you conditionally include an SSL block in a config template only for servers where a variable `enable_ssl` is true? Wrap the SSL lines in Jinja2 logic inside the .j2 file: `{% if enable_ssl %} ... {% endif %}` — the template module evaluates this per host, so only hosts with `enable_ssl: true` get that block rendered in.

17. Ansible Vault

Ansible Vault encrypts sensitive data — passwords, API keys, certificates — so secrets can be safely stored in Git alongside the rest of your automation code, instead of sitting in plain text.

```bash
# Create a NEW encrypted file (prompts for a vault password)
ansible-vault create secret.yaml

# Edit an already-encrypted file
ansible-vault edit secret.yaml

# View contents without permanently decrypting
ansible-vault view secret.yaml

# Encrypt an EXISTING plain-text file
ansible-vault encrypt secret.yaml

# Decrypt permanently back to plain text
ansible-vault decrypt secret.yaml

# Change the vault password
ansible-vault rekey secret.yaml

# Run a playbook that references vault-encrypted variables
ansible-playbook site.yml --ask-vault-pass
ansible-playbook site.yml --vault-password-file ~/.vault_pass.txt
```

Ansible Vault commands (corrected & expanded)

Fig 3: Ansible Vault workflow — a diagram: "Plain Secret (secret.yml, password: 123)" → "ansible-vault encrypt / create (vault password)" → "Encrypted File ($ANSIBLE_VAULT;1.1; AES256 ....)" → "Safe to commit to Git repo". A caption notes: `ansible-vault edit / decrypt / view` *requires the same vault password to read or modify. A plain secret file is encrypted with AES256 using a vault password before it is safe to commit to a Git repository.*

⚠️ **Important Warning:**

Never commit the VAULT PASSWORD itself to Git — only the encrypted file is safe to commit. Store the vault password in a secrets manager or a `--vault-password-file` kept OUTSIDE the repo (and outside version control), or use `--ask-vault-pass` interactively.

🎯 **Scenario-Based Interview Questions**

Q1: Your team needs to store a database password inside a playbook variable, and this repo is on a shared GitHub org. How do you keep the password safe? Put the password in a YAML file (e.g. `secrets.yml`) and encrypt it: `ansible-vault encrypt secrets.yml`. Reference the variable normally from your playbook (`{{ db_password }}`) and include the encrypted file with `vars_files`. Only the vault password holder(s) can ever read or edit the real value — the committed file itself is unreadable ciphertext.

Q2: A CI/CD pipeline needs to run an Ansible playbook automatically, with nobody present to type the vault password interactively. How do you handle this? Store the vault password in the CI system's secret store, write it to a temporary file at pipeline runtime, and pass `--vault-password-file <path>` to `ansible-playbook` — avoiding any interactive prompt.

Q3: You need to rotate (change) the vault password for an existing encrypted file without losing its contents. What command do you run? `ansible-vault rekey secret.yaml` — it prompts for the OLD password to decrypt, then a NEW password to re-encrypt, all in one step.

---

18. Include & Import

Large playbooks quickly become unmanageable as one giant file. include/import lets you split tasks and even whole plays across multiple files and pull them back together.

	IMPORT_TASKS / IMPORT_PLAYBOOK	INCLUDE_TASKS / INCLUDE_PLAYBOOK
Processing	STATIC — processed at parse time, before the play starts	DYNAMIC — processed at runtime, as the play executes
Tags/when scope	Applies immediately to every task inside	Conditions evaluated per included task, at the moment it runs
Loops	Cannot use a loop on the import statement itself	CAN loop over an include_tasks statement
Best for	Fixed, always-needed task sets	Conditional or dynamically-selected task sets

Fig H: import_tasks (static, resolved before the play runs) vs include_tasks (dynamic, resolved as the play executes) — two side-by-side boxes: "import_tasks — processed at PARSE time (before play starts)" leading to "Best for: fixed, always-needed tasks"; and "include_tasks — processed at RUN time (as play executes)" leading to "Best for: conditional / looped tasks".

YAML

```yaml
- name: main play
  hosts: all
  tasks:
    - import_tasks: install_common.yml
    - include_tasks: os_specific_setup.yml
      when: ansible_facts['os_family'] == "RedHat"
```

Reference: https://github.com/nareshdevopscloud/devops-ansible/tree/main/include_module

🎯 **Scenario-Based Interview Questions**

Q1: You have a huge 300-line playbook mixing package installs, user setup, and firewall rules for three different app types. How would you make this maintainable? Split it into logical task files (`install.yml`, `users.yml`, `firewall.yml`) and pull them into a slim main playbook using `import_tasks` / `include_tasks` — each file becomes independently readable, testable, and reusable across other playbooks.

Q2: Why might include_tasks be a better choice than import_tasks when a task file should only run for certain hosts based on a runtime-gathered fact? `include_tasks` is dynamic and evaluates its 'when' condition at actual runtime — appropriate when the decision depends on facts only known once the play is running. `import_tasks` is static and resolved before the play even starts, so it is better suited to task sets that are unconditionally always needed.

---

19. Ansible Roles

Roles are the standard way to package and reuse a complete piece of automation — tasks, handlers, templates, files, variables, and metadata — all in one self-contained, shareable directory structure.

```bash
ansible-galaxy init test
```

Scaffolds a full role directory structure named 'test'

Fig 4: Standard Ansible role directory layout generated by ansible-galaxy init:

```text
test/    (role name)
├── tasks/main.yml
│       -> main list of tasks for the role
├── handlers/main.yml
│       -> handlers triggered by notify
├── templates/
│       -> jinja2 (.j2) template files
├── files/
│       -> static files used by copy module
├── vars/main.yml
│       -> high priority role variables
├── defaults/main.yml
│       -> low priority default variables
├── meta/main.yml
│       -> role metadata & dependencies
└── README.md
```

🔴 **Ansible Concept**

A playbook uses a role simply by listing it: `roles: [ test ]` under a play — Ansible automatically finds and runs `tasks/main.yml`, loads `defaults/main.yml`, and makes templates/files available, with zero extra wiring. Roles can be shared and reused via Ansible Galaxy (https://galaxy.ansible.com) or a private Git repo, referenced in a `requirements.yml` file and installed with `ansible-galaxy install -r requirements.yml`.

Reference example: https://github.com/nareshdevopscloud/devops-ansible/tree/main/roles/first_role

💡 **Layman Explanation:**

A role is a pre-packaged meal kit — everything needed to 'make a web server' (packages, config templates, restart logic, default settings) is bundled in one labelled box. Any playbook can just say 'use the webserver role' and get the entire kit, instead of copy-pasting the same 40 lines of tasks into every new project.

🎯 **Scenario-Based Interview Questions**

Q1: Three different playbooks across three different projects all install and configure the same monitoring agent, with slightly duplicated task code in each. How do roles fix this? Extract the monitoring-agent logic into a single reusable role (tasks, templates, defaults). Each of the three playbooks simply references `roles: [ monitoring_agent ]` — one place to fix bugs or add features, automatically picked up everywhere it's used.

Q2: A role needs a default value for a variable (e.g. `agent_port: 9100`) that individual consuming playbooks should be able to override easily. Where does that default belong? `roles/<role>/defaults/main.yml` — this is the LOWEST precedence location for variables, specifically designed to be easily overridden by vars set anywhere else (play vars, group_vars, `-e`, etc.).

Q3: Your team wants to publish an internal 'company-baseline-security' role so every project can pull it in via Git instead of copy-pasting it. What's the mechanism? Push the role to a Git repo, then reference it in each consuming project's `requirements.yml` (source: git URL), and run `ansible-galaxy install -r requirements.yml` to pull it in before running the playbook.

---

20. Dynamic Inventory & Collections

20.1 Dynamic Inventory

A STATIC inventory (a plain hosts file you edit by hand) breaks down in cloud environments where servers are created and destroyed constantly by Auto Scaling Groups. A DYNAMIC inventory instead queries the cloud provider's API in real time to build the host list automatically.

YAML

```yaml
# inventory_aws_ec2.yml (dynamic inventory plugin config)
plugin: amazon.aws.aws_ec2
regions:
  - ap-south-1
keyed_groups:
  - key: tags.Role
    prefix: role
```

```bash
# run against it
ansible-inventory -i inventory_aws_ec2.yml --graph
ansible-playbook -i inventory_aws_ec2.yml site.yml
```

AWS EC2 dynamic inventory plugin example

20.2 Collections

Modules, plugins, and roles are now distributed as versioned bundles called **Collections** (e.g. `amazon.aws`, `community.general`) — this is how modern Ansible supports third-party integrations without bloating the core package.

```bash
ansible-galaxy collection install amazon.aws
ansible-galaxy collection install community.general
```

Installing collections

> 🔴 **Ansible Concept**
>
> Since Ansible 2.10+, most cloud/vendor-specific modules moved OUT of the core 'ansible' package and INTO separate Collections — this is why cloud automation almost always starts with `ansible-galaxy collection install ...`.

> 🎯 **Scenario-Based Interview Questions**
>
> **Q1: Your Auto Scaling Group adds and removes EC2 instances every few minutes based on load. A static `/etc/ansible/hosts` file quickly goes stale. What's the fix?** Switch to a dynamic inventory using the `amazon.aws.aws_ec2` plugin — it queries the live EC2 API at run time, so the host list is always accurate without any manual editing.
>
> **Q2: You try to use the amazon.aws EC2 module in a playbook and get a 'module not found' error, even though Ansible itself is installed and up to date. Why?** The `amazon.aws` collection is a separate package from core Ansible and must be installed explicitly: `ansible-galaxy collection install amazon.aws`.

21. Idempotency, Error Handling & Blocks

21.1 Idempotency (recap & formal definition)

📝 **Theory & Key Points:**

- Idempotency means: running the same automation once, or a hundred times, always leaves the system in the SAME end state, and only reports 'changed' on runs that actually did something.
- This is what makes it safe to re-run a playbook at any time — for drift correction, disaster recovery, or simply re-applying it as a scheduled 'compliance check'.

21.2 ignore_errors & failed_when

YAML

```yaml
- name: this task may fail on some hosts, continue anyway
  command: /opt/scripts/optional_cleanup.sh
  ignore_errors: true

- name: custom failure condition
  command: /opt/scripts/healthcheck.sh
  register: result
  failed_when: "'ERROR' in result.stdout"
```

21.3 Blocks — Group Tasks with rescue/always

```yaml
- name: safe deployment with rollback
  block:
    - name: deploy new app version
      copy:
        src: app_v2.jar
        dest: /opt/app/app.jar
    - name: restart app
      service:
        name: myapp
        state: restarted
  rescue:
    - name: roll back to previous version on failure
      copy:
        src: app_v1.jar
        dest: /opt/app/app.jar
    - name: restart app after rollback
      service:
        name: myapp
        state: restarted
  always:
    - name: send deployment notification
      debug:
        msg: "Deployment attempt finished"
```

block/rescue/always — Ansible's equivalent of try/catch/finally

🔴 **Ansible Concept**

`block` groups related tasks together (also useful for applying one `when` or `become` to several tasks at once). `rescue` runs ONLY if a task inside the block fails — perfect for automated rollback logic. `always` runs no matter what happened — success, failure, or rescue — ideal for cleanup or notifications.

Fig I: block/rescue/always — Ansible's try/catch/finally equivalent for safe, self-healing deployments. A diagram: "block: deploy + restart" branches to "SUCCESS → skip rescue" (green) or "FAILURE →

rescue: rollback" (red); both paths converge into "always: notify / cleanup" (purple).

🎯 **Scenario-Based Interview Questions**

Q1: A deployment playbook copies a new app JAR and restarts the service. If the restart fails (bad build), you want to AUTOMATICALLY roll back to the previous working version. How? Wrap the deploy + restart steps in a `block`. Add a `rescue` section that copies the previous known-good JAR back and restarts the service. Optionally add an `always` section to send a Slack/email notification regardless of outcome.

Q2: A cleanup task is expected to occasionally 'fail' on hosts where there's nothing to clean up, and that's fine — it shouldn't stop the whole playbook. How do you handle this? Add `ignore_errors: true` to that specific task so a non-zero exit there doesn't halt the rest of the play on that host.

Q3: A health-check script always exits 0 (success) even when it prints 'ERROR: service down' in its output — so Ansible always reports the task as successful, hiding real failures. Fix? Register the task's output, then use `failed_when` to define a CUSTOM failure condition based on the actual output text, e.g. `failed_when: "'ERROR' in result.stdout"` — now Ansible correctly flags it as failed even though the exit code was 0.

---

22. Best Practices & Quick Reference

22.1 Best Practices

- Keep secrets in Ansible Vault — never commit plain-text passwords/keys to Git.
- Prefer proper idempotent modules (`yum`, `copy`, `service`, `template`) over `command`/`shell` whenever a module already exists for the job.
- Organize large projects using Roles instead of one giant playbook.
- Always test with `--check` and `--diff` before applying against production.
- Use meaningful `name:` fields on every task/play — this becomes your audit trail and makes playbook output readable.
- Pin collection/role versions in `requirements.yml` for repeatable builds.
- Use `group_vars`/`host_vars` instead of hardcoding values inside tasks.
- Store inventories and playbooks in Git — treat automation as code, with code review.

22.2 Command Quick Reference

TASK	COMMAND
Check connectivity	`ansible all -m ping`
Run ad-hoc command	`ansible <group> -m <module> -a "<args>" -b`
Run a playbook	`ansible-playbook -i inventory site.yml`
Dry run / preview	`ansible-playbook site.yml --check --diff`
Run specific tags	`ansible-playbook site.yml --tags a,b`
Skip tags	`ansible-playbook site.yml --skip-tags c`
Limit to one host	`ansible-playbook site.yml --limit web01`
Encrypt a secrets file	`ansible-vault encrypt secrets.yml`
Run with vault secrets	`ansible-playbook site.yml --ask-vault-pass`
Scaffold a role	`ansible-galaxy init rolename`
Install a collection	`ansible-galaxy collection install amazon.aws`

🔴 **Ansible Concept**

Interview recap — be ready to explain in one sentence each: agentless push architecture, idempotency, inventory + groups, playbook = plays + tasks, handlers fire only on 'changed' via notify, vault encrypts secrets, roles package reusable automation, and dynamic inventory for elastic cloud fleets.

© MultiCloud DevOps Notes | Ansible Complete Notes — 45 pages, 22 Sections, 10 Diagrams, 21 Scenario Interview Q&A Sets

---

MultiCloud DevOps Handbook — compiled from the individual Git & GitHub, Terraform, Maven, SonarQube, CI/CD, Docker, Kubernetes, and Ansible study notes by Veera Sir, in reading order: Git → Terraform → Maven → SonarQube → CI/CD → Docker → Kubernetes → Ansible.